

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ
«ХАРКІВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ»

Кафедра «Програмна інженерія та
інтелектуальні технології управління»

ЗВІТ
з лабораторної роботи № 4
з навчальної дисципліни
«WEB-ДОДАТКИ НА ОСНОВІ JAVA»

ВИКОНАВ

Студентка групи КН-222а

Інесса РЕПЕШКО

ПЕРЕВІРИВ

Доцент кафедри ПІІТУ

кандидат технічних наук

Владислав КОВТУН

Харків 2024

ДОСЛІДЖЕННЯ ПРИНЦИПІВ ЗАСТОСУВАННЯ SPRING SECURITY ДЛЯ РЕАЛІЗАЦІЇ АВТЕНТИФІКАЦІЇ, АВТОРИЗАЦІЇ ТА СУПРОВОДЖЕННЯ СЕСІЙ

Мета роботи

- 1 Поглибити знання принципів побудови та застосування Spring Security.
- 2 Набути навичок створення Spring-застосунків, в яких реалізована автентифікація та авторизація з використанням можливостей Spring Security.
- 3 Набути навичок визначення ролей і користувачів системи.
- 4 Набути навичок створення системи авторизації засобами Spring Security.
- 5 Набути навичок застосування можливостей класів, що відповідають за підтримку сесій під час роботи із Spring-застосунками.
- 6 Набути навичок додавання тегів Spring Security на вебсторінках, що створюються з використанням Thymeleaf.
- 7 Удосконалити навички розробки user-friendly застосунків.

Завдання роботи

Додати до Spring-застосунку, створеного під час виконання ЛР№3, функціональність, яка забезпечує авторизацію та автентифікацію з використанням Spring Security.

Застосунок має забезпечувати:

- реєстрацію користувачів у системі збереженням їх логінів та паролів;
- вхід до системи лише для зареєстрованих користувачів;
- надання доступу до даних та можливість виконання CRUD-операцій відповідно до рівня доступу, визначеного під час реєстрації;
- відображення імені користувача, який увійшов до системи;
- обмеження часу перебування зареєстрованого користувача у системі.

Реалізацію даної лабораторної роботи можна поділити на декілька етапів описаних нижче.

- 1 Реалізація системи безпеки з використанням Spring Security:

1) конфігурація ролей та користувачів:

а) оголосити ролі користувачів (ADMIN, USER);

б) налаштувати зберігання користувачів у базі даних або в конфігураційних файлах для тестування;

2) реалізація доступу на основі ролей:

а) налаштувати доступ до ресурсів та операцій для різних ролей з використанням Spring Security;

б) користувачі ADMIN мають доступ до всіх функцій, користувачі USER – лише до перегляду даних;

3) конфігурація авторизації та автентифікації:

а) налаштувати форму входу та механізм автентифікації користувачів;

б) застосувати хешування паролів для зберігання у базі даних.

4) захист даних та інтерфейсу: сховати елементи управління (CRUD) для користувачів з роллю USER.

2 Використання Spring для підтримки сесій:

1) налаштування сесій: встановити механізм для управління сесіями та відстеження активності користувачів;

2) відображення імені користувача та ролі: відображати ім'я та роль користувача після успішної автентифікації;

3) обмеження часу активності користувачів: налаштувати обмеження часу сесії, автоматичне блокування доступу після бездіяльності.

3 Реалізація функціоналу додавання нових користувачів:

1) інтерфейс для додавання нових користувачів: розробити інтерфейс для створення нових користувачів, доступний тільки для ADMIN;

2) реалізація функціоналу додавання користувачів: створити механізм для реєстрації користувачів та перевірки введених даних;

3) конфігурація Spring Security для нових користувачів: забезпечити автентифікацію нових користувачів з використанням Spring Security.

Умови варіанту № 1 індивідуального завдання реалізованого у даному проєкті представлені на рис. 1.

Варіант №1 (ЗВО №1)			
Опис структури полів класу «Abiturient» (суттєвість 1)			
Ім'я поля	Тип даних	Опис	Властивості поля
id	Long	Сурогатний ключ	PK
p_code	String	Номер особової справи	Маска номеру "KH23-NNNN", унікальне для кожного студента
surname	String	Прізвище	Not null
name	String	Ім'я	Not null
pname	String	По-батькові	Not null
birthday	LocalDate	Дата народження	Not null
gender	boolean	Стать	Not null; true – чоловіча, false - жіноча
rating	double	Рейтинговий бал	Not null від 120.01 до 200.00
Опис структури полів класу «StudentDopInfo» (суттєвість 2)			
Ім'я поля	Тип даних	Опис	Властивості поля
id	Long	Сурогатний ключ	PK
student	Abiturient	Студент-власник особистої інформації	@OneToOne (FK до Student)
contract	boolean (можна String)	Контракт чи бюджет?	Not null. Перелік значень: «контракт»; «бюджет»
scholarship	boolean (можна String)	Є стипендія чи немає?	Not null. Перелік значень: «стипендія»; «ні». Якщо студент на контракті, то встановлюється значення «ні».
email	String	Адреса у корпоративній пошті НТУ «ХПІ»	Not null. Unique. Складається з транскрипції імені та прізвища студента, домен - @cs.khpi.edu.ua

Рисунок 1 – Умова варіанту № 1 індивідуального завдання

Хід роботи

1 Створення та налаштування проєкту

Створення нового Spring-проєкту в IntelliJ IDEA та його налаштування може виконуватись за наступними кроками.

- 1 Запустити IntelliJ IDEA і оберіть “New Project”.
- 2 Налаштувати основні параметри проєкту (рис. 2).
- 3 Натиснути “Next” і вибрати версію платформи та залежності для проєкту (рис. 3).

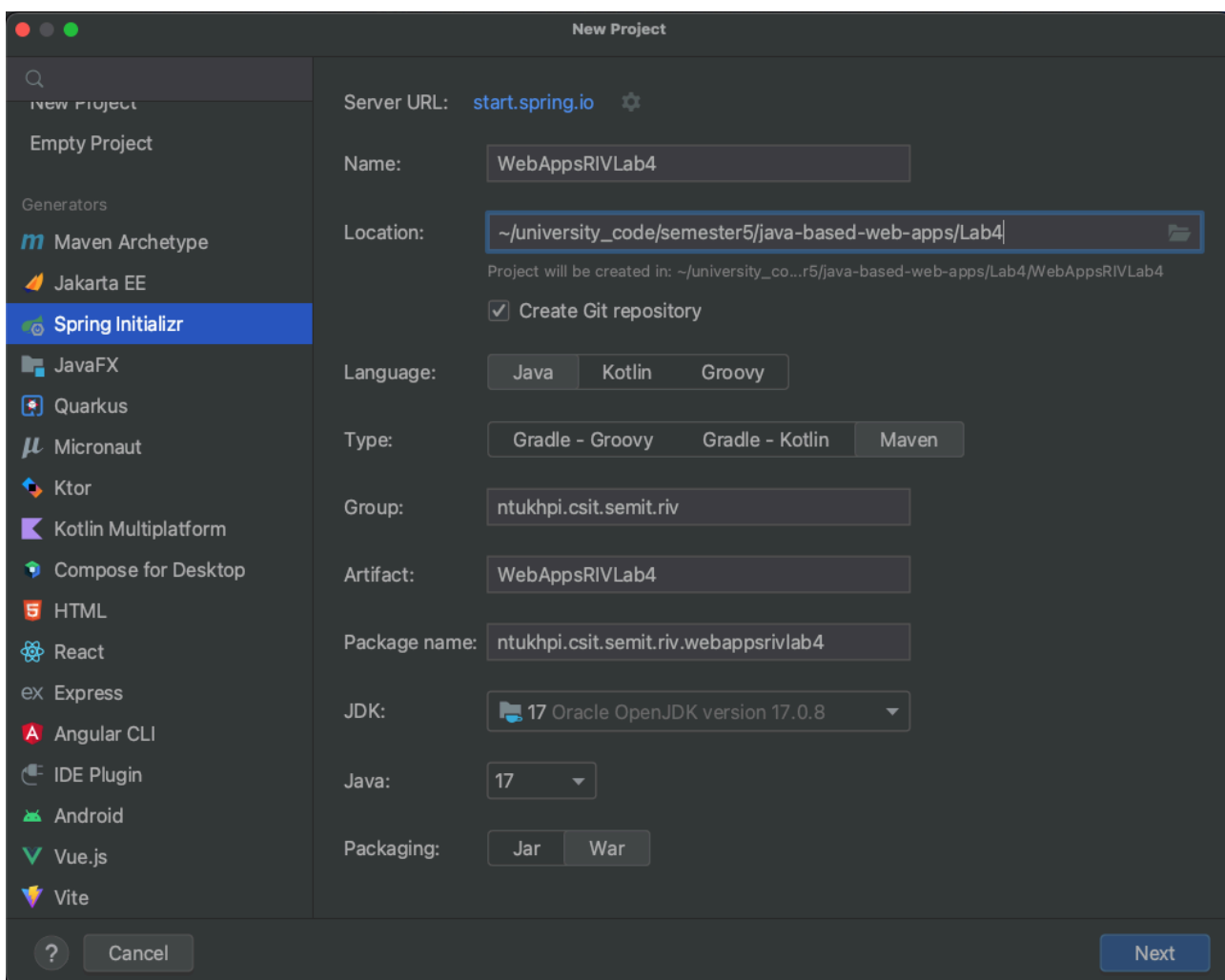


Рисунок 2 – Налаштування основних параметрів Spring-проєкту в IntelliJ IDEA

4 Після натискання “Create” проєкт буде створено і відкрито в IntelliJ IDEA зі стандартною структурою.

Примітка. Налаштування на рисунках 2-3 є лише рекомендованими, але за необхідності можна використовувати інші комбінації, переконавшись у сумісності обраних компонентів.

Також в процесі реалізації завдань лабораторної роботи були додані додаткові залежності до pom.xml файлу. У додатку A.1 представлений файл pom.xml конфігурації проєкту, що містить повний перелік залежностей реалізованого проєкту.

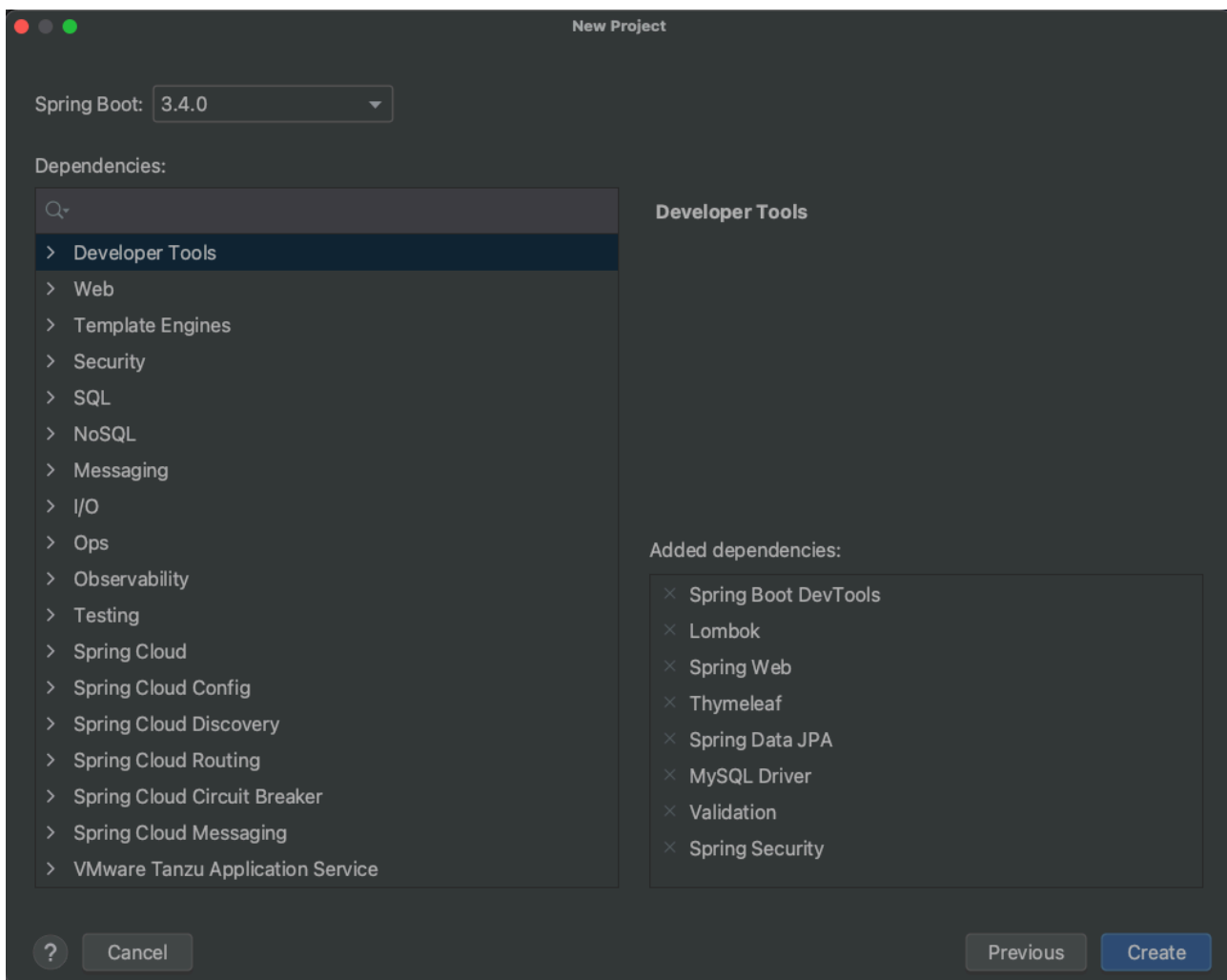


Рисунок 3 – Сторінка вибору версії платформи та залежностей проєкту в IntelliJ IDEA частина 1

2 Архітектура проєкту

Реалізований проєкт є Spring Boot вебзастосунком, реалізованим на Java. Він включає автентифікацію та авторизацію з використанням Spring Security. Застосунок працює з базою даних, у якій зберігаються дані абітурієнтів, студентів та користувачів. У даному Spring Boot проєкті реалізовано багат шарову архітектуру (Multi-layered Architecture) що представлена на рис. 4-6 далі в тексті роботи.

Проєкт розділений на декілька основних рівнів, що відповідають основним принципам розробки багат шарових (multilayered) застосунків описаних нижче.

1 Конфігураційний рівень (config):

- 1) містить конфігураційні класи, такі як EncoderConfig, SecurityConfig, і UserDetailsServiceImpl;

- 2) налаштовує системні параметри, зокрема, безпеку (Spring Security) та сервіси шифрування.

2 Рівень контролера (controller):

- 1) включає класи контролерів, такі як AuthController, HomeController, EntrantController, UserController та інші;

- 2) відповідає за обробку HTTP-запитів та взаємодію з користувачем.

3 Рівень моделі (model):

- 1) DAO (Data Access Objects): Містить класичні моделі, які представляють таблиці бази даних:

- a) entrant: клас Entrant;

- б) student: клас Student;

- в) user: клас UserEntity, що представляє користувачів системи;

- 2) enums: перелік (enum) ролей користувачів (Role), типу фінансування (FundingType) та статусу стипендії (ScholarshipStatus);

- 3) DTO (Data Transfer Objects): включає класи для передачі даних, такі як UserChangeCredentials, UserLogin, UserRegister тощо.

4 Рівень репозиторію (repository):

- 1) включає інтерфейси для взаємодії з базою даних, такі як EntrantRepository, StudentRepository, UserRepository;

- 2) репозиторії є посередниками між базою даних та сервісами, що забезпечують CRUD-операції.

5 Рівень сервісу (service): сервіси логіки бізнесу, що включають EntrantServiceImpl, StudentServiceImpl, UserServiceImpl та інші сервіси, що реалізують логіку взаємодії з репозиторіями та управління даними.

6 Валідаційний рівень (validator):

- 1) містить власні анотації та валідатори для валідації даних, зокрема ValidId, ValidBirthday, ValidUsername тощо;

2) валідаційні класи забезпечують відповідність даних певним правилам та обмеженням, встановленим у системі.

7 Рівень представлення (view) та ресурси (resources):

- 1) static: зберігає статичні файли, такі як іконки застосунку;
- 2) templates: HTML-файли, що використовуються у фронтенд частині застосунку та відображають інформацію для користувачів (форми входу, таблиці студентів, тощо);
- 3) конфігураційний файл `application.properties` визначає налаштування для Spring Boot проєкту (код файлу представлений у додатку A.2);
- 4) конфігураційний файл `logback.xml` визначає для налаштування логування (код файлу представлений у додатку A.3);
- 5) конфігураційний файл `Maven pom.xml` для налаштування залежностей проєкту (код файлу представлений у додатку A.1).

Реалізований проєкт також містить супровідні компоненти описані нижче.

1 Запуск Spring Boot вебзастосунку. Наступні класи важливі для конфігурації та запуску Spring Boot застосунку:

- 1) клас `WebAppsRIVLab4Application` є основним класом, який запускає застосунок, виконує початкову логіку для збереження Entrant та Student в базу даних і логування даних через `CommandLineRunner` (код файлу представлений у додатку A.4);
- 2) клас `ServletInitializer` забезпечує можливість розгортання Spring Boot застосунку як традиційного вебзастосунку (WAR-файл) у контейнері сервлетів (код файлу представлений у додатку A.5).

2 Тести. Папка “test” містить JUnit тести для перевірки функціональності сервісів та основного застосунку:

- 1) класи `EntrantServiceTest` та `StudentServiceTest` реалізують тестування CRUD операцій у сервісах обох сутностей `EntrantService` та `StudentService`;

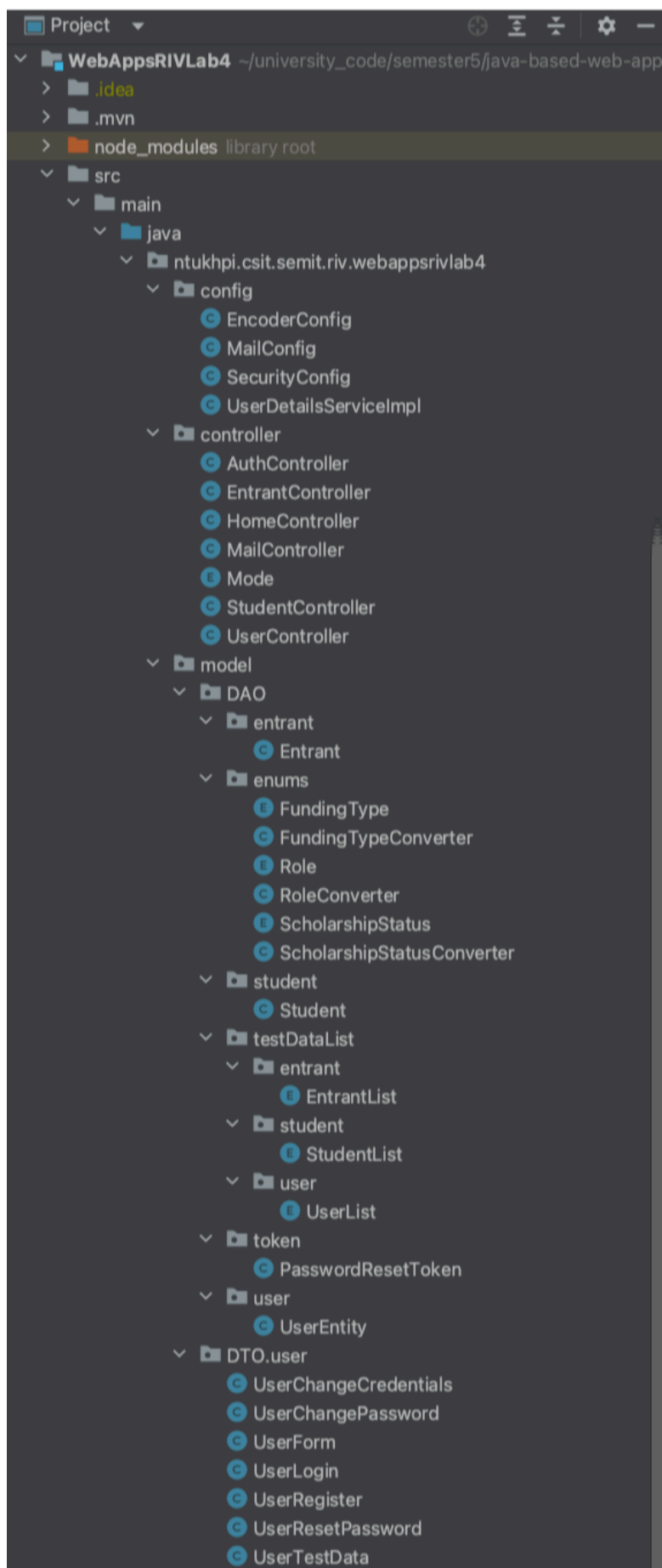


Рисунок 4 – Структура проекту WebAppsRIVLab4 частина 1

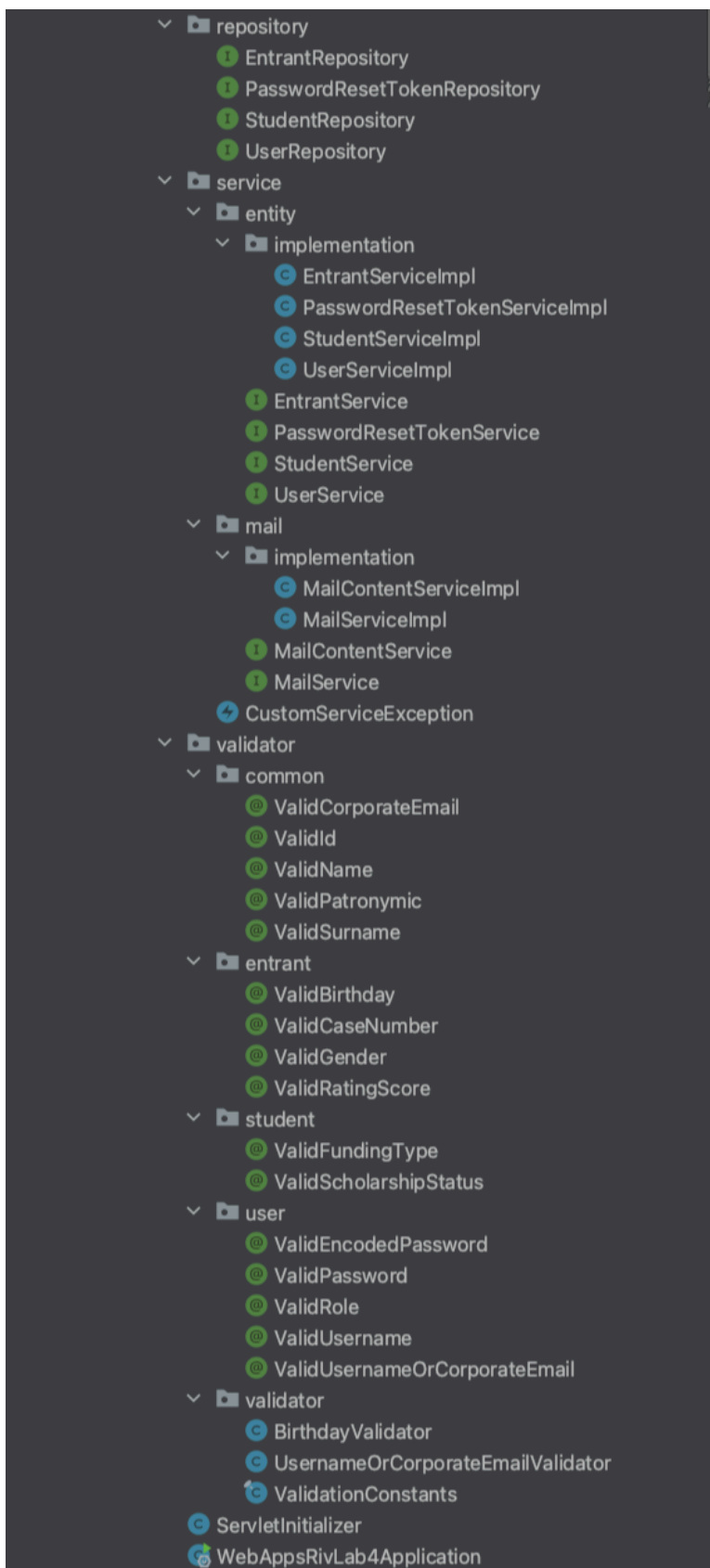


Рисунок 5 – Структура проекта WebAppsRivLab4 часть 2

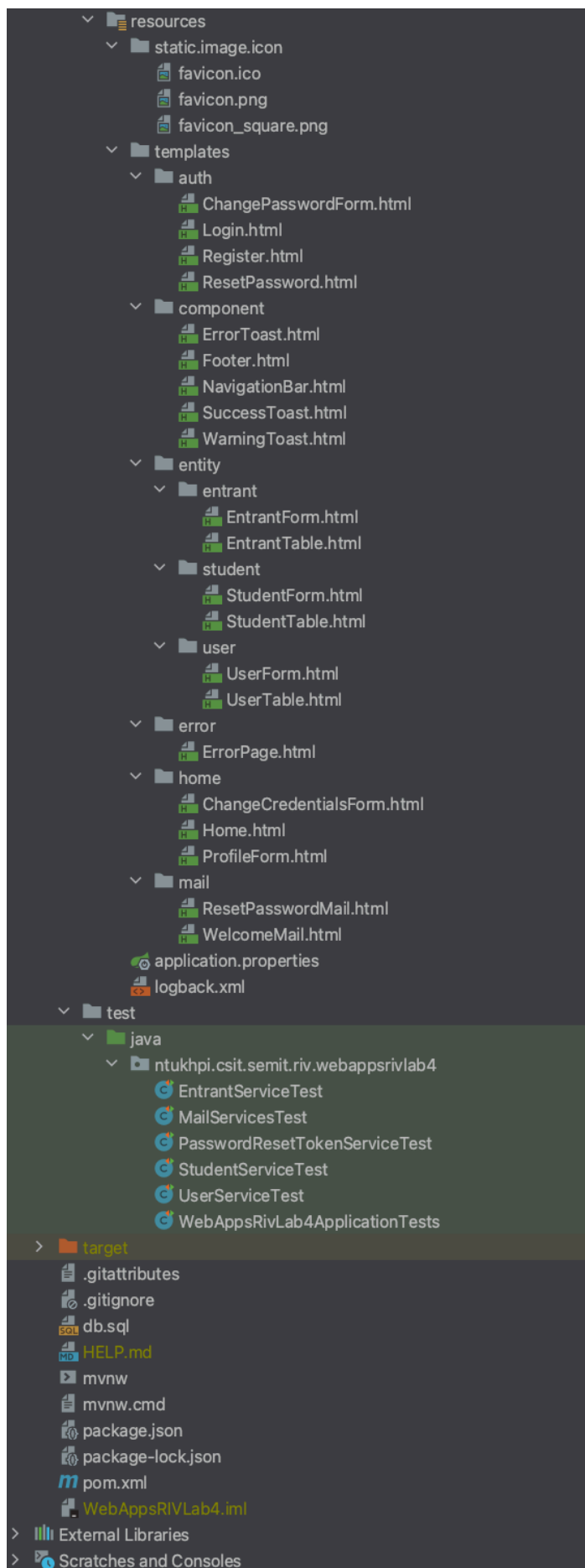


Рисунок 6 – Структура проекту WebAppsRIVLab4 частина 3

2) клас `WebAppsRIVLab4ApplicationTests` реалізує тестування загальної функціональності застосунку.

3 Конфігурація та допоміжні файли:

1) конфігураційний файл `application.properties` визначає налаштування для Spring Boot проєкту (код файлу представлений у додатку A.2);

2) конфігураційний файл `logback.xml` визначає для налаштування логування (код файлу представлений у додатку A.3);

3) конфігураційний файл Maven `pom.xml` для налаштування залежностей проєкту (код файлу представлений у додатку A.1).

Реалізована структура забезпечує чітке розділення відповідальностей, що полегшує його підтримку та масштабування.

3 Схема бази даних

Схема створеної бази даних на основі DBMS MySQL відображена далі в тексті роботи рис. 10. Структура створених таблиць `entrants`, `students` та `users` наведена на рис. 7-9 відповідно далі в тексті роботи.

columns (entrants)

	Name	Comment	Data Type	Not Null	Auto Increment	Column Kind	Default Expression	Hidden	On Update	Position	Collation
1	birthday		date	true		NORMAL	'2000-01-01'	false		1	null
2	gender		bit(1)	true		NORMAL	b'1'	false		2	null
3	rating_score		double	true		NORMAL	120.001	false		3	null
4	id		bigint	true	1	NORMAL		false		4	null
5	student_id		bigint	false		NORMAL		false		5	null
6	case_number		varchar(20)	true		NORMAL		false		6	null
7	name		varchar(50)	true		NORMAL		false		7	null
8	patronymic		varchar(50)	true		NORMAL		false		8	null
9	surname		varchar(50)	true		NORMAL		false		9	null

keys (entrants)

	Name	Primary
1	PRIMARY	true
2	UKj36ja5h4xym91tmpdw81xno84	false

indexes (entrants)

	Name	Comment	Unique	Type
1	PRIMARY		true	btree
2	UKj36ja5h4xym91tmpdw81xno84		true	btree

checks (entrants)

	Name	Predicate
1	entrants_chk_2	('rating_score' > 120.000) and ('rating_score' <= 200.000)
2	entrants_chk_1	'birthday' between _utf8mb4\1914-01-01\ and _utf8mb4\2008-01-01\
3	entrants_chk_3	regexp_like('case_number', _utf8mb4\'^[0DCS-00000`0DCS0IND0SSA0ESA]{1,10}{0-9}{2}-[0
4	entrants_chk_4	regexp_like('name', _utf8mb4\'^(?=.{1,50}\$)[0DCS-00000`0DCS0IND0SSA0ESA](?:[0DCS-0`0
5	entrants_chk_5	regexp_like('patronymic', _utf8mb4\'^(?=.{1,50}\$)[0DCS-00000`0DCS0IND0SSA0ESA](?:[0
6	entrants_chk_6	regexp_like('surname', _utf8mb4\'^(?=.{1,50}\$)[0DCS-00000`0DCS0IND0SSA0ESA](?:[0DCS-

Рисунок 7 – Структура таблиці `entrants`

columns (students)

	Name		Comment		Data Type		Not Null		Auto Increment		Column Kind		Default Expression		Hidden		On Update		Position		Collation	
1	entrant_id				bigint		false				NORMAL				false				1		null	
2	id				bigint		true		1		NORMAL				false				2		null	
3	corporate_email				varchar(123)		true				NORMAL				false				3		null	
4	funding_type				enum('BUDGET', 'CONTRACT')		true				NORMAL		'CONTRACT'		false				4		null	
5	scholarship_status				enum('ENHANCED', 'NONE', 'ORDINARY')		true				NORMAL		'NONE', 'NONE'		false				5		null	

keys (students)

	Name		Primary	
1	PRIMARY		true	
2	Ukebg37l1cg62sdw3n7d7gre9v		false	
3	UKng7nfdg9laddmqsl1be38mkx		false	

foreign keys (students)

	Name		Target Table		On Delete		On Update	
1	fk_student_entrant		entrants		cascade		cascade	

indexes (students)

	Name		Comment		Unique		Type	
1	Ukebg37l1cg62sdw3n7d7gre9v				true		btree	
2	PRIMARY				true		btree	
3	UKng7nfdg9laddmqsl1be38mkx				true		btree	

checks (students)

	Name		Predicate	
1	students_chk_1		regexp_like('corporate_email', '_utf8mb4\''(?=.{17,123}\$)[a-z](.?[a-z]+){1,99}@[a-z]{1,10}.khipi.edu.us\$', '_utf8mb4\''c') = 1	

Рисунок 8 – Структура таблиці students

columns (users)

	Name	Comment	Data Type	Not Null	Auto Increment	Column Kind
1	id		bigint	true	1	NORMAL
2	username		varchar(32)	true		NORMAL
3	name		varchar(50)	true		NORMAL
4	patronymic		varchar(50)	true		NORMAL
5	surname		varchar(50)	true		NORMAL
6	corporate_email		varchar(123)	true		NORMAL
7	password		varchar(255)	false		NORMAL
8	role		enum('ADMIN', 'ENTRANT_MANAGER', 'ENTRANT_VIEWER', 'STUDENT_MANAGER', 'STUDENT_VIEWER')	true		NORMAL

Default Expression

	Hidden	On Update	Position	Collation
	false		1	null
	false		2	null
	false		3	null
	false		4	null
	false		5	null
	false		6	null
	false		7	null
'STUDENT_VIEWER'	false		8	null

keys (users)

	Name	Primary
1	PRIMARY	true
2	UKr43af9ap4edm43mmtq01oddj6	false
3	UKaxfb69mtv3gvjwg69lj0jb93x	false

indexes (users)

	Name	Comment	Unique	Type
1	PRIMARY		true	btree
2	UKr43af9ap4edm43mmtq01oddj6		true	btree
3	UKaxfb69mtv3gvjwg69lj0jb93x		true	btree

checks (users)

	Name	Predicate
1	users_chk_5	regexp_like('corporate_email', '_utf8mb4\''^(?=.{17,123}\$)[a-z](.[a-z]+){1,99}@[a_
2	users_chk_2	regexp_like('name', '_utf8mb4\''^(?=.{1,50}\$)[0ĐČŠ-00000`0ĐČŠ0IND0SSA0ESA](?:[0ĐČŠ-0`0
3	users_chk_3	regexp_like('patronymic', '_utf8mb4\''^(?=.{1,50}\$)[0ĐČŠ-00000`0ĐČŠ0IND0SSA0ESA](?:[0Đ
4	users_chk_4	regexp_like('surname', '_utf8mb4\''^(?=.{1,50}\$)[0ĐČŠ-00000`0ĐČŠ0IND0SSA0ESA](?:[0ĐČŠ-
5	users_chk_1	regexp_like('username', '_utf8mb4\''^(?=.{8,32}\$)[a-zA-Z0-9_.-]+\`', '_utf8mb4\`c\`')_

Рисунок 9 – Структура таблиці users

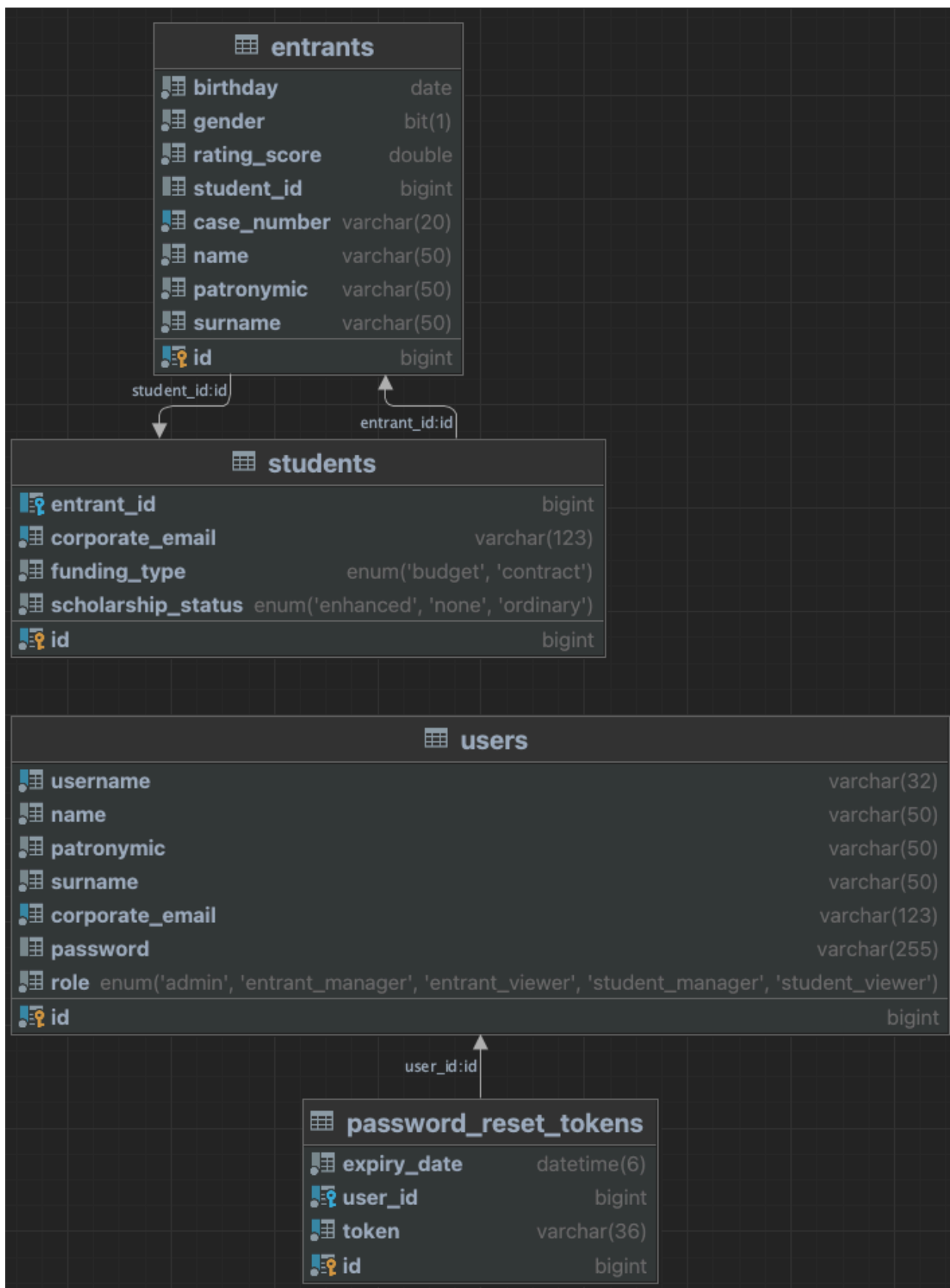


Рисунок 10 – Схема бази даних на основі DBMS MySQL 8.0.30

4 Реалізація конфігурації Spring Boot проєкту

Реалізація рівня конфігурації у Spring Boot проєкті здійснювалася для забезпечення безпеки, автентифікації та авторизації користувачів, а також ефективного управління сесіями та підключенням до необхідних сервісів. Це дозволяє чітко налаштовувати механізми шифрування паролів, керування ролями користувачів та контролю доступу до ресурсів на основі їхніх прав. Використання класів конфігурації, таких як `EncoderConfig`, `UserDetailsServiceImpl` та `SecurityConfig`, сприяє централізації та стандартизації налаштувань безпеки, що підвищує надійність системи та захищає дані від несанкціонованого доступу. Крім того, така організація конфігураційного рівня полегшує підтримку та масштабування проєкту, дозволяючи швидко адаптуватися до змін вимог безпеки та впроваджувати нові функціональні можливості без порушення існуючої логіки. В цілому, впровадження рівня конфігурації значно покращує загальну безпеку та стабільність вебзастосунку, забезпечуючи ефективне управління доступом та захистом даних користувачів.

У даній лабораторній роботі для рівня конфігурації було реалізовано наступні класи описані нижче.

1 **EncoderConfig.java**. Клас `EncoderConfig` відповідає за конфігурацію механізму шифрування паролів у Spring проєкті. Він визначає, як паролі будуть шифруватися перед їх зберіганням у базі даних з використанням алгоритму BCrypt, що є стандартом для безпечного зберігання паролів. Код даного класу представлений у додатку Б.1.

2 **UserDetailsServiceImpl.java**. Клас `UserDetailsServiceImpl` реалізує інтерфейс `UserDetailsService` і відповідає за завантаження даних користувача на основі імені користувача або корпоративної електронної пошти. Він взаємодіє з сервісом `UserService`, щоб отримати користувача з бази даних і перевести його дані в об'єкт `UserDetails`, який використовує Spring Security для аутентифікації та авторизації. Код даного класу представлений у додатку Б.2.

3 **SecurityConfig.java**. Клас `SecurityConfig` є головним конфігураційним класом для налаштування безпеки в проєкті. Він використовує Spring Security

для налаштування автентифікації користувачів, авторизації на основі ролей, управління сесіями та налаштування механізму “remember me”. Також він визначає, які URL-адреси доступні для певних ролей користувачів та встановлює параметри безпеки для сесій. Код даного класу представлений у додатку Б.3.

5 Реалізація контролера Spring Boot проєкту

Реалізація рівня контролера у Spring Boot проєкті здійснювалася для ефективного оброблення запитів користувачів, управління бізнес-логікою через сервіси та забезпечення належного відображення результатів через відповідні представлення. Це дозволяє чітко розділити відповідальності між різними шарами застосунку, що підвищує його підтримуваність та масштабованість. Використання контролерів сприяє організації коду за принципом єдиного обов'язку, що полегшує його розуміння та розвиток. Крім того, контролери забезпечують централізовану обробку помилок та ефективну взаємодію з користувачем, що покращує користувацький досвід та надійність системи. Впровадження рівня контролера також полегшує тестування та налагодження застосунку, оскільки дозволяє ізолювати обробку запитів від бізнес-логіки та доступу до даних. В цілому, реалізація рівня контролера підвищує гнучкість та ефективність розробки, забезпечуючи створення більш організованого, надійного та легко підтримуваного вебзастосунку.

Для реалізації рівня контролера у даний проєкт було перенесено наступні класи з минулої лабораторної роботи описані далі.

1 **Mode.java**. Клас або enum, який описує різні режими представлення, наприклад, для перегляду таблиці, додавання нового запису або редагування існуючого запису. Це дозволяє змінювати вигляд і поведінку інтерфейсу залежно від типу операції. Код даного класу представлений у додатку В.1.

2 **EntrantController.java**. Контролер для управління операціями, пов'язаними з вступниками (entrant). Він забезпечує CRUD-операції для об'єктів "вступник", а також підтримує фільтрацію та сортування даних, відображення

таблиці вступників, а також форму для додавання та редагування вступників. Код даного класу представлений у додатку В.2.

3 **StudentController.java**. Контролер для управління операціями з об'єктами "студент". Він відповідає за відображення списку студентів, додавання нових, редагування та видалення студентів, а також за перегляд їхніх деталей. Крім того, він взаємодіє з сервісом "Entrant" для асоціації студентів з відповідними вступниками. Код даного класу представлений у додатку В.3.

У даній лабораторній роботі для рівня контролера було реалізовано наступні класи описані нижче.

1 **AuthController.java** Цей контролер відповідає за обробку запитів, пов'язаних з авторизацією користувачів, включаючи вхід до системи, реєстрацію та вихід, а також управління сесіями користувачів. Він також відповідає за надання відповідей при невірному введенні або помилках авторизації. Код даного класу представлений у додатку В.4.

2 **HomeController.java**. Цей контролер відповідає за обробку запитів на головну сторінку застосунку. Він забезпечує відображення домашньої сторінки та здійснює перенаправлення на інші сторінки, залежно від ролі користувача. Код даного класу представлений у додатку В.5.

3 **UserController.java**. Контролер для управління операціями з користувачами (User). Він забезпечує CRUD-операції для користувачів, зокрема додавання нових користувачів, редагування їхніх даних, зміну ролей та видалення користувачів. Контролер також працює з формами для введення даних користувачів та здійснює валідацію введених даних. Код даного класу представлений у додатку В.6.

6 Реалізація моделі Spring Boot проєкту

Реалізація рівня моделі у Spring Boot проєкті здійснювалася з метою ефективного управління даними та забезпечення їхнього надійного зберігання в базі даних через чітку структуру об'єктів та їх взаємозв'язків. Використання JPA для мапінгу об'єктів на таблиці бази даних дозволяє автоматизувати стандартні

CRUD-операції, що підвищує продуктивність розробки та знижує ризик виникнення помилок. Крім того, впровадження класів DTO забезпечує безпечну та ефективну передачу даних між різними шарами застосунку, полегшуючи валідацію та обробку інформації. Така організація моделі сприяє абстрагуванню бізнес-логіки від деталей доступу до даних, що підвищує підтримуваність та масштабованість проєкту. В цілому, реалізація рівня моделі покращує надійність, гнучкість та якість програмного забезпечення, забезпечуючи стабільну основу для подальшого розвитку та розширення функціональності вебзастосунку.

Для реалізації рівня моделі у даний проєкт було перенесено класи DAO з минулої лабораторної роботи, що описані далі.

1 **Entrant.java**. Клас Entrant є сутністю, яка відображає абітурієнта у системі. Він містить основні атрибути абітурієнта, такі як номер справи, прізвище, ім'я, по-батькові, дата народження, стать, рейтинг, а також методи для доступу до цих даних і їх змінення. Код класу представлений у додатку Г.1.

2 **EntrantList.java**. Клас EntrantList є синглтоном, що містить попередньо визначений список абітурієнтів для тестування та демонстраційних цілей. Цей клас надає доступ до фіксованого набору даних, з можливістю додавати, видаляти та очищати список абітурієнтів. Код даного класу представлений у додатку Г.2.

3 **FundingType.java**. Клас FundingType є переліком і визначає тип фінансування для студентів або абітурієнтів, зокрема, "бюджет" або "контракт". Цей клас дозволяє в системі обробляти та зберігати інформацію про способи фінансування. Код класу представлений у додатку Г.3.

4 **FundingTypeConverter.java**. Клас FundingTypeConverter реалізує конвертацію типів фінансування між їх об'єктами і рядками, що використовуються для відображення даних у формах або в базі даних. Код класу представлений у додатку Г.4.

5 ScholarshipStatus.java. Клас ScholarshipStatus є переліком і описує статус стипендії, що може бути присвоєно студентам. Це може бути, наприклад, "звичайна", "підвищена" або "відсутня". Код класу представлений у додатку Г.5.

6 ScholarshipStatusConverter.java. Клас ScholarshipStatusConverter відповідає за конвертацію значень статусу стипендії з одного формату в інший, наприклад, з рядка в об'єкт. Код класу представлений у додатку Г.6.

7 Student.java. Клас Student є сутністю, що представляє студента в системі. Він містить основні атрибути студента, включаючи інформацію про фінансування, статус стипендії, корпоративну електронну пошту та дані абітурієнта, з яким студент пов'язаний. Код класу представлений у додатку Г.7.

8 StudentList.java. Клас StudentList є синглтоном, що містить попередньо визначений список студентів для тестування та демонстраційних цілей. Цей клас надає доступ до фіксованого набору даних, з можливістю додавати, видаляти та очищати список студентів. Код класу представлений у додатку Г.8.

У даній лабораторній роботі для рівня моделі було реалізовано наступні класи DAO описані нижче.

1 Role.java. Клас Role є переліком ролей користувачів у системі, де кожна роль визначає специфічні права доступу до сутностей і операцій:

1) роль "ADMIN" надає користувачу доступ до всіх CRUD операцій із сутностями користувачів, абітурієнтів та студентів;

2) роль "ENTRANT_MANAGER" дозволяє користувачу виконувати всі CRUD операції із сутностями абітурієнтів та переглядати (read операції) сутності студентів, що відповідає ролі працівника приймальної комісії чи бухгалтерії в університетській системі;

3) роль "STUDENT_MANAGER" надає користувачу доступ до всіх CRUD операцій із сутностями студентів та перегляд (read операції) сутностей абітурієнтів, що відповідає ролі працівника деканату чи іншого адміністративного персоналу;

4) роль "ENTRANT_VIEWER" дає користувачу лише переглядати сутності абітурієнтів, що може відповідати ролі, наприклад, консультанта

приймальної комісії або інших працівників, які потребують доступу до цієї інформації;

5) роль “STUDENT_VIEWER” надає користувачу доступ лише до перегляду (read операції) сутностей студентів, що може відповідати ролі старости або викладача, які мають потребу в інформації про студентів, але не в праві їх редагувати.

При самостійній реєстрації користувача в системі через реалізований функціонал реєстрації йому присвоюється роль “STUDENT_VIEWER”.

Створена ієрархія ролей дозволяє реалізувати контроль доступу до різних частин системи на основі ролей користувачів. Код класу Role представлений у додатку Г.9.

2 RoleConverter.java. Клас-конвертер для перетворення значень ролей у відповідні елементи переліку та навпаки. Він забезпечує зручне зберігання ролей у базі даних та їх правильну інтерпретацію в бізнес-логіці. Код класу представлений у додатку Г.10.

3 UserEntity.java. Клас сутності, який представляє користувача в системі. Він містить атрибути користувача, такі як ім'я, роль, електронна пошта та пароль, з вбудованими валідаціями для забезпечення безпеки та коректності введених даних. Він також включає методи для перетворення з DTO класів, таких як UserRegister і UserForm. Код класу представлений у додатку Г.11.

4 UserList.java. Синглтон-клас, який надає список попередньо визначених тестових даних користувачів для використання в тестуванні та демонстраційних цілях. Це забезпечує легкий доступ до тестових користувачів для перевірки функціональності додатка без необхідності кожного разу вводити дані вручну. Код класу представлений у додатку Г.12.

У даній лабораторній роботі для рівня моделі було реалізовано наступні класи DTO описані нижче.

1 UserLogin.java. Клас DTO для обробки даних, необхідних для автентифікації користувача в системі. Він включає в себе поля для введення

логіну та пароллю, що використовуються для перевірки особи користувача. Код класу представлений у додатку Г.13.

2 UserRegister.java. Цей клас відповідає за реєстрацію нового користувача в системі. Він містить необхідні поля для збирання даних про користувача, такі як логін, пароль, роль і електронна пошта, а також застосовує валідацію для перевірки коректності цих даних. Код класу представлений у додатку Г.14.

3 UserChangeCredentials.java. Цей клас є DTO для зміни облікових даних користувача. Він містить необхідні атрибути для зміни облікової інформації, зокрема старого та нового паролю, і використовує валідацію для забезпечення коректності введених даних. Код класу представлений у додатку Г.15.

4 UserForm.java. Цей клас представляє форму для перегляду, додавання, оновлення або видалення даних користувача. Він інкапсулює всі поля, необхідні для збору інформації про користувача, та застосовує валідацію для забезпечення коректності введених даних. Код класу представлений у додатку Г.16.

5 UserTestData.java. Цей клас використовується для створення тестових даних користувачів в процесі тестування та валідації. Він містить всю необхідну інформацію для імітації даних користувачів, включаючи логін, пароль, роль, електронну пошту та інші атрибути. Клас застосовує валідацію для забезпечення правильності даних перед використанням в тестах чи при реєстрації. Код класу представлений у додатку Г.17.

7 Реалізація репозиторію Spring Boot проєкту

Реалізація рівня репозиторію в Spring Boot проєкті здійснювалася для забезпечення ефективної та структурованої взаємодії з базою даних, спрощуючи процеси збереження, пошуку, оновлення та видалення об'єктів. Використання інтерфейсів JpaRepository дозволяє автоматизувати стандартні CRUD операції, а також створювати спеціалізовані запити з використанням анотацій @Query, що значно зменшує обсяг необхідного коду та підвищує продуктивність розробки. Крім того, рівень репозиторію забезпечує чітке

розділення відповідальностей між бізнес-логікою та доступом до даних, що сприяє покращенню підтримуваності та масштабованості проєкту. Це також полегшує тестування та налагодження системи, оскільки дозволяє ізолювати доступ до даних від інших компонентів. В цілому, впровадження рівня репозиторію підвищує надійність, гнучкість та ефективність управління даними, що сприяє створенню більш стабільного та легко підтримуваного вебзастосунок.

Для реалізації рівня репозиторію у даний проєкт було перенесено класи з минулої лабораторної роботи, що описані далі.

1 **EntrantRepository.java.** Клас `EntrantRepository` є репозиторієм для роботи з сутністю абітурієнтів у базі даних. Він розширює `JpaRepository`, що дозволяє виконувати базові CRUD операції, а також надає можливість здійснювати запити для фільтрації та сортування абітурієнтів за різними критеріями, зокрема за номером справи або за даними абітурієнтів. Код класу представлений у додатку Д.1.

2 **StudentRepository.java.** Репозиторій `StudentRepository` відповідає за роботу з сутністю студентів. Він забезпечує можливість виконувати базові CRUD операції, а також реалізує кастомні методи для пошуку та сортування студентів на основі різних критеріїв, таких як тип фінансування, статус стипендії та загальний пошук за іменем. Код даного класу представлений у додатку Д.2.

У даній лабораторній роботі для рівня репозиторію було реалізовано наступний клас описаний нижче.

UserRepository.java. `UserRepository` є репозиторієм для роботи з користувачами системи, де зберігаються дані користувачів, їх ролі та контактна інформація. Цей репозиторій надає методи для пошуку користувачів за логіном, корпоративною поштою або за комбінацією цих параметрів, а також включає можливості для фільтрації та сортування користувачів за ролями. Код класу представлений у додатку Д.3.

8 Реалізація сервісу Spring Boot проєкту

Реалізація рівня сервісу в Spring Boot проєкті була здійснена для ефективного управління бізнес-логікою та забезпечення чіткої взаємодії між контролерами та репозиторіями. Це дозволяє розділити відповідальність між різними шарами застосунку, що підвищує його підтримуваність та масштабованість. Використання сервісного рівня сприяє централізації обробки даних, забезпеченню транзакційної цілісності та ефективному управлінню помилками, що підвищує надійність системи. Крім того, це полегшує тестування бізнес-логіки, сприяє повторному використанню коду та полегшує інтеграцію нових функціональних можливостей. В цілому, впровадження рівня сервісу підвищує гнучкість та ефективність розробки, забезпечуючи створення більш організованого, надійного та легко підтримуваного вебзастосунку.

Для реалізації рівня сервісу у даний проєкт було перенесено класи з минулої лабораторної роботи, що описані далі.

1 **CustomServiceException.java.** Клас, що реалізує кастомне виключення для обробки помилок, специфічних для рівня сервісів. Розширює `RuntimeException`, що дозволяє кидати це виключення без необхідності обробки його в коді, надаючи додаткові можливості для інкапсуляції причин помилок та повідомлень в межах бізнес-логіки. Код класу представлений у додатку Е.1.

2 **EntrantService.java.** Інтерфейс, який визначає основні операції для роботи з абітурієнтами в системі, такі як створення, оновлення, пошук і видалення записів, а також фільтрація та сортування абітурієнтів за різними критеріями. Код класу представлений у додатку Е.2.

3 **EntrantServiceImpl.java.** Реалізація інтерфейсу `EntrantService`, що забезпечує виконання CRUD-операцій для сутностей абітурієнтів, зокрема збереження та оновлення даних абітурієнтів, а також обробку можливих винятків при виконанні операцій з даними. Код даного класу представлений у додатку Е.3.

4 **StudentService.java.** Інтерфейс для управління сутностями студентів, що містить методи для виконання CRUD-операцій, а також для пошуку,

фільтрації та сортування записів студентів в системі. Код класу представлений у додатку Е.4.

5 StudentServiceImpl.java. Реалізація інтерфейсу StudentService, що реалізує бізнес-логіку для роботи зі студентами, включаючи збереження, оновлення, видалення та отримання студентів, а також додаткову обробку помилок та транзакцій. Код класу представлений у додатку Е.5.

У даній лабораторній роботі для рівня сервісу було реалізовано наступні класи описані нижче.

1 UserService.java. Інтерфейс для управління користувачами в системі, що включає методи для реєстрації, оновлення, видалення користувачів та інші операції, зокрема пошук користувачів за різними параметрами, таких як роль чи електронна пошта. Код класу представлений у додатку Е.6.

2 UserServiceImpl.java. Реалізація інтерфейсу UserService, що реалізує функціональність для управління користувачами, а також забезпечує підтримку механізмів аутентифікації, зміни паролів і обробку винятків при роботі з даними користувачів. Код класу представлений у додатку Е.7.

9 Реалізація валідатора Spring Boot проєкту

Реалізація рівня валідації в Spring Boot проєкті здійснювалася для забезпечення високої якості та коректності вхідних даних, що надходять від користувачів та інших системних компонентів. Це дозволяє строго дотримуватися бізнес-логіки та вимог безпеки, запобігаючи виникненню помилок та некоректних даних у базі даних. Використання власних анотацій для валідації спрощує процес перевірки даних, забезпечуючи гнучкість та масштабованість системи. Крім того, автоматизована валідація підвищує ефективність розробки, зменшуючи кількість ручних перевірок та можливість людських помилок. Це сприяє підвищенню надійності та стабільності застосунку, а також полегшує його підтримку та подальше розширення. В цілому, впровадження рівня валідації значно покращує якість програмного забезпечення, забезпечуючи безпеку та задоволення вимог користувачів.

У даній лабораторній роботі для рівня валідації було реалізовано наступні класи описані нижче.

1 **BirthdayValidator.java**. Клас реалізує валідацію для перевірки дати народження, перевіряючи, чи знаходиться вона в межах допустимого віку (від 16 до 110 років) від поточної дати. Цей валідатор використовується для полів типу `LocalDate` та визначає, чи є дата народження в межах допустимого діапазону. Код класу представлений у додатку Ж.1.

2 **UsernameOrCorporateEmailValidator.java**. Цей клас забезпечує валідацію для перевірки, чи є значення або користувацьким ім'ям, або корпоративною електронною поштою. Валідація перевіряє формат користувацького імені або електронної пошти з використанням регулярних виразів, гарантуючи, що вхідні дані відповідають заданим стандартам. Код класу представлений у додатку Ж.2.

3 **ValidationConstants.java**. Клас містить константи, які використовуються в різних валідаторах для підтримки єдиного стандарту. Він включає регулярні вирази для валідації таких даних, як імена, електронні адреси, паролі та інші важливі параметри, що застосовуються в анотаціях. Код класу представлений у додатку Ж.3.

Далі наведені кастомні анотації для абітурієнтів, що відповідають за валідацію даних, специфічних для абітурієнтів, таких як дата народження, номер справи, стать та рейтинг, гарантуючи, що всі ці дані відповідають вимогам системи.

1 **ValidBirthday.java**. Це анотація для перевірки дати народження користувача. Вона забезпечує, що дата народження повинна бути в минулому і відповідати певним критеріям віку, що визначають мінімальний та максимальний вік. Код анотації представлений у додатку Ж.4.

2 **ValidCaseNumber.java**. Анотація для перевірки правильності формату номеру справи. Вона перевіряє, чи відповідає номер справи встановленому регулярному виразу, який визначає допустимий формат (літери та цифри з роздільниками). Код анотації представлений у додатку Ж.5.

3 **ValidGender.java**. Анотація для перевірки значення статі, яка вимагає, щоб поле не було порожнім або нульовим. Використовується для забезпечення, що значення статі є валідним і відповідає вимогам системи. Код анотації представлений у додатку Ж.6.

4 **ValidRatingScore.java**. Анотація для валідації рейтингових оцінок. Вона гарантує, що рейтинг є додатнім числом і знаходиться в межах від 120.000 до 200.000, забезпечуючи правильне введення рейтингів у систему та збереження їх відповідно до бізнес-логіки. Код анотації представлений у додатку Ж.7.

Далі наведено кастомні анотації для студентів, що включають валідації, що стосуються студентів, зокрема перевірку фінансування, статусу стипендії та інших полів, що визначають статус студента в системі.

1 **ValidFundingType.java**. Анотація для валідації типу фінансування, що перевіряє, чи тип фінансування є допустимим, як, наприклад, "бюджет" або "контракт". Вона використовується для перевірки вхідних даних, що пов'язані з фінансовими аспектами студентів чи абітурієнтів у системі. Код анотації представлений у додатку Ж.8.

2 **ValidScholarshipStatus.java**. Анотація для валідації статусу стипендії, перевіряючи, чи є вказаний статус дійсним серед можливих значень, наприклад, "отримую" або "не отримую". Вона дозволяє забезпечити коректність введених даних щодо стипендій для студентів. Код анотації представлений у додатку Ж.9.

Далі наведено кастомні анотації для користувачів, що застосовуються для валідації даних користувачів, таких як логін, пароль, електронна пошта та роль, що визначають доступ до різних функцій системи.

1 **ValidEncodedPassword.java**. Анотація для валідації закодованих паролів, яка перевіряє, чи довжина паролю є точною (60 символів), що важливо для роботи з паролями, закодованими з використанням алгоритмів хешування, таких як bcrypt. Ця анотація забезпечує, що в базу даних не потрапляють некоректно закодовані паролі. Код анотації представлений у додатку Ж.10.

2 **ValidPassword.java**. Анотація для валідації паролю користувача, яка забезпечує перевірку на мінімальну та максимальну довжину (від 8 до 50

символів), наявність хоча б однієї великої літери, цифри та спеціального символу. Вона потрібна для забезпечення безпеки паролів у системі, щоб користувачі використовували надійні паролі. Код даної анотації представлений у додатку Ж.11.

3 **ValidRole.java**. Анотація для валідації ролі користувача, перевіряючи, чи роль, яку задає користувач, є дійсною та відповідає одному з визначених значень. Ця анотація важлива для забезпечення коректної авторизації в системі та правильного розподілу прав доступу на основі ролей користувачів. Код анотації представлений у додатку Ж.12.

4 **ValidUsername.java**. Анотація для валідації імені користувача, яка перевіряє, чи є значення поля ненульовим, чи має довжину від 8 до 32 символів, та чи відповідає регулярному виразу для імені користувача (лише латинські літери, цифри та спеціальні символи, як '.', '_', '-'). Вона необхідна для забезпечення коректності введення імен користувачів у системі, зокрема під час реєстрації та авторизації. Код анотації представлений у додатку Ж.13.

5 **ValidUsernameOrCorporateEmail.java**. Анотація для валідації ідентифікатора користувача, який може бути або ім'ям користувача, або корпоративною email-адресою. Вона забезпечує перевірку того, що значення поля не є порожнім, має довжину від 8 до 123 символів та відповідає одному з двох форматів (ім'я користувача або корпоративна пошта). Ця анотація особливо корисна для сценаріїв автентифікації, коли користувач може увійти в систему, використовуючи будь-який з цих двох ідентифікаторів. Код анотації представлений у додатку Ж.14.

Далі наведено загальні кастомні анотації для всіх класів, що використовуються для загальної валідації даних, що застосовуються до всіх класів, наприклад, перевірка на порожнечу, правильність формату введення чи значення в межах допустимих діапазонів.

1 **ValidCorporateEmail.java**. Анотація `ValidCorporateEmail` забезпечує перевірку правильності корпоративної пошти. Вона гарантує, що значення поля не є порожнім, має довжину від 17 до 123 символів та відповідає визначеному

регулярному виразу для корпоративних електронних адрес, що має формат “name.surname@faculty.khpi.edu.ua”. Код даної анотації представлений у додатку Ж.15.

2 ValidId.java. Анотація ValidId використовується для перевірки валідності ідентифікаторів у системі. Вона забезпечує, щоб значення не було порожнім або нульовим, а також перевіряє, що ідентифікатор є позитивним числом. Код анотації представлений у додатку Ж.16.

3 ValidName.java. Анотація ValidName застосовується для перевірки імен користувачів, абітурієнтів чи студентів. Вона гарантує, що ім’я не є порожнім, має довжину від 1 до 50 символів та відповідає регулярному виразу, що дозволяє використовувати лише літери українського алфавіту, включаючи складні імена з пробілами чи дефісами. Код даної анотації представлений у додатку Ж.17.

4 ValidPatronymic.java. Анотація ValidPatronymic використовується для перевірки патронімів (по батькові). Вона забезпечує, що значення не є порожнім, довжина знаходиться в межах від 1 до 50 символів і відповідає регулярному виразу для патронімів, що допускає літери українського алфавіту, а також пробіли або дефіси для складних форм. Код анотації представлений у додатку Ж.18.

5 ValidSurname.java. Анотація ValidSurname застосовується для валідації прізвищ. Вона перевіряє, щоб прізвище не було порожнім, його довжина знаходилася в межах від 1 до 50 символів і відповідало визначеному шаблону, що дозволяє використовувати тільки літери українського алфавіту, з можливістю включення пробілів та дефісів для складних прізвищ. Код анотації представлений у додатку Ж.19.

10 Реалізація тестування Spring Boot проєкту

Реалізація рівня тестування у Spring Boot проєкті здійснювалася для забезпечення високої надійності та коректності роботи основних компонентів застосунку, таких як сервіси, контролери та репозиторії. Тестування дозволяє

виявляти та виправляти помилки на ранніх етапах розробки, що запобігає їх появі в продакшн-середовищі та підвищує загальну стабільність системи. Крім того, автоматизовані тести сприяють підтримці якості коду, полегшують процеси рефакторингу та масштабування проєкту, а також забезпечують впевненість у правильній реалізації бізнес-логіки та функціональних вимог. Використання тестування також покращує співпрацю між розробниками, оскільки чітко визначені тести служать документацією для очікуваної поведінки компонентів. В цілому, інтеграція рівня тестування підвищує ефективність розробки, знижує ризики помилок та сприяє створенню більш надійного та якісного вебзастосунку.

У даній лабораторній роботі для рівня сервісу було реалізовано наступні класи описані нижче.

1 EntrantServiceTest. Цей клас містить тести для класу EntrantService, що обробляє сутність абітурієнта. Тести перевіряють створення, збереження, оновлення, видалення абітурієнтів, а також перевіряють правильність роботи функціональності на предмет дублювання даних, таких як дублікати номера справи. Код класу представлений у додатку И.1.

2 StudentServiceTest. Тести для класу StudentService, що забезпечує управління студентами. Перевіряється збереження студентів, оновлення їх даних (наприклад, зміна корпоративної електронної пошти), видалення студентів та обробка помилок, таких як спроба зберегти студента з дубльованою корпоративною електронною поштою. Код класу представлений у додатку И.2.

3 UserServiceTest. Цей клас тестує функціональність для класу UserService, який відповідає за управління користувачами. Тести перевіряють основні CRUD-операції для користувачів, такі як створення, читання, оновлення, видалення, а також обробку помилок для дублювання імен користувачів і корпоративних електронних адрес. Код класу представлений у додатку И.3.

Результати виконання тестових випадків наведених наборів тестів представлені на рис. 11-13 далі в тексті роботи.

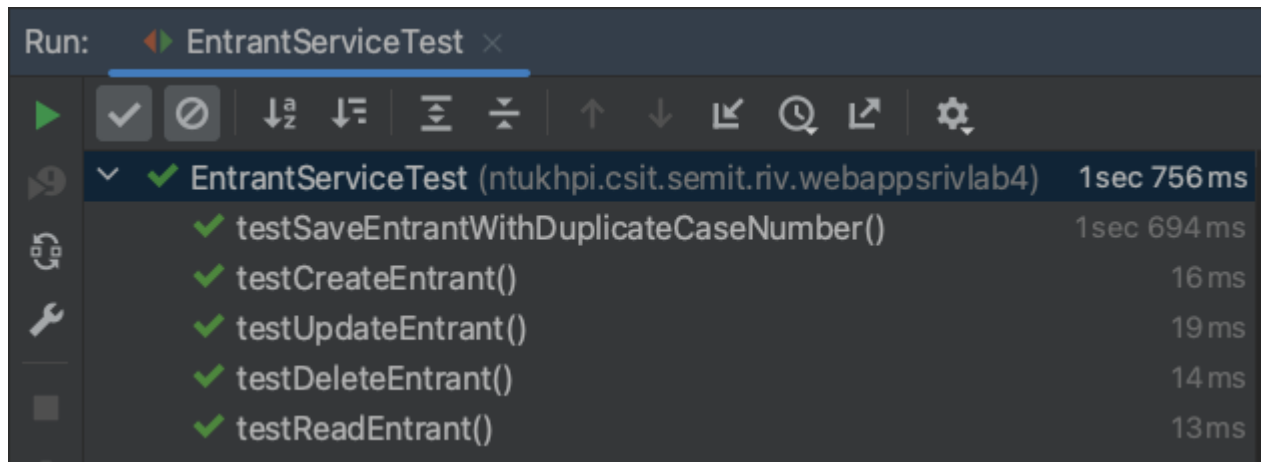


Рисунок 11 – Результати виконання тестового набору EntrantServiceTest.java

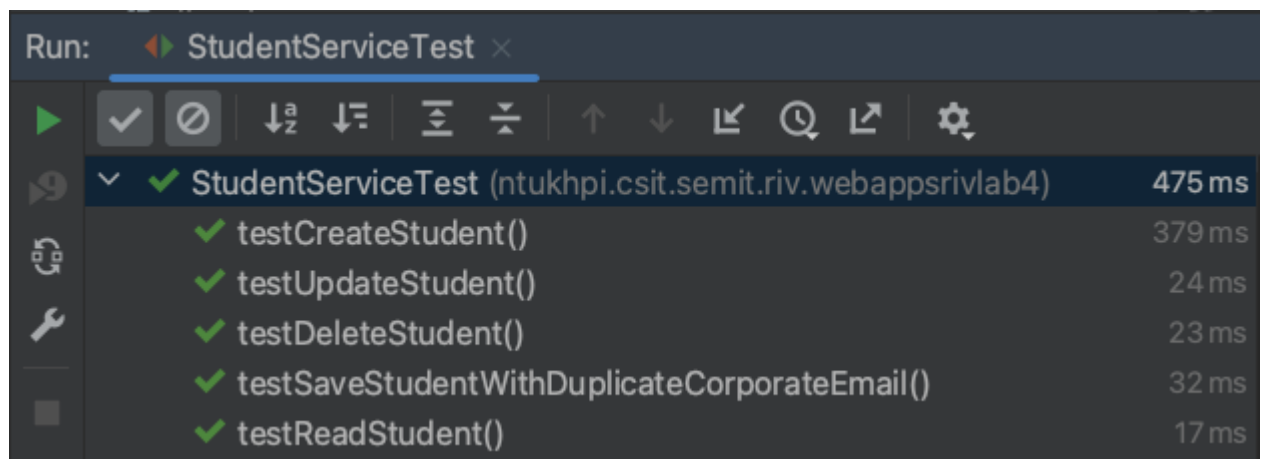


Рисунок 12 – Результати виконання тестового набору StudentServiceTest.java

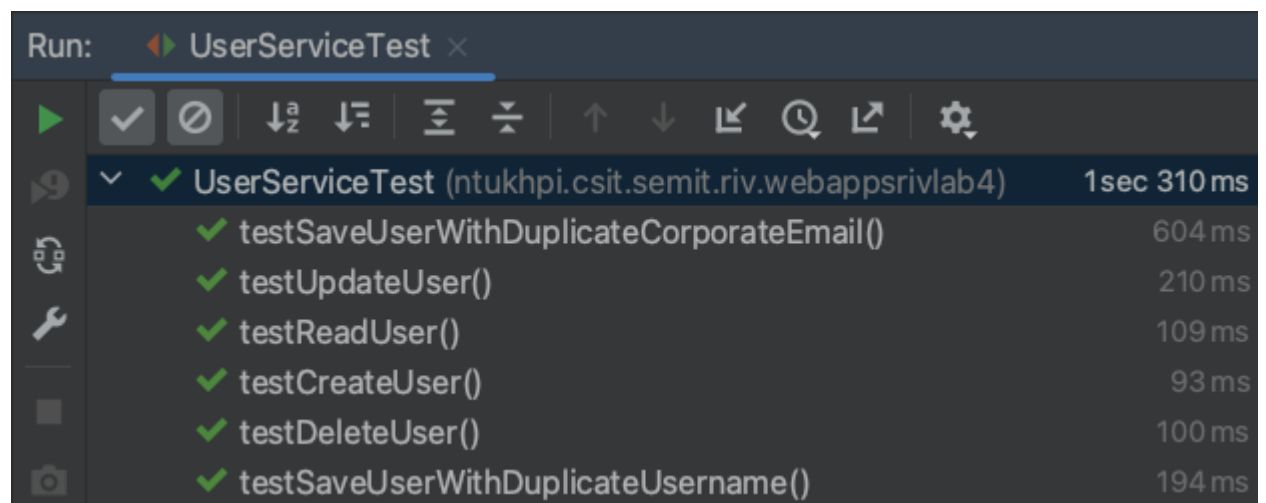


Рисунок 13 – Результати виконання тестового набору UserServiceTest.java

11 Реалізація представлення Spring Boot проєкту

Реалізовані файли відповідають за реалізацію рівня представлення (view) в Spring проєкті і представляють собою HTML-шаблони для різних сторінок, що використовуються на етапі авторизації в вебзастосунку. Вони створені з використанням Thymeleaf як шаблонізатора, що дозволяє динамічно заповнювати дані з серверу і забезпечувати інтерактивність на клієнтській стороні. Дані сторінки можна поділити на декілька груп за призначенням:

- компоненти, які включаються на інших сторінках;
- помилки;
- авторизація користувачів;
- домашня сторінка та профіль користувача;
- таблиці і форми для окремих сутностей.

Файли компонентів включають шаблони HTML, які використовуються для відображення повідомлень про помилки та загальних елементів інтерфейсу, таких як навігаційна панель та футер. Всі ці сторінки інтегруються з використанням Thymeleaf – шаблонізатора, що дозволяє динамічно підставляти дані та керувати відображенням елементів залежно від контексту (наприклад, помилок, успішних повідомлень, та інших). Опис даних сторінок наведено нижче.

1 NavBar.html. Навігаційна панель, яка відображається на всіх сторінках сайту та дозволяє користувачам переміщатися між основними розділами сайту, такими як домашня сторінка, профіль, налаштування тощо. Код компонента представлений у додатку К.1.

2 Footer.html. Загальний футер сторінки, що включає інформацію про копірайти, посилання на політику конфіденційності або інші додаткові ресурси, доступні на сайті. Код компонента представлений у додатку К.2.

3 ErrorToast.html. Шаблон для відображення спливаючого повідомлення про помилку на верхній частині екрану. Це спливаюче повідомлення містить текст помилки, який відображається, якщо виникає проблема в процесі виконання операції. Код компонента представлений у додатку К.3.

4 **WarningToast.html**. Сторінка, що показує попередження для користувача, коли відбувається подія, яку потрібно врахувати, але яка не є критичною для виконання операції. Код даного компонента представлений у додатку К.4.

5 **SuccessToast.html**. Це шаблон для відображення спливаючого повідомлення про успішну операцію. Після виконання дій, таких як реєстрація чи зміна даних, користувач бачить підтвердження у вигляді цього повідомлення. Код компонента представлений у додатку К.5.

ErrorPage.htm – універсальний шаблон сторінки помилки у вебзастосунку. Даний шаблон створений для відображення сторінки з помилкою, яка з'являється, коли виникає критична помилка, наприклад, відсутність запрошеної сторінки. Ця сторінка містить код помилки, опис та кнопки для повернення на попередню сторінку чи на домашню сторінку застосунку. Код сторінки представлений у додатку К.6.

Сторінки етапу авторизації включають реєстрацію, вхід до системи, скидання пароля та зміну пароля. Вони взаємодіють із бекендом для виконання відповідних операцій та відображають повідомлення про успіх або помилки через спеціальні компоненти (наприклад, сповіщення для помилок, попереджень або успіхів). Опис даних сторінок наведено нижче.

1 **Login.html**. Це форма для входу користувача в систему з використанням логіну та пароля. Вона підтримує можливість показу повідомлень про помилки входу і забезпечує функціональність автентифікації користувача, спрямовану на захист ресурсів застосунку. Код сторінки представлений у додатку К.7.

2 **Register.html**. Шаблон для реєстрації нового користувача, який включає поля для вводу імені, прізвища, по-батькові, корпоративної електронної пошти, логіну та пароля. Після заповнення користувач може зареєструвати свій акаунт, і дані будуть надіслані на сервер для обробки. Код сторінки представлений у додатку К.8.

Наступні файли включають шаблони HTML, які використовуються для відображення домашньої сторінки та профілю користувача. Вони надають

інтерфейс для взаємодії користувачів із системою, включаючи можливість переглядати та змінювати особисті дані, змінювати облікові дані. Ці сторінки відповідають за відображення профілю користувача та управління його даними в системі, і вони активно використовують можливості Spring Security та Thymeleaf для роботи з даними та обробки форм. Опис даних сторінок наведено нижче.

1 **Home.html.** Це сторінка домашнього екрану, яка містить основну інформацію та навігаційні елементи для користувача. Вона забезпечує доступ до основних функцій застосунку, таких як профіль користувача та інші важливі ресурси. Код сторінки представлений у додатку К.9.

2 **ProfileForm.html.** Це форма профілю користувача, яка дозволяє переглядати та редагувати основні дані користувача, такі як ім'я, прізвище, ім'я по батькові, корпоративний email та інші важливі параметри профілю. Вона також містить можливість змінювати персональні дані з використанням фреймворку Thymeleaf. Код сторінки представлений у додатку К.10.

3 **ChangeCredentialsForm.html.** Ця сторінка надає форму для зміни облікових даних користувача, таких як ім'я користувача та пароль. Вона включає валідацію для вводу даних, щоб забезпечити правильність введених даних перед відправленням на сервер. Код даної сторінки представлений у додатку К.11.

Наступні файли також є частиною рівня представлення (View) у Spring MVC застосунку та реалізують представлення даних у вебзастосунку, зокрема для відображення та управління сутностями (абітурієнти, студенти, користувачі) через відповідні таблиці та форми. Вони виконують роль інтерфейсу між користувачем та сервером, де дані зберігаються, обробляються та представляються у вигляді таблиць або форм. Вони забезпечують зручний користувацький інтерфейс для взаємодії з даними, такими як перегляд, додавання, редагування та видалення сутностей. Опис даних сторінок наведено нижче.

1 **EntrantForm.html**. Цей файл є формою для створення або редагування абітурієнта, де відображаються поля для введення інформації про абітурієнта, такі як прізвище, ім'я, по-батькові, дата народження тощо, та надається можливість взаємодіяти з цією інформацією в залежності від режиму (додавання, редагування, видалення або перегляд). Код сторінки представлений у додатку К.12.

2 **EntrantTable.html**. Файл відповідає за відображення таблиці з усіма абітурієнтами в системі, де можна переглядати та сортувати записи абітурієнтів. Він також включає можливість фільтрації та пошуку абітурієнтів за певними параметрами, такими як ім'я або роль. Код даної сторінки представлений у додатку К.13.

3 **StudentForm.html**. Це форма для створення або редагування запису студента в системі. Вона включає поля для введення інформації, такої як прізвище, ім'я, статус фінансування, а також можливість редагування цих даних залежно від обраного режиму (додавання, редагування, видалення або перегляд). Код сторінки представлений у додатку К.14.

4 **StudentTable.html**. Це файл, який відображає таблицю з усіма студентами в системі, де можна переглядати, сортувати та фільтрувати записи про студентів за різними критеріями, такими як ім'я, спеціальність чи статус. Код сторінки представлений у додатку К.15.

5 **UserForm.html**. Форма для створення або редагування користувача в системі, де можна ввести або змінити корпоративну електронну пошту, ім'я користувача, прізвище та роль. Ця форма має різні режими (додавання, редагування, видалення або перегляд), які дозволяють користувачеві лише робити різні дії з даними залежно від рівня доступу. Код сторінки представлений у додатку К.16.

6 **UserTable.html**. Це таблиця, яка містить список всіх користувачів у системі. Тут можна переглядати деталі кожного користувача, сортувати записи та виконувати дії, такі як редагування або видалення, для користувачів з відповідними правами доступу. Код сторінки представлений у додатку К.17.

У реалізації рівня представлення в Spring Boot проєкті було створено набір HTML-шаблонів з використанням Thymeleaf, що забезпечує динамічне відображення даних та інтерактивність вебзастосунку. Дана реалізація забезпечує взаємодію користувача з системою на етапі авторизації та маніпулювання даними, а також надає зручний та інтуїтивно зрозумілий інтерфейс. Використання Thymeleaf як шаблонізатора дозволяє легко підтримувати та оновлювати інтерфейс, сприяє чіткому розподілу відповідальності між фронтендом і бекендом, а також підвищує продуктивність розробки завдяки повторному використанню компонентів. Крім того, це підвищує гнучкість застосунку, дозволяючи швидко адаптуватися до змін вимог користувачів та забезпечує покращену підтримку безпеки та валідації даних, що в цілому сприяє підвищенню якості та надійності вебзастосунку.

Результати роботи вебзастосунку реалізованого до завдання даної лабораторної роботи наведені на рис. Л.1.1-Л.5.10 у відповідних підрозділах додатку Л.

Висновки

Отримані результати під час виконання даної лабораторної роботи відображають переваги застосування фреймворку Spring для Java-розробників. Використання Spring Security у розробці Spring Boot застосунку надає Java-розробникам суттєві переваги при реалізації автентифікації, авторизації та керування сесіями користувачів.

Spring Security забезпечує потужний та гнучкий механізм безпеки, який легко інтегрується з іншими компонентами Spring екосистеми завдяки своєму модульному підходу та широким можливостям налаштування. Під час виконання даної лабораторної роботи було зауважено особливу перевагу в реалізації ієрархії ролей, яка спрощує конфігурацію контролю доступу, знижує дублювання коду та забезпечує чітке розмежування прав доступу між різними типами користувачів.

Spring Security забезпечує гнучкість та масштабованість завдяки можливості централізованого управління безпекою, інтеграції з іншими компонентами Spring екосистеми та підтримці різноманітних методів аутентифікації, таких як базова аутентифікація, OAuth2 та JWT. Крім того, автоматичний захист від поширених загроз, таких як CSRF та XSS, підвищує загальний рівень безпеки застосунку. Завдяки глибокій інтеграції з Spring Framework, активній підтримці спільноти та регулярним оновленням, Spring Security дозволяє підтримувати високий рівень безпеки та відповідати сучасним вимогам до захисту даних. Впровадження Spring Security сприяє підвищенню продуктивності розробників, оскільки значно спрощує процес налаштування безпеки, дозволяючи зосередитись на реалізації бізнес-логіки.

Таким чином, використання Spring Security значно спрощує процес розробки безпечних веб-застосунків, забезпечуючи надійний захист, гнучкість у впровадженні політик безпеки та покращуючи підтримуваність, масштабованість, загальну якість та надійність програмного забезпечення

ДОДАТОК А

Код файлів конфігурації Spring Boot проєкту

А.1 Код файлу pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.3.5</version>
    <relativePath/> <!-- lookup parent from repository -->
  </parent>
  <groupId>ntukhpi.csit.semit.riv</groupId>
  <artifactId>WebAppsRIVLab4</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <packaging>war</packaging>
  <name>WebAppsRIVLab4</name>
  <description>WebAppsRIVLab4</description>
  <url/>
  <licenses>
    <license/>
  </licenses>
  <developers>
    <developer/>
  </developers>
  <scm>
    <connection/>
    <developerConnection/>
```

```

    <tag/>
    <url/>
</scm>
<properties>
    <java.version>17</java.version>
</properties>
<dependencies>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-data-jpa</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-security</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.session</groupId>
        <artifactId>spring-session-core</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-thymeleaf</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-validation</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
    <dependency>

```

```

        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-test</artifactId>
        <scope>test</scope>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-mail</artifactId>
    </dependency>
    <dependency>
        <groupId>org.thymeleaf.extras</groupId>
<artifactId>thymeleaf-extras-springsecurity6</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-devtools</artifactId>
        <scope>runtime</scope>
        <optional>true</optional>
    </dependency>
    <dependency>
        <groupId>com.mysql</groupId>
        <artifactId>mysql-connector-j</artifactId>
        <scope>runtime</scope>
    </dependency>
    <dependency>
        <groupId>org.projectlombok</groupId>
        <artifactId>lombok</artifactId>
        <optional>true</optional>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-tomcat</artifactId>
        <scope>provided</scope>
    </dependency>
    <dependency>
        <groupId>org.springframework.security</groupId>

```

```

        <artifactId>spring-security-test</artifactId>
        <scope>test</scope>
    </dependency>
    <dependency>
        <groupId>org.apache.commons</groupId>
        <artifactId>commons-lang3</artifactId>
        <version>3.15.0</version>
    </dependency>
    <dependency>
        <groupId>ch.qos.logback</groupId>
        <artifactId>logback-classic</artifactId>
        <version>1.4.12</version>
    </dependency>
</dependencies>

<build>
    <plugins>
        <plugin>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-maven-plugin</artifactId>
            <configuration>
                <excludes>
                    <exclude>
                        <groupId>org.projectlombok</groupId>
                        <artifactId>lombok</artifactId>
                    </exclude>
                </excludes>
            </configuration>
        </plugin>
    </plugins>
</build>
</project>

```

A.2 Код файла application.properties

```

spring.application.name=WebAppsRIVLab4
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
spring.datasource.url=jdbc:mysql://localhost:3307/webappsrivlab4?serverTimezone=Europe/Kiev&useSSL=false
spring.datasource.username=root
spring.datasource.password=rootpassword

spring.jpa.hibernate.ddl-auto=create
#spring.jpa.hibernate.ddl-auto=update
logging.level.org.hibernate.SQL=info
#logging.level.org.hibernate.SQL=debug
#spring.jpa.show-sql=true
#spring.jpa.properties.hibernate.format_sql=true
spring.jpa.open-in-view=false

logging.level.org.springframework.security=trace
#logging.level.org.springframework.security=debug

```

A.3 Код файла logback.xml

```

<configuration>
    <appender name="CONSOLE"
class="ch.qos.logback.core.ConsoleAppender">
        <encoder>
            <pattern>%d{yyyy-MM-dd HH:mm:ss.SSS} [%-5level]
%-55logger{36} - %msg%n</pattern>
        </encoder>
    </appender>

    <!--      <logger name="ntukhpi.csit.semit.riv.webappsrivlab4"
level="DEBUG" />-->
    <!--      <logger name="ntukhpi.csit.semit.riv.webappsrivlab4"
level="TRACE" />-->

```

```

    <logger name="ntukhpi.csit.semit.riv.webappsrivlab4"
level="INFO" />

    <logger name="org.springframework.security" level="trace"
additivity="false">
        <appender-ref ref="Console" />
    </logger>

    <root level="INFO">
        <appender-ref ref="CONSOLE" />
    </root>
</configuration>

```

A.4 Код класу WebAppsRIVLab4Application.java

```

package ntukhpi.csit.semit.riv.webappsrivlab4;

import
ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.entrant.Entrant;
import
ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.student.Student;
import
ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.testDataList.entra
nt.EntrantList;
import
ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.testDataList.stude
nt.StudentList;
import
ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.user.UserEntity;
import
ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.testDataList.user.
UserList;
import
ntukhpi.csit.semit.riv.webappsrivlab4.service.entity.EntrantServic
e;

```

```

import
ntukhpi.csit.semit.riv.webappsrivlab4.service.entity.StudentService;
import
ntukhpi.csit.semit.riv.webappsrivlab4.service.entity.UserService;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.CommandLineRunner;
import org.springframework.boot.SpringApplication;
import
org.springframework.boot.autoconfigure.SpringBootApplication;

import java.util.List;

/**
 * Main entry point for the Spring Boot application
 * `WebAppsRivLab4Application`.
 * This class is responsible for starting the application,
 * initializing necessary services,
 * and populating the database with initial test data.
 * <p>
 * Key functionalities:
 * - Loads initial data into the application by interacting with
 * the `UserService`, `EntrantService`, and `StudentService`.
 * - Saves test data for users, entrants, and students into the
 * database.
 * - Associates each entrant with a student and updates the entrant
 * records accordingly.
 * <p>
 * Dependencies:
 * - `EntrantService`: Manages operations related to the `Entrant`
 * entities.
 * - `StudentService`: Handles operations related to the `Student`
 * entities.

```

```

* - `UserService`: Manages operations related to the `User`
entities.
* <p>
* The `run` method is executed on application startup to populate
the application with test data, saving all user, entrant, and
student records.
* <p>
*
* @author Inessa Repeshko CS-222a
* @see EntrantService
* @see StudentService
* @see UserService
* @see Entrant
* @see UserEntity
* @see Student
*/

@SpringBootApplication
public class WebAppsRivLab4Application implements
CommandLineRunner {
    private static final Logger logger =
LoggerFactory.getLogger(WebAppsRivLab4Application.class);
    private final EntrantService entrantService;
    private final StudentService studentService;
    private final UserService userService;

    @Autowired
    public WebAppsRivLab4Application(EntrantService entrantService,
StudentService studentService, UserService userService) {
        this.entrantService = entrantService;
        this.studentService = studentService;
        this.userService = userService;
    }

    @Override
    public void run(String... args) {

```



```

        List<UserEntity> userEntityList =
UserList.INSTANCE.getUsers().stream()
            .map(UserEntity::getUserEntityFromUserTestData)
            .toList();
        userService.saveAllUsers(userEntityList);

        List<Entrant> entrantList =
EntrantList.INSTANCE.getEntrants();
        entrantService.saveAllEntrants(entrantList);

studentService.saveAllStudents(StudentList.INSTANCE.getStudents())
;

        entrantList.forEach(entrant -> {
            entrant.setStudentId(
entrantService.findStudentIdForEntrant(entrant.getId()));
            entrantService.updateEntrant(entrant);
        });
    }

    public static void main(String[] args) {
        SpringApplication.run(WebAppsRivLab4Application.class,
args);
    }
}

```

A.5 Код класы ServletInitializer.java

```

package ntukhpi.csit.semit.riv.webappsrivlab4;
import org.springframework.boot.builder.SpringApplicationBuilder;
import
org.springframework.boot.web.servlet.support.SpringBootServletInit
ializer;

```

```

/**
 * Servlet initializer for configuring the Spring Boot application
 * when deployed in a traditional servlet container.
 * This class extends `SpringBootServletInitializer` and overrides
 * the `configure` method to specify the main application class.
 * <p>
 * Key functionalities:
 * - Configures the Spring Boot application when running within a
 * servlet container.
 * - Ensures the correct setup of the Spring application context
 * during deployment.
 * <p>
 * This class is essential for deploying Spring Boot applications
 * to an external servlet container (like Tomcat or Jetty).
 * It allows the application to function as a traditional WAR (Web
 * Application Archive) file.
 *
 * @author Inessa Repeshko CS-222a
 * @see SpringBootServletInitializer
 * @see WebAppsRivLab4Application
 */

public class ServletInitializer extends
SpringBootServletInitializer {

    @Override
    protected SpringApplicationBuilder
configure(SpringApplicationBuilder application) {
        return
application.sources(WebAppsRivLab4Application.class);
    }
}

```

ДОДАТОК Б

Код файлів конфігурації Spring Security у Spring Boot проєкті

Б.1 Код файлу EncoderConfig.java

```
package ntukhpi.csit.semit.riv.webappsrivlab4.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import
org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import
org.springframework.security.crypto.password.PasswordEncoder;

/**
 * Configuration class for defining the PasswordEncoder bean.
 * This class is annotated with @Configuration, indicating that it
 * declares one or more @Bean methods
 * and can be processed by the Spring container to generate bean
 * definitions and service requests.
 * <p>
 * The defined PasswordEncoder bean uses BCryptPasswordEncoder to
 * hash passwords securely.
 * BCrypt is a strong hashing algorithm widely used for password
 * storage to ensure
 * passwords are not stored in plaintext and are resistant to brute
 * force attacks.
 *
 * @author Inessa Repeshko CS-222a
 * @see Configuration
 * @see PasswordEncoder
 * @see BCryptPasswordEncoder
 */
```

```

@Configuration
public class EncoderConfig {
    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }
}

```

Б.2 Код файла UserDetailsServiceImpl.java

```

package ntukhpi.csit.semit.riv.webappsrivlab4.config;

import
ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.user.UserEntity;
import
ntukhpi.csit.semit.riv.webappsrivlab4.service.entity.UserService;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.security.core.userdetails.UserDetails;
import
org.springframework.security.core.userdetails.UserDetailsService;
import
org.springframework.security.core.userdetails.UsernameNotFoundException;
import org.springframework.stereotype.Service;

/**
 * Implementation of the UserDetailsService interface for user
 * authentication and authorization.
 * This service is responsible for loading user-specific data, such
 * as username, password, and roles,
 * from the database to be used by Spring Security during the
 * authentication process.

```

```

* <p>
* The class utilizes the UserService to fetch user details based
on a given username or corporate email.
* If the user is not found, a UsernameNotFoundException is thrown
to indicate authentication failure.
* <p>
* Key responsibilities:
* - Fetching the UserEntity object from the database.
* - Converting UserEntity data into a UserDetails object required
by Spring Security.
* <p>
* This implementation ensures secure user management and seamless
integration with Spring Security.
*
* @author Inessa Repeshko CS-222a
* @see UserService
* @see UserEntity
* @see UserDetails
* @see UsernameNotFoundException
*/

@Service
public class UserDetailsServiceImpl implements UserDetailsService
{
    private static final Logger logger =
LoggerFactory.getLogger(UserDetailsServiceImpl.class);

    private final UserService userService;

    @Autowired
    public UserDetailsServiceImpl(UserService userService) {
        this.userService = userService;
    }

    @Override

```

```

    public UserDetails loadUserByUsername(String userIdentifier)
throws UsernameNotFoundException {
    UserEntity userEntity =
userService.findUserByUsernameOrCorporateEmail(userIdentifier);

    if (userEntity == null) {
        throw new UsernameNotFoundException("No userEntity
found with the specified data.");
    }

    return
org.springframework.security.core.userdetails.User.builder()
        .username(userEntity.getUsername())
        .password(userEntity.getPassword())
        .roles(userEntity.getRole().name())
        .build();
}
}

```

Б.3 Код файла SecurityConfig.java

```

package ntukhpi.csit.semit.riv.webappsrivlab4.config;

import jakarta.servlet.SessionTrackingMode;
import ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.enums.Role;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Autowired;
import
org.springframework.boot.web.servlet.ServletContextInitializer;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import
org.springframework.security.access.hierarchicalroles.RoleHierarch
y;

```

```

import
org.springframework.security.access.hierarchicalroles.RoleHierarch
yImpl;
import
org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.config.Customizer;
import
org.springframework.security.config.annotation.authentication.conf
iguration.AuthenticationConfiguration;
import
org.springframework.security.config.annotation.web.builders.HttpSe
curity;
import
org.springframework.security.config.annotation.web.configuration.E
nableWebSecurity;
import
org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.web.SecurityFilterChain;
import
org.springframework.security.web.session.HttpSessionEventPublisher
;

import java.util.EnumSet;

/**
 * Configuration class for security settings in the application.
 * This class is annotated with @Configuration and
 * @EnableWebSecurity, making it the central configuration
 * for securing the application using Spring Security.
 * <p>
 * Key functionalities provided by this class:
 * <p>
 * - AuthenticationManager: Configures the authentication
manager to handle user authentication.
 * - SecurityFilterChain: Defines the security filter chain to
manage HTTP security, including login, logout,

```

```

* session management, and access control.
* - Session Management: Configures session timeout, tracking
modes, and invalid session handling.
* - Role Hierarchy: Defines hierarchical roles to simplify
access control configuration.
* - Remember-Me Functionality: Implements a "remember me"
mechanism for user authentication persistence.
* - Authorization Rules: Specifies access control for various
endpoints based on roles.
* - HttpSessionEventPublisher: Publishes HTTP session events
for tracking and management purposes.
* <p>
* Notable configurations:
* - Login and logout URLs, along with session expiration handling.
* - Custom session tracking using cookies.
* - Use of BCrypt for password hashing (via UserDetailsService).
* - Role hierarchy to simplify role-based access control logic.
*
* @author Inessa Repeshko CS-222a
* @see AuthenticationManager
* @see SecurityFilterChain
* @see HttpSecurity
* @see Role
* @see UserDetailsService
* @see HttpSessionEventPublisher
* @see SessionTrackingMode
* @see ServletContextInitializer
*/

@Configuration
@EnableWebSecurity
public class SecurityConfig {
    private static final Logger logger =
LoggerFactory.getLogger(SecurityConfig.class);
    private final UserDetailsService userDetailsService;

```



```

@Autowired
public SecurityConfig(UserDetailsService userDetailsService) {
    this.userDetailsService = userDetailsService;
}

@Bean
public AuthenticationManager
authenticationManager(AuthenticationConfiguration authConfig)
throws Exception {
    return authConfig.getAuthenticationManager();
}

@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity
http) throws Exception {
    http
        .csrf(Customizer.withDefaults())
        .formLogin(form -> form
            .loginPage("/login")

.usernameParameter("corporateEmailOrUsername")
            .passwordParameter("password")
            .loginProcessingUrl("/login")
            .defaultSuccessUrl("/home", false)
            .failureUrl("/login?error=true")
            .failureUrl("/errors/unauthorized")
            .permitAll()
        )
        .logout(logout -> logout
            .logoutUrl("/logout")
            .logoutSuccessUrl("/login")
            .invalidateHttpSession(true)
            .clearAuthentication(true)
            .deleteCookies("JSESSIONID")
            .deleteCookies("remember-me")
            .permitAll()

```

```

        )
        .sessionManagement(session -> session

.invalidSessionUrl("/login?sessionExpired=true")
        .maximumSessions(1)

.expiredUrl("/login?sessionInvalidated=true")
    )
    .rememberMe(rememberMe -> rememberMe
        .key("rememberMeSecretKey")
        .tokenValiditySeconds(7 * 24 * 60 * 60)
        .rememberMeParameter("remember-me")
        .userDetailsService(userDetailsService)
    )
    .userDetailsService(userDetailsService)
    .authorizeHttpRequests(auth -> auth
        .requestMatchers("/image/**").permitAll()
        .requestMatchers("/", "/login",
"/register", "/reset-password", "/change-password",
"/errors/**").permitAll()
        .requestMatchers("/home", "/profile",
"/profile/change-credentials").authenticated()
        .requestMatchers("/users",
"/users/**").hasRole(Role.ADMIN.name())

.requestMatchers("/entrants").hasRole(Role.ENTRANT_VIEWER.name())

.requestMatchers("/students").hasRole(Role.STUDENT_VIEWER.name())

.requestMatchers("/entrants/{id}/view").hasRole(Role.ENTRANT_VIEWE
R.name())

.requestMatchers("/students/{id}/view").hasRole(Role.STUDENT_VIEWE
R.name())

```

```

        .requestMatchers("/entrants/add",
"/entrants/{id}/edit",
"/entrants/{id}/delete").hasRole(Role.ENTRANT_MANAGER.name())
        .requestMatchers("/students/add",
"/students/{id}/edit",
"/students/{id}/delete").hasRole(Role.STUDENT_MANAGER.name())
        .anyRequest().authenticated()
    );

    return http.build();
}

@Bean
public ServletContextInitializer servletContextInitializer() {
    return servletContext -> {

servletContext.setSessionTrackingModes(EnumSet.of(SessionTrackingM
ode.COOKIE));

        servletContext.setSessionTimeout(90);
        servletContext.getSessionCookieConfig().setMaxAge(60 *
60);

    };
}

@Bean
public HttpSessionEventPublisher httpSessionEventPublisher() {
    return new HttpSessionEventPublisher();
}

@Bean
static RoleHierarchy roleHierarchy() {
    return RoleHierarchyImpl.withDefaultRolePrefix()

.role(Role.ADMIN.name()).implies(Role.ENTRANT_MANAGER.name(),
Role.STUDENT_MANAGER.name())

```

```
.role(Role.ENTRANT_MANAGER.name()).implies(Role.ENTRANT_VIEWER.name()  
e(), Role.STUDENT_VIEWER.name())  
  
.role(Role.STUDENT_MANAGER.name()).implies(Role.STUDENT_VIEWER.name()  
e(), Role.ENTRANT_VIEWER.name())  
        .build();  
    }  
}
```

ДОДАТОК В

Код файлів контроллера Spring Boot проєкту

В.1 Код файлу Mode.java

```
package ntukhpi.csit.semit.riv.webappsrivlab4.controller;

import lombok.Getter;

/**
 * Enum class representing the various operation modes for
 * controllers and views.
 * This class is used to define specific modes that dictate the
 * behavior of UI elements and logic flow
 * in different parts of the application.
 * <p>
 * Modes:
 * - ADD: Represents the mode for creating a new entity.
 * - VIEW_FORM: Represents the mode for viewing detailed
 * information about an individual entity.
 * - VIEW_TABLE: Represents the mode for viewing a table or list of
 * entities.
 * - EDIT: Represents the mode for modifying an existing entity.
 * - DELETE: Represents the mode for deleting an entity.
 * <p>
 * Each mode has a descriptive string value that is accessible
 * using Lombok's generated `getValue()` method,
 * which allows for seamless integration with templates and logging
 * mechanisms.
 * <p>
 * Lombok's `@Getter` annotation is used to automatically generate
 * a getter for the `value` field.
 *
 */
```

```

* @author Inessa Repeshko CS-222a
* @see lombok.Getter
* @see Enum
* @see String
*/

```

```
@Getter
```

```

public enum Mode {
    ADD("Add"),
    VIEW_FORM("View"),
    VIEW_TABLE("View"),
    EDIT("Edit"),
    DELETE("Delete");

    private final String value;

    Mode(String value) {
        this.value = value;
    }
}

```

В.2 Код файла EntrantController.java

```

package ntukhpi.csit.semit.riv.webappsrivlab4.controller;

import jakarta.validation.ConstraintViolationException;
import jakarta.validation.Valid;
import
ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.entrant.Entrant;
import
ntukhpi.csit.semit.riv.webappsrivlab4.service.entity.EntrantService;
import
ntukhpi.csit.semit.riv.webappsrivlab4.service.CustomServiceException;

```

```

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.format.annotation.DateTimeFormat;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.validation.BindingResult;
import org.springframework.validation.annotation.Validated;
import org.springframework.web.bind.annotation.*;
import
org.springframework.web.servlet.mvc.support.RedirectAttributes;
import org.springframework.web.util.UriComponentsBuilder;

import java.time.LocalDate;
import java.util.List;

/**
 * Controller class for managing operations on Entrants.
 * This class provides methods to handle CRUD (Create, Read,
 * Update, Delete) operations
 * for Entrant entities, along with features like filtering and
 * sorting.
 * <p>
 * Key functionalities:
 * - Viewing the list of entrants with filtering and sorting
 * options.
 * - Adding new entrants to the database.
 * - Editing existing entrants and saving updates.
 * - Viewing entrant details in a read-only mode.
 * - Deleting entrants from the database with confirmation.
 * <p>
 * Each operation includes exception handling to manage potential
 * validation errors
 * or service layer exceptions, ensuring user-friendly error
 * messages and smooth redirection.
 * <p>

```

```

* Dependencies:
* - EntrantService: Handles business logic and interactions with
the data layer.
* <p>
* Model attributes such as `mode` are used to determine the
current operation
* (e.g., VIEW_TABLE, ADD, EDIT, DELETE) and adapt the view
dynamically.
*
* @author Inessa Repeshko CS-222a
* @see Entrant
* @see EntrantService
* @see CustomServiceException
* @see BindingResult
* @see ConstraintViolationException
* @see Mode
* @see UriComponentsBuilder
* @see RedirectAttributes
*/

@Controller
@Validated
public class EntrantController {
    private static final Logger logger =
LoggerFactory.getLogger(EntrantController.class);
    private final EntrantService entrantService;

    @Autowired
    public EntrantController(EntrantService entrantService) {
        this.entrantService = entrantService;
    }

    @GetMapping("/entrants")
    public String showEntrantTable(@RequestParam(value = "search",
required = false) String search,

```



```

        @RequestParam(value =
"birthdayStart", required = false) @DateTimeFormat(iso =
DateTimeFormat.ISO.DATE) LocalDate birthdayStart,
        @RequestParam(value =
"birthdayEnd", required = false) @DateTimeFormat(iso =
DateTimeFormat.ISO.DATE) LocalDate birthdayEnd,
        @RequestParam(value =
"ratingScoreMin", required = false) Double ratingScoreMin,
        @RequestParam(value =
"ratingScoreMax", required = false) Double ratingScoreMax,
        @RequestParam(value = "sort",
required = false, defaultValue = "id-asc") String sort,
        Model model) {
    model.addAttribute("fields",
Entrant.getFieldNamesAsFormattedStrings());
    model.addAttribute("mode", Mode.VIEW_TABLE);

    try {
        List<Entrant> entrants =
entrantService.getFilteredAndSortedEntrants(
            search,
            birthdayStart,
            birthdayEnd,
            ratingScoreMin,
            ratingScoreMax,
            sort);

        model.addAttribute("entrants", entrants);
    } catch (CustomServiceException |
ConstraintViolationException e) {
        model.addAttribute("errorTitle", "Failed to load
table");
        model.addAttribute("errorMessage", e.getMessage());
    }

    return "entity/entrant/EntrantTable";
}

```

```

    }

    @GetMapping("/entrants/add")
    public String showCreateEntrantForm(Model model) {
        Entrant entrant = new Entrant(
            "KH22-0",
            "Репешко",
            "Інесса",
            "Віталіївна",
            "2000-01-01",
            "FALSE",
            "120.001"
        );
        /*Entrant entrant = new Entrant();*/

        model.addAttribute("entrant", entrant);
        model.addAttribute("mode", Mode.ADD);

        return "entity/entrant/EntrantForm";
    }

    @PostMapping("/entrants/add")
    public String saveEntrant(@Valid @ModelAttribute("entrant")
Entrant entrantToSave,
                                BindingResult result,
                                RedirectAttributes
redirectAttributes,
                                Model model) {
        model.addAttribute("mode", Mode.ADD);

        try {
            if (result.hasErrors()) {
                result.getAllErrors().forEach(e ->
logger.error(e.toString()));
                throw new
CustomServiceException(result.getAllErrors().toString());
            }
        }
    }

```

```

    }

    Entrant savedEntrant =
entrantService.saveEntrant(entrantToSave);

    redirectAttributes.addFlashAttribute("entrant",
savedEntrant);
    redirectAttributes.addFlashAttribute("successTitle",
"Successfully saved");
    redirectAttributes.addFlashAttribute("successMessage",
        "The record successfully saved to the
database.");
    redirectAttributes.addFlashAttribute("isSaved", true);

    String redirectUrl =
UriComponentsBuilder.fromPath("/entrants/{id}/edit")
        .buildAndExpand(savedEntrant.getId())
        .toUriString();

    return "redirect:" + redirectUrl;
} catch (CustomServiceException |
ConstraintViolationException e) {
    model.addAttribute("entrant", entrantToSave);
    model.addAttribute("errorTitle", "Failed to save");
    model.addAttribute("errorMessage", e.getMessage());

    return "entity/entrant/EntrantForm";
}
}

@GetMapping("/entrants/{id}/view")
public String showViewEntrantForm(@PathVariable Long id,
    Model model) {
    model.addAttribute("mode", Mode.VIEW_FORM);

    try {

```

```

        Entrant entrantToView =
entrantService.findEntrantById(id);

        model.addAttribute("entrant", entrantToView);

        return "entity/entrant/EntrantForm";
    } catch (CustomServiceException |
ConstraintViolationException e) {
        model.addAttribute("errorTitle", "Page Not Found
Error");

        model.addAttribute("errorCode", 404);
        model.addAttribute("errorMessage", "Oops! Looks like
the page doesn't exist.");
        model.addAttribute("errorException", e.getMessage());

        return "error/ErrorPage";
    }
}

@GetMapping("/entrants/{id}/edit")
public String showUpdateEntrantForm(@PathVariable Long id,
                                    Model model) {
    model.addAttribute("mode", Mode.EDIT);

    try {
        Entrant entrantToUpdate =
entrantService.findEntrantById(id);

        model.addAttribute("entrant", entrantToUpdate);

        return "entity/entrant/EntrantForm";
    } catch (CustomServiceException |
ConstraintViolationException e) {
        model.addAttribute("errorTitle", "Page Not Found
Error");

        model.addAttribute("errorCode", 404);

```

```

        model.addAttribute("errorMessage", "Oops! Looks like
the page doesn't exist.");

        model.addAttribute("errorException", e.getMessage());

        return "error/ErrorPage";
    }
}

@PostMapping("/entrants/{id}/edit")
public String updateEntrant(@Valid @ModelAttribute("entrant")
Entrant entrantToUpdate,

                           @PathVariable Long id,
                           BindingResult result,
                           Model model) {

    model.addAttribute("mode", Mode.EDIT);

    try {
        if (result.hasErrors()) {
            result.getAllErrors().forEach(e ->
logger.error(e.toString()));
            throw new
CustomServiceException(result.getAllErrors().toString());
        }

        Entrant updatedEntrant = null;

        if (id.equals(entrantToUpdate.getId())) {
            updatedEntrant =
entrantService.updateEntrant(entrantToUpdate);
        } else {
            throw new CustomServiceException("The ID of the
entrant to update does not match the ID in the route." +
            " Check the data and try again.");
        }

        model.addAttribute("entrant", updatedEntrant);
    }
}

```

```

        model.addAttribute("successTitle", "Successfully
updated");

        model.addAttribute("successMessage",
            "The record successfully updated in the
database.");

        return "entity/entrant/EntrantForm";
    } catch (CustomServiceException |
ConstraintViolationException e) {
        model.addAttribute("entrant", entrantToUpdate);
        model.addAttribute("errorTitle", "Failed to update");
        model.addAttribute("errorMessage", e.getMessage());

        return "entity/entrant/EntrantForm";
    }
}

@GetMapping("/entrants/{id}/delete")
public String showDeleteEntrantForm(@PathVariable Long id,
                                    Model model) {
    model.addAttribute("mode", Mode.DELETE);

    try {
        Entrant entrantToDelete =
entrantService.findEntrantById(id);

        model.addAttribute("entrant", entrantToDelete);

        return "entity/entrant/EntrantForm";
    } catch (CustomServiceException |
ConstraintViolationException e) {
        model.addAttribute("errorTitle", "Page Not Found
Error");

        model.addAttribute("errorCode", 404);
        model.addAttribute("errorMessage", "Oops! Looks like
the page doesn't exist.");
    }
}

```

```

        model.addAttribute("errorException", e.getMessage());

        return "error/ErrorPage";
    }
}

@PostMapping("/entrants/{id}/delete")
public String deleteEntrant(@Valid @ModelAttribute("entrant")
Entrant entrantToDelete,

                           @PathVariable Long id,
                           BindingResult result,
                           RedirectAttributes
redirectAttributes,

                           Model model) {
    model.addAttribute("mode", Mode.DELETE);

    try {
        if (result.hasErrors()) {
            result.getAllErrors().forEach(e ->
logger.error(e.toString()));
            throw new
CustomServiceException(result.getAllErrors().toString());
        }

        if (id.equals(entrantToDelete.getId())) {
entrantService.deleteEntrantById(entrantToDelete.getId());
        } else {
            throw new CustomServiceException("The ID of the
entrant to delete does not match the ID in the route." +
            " Check the data and try again.");
        }

        redirectAttributes.addFlashAttribute("successTitle",
"Successfully deleted");
        redirectAttributes.addFlashAttribute("successMessage",

```

```

        "The record successfully deleted from the
database.");

        return "redirect:/entrants";
    } catch (CustomServiceException |
ConstraintViolationException e) {
        model.addAttribute("entrant", entrantToDelete);
        model.addAttribute("errorTitle", "Failed to delete");
        model.addAttribute("errorMessage", e.getMessage());

        return "entity/entrant/EntrantForm";
    }
}
}

```

B.3 Код файла StudentController.java

```

package ntukhpi.csit.semit.riv.webappsrivlab4.controller;

import jakarta.validation.ConstraintViolationException;
import jakarta.validation.Valid;
import
ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.entrant.Entrant;
import
ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.enums.FundingType;
import
ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.enums.ScholarshipS
tatus;
import
ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.student.Student;
import
ntukhpi.csit.semit.riv.webappsrivlab4.service.entity.EntrantServic
e;

```



```

import
ntukhpi.csit.semit.riv.webappsrivlab4.service.entity.StudentService;
import
ntukhpi.csit.semit.riv.webappsrivlab4.service.CustomServiceException;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.validation.BindingResult;
import org.springframework.web.bind.annotation.*;
import
org.springframework.web.servlet.mvc.support.RedirectAttributes;
import org.springframework.web.util.UriComponentsBuilder;

import java.util.List;

/**
 * Controller class for managing operations related to students.
 * This class provides methods for handling CRUD (Create, Read,
 * Update, Delete) operations
 * for the `Student` entity, as well as filtering and sorting
 * functionalities.
 * <p>
 * Key functionalities:
 * - Viewing a list of students with optional filtering and
 * sorting.
 * - Adding new students, including associating them with entrants.
 * - Editing and updating existing students' information.
 * - Viewing details of a specific student.
 * - Deleting students and removing their association with
 * entrants.
 * <p>

```

```

* Exception handling ensures that validation errors and
service-level exceptions are
* properly managed and communicated to the user with meaningful
messages.
* <p>
* Dependencies:
* - `StudentService`: Handles business logic for managing
students.
* - `EntrantService`: Handles business logic for managing
associated entrants.
* <p>
* View modes are managed using the `Mode` enum to dynamically
adapt UI behavior
* based on the current operation (e.g., VIEW_TABLE, ADD, EDIT,
DELETE).
* <p>
* Relationships between `Student` and `Entrant` entities are
maintained,
* ensuring data consistency during operations such as creation and
deletion.
*
* @author Inessa Repeshko CS-222a
* @see Student
* @see Entrant
* @see StudentService
* @see EntrantService
* @see Mode
* @see CustomServiceException
*/

@Controller
public class StudentController {
    private static final Logger logger =
LoggerFactory.getLogger(StudentController.class);
    private final StudentService studentService;
    private final EntrantService entrantService;

```

```

    @Autowired
    public StudentController(StudentService studentService,
EntrantService entrantService) {
        this.studentService = studentService;
        this.entrantService = entrantService;
    }

    @GetMapping("/students")
    public String showStudentTable(@RequestParam(value = "search",
required = false) String search,
                                   @RequestParam(value =
"fundingType", required = false) String fundingType,
                                   @RequestParam(value =
"scholarshipStatus", required = false) String scholarshipStatus,
                                   @RequestParam(value = "sort",
required = false, defaultValue = "id-asc") String sort,
                                   Model model) {
        model.addAttribute("fields",
Student.getFieldNamesAsFormattedStrings());
        model.addAttribute("mode", Mode.VIEW_TABLE);

        try {
            List<Student> students =
studentService.getFilteredAndSortedStudents(
                search,
                fundingType,
                scholarshipStatus,
                sort);

            model.addAttribute("students", students);
        } catch (CustomServiceException |
ConstraintViolationException e) {
            model.addAttribute("errorTitle", "Failed to load
table");

            model.addAttribute("errorMessage", e.getMessage());

```

```

    }

    return "entity/student/StudentTable";
}

@GetMapping("/students/add")
public String showCreateStudentForm(Model model) {
    Student student = new Student();
    student.setFundingType(FundingType.BUDGET);
    student.setScholarshipStatus(ScholarshipStatus.ORDINARY);
    student.setCorporateEmail("regina.marchuk@cs.khpi.edu.ua");

    List<Entrant> entrantList =
entrantService.findEntrantsWithoutStudents();

    model.addAttribute("student", student);
    model.addAttribute("entrantList", entrantList);
    model.addAttribute("mode", Mode.ADD);

    return "entity/student/StudentForm";
}

@PostMapping("/students/add")
public String saveStudent(@Valid @ModelAttribute("student")
Student studentToSave,

                        BindingResult result,
                        RedirectAttributes
redirectAttributes,

                        Model model) {
    model.addAttribute("mode", Mode.ADD);

    try {
        if (result.hasErrors()) {
            result.getAllErrors().forEach(e ->
logger.error(e.toString()));

```

```

        throw new
CustomServiceException(result.getAllErrors().toString());
    }

    studentToSave.setEntrant(entrantService.findEntrantById(studentToS
ave.getEntrant().getId()));

    Student savedStudent =
studentService.saveStudent(studentToSave);

    Entrant entrantToUpdate =
entrantService.findEntrantById(studentToSave.getEntrant().getId())
;

    entrantToUpdate.setStudentId(studentToSave.getId());
    entrantService.updateEntrant(entrantToUpdate);

    redirectAttributes.addFlashAttribute("student",
savedStudent);

    redirectAttributes.addFlashAttribute("successTitle",
"Successfully saved");
    redirectAttributes.addFlashAttribute("successMessage",
        "The record successfully saved to the
database.");
    redirectAttributes.addFlashAttribute("isSaved", true);

    String redirectUrl =
UriComponentsBuilder.fromPath("/students/{id}/edit")
        .buildAndExpand(savedStudent.getId())
        .toUriString();

    return "redirect:" + redirectUrl;
} catch (CustomServiceException |
ConstraintViolationException e) {

    List<Entrant> entrantList =
entrantService.findEntrantsWithoutStudents();

```

```

        model.addAttribute("entrantList", entrantList);
        model.addAttribute("student", studentToSave);
        model.addAttribute("errorTitle", "Failed to save");
        model.addAttribute("errorMessage", e.getMessage());

        return "entity/student/StudentForm";
    }
}

@GetMapping("/students/{id}/view")
public String showViewStudentForm(@PathVariable Long id,
                                   Model model) {
    model.addAttribute("mode", Mode.VIEW_FORM);

    try {
        Student studentToUpdate =
studentService.findStudentById(id);
        List<Entrant> entrantList =
entrantService.findEntrantsWithoutStudents();

        model.addAttribute("student", studentToUpdate);
        model.addAttribute("entrantList", entrantList);

        return "entity/student/StudentForm";
    } catch (CustomServiceException |
ConstraintViolationException e) {
        model.addAttribute("errorTitle", "Page Not Found
Error");
        model.addAttribute("errorCode", 404);
        model.addAttribute("errorMessage", "Oops! Looks like
the page doesn't exist.");
        model.addAttribute("errorException", e.getMessage());

        return "error/ErrorPage";
    }
}

```

```

@GetMapping("/students/{id}/edit")
public String showUpdateStudentForm(@PathVariable Long id,
                                     Model model) {
    model.addAttribute("mode", Mode.EDIT);

    try {
        Student studentToUpdate =
studentService.findStudentById(id);
        List<Entrant> entrantList =
entrantService.findEntrantsWithoutStudents();

        model.addAttribute("student", studentToUpdate);
        model.addAttribute("entrantList", entrantList);

        return "entity/student/StudentForm";
    } catch (CustomServiceException |
ConstraintViolationException e) {
        model.addAttribute("errorTitle", "Page Not Found
Error");
        model.addAttribute("errorCode", 404);
        model.addAttribute("errorMessage", "Oops! Looks like
the page doesn't exist.");
        model.addAttribute("errorException", e.getMessage());

        return "error/ErrorPage";
    }
}

@PostMapping("/students/{id}/edit")
public String updateStudent(@Valid @ModelAttribute("student")
Student studentToUpdate,
                            @PathVariable Long id,
                            BindingResult result,
                            Model model) {
    model.addAttribute("mode", Mode.EDIT);

```

```

        try {
            if (result.hasErrors()) {
                result.getAllErrors().forEach(e ->
logger.error(e.toString()));
                throw new
CustomServiceException(result.getAllErrors().toString());
            }

            Student updatedStudent;

            if (id.equals(studentToUpdate.getId())) {

studentToUpdate.setEntrant(entrantService.findEntrantById(studentT
oUpdate.getEntrant().getId()));
                updatedStudent =
studentService.updateStudent(studentToUpdate);
            } else {
                throw new CustomServiceException("The ID of the
student to update does not match the ID in the route." +
                " Check the data and try again.");
            }

            List<Entrant> entrantList =
entrantService.findEntrantsWithoutStudents();

            model.addAttribute("student", updatedStudent);
            model.addAttribute("entrantList", entrantList);
            model.addAttribute("successTitle", "Successfully
updated");
            model.addAttribute("successMessage",
                "The record successfully updated in the
database.");

            return "entity/student/StudentForm";

```



```

        } catch (CustomServiceException |
ConstraintViolationException e) {

            List<Entrant> entrantList =
entrantService.findEntrantsWithoutStudents();

            model.addAttribute("entrantList", entrantList);
            model.addAttribute("student", studentToUpdate);
            model.addAttribute("errorTitle", "Failed to update");
            model.addAttribute("errorMessage", e.getMessage());

            return "entity/student/StudentForm";
        }
    }

    @GetMapping("/students/{id}/delete")
    public String showDeleteStudentForm(@PathVariable Long id,
                                         Model model) {

        model.addAttribute("mode", Mode.DELETE);

        try {

            Student studentToDelete =
studentService.findStudentById(id);

            List<Entrant> entrantList =
entrantService.findEntrantsWithoutStudents();

            model.addAttribute("entrantList", entrantList);
            model.addAttribute("student", studentToDelete);

            return "entity/student/StudentForm";
        } catch (CustomServiceException |
ConstraintViolationException e) {

            model.addAttribute("errorTitle", "Page Not Found
Error");

            model.addAttribute("errorCode", 404);

            model.addAttribute("errorMessage", "Oops! Looks like
the page doesn't exist.");

```

```

        model.addAttribute("errorException", e.getMessage());

        return "error/ErrorPage";
    }
}

@PostMapping("/students/{id}/delete")
public String deleteStudent(@Valid @ModelAttribute("student")
Student studentToDelete,

                            @PathVariable Long id,
                            BindingResult result,
                            RedirectAttributes
redirectAttributes,

                            Model model) {
    model.addAttribute("mode", Mode.DELETE);

    try {
        if (result.hasErrors()) {
            result.getAllErrors().forEach(e ->
logger.error(e.toString()));
            throw new
CustomServiceException(result.getAllErrors().toString());
        }

        if (id.equals(studentToDelete.getId())) {
studentService.deleteStudent(studentToDelete.getId());
        } else {
            throw new CustomServiceException("The ID of the
student to delete does not match the ID in the route." +
            " Check the data and try again.");
        }

        redirectAttributes.addFlashAttribute("successTitle",
"Successfully deleted");
        redirectAttributes.addFlashAttribute("successMessage",

```

```

        "The record successfully deleted from the
        database.");

        return "redirect:/students";
    } catch (CustomServiceException |
ConstraintViolationException e) {
        List<Entrant> entrantList =
entrantService.findEntrantsWithoutStudents();

        model.addAttribute("entrantList", entrantList);
        model.addAttribute("student", studentToDelete);
        model.addAttribute("errorTitle", "Failed to delete");
        model.addAttribute("errorMessage", e.getMessage());

        return "entity/student/StudentForm";
    }
}
}

```

B.4 Код файла AuthController.java

```

package ntukhpi.csit.semit.riv.webappsrivlab4.controller;

import jakarta.servlet.http.HttpServletRequest;
import jakarta.validation.ConstraintViolationException;
import jakarta.validation.Valid;
import jakarta.validation.constraints.NotNull;
import ntukhpi.csit.semit.riv.webappsrivlab4.model.DTO.user.*;
import
ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.token.PasswordRese
tToken;
import
ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.user.UserEntity;

```

```

import
ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.testDataList.user.
UserList;
import
ntukhpi.csit.semit.riv.webappsrivlab4.service.entity.PasswordReset
TokenService;
import
ntukhpi.csit.semit.riv.webappsrivlab4.service.entity.UserService;
import
ntukhpi.csit.semit.riv.webappsrivlab4.service.CustomServiceExcepti
on;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.validation.BindingResult;
import org.springframework.validation.annotation.Validated;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.ModelAttribute;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestParam;
import
org.springframework.web.servlet.mvc.support.RedirectAttributes;

import java.util.List;

/**
 * Controller class for managing user authentication and
 * authorization.
 * This class handles HTTP requests related to user login,
 * registration, password reset, and password change processes.
 * It serves as a bridge between the service layer and the view
 * layer, managing the flow of user-related data.
 * <p>
 * Key functionalities:

```

```

* - Login: Displays the login page and handles authentication
errors.
* - Registration: Handles new user registration and ensures the
validity of input data.
* - Password Reset: Allows users to request and handle password
reset operations via email.
* - Password Change: Facilitates password update based on valid
tokens.
* <p>
* Exception handling is incorporated to provide user-friendly
error messages for various scenarios,
* such as invalid credentials, expired sessions, and incorrect
token usage.
* <p>
* Dependencies:
* - UserService: For user-related operations such as registration,
login, and updates.
* - PasswordResetTokenService: For managing password reset tokens
and their validation.
*
* @author Inessa Repeshko CS-222a
* @see UserService
* @see PasswordResetTokenService
* @see PasswordResetToken
* @see UserEntity
* @see UserTestData
* @see UserLogin
* @see UserRegister
* @see UserResetPassword
* @see UserChangePassword
* @see CustomServiceException
*/

@Controller
@Validated
public class AuthController {

```

```

    private static final Logger logger =
LoggerFactory.getLogger(AuthController.class);

    private final UserService userService;
    private final PasswordResetTokenService tokenService;

    private final List<UserTestData> userTestDataList;

    @Autowired
    public AuthController(UserService userService,
                           PasswordResetTokenService tokenService) {
        this.userService = userService;
        this.tokenService = tokenService;
        this.userTestDataList = UserList.INSTANCE.getUsers();
    }

    @GetMapping("/")
    public String showStartPage() {
        return "redirect:/login";
    }

    @GetMapping("/login")
    public String showLoginPage(@RequestParam(value = "error",
required = false) String error,
                               @RequestParam(value =
"sessionExpired", required = false) String sessionExpired,
                               Model model) {
        if (error != null) {
            model.addAttribute("errorTitle", "Failed
authorization");
            model.addAttribute("errorMessage", "Invalid user
identifier or password entered. " +
                               "Please, check your credentials and try
again!");
        }

        if (Boolean.parseBoolean(sessionExpired)) {

```

```

        model.addAttribute("warningTitle", "Session expired");
        model.addAttribute("warningMessage", "Your session
expired. " +
            "Please, sign in again!");
    }

    UserTestData userTestData = userTestDataList.get(0);
    UserLogin userLogin =
UserLogin.getUserLoginFromUserTestData(userTestData);

    model.addAttribute("user", userLogin);

    return "auth/Login";
}

@GetMapping("/register")
public String showRegisterPage(Model model) {
    UserTestData userTestData = userTestDataList.get(0);
    UserRegister userRegister =
UserRegister.getUserRegisterFromUserTestData(userTestData);

    model.addAttribute("user", userRegister);

    return "auth/Register";
}

@PostMapping("/register")
public String saveUser(@Valid @ModelAttribute("user")
UserRegister userRegister,
                        BindingResult result,
                        RedirectAttributes redirectAttributes,
                        Model model) {
    try {
        if (result.hasErrors()) {
            result.getAllErrors().forEach(e ->
logger.error(e.toString()));

```

```

        throw new
CustomServiceException(result.getAllErrors().toString());
    }

    if (userRegister.getPassword() != null
        && !userRegister.getPassword().isBlank()
        && userRegister.getConfirmPassword() != null
        &&
!userRegister.getConfirmPassword().isBlank()) {
        if
(!userRegister.getPassword().equals(userRegister.getConfirmPasswor
d())) {
            throw new CustomServiceException("The password
and password confirmation are not identical. " +
                "Please check the data and try
again!");
        }
    } else {
        throw new CustomServiceException("Invalid password
data entered. " +
            "Please check the data and try again!");
    }

    UserEntity savedUserEntity = userService.saveUser(
UserEntity.getUserEntityFromUserRegister(userRegister));
    UserLogin userLogin =
UserLogin.getUserLoginFromUserEntity(savedUserEntity);

    redirectAttributes.addFlashAttribute("user",
userLogin);
    redirectAttributes.addFlashAttribute("successTitle",
"Successfully registered.");
    redirectAttributes.addFlashAttribute("successMessage",
        "You have successfully registered!");
    redirectAttributes.addFlashAttribute("isSaved", true);

```



```

        return "redirect:/login";
    } catch (CustomServiceException |
ConstraintViolationException e) {
        model.addAttribute("user", userRegister);
        model.addAttribute("action", "save");
        model.addAttribute("errorTitle", "Failed to save");
        model.addAttribute("errorMessage", e.getMessage());

        return "auth/Register";
    }
}

@GetMapping("/reset-password")
public String showResetPasswordPage(Model model) {
    UserTestData userTestData = userTestDataList.get(1);
    UserResetPassword userResetPassword =
UserResetPassword.getUserResetPasswordFromUserTestData(userTestDat
a);

    model.addAttribute("user", userResetPassword);

    return "auth/ResetPassword";
}

@PostMapping("/reset-password")
public String sendResetPasswordMail(@Valid
@ModelAttribute("user") UserResetPassword userResetPassword,
                                   BindingResult result,
                                   Model model,
                                   HttpServletRequest request)
{
    try {
        if (result.hasErrors()) {
            result.getAllErrors().forEach(e ->
logger.error(e.toString()));

```

```

        throw new
CustomServiceException(result.getAllErrors().toString());
    }

    UserEntity userEntity = null;

    try {
        userEntity =
userService.findUserByCorporateEmail(userResetPassword.getCorporat
eEmail());

        if
(tokenService.findTokenByUserId(userEntity.getId()) != null) {
tokenService.deleteTokenByUserId(userEntity.getId());
        }
    } catch (CustomServiceException e) {
        logger.error("Unable to find a user to send a
password recovery email. " + e.getMessage());
    }

    MailController.sendResetPasswordMail(userEntity,
request);

    model.addAttribute("successTitle", "Successfully sent
an email");
    model.addAttribute("successMessage",
        "If the email address you entered is associated
with an account, " +
            "a password reset link has been sent to
your email.\n" +
            "Please check your inbox and follow the
instructions to reset your password.");

    return "auth/ResetPassword";
} catch (Exception e) {

```

```

        model.addAttribute("user", userResetPassword);
        model.addAttribute("errorTitle", "Failed to send an
email");
        model.addAttribute("errorMessage", e.getMessage());

        return "auth/ResetPassword";
    }
}

@GetMapping("/change-password")
public String changePassword(@RequestParam(value = "token",
required = true) String token,
                             RedirectAttributes
redirectAttributes,
                             Model model) {

    try {
        PasswordResetToken foundToken =
tokenService.findTokenByToken(token);

        if (tokenService.isValidToken(foundToken)) {

            UserEntity userEntity =
userService.findUserByExample(foundToken.getUser());

            if (userEntity == null || userEntity.getId() ==
null) {

                String message = "The user data in the token is
invalid.";

                logger.error(message);
                throw new CustomServiceException(message);
            }

            UserChangePassword userChangePassword = new
UserChangePassword();

            model.addAttribute("token", token);

```

```

        model.addAttribute("user", userChangePassword);

        return "auth/ChangePasswordForm";
    } else {
        String message = "The specified token is invalid or
expired.";

        logger.error(message);

        throw new CustomServiceException(message);
    }
} catch (CustomServiceException |
ConstraintViolationException e) {
    redirectAttributes.addFlashAttribute("errorTitle",
"Access failed");

    redirectAttributes.addFlashAttribute("errorMessage",
        "The specified token is invalid or expired. " +
        "Please, try to restore access to your
account again!");

    return "redirect:/login";
}
}

@PostMapping("/change-password")
public String showChangePasswordPage(@NotNull
@ModelAttribute("token") String token,
                                     @Valid
@ModelAttribute("user") UserChangePassword userChangePassword,
                                     BindingResult result,
                                     RedirectAttributes
redirectAttributes,
                                     Model model) {

    try {
        if (result.hasErrors()) {
            result.getAllErrors().forEach(e ->
logger.error(e.toString()));

```

```

        throw new
CustomServiceException(result.getAllErrors().toString());
    }

    PasswordResetToken foundToken =
tokenService.findTokenByToken(token);

    if (tokenService.isValidToken(foundToken)) {
        UserEntity userEntityToUpdate =
userService.findUserById(foundToken.getUser().getId());

        if (userEntityToUpdate == null ||
userEntityToUpdate.getId() == null) {
            String message = "The user data in the token is
invalid.";

            logger.error(message);
            throw new CustomServiceException(message);
        }

        if (userChangePassword.getPassword() != null
            &&
!userChangePassword.getPassword().isBlank()
            && userChangePassword.getConfirmPassword()
!= null
            &&
!userChangePassword.getConfirmPassword().isBlank()) {
            if
(!userChangePassword.getPassword().equals(userChangePassword.getCo
nfirmPassword())) {
                throw new CustomServiceException("The
password and password confirmation are not identical. " +
                    "Please check the data and try
again!");
            }
        } else {

```

```

            throw new CustomServiceException("Invalid
password data entered. " +
            "Please check the data and try
again!");
        }

userEntityToUpdate.setPassword(userChangePassword.getPassword());
userService.updateUser(userEntityToUpdate);

redirectAttributes.addFlashAttribute("token",
token);

redirectAttributes.addFlashAttribute("successTitle", "Successfully
updated password");

redirectAttributes.addFlashAttribute("successMessage",
    "Your password successfully updated.
Please, sign in!");

        return "redirect:/login";
    } else {
        String message = "The passed token is invalid or
expired.";

        logger.error(message);
        throw new CustomServiceException(message);
    }
} catch (CustomServiceException |
ConstraintViolationException e) {
    model.addAttribute("errorTitle", "Access failed");
    model.addAttribute("errorMessage",
        "The passed token is invalid or expired. " +
        "Please, try to restore access to your
account again!");

    return "auth/ChangePasswordForm";
}

```

```

    }

}

}

```

В.5 Код файла HomeController.java

```

package ntukhpi.csit.semit.riv.webappsrivlab4.controller;

import jakarta.servlet.http.Cookie;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import jakarta.validation.ConstraintViolationException;
import jakarta.validation.Valid;
import ntukhpi.csit.semit.riv.webappsrivlab4.config.EncoderConfig;
import
ntukhpi.csit.semit.riv.webappsrivlab4.model.DTO.user.UserChangeCre
dentials;
import
ntukhpi.csit.semit.riv.webappsrivlab4.model.DTO.user.UserForm;
import ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.enums.Role;
import
ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.user.UserEntity;
import
ntukhpi.csit.semit.riv.webappsrivlab4.service.entity.UserService;
import
ntukhpi.csit.semit.riv.webappsrivlab4.service.CustomServiceExcepti
on;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Autowired;
import
org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.validation.BindingResult;

```

```

import org.springframework.validation.annotation.Validated;
import org.springframework.web.bind.annotation.*;

/**
 * Controller class for managing the user's home and
 * profile-related operations.
 * This class provides endpoints for viewing the home page,
 * managing the user's profile,
 * and updating credentials such as username and password.
 * <p>
 * Key functionalities:
 * - Displaying the home page.
 * - Viewing and updating the user's profile information.
 * - Changing user credentials (username and password) securely.
 * - Handling unauthorized access errors with a user-friendly error
 * page.
 * <p>
 * Exception handling ensures that validation errors and
 * service-level exceptions
 * are properly managed, providing meaningful error messages to the
 * user.
 * <p>
 * Dependencies:
 * - UserService: Provides user-related business logic and database
 * interactions.
 * - EncoderConfig: Handles password encryption and verification.
 * <p>
 * SecurityContextHolder is used to manage the authenticated user's
 * session,
 * ensuring that user-specific data is retrieved dynamically.
 *
 * @author Inessa Repeshko CS-222a
 * @see UserService
 * @see EncoderConfig
 * @see UserEntity
 * @see UserForm

```



```

* @see UserChangeCredentials
* @see SecurityContextHolder
* @see CustomServiceException
* @see ConstraintViolationException
* @see Cookie
* @see HttpServletRequest
* @see HttpServletResponse
*/

@Controller
@Validated
public class HomeController {

    private static final Logger logger =
LoggerFactory.getLogger(HomeController.class);

    private final UserService userService;
    private final EncoderConfig encoderConfig;

    @Autowired
    public HomeController(UserService userService, EncoderConfig
encoderConfig) {

        this.userService = userService;
        this.encoderConfig = encoderConfig;
    }

    public UserEntity getCurrentUser() {
        return userService.findUserByUsername(
SecurityContextHolder.getContext().getAuthentication()
                .getName());
    }

    @GetMapping("/home")
    public String showHomePage(Model model) {
        /*model.addAttribute("user", getCurrentUser());*/

        return "home/Home";
    }
}

```

```

    }

    @GetMapping("/profile")
    public String showProfilePage(Model model) {
        UserForm userForm =
UserForm.getUserFormFromUserEntity(getCurrentUser());

        model.addAttribute("user", userForm);

        return "home/ProfileForm";
    }

    @GetMapping("/profile/change-credentials")
    public String showProfileChangePasswordPage(Model model) {
        UserChangeCredentials userChangeCredentials = new
UserChangeCredentials();

        model.addAttribute("userChangeCredentials",
userChangeCredentials);

        return "home/ChangeCredentialsForm";
    }

    @PostMapping("/profile/change-credentials")
    public String updateCredentials(@Valid
@ModelAttribute("userChangeCredentials") UserChangeCredentials
userChangeCredentials,

                                   BindingResult result,
                                   Model model,
                                   HttpServletRequest request,
                                   HttpServletResponse response) {

        try {
            if (result.hasErrors()) {
                result.getAllErrors().forEach(e ->
logger.error(e.toString()));

```

```

        throw new
CustomServiceException(result.getAllErrors().toString());
    }

    UserEntity userEntityToUpdate = getCurrentUser();
    UserChangeCredentials updatedUserCredentials = null;

    if (userChangeCredentials.getPassword() != null
        &&
!userChangeCredentials.getPassword().isBlank()
        && userChangeCredentials.getConfirmPassword()
!= null
        &&
!userChangeCredentials.getConfirmPassword().isBlank()) {
        if
(!userChangeCredentials.getPassword().equals(userChangeCredentials
.getConfirmPassword())) {
            throw new CustomServiceException("The new
password and password confirmation are not identical." +
                " Please, check the data and try
again!");
        }

        if (encoderConfig.passwordEncoder().matches(
            userChangeCredentials.getPassword(),
            userEntityToUpdate.getPassword())) {
            throw new CustomServiceException("The new
password cannot be identical to the current password." +
                " Please, check the data and try
again!");
        }

userEntityToUpdate.setPassword(encoderConfig.passwordEncoder().enc
ode(userChangeCredentials.getPassword()));
    } else {

```

```

        throw new CustomServiceException("Invalid password
data entered. " +
        "Please, check the data and try again!");
    }

    if (userChangeCredentials.getUsername() != null
        &&
        !userChangeCredentials.getUsername().isBlank()) {
        if
        (!userChangeCredentials.getUsername().equals(userEntityToUpdate.ge
tUsername())) {
            if
            (userService.findUserByUsername(userChangeCredentials.getUsername(
)) == null) {

                userEntityToUpdate.setUsername(userChangeCredentials.getUsername()
);

            } else {
                throw new CustomServiceException("The
provided name already belongs to another user. " +
                "Please, check the data and try
again!");
            }
        }
    } else {
        throw new CustomServiceException("Invalid username
entered. " +
        "Please, check the data and try again!");
    }

    updatedUserCredentials =
    UserChangeCredentials.getUserChangeCredentialsFromUserEntity(
        userService.updateUser(userEntityToUpdate));

```

```

        model.addAttribute("userChangeCredentials",
updatedUserCredentials);

        model.addAttribute("successTitle", "Successfully
updated");

        model.addAttribute("successMessage",
            "User credentials successfully updated in the
database.");

        SecurityContextHolder.clearContext();
        request.getSession().invalidate();

        Cookie cookie = new Cookie("JSESSIONID", null);
        cookie.setHttpOnly(true);
        cookie.setPath("/");
        cookie.setMaxAge(0);
        response.addCookie(cookie);

        return "home/ChangeCredentialsForm";
    } catch (CustomServiceException |
ConstraintViolationException e) {
        model.addAttribute("roleList", Role.getAllRoles());
        model.addAttribute("userChangeCredentials",
userChangeCredentials);

        model.addAttribute("errorTitle", "Failed to update");
        model.addAttribute("errorMessage", e.getMessage());

        return "home/ChangeCredentialsForm";
    }
}

@GetMapping("/errors/unauthorized")
public String showUnauthorizedErrorPage(Model model) {
    model.addAttribute("errorTitle", "Unauthorized Error");
    model.addAttribute("errorCode", 401);
    model.addAttribute("errorMessage", "Unauthorized access.
Please check your credentials.");
}

```

```

        return "error/ErrorPage";
    }
}

```

B.6 Код файла UserController.java

```

package ntukhpi.csit.semit.riv.webappsrivlab4.controller;

import jakarta.servlet.http.HttpServletRequest;
import jakarta.validation.ConstraintViolationException;
import jakarta.validation.Valid;
import ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.enums.Role;
import
ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.user.UserEntity;
import
ntukhpi.csit.semit.riv.webappsrivlab4.model.DTO.user.UserForm;
import
ntukhpi.csit.semit.riv.webappsrivlab4.service.entity.UserService;
import
ntukhpi.csit.semit.riv.webappsrivlab4.service.CustomServiceExcepti
on;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.validation.BindingResult;
import org.springframework.validation.annotation.Validated;
import org.springframework.web.bind.annotation.*;
import
org.springframework.web.servlet.mvc.support.RedirectAttributes;
import org.springframework.web.util.UriComponentsBuilder;

import java.util.List;

```

```

/**
 * Controller class for managing user-related operations.
 * This class provides methods for handling CRUD (Create, Read,
 * Update, Delete) operations
 * for the `User` entity, as well as filtering and sorting
 * functionalities.
 *
 * <p>
 * Key functionalities:
 * - Viewing a list of users with optional filtering and sorting.
 * - Adding new users and sending welcome emails with password
 * setup instructions.
 * - Editing and updating user details, including roles and
 * personal information.
 * - Deleting users while ensuring proper validation and error
 * handling.
 *
 * <p>
 * Exception handling ensures that validation errors and
 * service-level exceptions are
 * properly managed and communicated to the user with meaningful
 * messages.
 *
 * <p>
 * Dependencies:
 * - `UserService`: Handles business logic for managing users.
 * - `MailController`: Handles email notifications, such as welcome
 * emails.
 *
 * <p>
 * View modes are managed using the `Mode` enum to dynamically
 * adapt UI behavior
 * based on the current operation (e.g., VIEW_TABLE, ADD, EDIT,
 * DELETE).
 *
 * <p>
 * Relationships between users and other entities are maintained to
 * ensure
 * data consistency during operations such as creation and
 * deletion.
 *
 */

```

```

* @author Inessa Repeshko CS-222a
* @see UserForm
* @see UserEntity
* @see UserService
* @see MailController
* @see Mode
* @see CustomServiceException
*/

@Controller
@Validated
public class UserController {
    private static final Logger logger =
LoggerFactory.getLogger(UserController.class);
    private final UserService userService;

    @Autowired
    public UserController(UserService userService) {
        this.userService = userService;
    }

    @GetMapping("/users")
    public String showUserTable(@RequestParam(value = "search",
required = false) String search,
                                @RequestParam(value = "role",
required = false) String role,
                                @RequestParam(value = "sort",
required = false, defaultValue = "id-asc") String sort,
                                Model model) {
        model.addAttribute("mode", Mode.VIEW_TABLE);
        model.addAttribute("fields",
UserForm.getFieldNamesAsFormattedStrings());

        try {
            List<UserEntity> userEntities =
userService.getFilteredAndSortedUsers(search, role, sort);

```



```

        List<UserForm> users = userEntities.stream()
            .map(UserForm::getUserFormFromUserEntity)
            .toList();

        model.addAttribute("users", users);
    } catch (CustomServiceException |
ConstraintViolationException e) {
        model.addAttribute("errorTitle", "Failed to load
table");
        model.addAttribute("errorMessage", e.getMessage());
    }

    return "entity/user/UserTable";
}

@GetMapping("/users/add")
public String showAddUserForm(Model model) {
    model.addAttribute("mode", Mode.ADD);

    UserForm user = new UserForm(
        null,
        "Марчук",
        "Регіна",
        "Віталіївна",
        "regina.marchuk@cs.khpi.edu.ua",
        "regina.marchuk",
        Role.STUDENT_VIEWER);

    model.addAttribute("user", user);
    model.addAttribute("roleList", Role.getAllRoles());

    return "entity/user/UserForm";
}

@PostMapping("/users/add")

```

```

    public String saveUser(@Valid @ModelAttribute("user") UserForm
userToSave,

                                BindingResult result,
                                RedirectAttributes redirectAttributes,
                                Model model,
                                HttpServletRequest request) {

        model.addAttribute("mode", Mode.ADD);

        try {
            if (result.hasErrors()) {
                result.getAllErrors().forEach(e ->
logger.error(e.toString()));
                throw new
CustomServiceException(result.getAllErrors().toString());
            }

            UserEntity savedUserEntity = userService.saveUser(
                UserEntity.getUserEntityFromUserForm(
                    userToSave,
                    null));

            MailController.sendWelcomeMail(savedUserEntity,
request);

            redirectAttributes.addFlashAttribute("user",
savedUserEntity);
            redirectAttributes.addFlashAttribute("successTitle",
"Successfully saved");
            redirectAttributes.addFlashAttribute("successMessage",
                "The record successfully saved to the
database.\n"
                    + "A password setup email was sent to
'" + savedUserEntity.getCorporateEmail() + "'.");
            redirectAttributes.addFlashAttribute("isSaved", true);

```

```

        String redirectUrl =
UriComponentsBuilder.fromPath("/users/{id}/edit")
                        .buildAndExpand(savedUserEntity.getId())
                        .toUriString();

        return "redirect:" + redirectUrl;
    } catch (CustomServiceException |
ConstraintViolationException e) {
        model.addAttribute("user", userToSave);
        model.addAttribute("roleList", Role.getAllRoles());
        model.addAttribute("errorTitle", "Failed to save");
        model.addAttribute("errorMessage", e.getMessage());

        return "entity/user/UserForm";
    }
}

@GetMapping("/users/{id}/edit")
public String showEditUserForm(@PathVariable Long id,
                                Model model) {
    model.addAttribute("mode", Mode.EDIT);

    try {
        UserForm userToUpdate =
UserForm.getUserFormFromUserEntity(
                                userService.findUserById(id));

        model.addAttribute("user", userToUpdate);
        model.addAttribute("roleList", Role.getAllRoles());

        return "entity/user/UserForm";
    } catch (CustomServiceException |
ConstraintViolationException e) {
        model.addAttribute("errorTitle", "Page Not Found
Error");

        model.addAttribute("errorCode", 404);
    }
}

```

```

        model.addAttribute("errorMessage", "Oops! Looks like
the page doesn't exist.");

        model.addAttribute("errorException", e.getMessage());

        return "error/ErrorPage";
    }
}

@PostMapping("/users/{id}/edit")
public String updateUser(@Valid @ModelAttribute("user")
UserForm userToUpdate,

                        @PathVariable Long id,
                        BindingResult result,
                        Model model) {

    model.addAttribute("mode", Mode.EDIT);

    try {
        if (result.hasErrors()) {
            result.getAllErrors().forEach(e ->
logger.error(e.toString()));
            throw new
CustomServiceException(result.getAllErrors().toString());
        }

        UserForm updatedUser;
        UserEntity userEntity =
userService.findUserById(userToUpdate.getId());

        if (id.equals(userToUpdate.getId())
            && userEntity != null
            && id.equals(userEntity.getId())) {
            UserEntity userEntityToSave =
UserEntity.getUserEntityFromUserForm(
                userToUpdate,
                userEntity.getPassword());

```

```

        UserEntity updatedUserEntity =
userService.updateUser(userEntityToSave);

        updatedUser =
UserForm.getUserFormFromUserEntity(updatedUserEntity);
    } else {

        throw new CustomServiceException("The ID of the
user to update does not match the ID in the route " +
            "or the user with the specified ID does not
exist. Check the data and try again.");
    }

    model.addAttribute("user", updatedUser);
    model.addAttribute("roleList", Role.getAllRoles());
    model.addAttribute("successTitle", "Successfully
updated");
    model.addAttribute("successMessage",
        "The record successfully updated in the
database.");

    return "entity/user/UserForm";
} catch (CustomServiceException |
ConstraintViolationException e) {
    model.addAttribute("user", userToUpdate);
    model.addAttribute("roleList", Role.getAllRoles());
    model.addAttribute("errorTitle", "Failed to update");
    model.addAttribute("errorMessage", e.getMessage());

    return "entity/user/UserForm";
}
}

@GetMapping("/users/{id}/delete")
public String showDeleteUserForm(@PathVariable Long id,
                                Model model) {
    model.addAttribute("mode", Mode.DELETE);

```

```

        try {
            UserForm userToDelete =
UserForm.getUserFormFromUserEntity(
                userService.findUserById(id));

            model.addAttribute("roleList", Role.getAllRoles());
            model.addAttribute("user", userToDelete);

            return "entity/user/UserForm";
        } catch (CustomServiceException |
ConstraintViolationException e) {
            model.addAttribute("errorTitle", "Page Not Found
Error");

            model.addAttribute("errorCode", 404);
            model.addAttribute("errorMessage", "Oops! Looks like
the page doesn't exist.");
            model.addAttribute("errorException", e.getMessage());

            return "error/ErrorPage";
        }
    }

    @PostMapping("/users/{id}/delete")
    public String deleteUser(@ModelAttribute("user") UserForm
userToDelete,

                            @PathVariable Long id,
                            BindingResult result,
                            RedirectAttributes redirectAttributes,
                            Model model) {
        model.addAttribute("mode", Mode.DELETE);

        try {
            if (result.hasErrors()) {
                result.getAllErrors().forEach(e ->
logger.error(e.toString()));

```

```

        throw new
CustomServiceException(result.getAllErrors().toString());
    }

    UserEntity userEntity =
userService.findUserById(userToDelete.getId());

    if (id.equals(userToDelete.getId())
        && userEntity != null
        && id.equals(userEntity.getId())) {
        userService.deleteUserById(userEntity.getId());
    } else {
        throw new CustomServiceException("The ID of the
user to delete does not match the ID in the route " +
            "or the user with the specified ID does not
exist. Check the data and try again.");
    }

    redirectAttributes.addFlashAttribute("successTitle",
"Successfully deleted");
    redirectAttributes.addFlashAttribute("successMessage",
        "The record successfully deleted from the
database.");

    return "redirect:/users";
} catch (CustomServiceException |
ConstraintViolationException e) {
    model.addAttribute("user", userToDelete);
    model.addAttribute("roleList", Role.getAllRoles());
    model.addAttribute("errorTitle", "Failed to delete");
    model.addAttribute("errorMessage", e.getMessage());
    return "entity/user/UserForm";
}
}
}

```

ДОДАТОК Г

Код файлів моделі Spring Boot проєкту

Г.1 Код файлу Entrant.java

```
package ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.entrant;

import jakarta.persistence.*;
import jakarta.validation.constraints.*;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;
import
ntukhpi.csit.semit.riv.webappsrivlab4.validator.common.ValidName;
import
ntukhpi.csit.semit.riv.webappsrivlab4.validator.common.ValidPatron
ymic;
import
ntukhpi.csit.semit.riv.webappsrivlab4.validator.common.ValidSurnam
e;
import
ntukhpi.csit.semit.riv.webappsrivlab4.validator.validator.Validati
onConstants;
import
ntukhpi.csit.semit.riv.webappsrivlab4.validator.entrant.ValidBirth
day;
import
ntukhpi.csit.semit.riv.webappsrivlab4.validator.entrant.ValidCaseN
umber;
import
ntukhpi.csit.semit.riv.webappsrivlab4.validator.entrant.ValidGende
r;
```



```

import
ntukhpi.csit.semit.riv.webappsrivlab4.validator.entrant.ValidRatingScore;
import org.hibernate.annotations.Check;
import org.hibernate.annotations.ColumnDefault;
import org.hibernate.validator.constraints.Length;
import org.springframework.format.annotation.DateTimeFormat;
import org.springframework.format.annotation.NumberFormat;

import java.time.LocalDate;
import java.time.format.DateTimeParseException;
import java.util.Arrays;
import java.util.List;
import java.util.regex.Pattern;

/**
 * Entity class representing an entrant in the system.
 * This class is mapped to the `entrants` table in the database and
 * provides
 * attributes and validation for managing entrant-related data.
 * <p>
 * Key attributes:
 * - `id`: The primary key for the entrant entity.
 * - `caseNumber`: A unique identifier for each entrant, validated
 * with specific regex patterns.
 * - `surname`, `name`, `patronymic`: Personal details of the
 * entrant with length and pattern validations.
 * - `birthday`: The date of birth, constrained to a specific range
 * (1914-01-01 to 2008-01-01).
 * - `gender`: A boolean field representing the gender of the
 * entrant.
 * - `ratingScore`: The entrant's rating score, constrained to be
 * within the range of 120.000 to 200.000.
 * - `studentId`: Optional field linking the entrant to a student
 * entity.
 * <p>

```

```

* Additional features:
* - Custom constructors for initializing entrant data with string
values.
* - Validation annotations ensure data integrity at the
application and database levels.
* - Utility methods, such as `getFieldNamesAsFormattedStrings`,
provide metadata for UI or other purposes.
* <p>
* This entity integrates with Hibernate and Spring Validation to
ensure robust data handling.
*
* @author Inessa Repeshko CS-222a
* @see LocalDate
* @see Arrays
* @see List
* @see ValidationConstants
*/

@Entity
@Table(name = "entrants")
@Data
@AllArgsConstructor
@NoArgsConstructor
public class Entrant {
    @Column(name = "id", insertable = false, updatable = false)
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Positive(message = "The id must be a positive number.")
    private Long id;

    @Column(name = "case_number", unique = true, nullable = false,
length = 20)
    @Check(constraints = "REGEXP_LIKE(case_number,
'^[A-ЩФЮЯҐЄІİ]{1,10}[0-9]{2}-[0-9]{1,4}$', 'c') = 1")
    @Length(min = 5, max = 20, message = "The case number length
should be between 5 to 20 characters.")

```

```

@ValidCaseNumber
private String caseNumber;

@Column(name = "surname", nullable = false, length = 50)
@Check(constraints = "REGEXP_LIKE(surname, '" +
ValidationConstants.REGEX_NAMES + "', 'c') = 1")
@Length(min = 1, max = 50, message = "The surname length should
be between 1 to 50 characters.")
@ValidSurname
private String surname;

@Column(name = "name", nullable = false, length = 50)
@Check(constraints = "REGEXP_LIKE(name, '" +
ValidationConstants.REGEX_NAMES + "', 'c') = 1")
@Length(min = 1, max = 50, message = "The name length should be
between 1 to 50 characters.")
@ValidName
private String name;

@Column(name = "patronymic", nullable = false, length = 50)
@Check(constraints = "REGEXP_LIKE(patronymic, '" +
ValidationConstants.REGEX_NAMES + "', 'c') = 1")
@Length(min = 1, max = 50, message = "The patronymic length
should be between 1 to 50 characters.")
@ValidPatronymic
private String patronymic;

@Column(name = "birthday", nullable = false)
@ColumnDefault(value = "'2000-01-01'")
@Check(constraints = "birthday BETWEEN '1914-01-01' AND
'2008-01-01'")
@NotNull(message = "The birthday must not be null.")
@ValidBirthday
@DateTimeFormat(iso = DateTimeFormat.ISO.DATE)
private LocalDate birthday;

```

```

@Column(name = "gender", nullable = false)
@ColumnDefault(value = "1")
@ValidGender
private Boolean gender;

@Column(name = "rating_score", nullable = false)
@ColumnDefault(value = "120.001")
@Check(constraints = "rating_score > 120.000 AND rating_score
<= 200.000")
@NotNull(message = "The rating score must not be null.")
@ValidRatingScore
@NumberFormat(pattern = "###.###")
private Double ratingScore;

@Column(name = "student_id", nullable = true)
@Positive(message = "The student id must be a positive
number.")
private Long studentId;

public Entrant(@NotNull String caseNumber,
               @NotNull String surname,
               @NotNull String name,
               @NotNull String patronymic,
               @NotNull String birthday,
               @NotNull String gender,
               @NotNull String ratingScore) {
    setFieldsFromStringValues(caseNumber, surname, name,
patronymic, birthday, gender, ratingScore);
}

public Entrant(@NotNull String id,
               @NotNull String caseNumber,
               @NotNull String surname,
               @NotNull String name,
               @NotNull String patronymic,
               @NotNull String birthday,

```

```

        @NotNull String gender,
        @NotNull String ratingScore) {

    try {
        setId(Long.parseLong(id));
    } catch (NumberFormatException e) {
        throw new IllegalArgumentException("The id must be a
numeric value greater than 0.");
    }

    setFieldsFromStringValues(caseNumber, surname, name,
patronymic, birthday, gender, ratingScore);
}

    private void setFieldsFromStringValues(@NotNull String
caseNumber,

                                           @NotNull String surname,
                                           @NotNull String name,
                                           @NotNull String
patronymic,

                                           @NotNull String
birthday,

                                           @NotNull String gender,
                                           @NotNull String
ratingScore) {
        setCaseNumber(caseNumber);
        setSurname(surname);
        setName(name);
        setPatronymic(patronymic);

        try {
            setBirthday(LocalDate.parse(birthday,
ValidationConstants.DATE_FORMATTER_EN));
        } catch (DateTimeParseException e) {
            throw new IllegalArgumentException("The birthday should
be in the format 'yyyy-MM-dd'.");
        }
    }
}

```

```

        if
(gender.trim().matches(Pattern.compile(ValidationConstants.REGEX_G
ENDER).pattern())) {
            setGender(Boolean.parseBoolean(gender));
        } else {
            throw new IllegalArgumentException("The gender must be
boolean value in numeric or string format");
        }

        try {
            setRatingScore(Double.parseDouble(ratingScore));
        } catch (NumberFormatException e) {
            throw new IllegalArgumentException("The rating score
must be a numeric value.");
        }
    }

    public static String[][] getFieldNamesAsFormattedStrings() {
        List<String[]> fieldNames = Arrays.asList(
            new String[]{"caseNumber", "Case Number"},
            new String[]{"fullName", "Full Name"},
            new String[]{"birthday", "Birthday"},
            new String[]{"gender", "Gender"},
            new String[]{"ratingScore", "Rating Score"}
        );

        return fieldNames.toArray(String[][]::new);
    }
}

```

Г.2 Код файла EntrantList.java

```
package
ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.testDataList.entra
nt;

import
ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.entrant.Entrant;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

/**
 * Singleton class representing a list of predefined entrant test
 * data.
 * This class provides a collection of `Entrant` objects for
 * testing and demonstration purposes.
 * The list is initialized with sample data, which can be
 * manipulated (added, removed, or cleared)
 * as needed.
 * <p>
 * Key functionalities:
 * - Provides access to a predefined list of `Entrant` objects.
 * - Allows adding, removing, and clearing entrants dynamically.
 * - Ensures that the list is shared across the application using
 * the singleton pattern.
 * <p>
 * Sample data includes various combinations of entrant attributes
 * such as case number, name,
 * birthday, gender, and rating score. This facilitates testing of
 * functionalities dependent on
 * the `Entrant` entity.
 * <p>
```

```

* The singleton pattern is implemented using an enum (`INSTANCE`)
to ensure thread safety
* and prevent multiple instantiations.
*
* @author Inessa Repeshko CS-222a
*/

public enum EntrantList {
    INSTANCE;

    private final List<Entrant> entrants;

    EntrantList() {
        this.entrants = new ArrayList<>();

        List<String[]> entrantData = Arrays.asList(
            new String[]{"KH22-4814", "Бабічев", "Олексій",
"Вікторович", "2005-05-22", "TRUE", "168.111"},
            new String[]{"KH22-5835", "Волков", "Дмитро",
"Олександрович", "2005-07-16", "TRUE", "151.202"},
            new String[]{"KH22-5511", "Геєнко", "Анна",
"Олексіївна", "2005-12-03", "FALSE", "150.013"},
            new String[]{"KH22-9299", "Голоп'яров", "Микита",
"Сергійович", "2005-11-07", "TRUE", "161.484"},
            new String[]{"KH22-9012", "Гриценко", "Максим",
"Георгійович", "2005-12-04", "TRUE", "191.945"},
            new String[]{"KH22-6636", "Жарий", "Вадим",
"Віталійович", "2005-10-25", "TRUE", "128.346"},
            new String[]{"KH22-5140", "Жупанов", "Андрій",
"Володимирович", "2005-06-22", "TRUE", "183.627"},
            new String[]{"KH22-4896", "Кльоз", "Михайло",
"Романович", "2005-01-15", "TRUE", "127.288"},
            new String[]{"KH22-4694", "Коваль", "Микита",
"Артемович", "2005-10-11", "TRUE", "139.739"},

```



```

        new String[]{"KH22-5529", "Ковтун", "Єгор",
"Вячеславович", "2005-04-10", "TRUE", "187.790"},
        new String[]{"KH22-6874", "Колесниченко", "Денис",
"Євгенович", "2005-03-26", "TRUE", "165.641"},
        new String[]{"KH22-2947", "Ласкевич", "Андрій",
"Сергійович", "2005-09-05", "TRUE", "175.392"},
        new String[]{"KH22-3856", "Лукаш", "Антон",
"Олександрович", "2005-11-24", "TRUE", "184.473"},
        new String[]{"KH22-7534", "Палій", "Нікіта",
"Андрійович", "2005-05-05", "TRUE", "168.004"},
        new String[]{"KH22-3843", "Прокопов", "Владислав",
"Сергійович", "2004-12-12", "TRUE", "164.665"},
        new String[]{"KH22-7819", "Репешко", "Інесса",
"Віталіївна", "2005-05-06", "FALSE", "185.306"},
        new String[]{"KH22-9115", "Скиба", "Андрій",
"Григорович", "2004-02-26", "TRUE", "125.907"},
        new String[]{"KH22-3812", "Цибань", "Ольга",
"Юріївна", "2004-05-19", "FALSE", "190.298"},
        new String[]{"KH22-1828", "Шевченко", "Данило",
"Валерійович", "2004-10-25", "TRUE", "166.849"},
        new String[]{"KH22-9514", "Антипенко", "Катерина",
"Віталіївна", "2005-01-06", "FALSE", "168.460"},
        new String[]{"KH22-2929", "Багрянцева", "Марина",
"Русланівна", "2005-10-01", "FALSE", "157.631"},
        new String[]{"KH22-1637", "Бондаренко", "Ярослав",
"Юрійович", "2005-04-24", "TRUE", "199.532"},
        new String[]{"KH22-6681", "Булгаков", "Віктор",
"Олександрович", "2005-05-17", "TRUE", "153.683"},
        new String[]{"KH22-7557", "Васильєв", "Михайло",
"Олександрович", "2005-01-25", "TRUE", "170.864"},
        new String[]{"KH22-8759", "Деркач", "Дарина",
"Євгенівна", "2005-03-03", "FALSE", "171.715"},
        new String[]{"KH22-7309", "Ілюхін", "Тимур",
"Ігорович", "2005-01-24", "TRUE", "125.776"},
        new String[]{"KH22-8254", "Криженко", "Аліна",
"Степанівна", "2005-08-29", "FALSE", "138.747"},

```

```

        new String[]{"KH22-3825", "Посмашний", "Ярослав",
"Дмитрович", "2000-02-25", "TRUE", "160.718"},
        new String[]{"KH22-8153", "Рожкова", "Вікторія",
"Андріївна", "1993-10-04", "FALSE", "188.309"},
        new String[]{"KH22-4788", "Сизоненко", "Єлизавета",
"Олександрівна", "1997-02-17", "FALSE", "124.240"},
        new String[]{"KH22-8255", "Криженко", "Аліна",
"Степанівна", "2005-08-29", "FALSE", "138.747"},
        new String[]{"KH22-3826", "Посмашний", "Ярослав",
"Дмитрович", "2000-02-25", "TRUE", "160.718"},
        new String[]{"KH22-8154", "Рожкова", "Вікторія",
"Андріївна", "1993-10-04", "FALSE", "188.309"},
        new String[]{"KH22-6682", "Булгаков", "Віктор",
"Олександрович", "2005-05-17", "TRUE", "153.683"}
    );

    entrantData.forEach(
        data -> addEntrant(new Entrant(
            data[0],
            data[1],
            data[2],
            data[3],
            data[4],
            data[5],
            data[6])));

    addEntrant(new Entrant(
        "KH22-9998",
        "Марчук",
        "Регіна",
        "Віталіївна",
        "1996-06-06",
        "FALSE",
        "200"));

    /*addEntrant(new Entrant(

```

```

        "АВВГГІІ99-0000",
        "Микол-Іванович Петренко-О'Браєн",
        "Іван Петро",
        "Петрович Іванович-О'Браєн",
        "1996-06-06",
        "TRUE",
        "200"));*/

    }

    public List<Entrant> getEntrants() {
        return new ArrayList<>(entrants);
    }

    public void addEntrant(Entrant entrant) {
        entrants.add(entrant);
    }

    public void removeEntrant(Entrant entrant) {
        entrants.remove(entrant);
    }

    public void clearEntrants() {
        entrants.clear();
    }
}

```

Г.3 Код файлу FundingType.java

```

package ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.enums;

import lombok.Getter;

import java.util.Arrays;
import java.util.List;

```

```

/**
 * Enum class representing the funding type for students.
 * This enum defines two funding types:
 * - CONTRACT: Represents students on a contract basis
 * ("контракт").
 * - BUDGET: Represents students funded by the government budget
 * ("бюджет").
 * <p>
 * Key features:
 * - `getValue()`: Provides the Ukrainian human-readable value of
 * the funding type.
 * - `getValues()`: Returns a list of all funding types available
 * in the system.
 * - `fromValue(String value)`: Converts a string to the
 * corresponding `FundingType` constant.
 * Throws an `IllegalArgumentException` if the value does not match
 * any funding type.
 * <p>
 * Lombok's `@Getter` annotation is used to generate the getter for
 * the `value` field automatically.
 * This enum ensures consistent handling of funding types
 * throughout the application.
 *
 * @author Inessa Repeshko CS-222a
 * @see ScholarshipStatus
 * @see Enum
 * @see List
 * @see Arrays
 * @see IllegalArgumentException
 */

@Getter
public enum FundingType {
    CONTRACT("контракт"),
    BUDGET("бюджет");

```

```

private final String value;

FundingType(String value) {
    this.value = value;
}

public static List<FundingType> getValues() {
    return Arrays.stream(FundingType.values()).toList();
}

public static FundingType fromValue(String value) {
    for (FundingType type : FundingType.values()) {
        if (type.getValue().equals(value)) {
            return type;
        }
    }

    throw new IllegalArgumentException("Unknown FundingType
value: " + value);
}
}

```

Г.4 Код файла FundingTypeConverter.java

```

package ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.enums;

import jakarta.persistence.AttributeConverter;
import jakarta.persistence.Converter;

/**
 * JPA AttributeConverter for the `FundingType` enum.
 * This class provides the logic for converting between the
 * `FundingType` enum and its
 * corresponding database representation (a `String`).
 * <p>

```

```

* Key functionalities:
* - `convertToDatabaseColumn(FundingType fundingType)`: Converts a
`FundingType` enum value to its
* corresponding `String` representation for storage in the
database.
* - `convertToEntityAttribute(String dbData)`: Converts a `String`
value from the database back to
* the corresponding `FundingType` enum constant. Throws an
`IllegalArgumentException` if the
* string value does not match any valid funding type.
* <p>
* This converter is automatically applied to all `FundingType`
fields in entities due to
* the `@Converter(autoApply = true)` annotation.
* <p>
* Ensures that the `FundingType` enum is seamlessly persisted and
retrieved in JPA entities,
* maintaining data integrity and simplifying application logic.
*
* @author Inessa Repeshko CS-222a
* @see FundingType
* @see jakarta.persistence.AttributeConverter
* @see IllegalArgumentException
*/

@Converter(autoApply = true)
public class FundingTypeConverter implements
AttributeConverter<FundingType, String> {

    @Override
    public String convertToDatabaseColumn(FundingType fundingType)
{
        if (fundingType == null) {
            return null;
        }

```

```

        return fundingType.getValue();
    }

    @Override
    public FundingType convertToEntityAttribute(String dbData) {
        if (dbData == null || dbData.isEmpty()) {
            return null;
        }

        for (FundingType type : FundingType.values()) {
            if (type.getValue().equals(dbData)) {
                return type;
            }
        }

        throw new IllegalArgumentException("Unknown FundingType
value: " + dbData);
    }
}

```

Г.5 Код файла ScholarshipStatus.java

```

package ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.enums;

import lombok.Getter;

import java.util.Arrays;
import java.util.List;

/**
 * Enum class representing the scholarship status for students.
 * This enum defines the possible scholarship statuses in the
 * system:
 * - ENHANCED: Represents an enhanced scholarship ("підвищена").
 * - ORDINARY: Represents an ordinary scholarship ("звичайна").

```

```

* - NONE: Indicates the absence of a scholarship ("відсутня").
* <p>
* Key functionalities:
* - `getValues()`: Returns a list of all scholarship status
constants.
* - `fromValue(String value)`: Converts a string value to the
corresponding `ScholarshipStatus` constant.
* Throws an `IllegalArgumentException` if the value does not match
any defined status.
* <p>
* Lombok's `@Getter` annotation is used to automatically generate
a getter for the `value` field.
* This enum ensures consistent handling of scholarship statuses
across the application.
*
* @author Inessa Repeshko CS-222a
* @see FundingType
* @see Enum
* @see List
* @see Arrays
* @see IllegalArgumentException
*/

@Getter
public enum ScholarshipStatus {
    ENHANCED("підвищена"),
    ORDINARY("звичайна"),
    NONE("відсутня");

    private final String value;

    ScholarshipStatus(String value) {
        this.value = value;
    }

    public static List<ScholarshipStatus> getValues() {

```



```

        return Arrays.stream(ScholarshipStatus.values()).toList();
    }

    public static ScholarshipStatus fromValue(String value) {
        for (ScholarshipStatus status : ScholarshipStatus.values())
        {
            if (status.getValue().equals(value)) {
                return status;
            }
        }

        throw new IllegalArgumentException("Unknown
ScholarshipStatus value: " + value);
    }
}

```

Г.6 Код файла ScholarshipStatusConverter.java

```

package ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.enums;

import jakarta.persistence.AttributeConverter;
import jakarta.persistence.Converter;

/**
 * JPA AttributeConverter for the `ScholarshipStatus` enum.
 * This class provides the logic for converting between the
 * `ScholarshipStatus` enum
 * and its corresponding database representation (a `String`).
 * <p>
 * Key functionalities:
 * - `convertToDatabaseColumn(ScholarshipStatus
scholarshipStatus)` : Converts a `ScholarshipStatus`
 * enum value to its corresponding `String` representation for
storage in the database.

```

```

* - `convertToEntityAttribute(String dbData)`: Converts a `String`
value from the database back to
* the corresponding `ScholarshipStatus` enum constant. Throws an
`IllegalArgumentException` if the
* string value does not match any defined scholarship status.
* <p>
* This converter is automatically applied to all
`ScholarshipStatus` fields in entities due to
* the `@Converter(autoApply = true)` annotation.
* <p>
* Ensures that the `ScholarshipStatus` enum is seamlessly
persisted and retrieved in JPA entities,
* maintaining data integrity and simplifying application logic.
*
* @author Inessa Repeshko CS-222a
* @see ScholarshipStatus
* @see jakarta.persistence.AttributeConverter
* @see IllegalArgumentException
*/

@Converter(autoApply = true)
public class ScholarshipStatusConverter implements
AttributeConverter<ScholarshipStatus, String> {

    @Override
    public String convertToDatabaseColumn(ScholarshipStatus
scholarshipStatus) {
        if (scholarshipStatus == null) {
            return null;
        }

        return scholarshipStatus.getValue();
    }

    @Override

```

```

    public ScholarshipStatus convertToEntityAttribute(String
dbData) {
        if (dbData == null || dbData.isEmpty()) {
            return null;
        }

        for (ScholarshipStatus status : ScholarshipStatus.values())
        {
            if (status.getValue().equals(dbData)) {
                return status;
            }
        }

        throw new IllegalArgumentException("Unknown
ScholarshipStatus value: " + dbData);
    }
}

```

Г.7 Код файла Student.java

```

package ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.student;

import jakarta.persistence.*;
import jakarta.validation.Valid;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;
import
ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.entrant.Entrant;
import
ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.enums.FundingType;
import
ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.enums.FundingTypeC
onverter;

```

```

import
ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.enums.ScholarshipsS
tatus;
import
ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.enums.ScholarshipsS
tatusConverter;
import
ntukhpi.csit.semit.riv.webappsrivlab4.validator.student.ValidFundi
ngType;
import
ntukhpi.csit.semit.riv.webappsrivlab4.validator.student.ValidSchol
arshipStatus;
import
ntukhpi.csit.semit.riv.webappsrivlab4.validator.validator.Validati
onConstants;
import
ntukhpi.csit.semit.riv.webappsrivlab4.validator.common.ValidCorpor
ateEmail;
import org.hibernate.annotations.Check;
import org.hibernate.annotations.ColumnDefault;
import org.hibernate.validator.constraints.Length;

import java.util.Arrays;
import java.util.List;

/**
 * Entity class representing a student in the system.
 * This class is mapped to the `students` table in the database and
 * provides attributes
 * and validation for managing student-related data.
 * <p>
 * Key attributes:
 * - `id`: The primary key for the student entity.
 * - `entrant`: A one-to-one relationship with the `Entrant`
 * entity, with cascading behavior

```

```

* for refresh and removal operations. Ensures data consistency
through foreign key constraints.
* - `fundingType`: Indicates the funding type (e.g., CONTRACT or
BUDGET) with validation and
* persistence using the `FundingTypeConverter`.
* - `scholarshipStatus`: Specifies the scholarship status (e.g.,
ENHANCED, ORDINARY, or NONE),
* with validation and persistence using the
`ScholarshipStatusConverter`.
* - `corporateEmail`: The student's unique corporate email
address, validated against
* a specific format and length constraints.
* <p>
* Additional features:
* - Custom constructor for initializing a `Student` object with
string-based attributes.
* - Validation annotations to ensure data integrity at both
application and database levels.
* - Utility method `getFieldNamesAsFormattedStrings` provides
metadata for UI display purposes.
* <p>
* Relationships, constraints, and validations ensure seamless
integration with Hibernate
* and Spring Validation frameworks, maintaining robust and
consistent data handling.
*
* @author Inessa Repeshko CS-222a
*/

@Entity
@Table(name = "students")
@Data
@AllArgsConstructor
@NoArgsConstructor
public class Student {

```

```

@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
@Column(name = "id")
private Long id;

@OneToOne(targetEntity = Entrant.class, cascade =
{CascadeType.REFRESH, CascadeType.REMOVE})
@JoinColumn(name = "entrant_id", referencedColumnName = "id",
    foreignKey = @ForeignKey(
        name = "fk_student_entrant",
        foreignKeyDefinition = "FOREIGN KEY
(entrant_id) REFERENCES entrants(id) ON DELETE CASCADE ON UPDATE
CASCADE"))
@Valid
private Entrant entrant;

@Column(name = "funding_type", nullable = false)
@Enumerated(EnumType.STRING)
@Convert(converter = FundingTypeConverter.class)
@ColumnDefault(value = "'CONTRACT'")
@ValidFundingType
private FundingType fundingType;

@Column(name = "scholarship_status", nullable = false)
@Enumerated(EnumType.STRING)
@Convert(converter = ScholarshipStatusConverter.class)
@ColumnDefault(value = "'NONE'")
@ValidScholarshipStatus
private ScholarshipStatus scholarshipStatus;

@Column(name = "corporate_email", nullable = false, unique =
true, length = 123)
@Check(constraints = "REGEXP_LIKE(corporate_email, '" +
ValidationConstants.REGEX_CORPORATE_EMAIL + "', 'c') = 1")
@Length(min = 17, max = 123, message = "The corporate email
length should be between 17 to 123 characters.")

```

```

        @org.hibernate.validator.constraints.Email(regexp =
ValidationConstants.REGEX_CORPORATE_EMAIL,
            message = "The corporate email should match the format
'name.surname@faculty.khpi.edu.ua'.")
        @ValidCorporateEmail
        private String corporateEmail;

        public Student(Entrant entrant,
                        String fundingType,
                        String scholarshipStatus,
                        String corporateEmail) {
            setEntrant(entrant);
            setFundingType(FundingType.fromValue(fundingType));
            setScholarshipStatus(ScholarshipStatus.fromValue(scholarshipStatus
));
            setCorporateEmail(corporateEmail);
        }

        public static String[][] getFieldNamesAsFormattedStrings() {
            List<String[]> fieldNames = Arrays.asList(
                new String[]{"fullName", "Full Name"},
                new String[]{"fundingType", "Funding Type"},
                new String[]{"scholarshipStatus", "Scholarship
Status"},
                new String[]{"corporateEmail", "Corporate Email"}
            );

            return fieldNames.toArray(String[][]::new);
        }
    }
}

```

Г.8 Код файла StudentList.java

```
package
ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.testDataList.stude
nt;

import
ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.entrant.Entrant;
import
ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.student.Student;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

/**
 * Singleton class representing a list of predefined student test
 * data.
 * This class provides a collection of `Student` objects for
 * testing and demonstration purposes.
 * The list is initialized with sample data, which can be
 * dynamically manipulated (added, removed, or cleared).
 * <p>
 * Key functionalities:
 * - Provides access to a predefined list of `Student` objects with
 * associated `Entrant` data.
 * - Allows adding, removing, and clearing students dynamically.
 * - Ensures a single instance of the list is shared across the
 * application using the singleton pattern.
 * <p>
 * Sample data includes various combinations of student attributes
 * such as funding type, scholarship status,
 * corporate email, and linked entrant details. This facilitates
 * testing functionalities that depend on the
 * `Student` entity.
```



```

* <p>
* The singleton pattern is implemented using an enum (`INSTANCE`)
to ensure thread safety
* and prevent multiple instantiations.
*
* @author Inessa Repeshko CS-222a
* @see Student
* @see Entrant
* @see ArrayList
* @see List
* @see Enum
*/

public enum StudentList {
    INSTANCE;

    private final List<Student> students;

    StudentList() {
        this.students = new ArrayList<>();

        List<String[]> studentData = Arrays.asList(
            new String[]{"KH22-4814", "Бабічев", "Олексій",
"Вікторович", "2005-05-22", "TRUE", "168.111", "бюджет",
"звичайна", "oleksii.babichiev@cs.khpi.edu.ua"},
            new String[]{"KH22-5835", "Волков", "Дмитро",
"Олександрович", "2005-07-16", "TRUE", "151.202", "бюджет",
"відсутня", "dmytro.o.volkov@cs.khpi.edu.ua"},
            new String[]{"KH22-5511", "Геєнко", "Анна",
"Олексіївна", "2005-12-03", "FALSE", "150.013", "бюджет",
"відсутня", "anna.hieienko@cs.khpi.edu.ua"},
            new String[]{"KH22-9299", "Голоп'яров", "Микита",
"Сергійович", "2005-11-07", "TRUE", "161.484", "бюджет",
"звичайна", "mykyta.holoporov@cs.khpi.edu.ua"},
            new String[]{"KH22-9012", "Гриценко", "Максим",
"Георгійович", "2005-12-04", "TRUE", "191.945", "бюджет",
"підвищена", "maksym.hrytsenko@cs.khpi.edu.ua"},

```

```

        new String[]{"KH22-6636", "Жарий", "Вадим",
"Віталійович", "2005-10-25", "TRUE", "128.346", "контракт",
"відсутня", "vadym.zharyi@cs.khpi.edu.ua"},

        new String[]{"KH22-5140", "Жупанов", "Андрій",
"Володимирович", "2005-06-22", "TRUE", "183.627", "бюджет",
"звичайна", "andrii.zhupanov@cs.khpi.edu.ua"},

        new String[]{"KH22-4896", "Кльоз", "Михайло",
"Романович", "2005-01-15", "TRUE", "127.288", "контракт",
"відсутня", "mykhailo.kloz@cs.khpi.edu.ua"},

        new String[]{"KH22-4694", "Коваль", "Микита",
"Артемович", "2005-10-11", "TRUE", "139.739", "бюджет",
"відсутня", "mykyta.koval@cs.khpi.edu.ua"},

        new String[]{"KH22-5529", "Ковтун", "Єгор",
"Вячеславович", "2005-04-10", "TRUE", "187.790", "бюджет",
"підвищена", "yehor.kovtun@cs.khpi.edu.ua"},

        new String[]{"KH22-6874", "Колесниченко", "Денис",
"Євгенович", "2005-03-26", "TRUE", "165.641", "бюджет",
"звичайна", "denys.kolesnychenko@cs.khpi.edu.ua"},

        new String[]{"KH22-2947", "Ласкевич", "Андрій",
"Сергійович", "2005-09-05", "TRUE", "175.392", "бюджет",
"звичайна", "andrii.laskevych@cs.khpi.edu.ua"},

        new String[]{"KH22-3856", "Лукаш", "Антон",
"Олександрович", "2005-11-24", "TRUE", "184.473", "бюджет",
"звичайна", "anton.lukash@cs.khpi.edu.ua"},

        new String[]{"KH22-7534", "Палій", "Нікіта",
"Андрійович", "2005-05-05", "TRUE", "168.004", "бюджет",
"відсутня", "nikita.palii@cs.khpi.edu.ua"},

        new String[]{"KH22-3843", "Прокопов", "Владислав",
"Сергійович", "2004-12-12", "TRUE", "164.665", "бюджет",
"звичайна", "vladyslav.prokopov@cs.khpi.edu.ua"},

        new String[]{"KH22-7819", "Репешко", "Інесса",
"Віталіївна", "2005-05-06", "FALSE", "185.306", "бюджет",
"підвищена", "inessa.repeshko@cs.khpi.edu.ua"},

        new String[]{"KH22-9115", "Скиба", "Андрій",
"Григорович", "2004-02-26", "TRUE", "125.907", "контракт",
"відсутня", "andrii.h.skyba@cit.khpi.edu.ua"},

```

```

        new String[]{"KH22-3812", "Цибань", "Ольга",
"Юріївна", "2004-05-19", "FALSE", "190.298", "бюджет", "відсутня",
"olha.tsyban@cs.khpi.edu.ua"},

        new String[]{"KH22-1828", "Шевченко", "Данило",
"Валерійович", "2004-10-25", "TRUE", "166.849", "бюджет",
"звичайна", "danylo.v.shevchenko@cs.khpi.edu.ua"},

        new String[]{"KH22-9514", "Антипенко", "Катерина",
"Віталіївна", "2005-01-06", "FALSE", "168.460", "бюджет",
"звичайна", "kateryna.antypenko@cs.khpi.edu.ua"},

        new String[]{"KH22-2929", "Багрянцева", "Марина",
"Русланівна", "2005-10-01", "FALSE", "157.631", "бюджет",
"звичайна", "marina.bahriantseva@cs.khpi.edu.ua"},

        new String[]{"KH22-1637", "Бондаренко", "Ярослав",
"Юрійович", "2005-04-24", "TRUE", "199.532", "бюджет",
"підвищена", "yaroslav.y.bondarenko@cs.khpi.edu.ua"},

        new String[]{"KH22-6681", "Булгаков", "Віктор",
"Олександрович", "2005-05-17", "TRUE", "153.683", "бюджет",
"відсутня", "viktor.bulhakov@cs.khpi.edu.ua"},

        new String[]{"KH22-7557", "Васильєв", "Михайло",
"Олександрович", "2005-01-25", "TRUE", "170.864", "бюджет",
"відсутня", "mykhailo.vasyliiev@cs.khpi.edu.ua"},

        new String[]{"KH22-8759", "Деркач", "Дарина",
"Євгенівна", "2005-03-03", "FALSE", "171.715", "бюджет",
"звичайна", "daryna.derkach@cs.khpi.edu.ua"},

        new String[]{"KH22-7309", "Ілюхін", "Тимур",
"Ігорович", "2005-01-24", "TRUE", "125.776", "контракт",
"відсутня", "tymur.iliukhin@cs.khpi.edu.ua"},

        new String[]{"KH22-8254", "Криженко", "Аліна",
"Степанівна", "2005-08-29", "FALSE", "138.747", "бюджет",
"відсутня", "alina.kryzhenko@cs.khpi.edu.ua"},

        new String[]{"KH22-3825", "Посмашний", "Ярослав",
"Дмитрович", "2000-02-25", "TRUE", "160.718", "бюджет",
"звичайна", "yaroslav.posmashnyi@cs.khpi.edu.ua"},

        new String[]{"KH22-8153", "Рожкова", "Вікторія",
"Андріївна", "1993-10-04", "FALSE", "188.309", "бюджет",
"підвищена", "viktoriia.rozhkova@cs.khpi.edu.ua"},

```

```

        new String[]{"KH22-4788", "Сизоненко", "Єлизавета",
"Олександрівна", "1997-02-17", "FALSE", "124.240", "контракт",
"відсутня", "yelyzaveta.syzonenko@cs.khpi.edu.ua"}
    );

    studentData.forEach(
        data -> addStudent(new Student(
            new Entrant(data[0], data[1], data[2],
data[3], data[4], data[5], data[6]),
            data[7], data[8], data[9])));
    }

    public List<Student> getStudents() {
        return new ArrayList<>(students);
    }

    public void addStudent(Student student) {
        students.add(student);
    }

    public void removeStudent(Student student) {
        students.remove(student);
    }

    public void clearStudents() {
        students.clear();
    }
}

```

Г.9 Код файла Role.java

```

package ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.enums;

import lombok.Getter;

```

```

import java.util.Arrays;
import java.util.List;
import java.util.stream.Collectors;

/**
 * Enum class representing roles in the system.
 * This enum defines user roles and their associated human-readable
 * values in Ukrainian.
 * <p>
 * Roles:
 * - ADMIN: Represents the administrator role ("Адміністратор").
 * - ENTRANT_MANAGER: Represents a manager responsible for entrants
 * ("Менеджер абітурієнтів").
 * - STUDENT_MANAGER: Represents a manager responsible for students
 * ("Менеджер студентів").
 * - ENTRANT_VIEWER: Represents a viewer with access to entrant
 * data ("Переглядач абітурієнтів").
 * - STUDENT_VIEWER: Represents a viewer with access to student
 * data ("Переглядач студентів").
 * <p>
 * Key functionalities:
 * - `getFormattedName()`: Converts the enum name into a
 * human-readable format (e.g., "Entrant Manager").
 * - `getAllRoles()`: Returns a list of all `Role` constants.
 * - `getAllFormattedNames()`: Returns a list of all formatted role
 * names.
 * - `getAllValues()`: Returns a list of all role values in
 * Ukrainian.
 * - `fromValue(String value)`: Converts a string value to the
 * corresponding `Role` constant.
 * - `fromName(String name)`: Converts a string name to the
 * corresponding `Role` constant.
 * <p>
 * Lombok's `@Getter` annotation is used to automatically generate
 * getters for the `value` field.

```

```

* This enum ensures consistent handling of user roles across the
application.
*
* @author Inessa Repeshko CS-222a
* @see RoleConverter
* @see java.util.List
* @see java.util.stream.Collectors
*/

@Getter
public enum Role {
    ADMIN("Адміністратор"),
    ENTRANT_MANAGER("Менеджер абітурієнтів"),
    STUDENT_MANAGER("Менеджер студентів"),
    ENTRANT_VIEWER("Переглядач абітурієнтів"),
    STUDENT_VIEWER("Переглядач студентів");

    private final String value;

    Role(String value) {
        this.value = value;
    }

    public String getFormattedName() {
        return Arrays.stream(this.name().split("_"))
            .map(word -> word.charAt(0) +
word.substring(1).toLowerCase())
            .collect(Collectors.joining(" "));
    }

    public static List<Role> getAllRoles() {
        return Arrays.stream(Role.values()).toList();
    }

    public static List<String> getAllFormattedNames() {

```

```

        return
Arrays.stream(Role.values()).map(Role::getFormattedName).toList();
    }

    public static List<String> getAllValues() {
        return
Arrays.stream(Role.values()).map(Role::getValue).toList();
    }

    public static Role fromValue(String value) {
        for (Role role : Role.values()) {
            if (role.getValue().equals(value)) {
                return role;
            }
        }

        throw new IllegalArgumentException("Unknown Role value: " +
value);
    }

    public static Role fromName(String name) {
        for (Role role : Role.values()) {
            if (role.name().equals(name)) {
                return role;
            }
        }

        throw new IllegalArgumentException("Unknown Role name: " +
name);
    }
}

```

Г.10 Код файла RoleConverter.java

```

package ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.enums;

import jakarta.persistence.AttributeConverter;
import jakarta.persistence.Converter;

/**
 * JPA AttributeConverter for the `Role` enum.
 * This class provides the logic for converting between the `Role`
 * enum and its
 * corresponding database representation (a `String`).
 *
 * Key functionalities:
 * - `convertToDatabaseColumn(Role role)` : Converts a `Role` enum
 * value to its
 * corresponding `String` representation for storage in the
 * database.
 * - `convertToEntityAttribute(String dbData)` : Converts a `String`
 * value from the database back to
 * the corresponding `Role` enum constant. Throws an
 * `IllegalArgumentException` if the
 * string value does not match any defined role.
 *
 * This converter is automatically applied to all `Role` fields in
 * entities due to
 * the `@Converter(autoApply = true)` annotation.
 *
 * Ensures that the `Role` enum is seamlessly persisted and
 * retrieved in JPA entities,
 * maintaining data integrity and simplifying application logic.
 *
 * @author Inessa Repeshko CS-222a
 * @see Role
 * @see jakarta.persistence.AttributeConverter

```



```

*/

@Converter(autoApply = true)
public class RoleConverter implements AttributeConverter<Role,
String> {

    @Override
    public String convertToDatabaseColumn(Role role) {
        if (role == null) {
            return null;
        }

        return role.getValue();
    }

    @Override
    public Role convertToEntityAttribute(String dbData) {
        if (dbData == null || dbData.isEmpty()) {
            return null;
        }

        for (Role role : Role.values()) {
            if (role.getValue().equals(dbData)) {
                return role;
            }
        }

        throw new IllegalArgumentException("Unknown Role value: " +
dbData);
    }
}

```

Г.11 Код файла UserEntity.java

```
package ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.user;

import jakarta.persistence.*;
import jakarta.validation.Valid;
import jakarta.validation.constraints.*;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;
import
ntukhpi.csit.semit.riv.webappsrivlab4.model.DTO.user.UserForm;
import
ntukhpi.csit.semit.riv.webappsrivlab4.model.DTO.user.UserRegister;
import
ntukhpi.csit.semit.riv.webappsrivlab4.model.DTO.user.UserTestData;
import ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.enums.Role;
import
ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.enums.RoleConverte
r;
import
ntukhpi.csit.semit.riv.webappsrivlab4.validator.common.ValidName;
import
ntukhpi.csit.semit.riv.webappsrivlab4.validator.common.ValidPatron
ymic;
import
ntukhpi.csit.semit.riv.webappsrivlab4.validator.common.ValidSurnam
e;
import
ntukhpi.csit.semit.riv.webappsrivlab4.validator.user.ValidEncodedP
assword;
import
ntukhpi.csit.semit.riv.webappsrivlab4.validator.user.ValidRole;
```

```

import
ntukhpi.csit.semit.riv.webappsrivlab4.validator.validator.Validati
onConstants;
import
ntukhpi.csit.semit.riv.webappsrivlab4.validator.common.ValidCorpor
ateEmail;
import
ntukhpi.csit.semit.riv.webappsrivlab4.validator.user.ValidUsername
;
import org.hibernate.annotations.Check;
import org.hibernate.annotations.ColumnDefault;
import org.hibernate.validator.constraints.Length;

/**
 * Entity class representing a user in the system.
 * This class maps to the "users" table in the database and
 * contains user-specific information, including
 * authentication and authorization details.
 * <p>
 * Key functionalities:
 * - Represents user attributes such as username, password, role,
 * email, and personal details (surname, name, patronymic).
 * - Provides factory methods to create `UserEntity` instances from
 * DTO objects (`UserForm`, `UserRegister`, and `UserTestData`).
 * - Includes validations for username, email, and personal details
 * to ensure data integrity.
 * <p>
 * Key attributes:
 * - `id`: Unique identifier for the user.
 * - `username`: Unique username for the user.
 * - `password`: Encrypted password for authentication.
 * - `role`: Role of the user, defining access levels.
 * - `corporateEmail`: Corporate email associated with the user.
 * - `surname`, `name`, `patronymic`: Personal details of the user.
 * <p>
 * Relationships:

```

```

* - Utilizes the `Role` enum for role-based access control.
* - Supports integration with DTO classes (`UserForm`,
`UserRegister`, `UserTestData`) for data transfer.
* <p>
* Constraints:
* - Username must be unique and follow a specific pattern.
* - Email must match the corporate email format and be unique.
* - Personal details have length and format constraints.
*
* @author Inessa Repeshko CS-222a
* @see Role
* @see RoleConverter
* @see UserForm
* @see UserRegister
* @see UserTestData
*/

@Entity
@Table(name = "users")
@Data
@AllArgsConstructor
@NoArgsConstructor
public class UserEntity {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "id")
    @Positive(message = "The id must be a positive number.")
    private Long id;

    @Column(name = "username", unique = true, nullable = false,
length = 32)
    @Check(constraints = "REGEXP_LIKE(username, '" +
ValidationConstants.REGEX_USERNAME + "', 'c') = 1")
    @Length(min = 8, max = 32, message = "The username should be
between 8 to 32 characters.")
    @ValidUsername

```

```

private String username;

@Column(name = "password", nullable = true)
private String password;

@Column(name = "role", nullable = false)
@Enumerated(EnumType.STRING)
@Convert(converter = RoleConverter.class)
@ColumnDefault(value = "'STUDENT_VIEWER'")
@ValidRole
private Role role;

@Column(name = "corporate_email", nullable = false, unique =
true, length = 123)
@Check(constraints = "REGEXP_LIKE(corporate_email, '" +
ValidationConstants.REGEX_CORPORATE_EMAIL + "', 'c') = 1")
@Length(min = 17, max = 123, message = "The corporate email
length should be between 1 to 50 characters.")
@org.hibernate.validator.constraints.Email(regex =
ValidationConstants.REGEX_CORPORATE_EMAIL, message = "The
corporate email should match the format
'name.surname@faculty.khpi.edu.ua'.")
@ValidCorporateEmail
private String corporateEmail;

@Column(name = "surname", nullable = false, length = 50)
@Check(constraints = "REGEXP_LIKE(surname, '" +
ValidationConstants.REGEX_NAMES + "', 'c') = 1")
@Length(min = 1, max = 50, message = "The surname length should
be between 1 to 50 characters.")
@ValidSurname
private String surname;

@Column(name = "name", nullable = false, length = 50)
@Check(constraints = "REGEXP_LIKE(name, '" +
ValidationConstants.REGEX_NAMES + "', 'c') = 1")

```

```

    @Length(min = 1, max = 50, message = "The name length should be
between 1 to 50 characters.")
    @ValidName
    private String name;

    @Column(name = "patronymic", nullable = false, length = 50)
    @Check(constraints = "REGEXP_LIKE(patronymic, '" +
ValidationConstants.REGEX_NAMES + "', 'c') = 1")
    @Length(min = 1, max = 50, message = "The patronymic length
should be between 1 to 50 characters.")
    @ValidPatronymic
    private String patronymic;

    public UserEntity(String username, String password, Role role,
String corporateEmail, String surname, String name, String
patronymic) {
        setUsername(username);
        setPassword(password);
        setRole(role);
        setCorporateEmail(corporateEmail);
        setSurname(surname);
        setName(name);
        setPatronymic(patronymic);
    }

    public UserEntity(String username, Role role, String
corporateEmail, String surname, String name, String patronymic) {
        setUsername(username);
        setRole(role);
        setCorporateEmail(corporateEmail);
        setSurname(surname);
        setName(name);
        setPatronymic(patronymic);
    }

```

```

    public static UserEntity getUserEntityFromUserTestData(@Valid
UserTestData userTestData) {
        return new UserEntity(
            null,
            userTestData.getUsername(),
            userTestData.getPassword(),
            userTestData.getRole(),
            userTestData.getCorporateEmail(),
            userTestData.getSurname(),
            userTestData.getName(),
            userTestData.getPatronymic()
        );
    }

    public static UserEntity getUserEntityFromUserRegister(@Valid
UserRegister userRegister) {
        return new UserEntity(
            null,
            userRegister.getUsername(),
            userRegister.getPassword(),
            Role.STUDENT_VIEWER,
            userRegister.getCorporateEmail(),
            userRegister.getSurname(),
            userRegister.getName(),
            userRegister.getPatronymic()
        );
    }

    public static UserEntity getUserEntityFromUserForm(@Valid
UserForm userForm,
@ValidEncodedPassword String password) {
        return new UserEntity(
            userForm.getId(),
            userForm.getUsername(),
            password,

```

```

        userForm.getRole() != null ? userForm.getRole() :
Role.STUDENT_VIEWER,
        userForm.getCorporateEmail(),
        userForm.getSurname(),
        userForm.getName(),
        userForm.getPatronymic()
    );
}
}

```

Г.12 Код файла UserList.java

```

package
ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.testDataList.user;

import
ntukhpi.csit.semit.riv.webappsrivlab4.model.DTO.user.UserTestData;
import ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.enums.Role;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

/**
 * Singleton class representing a list of predefined user test
 * data.
 * This class provides a collection of `UserTestData` objects for
 * testing and demonstration purposes.
 * The list is initialized with sample data, which can be
 * manipulated (added, removed, or cleared) as needed.
 * <p>
 * Key functionalities:
 * - Provides access to a predefined list of `UserTestData`
 * objects.
 * - Supports dynamic operations to add, remove, or clear users.

```



```

* - Ensures a single instance of the list is shared across the
application using the singleton pattern.
* <p>
* Sample data includes users with attributes such as username,
password, role, email, and personal details (surname, name,
patronymic).
* This facilitates testing functionalities dependent on the
`UserTestData` structure and user-related logic.
* <p>
* The singleton pattern is implemented using an enum (`INSTANCE`)
to ensure thread safety and prevent multiple instantiations.
*
* @author Inessa Repeshko CS-222a
* @see UserTestData
* @see Role
* @see java.util.List
* @see java.util.ArrayList
* @see Enum
*/

public enum UserList {
    INSTANCE;

    private final List<UserTestData> users;

    UserList() {
        this.users = new ArrayList<>();

        List<String[]> userData = Arrays.asList(
            new String[]{"shavlii_oleksii", "Admin123!",
Role.ADMIN.name(), "shavlii.oleksii@ntu.khpi.edu.ua", "Шавлій",
"Олексій", "Олександрович"},
            new String[]{"burulka-liudmyla", "$Luda1990",
Role.ENTRANT_MANAGER.name(), "burulka.liudmyla@ntu.khpi.edu.ua",
"Бурулька", "Людмила", "Ярославівна"},

```

```

        new String[]{"voloshkov-tymur", "Tymur9876*_",
Role.STUDENT_MANAGER.name(), "voloshkov.tymur@ntu.khpi.edu.ua",
"Волошков", "Тимур", "Сергійович"},

        new String[]{"kulish.volodymyr", "12Vova_+A",
Role.ENTRANT_VIEWER.name(), "kulish.volodymyr@cs.khpi.edu.ua",
"Куліш", "Володимир", "Анатолійович"},

        new String[]{"stepova.alina", "Alina0987!",
Role.STUDENT_VIEWER.name(), "stepova.alina@cs.khpi.edu.ua",
"Степова", "Аліна", "Трофимівна"},

        new String[]{"ivanenko_anna", "Anna2023@",
Role.ADMIN.name(), "ivanenko.anna@ntu.khpi.edu.ua", "Іваненко",
"Анна", "Петрівна"},

        new String[]{"tarasenko-oleh", "Oleh9876*!",
Role.ENTRANT_MANAGER.name(), "tarasenko.oleh@ntu.khpi.edu.ua",
"Тарасенко", "Олег", "Іванович"},

        new String[]{"melnyk_daryna", "Daryna!555",
Role.STUDENT_MANAGER.name(), "melnyk.daryna@ntu.khpi.edu.ua",
"Мельник", "Дарина", "Анатоліївна"},

        new String[]{"horbunov.vladyslav",
"Vladyslav_2022", Role.ENTRANT_VIEWER.name(),
"horbunov.vladyslav@cs.khpi.edu.ua", "Горбунов", "Владислав",
"Михайлович"},

        new String[]{"romanova-olena", "Olena123#@",
Role.STUDENT_VIEWER.name(), "romanova.olena@cs.khpi.edu.ua",
"Романова", "Олена", "Олексіївна"},

        new String[]{"vasylchenko.mykhailo",
"Mykhailo_1998!", Role.ADMIN.name(),
"vasylchenko.mykhailo@ntu.khpi.edu.ua", "Васильченко", "Михайло",
"Олександрович"},

        new String[]{"shapoval-kseniia", "Kseniia789#",
Role.ENTRANT_MANAGER.name(), "shapoval.kseniia@ntu.khpi.edu.ua",
"Шаповал", "Ксенія", "Василівна"},

        new String[]{"kozak.yevhenii", "Yevhenii@2021",
Role.STUDENT_MANAGER.name(), "kozak.yevhenii@ntu.khpi.edu.ua",
"Козак", "Євгеній", "Олегович"},

```

```

        new String[]{"bondar.inna", "Inna999+*",
Role.ENTRANT_VIEWER.name(), "bondar.inna@cs.khpi.edu.ua",
"Бондар", "Інна", "Ігорівна"},
        new String[]{"kravchenko-artem", "Artem456*#",
Role.STUDENT_VIEWER.name(), "kravchenko.artem@cs.khpi.edu.ua",
"Кравченко", "Артем", "Богданович"}
    );

    userData.forEach(user -> addUser(new UserTestData(user[0],
user[1], Role.fromName(user[2]), user[3], user[4], user[5],
user[6])));
}

public List<UserTestData> getUsers() {
    return new ArrayList<>(users);
}

public void addUser(UserTestData user) {
    users.add(user);
}

public void removeUser(UserTestData user) {
    users.remove(user);
}

public void clearUsers() {
    users.clear();
}
}

```

Г.13 Код файла UserLogin.java

```

package ntukhpi.csit.semit.riv.webappsrivlab4.model.DTO.user;

import lombok.AllArgsConstructor;

```

```

import lombok.Data;
import lombok.NoArgsConstructor;
import
ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.user.UserEntity;
import
ntukhpi.csit.semit.riv.webappsrivlab4.validator.user.ValidPassword
;
import
ntukhpi.csit.semit.riv.webappsrivlab4.validator.user.ValidUsername
OrCorporateEmail;

/**
 * Data Transfer Object (DTO) for user login functionality.
 * This class encapsulates login-related information for
 authentication purposes.
 * <p>
 * Key functionalities:
 * - Represents the data required for user login.
 * - Supports both corporate email and username as valid login
 identifiers.
 * - Ensures secure handling of user passwords with validation.
 * <p>
 * Key attributes:
 * - `corporateEmailOrUsername`: Identifier for login, can be
 either corporate email or username.
 * - `password`: User's password for authentication.
 * <p>
 * Validation:
 * - `@ValidUsernameOrCorporateEmail`: Ensures the input is a valid
 username or corporate email.
 * - `@ValidPassword`: Ensures the password meets defined security
 requirements.
 * <p>
 * Utility Methods:
 * - `getUserLoginFromUserTestData`: Constructs a `UserLogin`
 object from `UserTestData`.

```

```

* - `getUserLoginFromUserEntity`: Constructs a `UserLogin` object
from a `UserEntity`.
* <p>
* Relationships:
* - Tightly integrated with `UserEntity` for mapping user login
information.
* - Supports processes like authentication and user login
validation.
*
* @author Inessa Repeshko CS-222a
* @see UserEntity
* @see UserTestData
* @see ValidUsernameOrCorporateEmail
* @see ValidPassword
*/

@Data
@AllArgsConstructor
@NoArgsConstructor
public class UserLogin {
    @ValidUsernameOrCorporateEmail
    private String corporateEmailOrUsername;

    @ValidPassword
    private String password;

    public static UserLogin
getUserLoginFromUserTestData(UserTestData userTestData) {
        return new UserLogin(
            userTestData.getCorporateEmail() != null ?
userTestData.getCorporateEmail() : userTestData.getUsername(),
            userTestData.getPassword()
        );
    }
}

```

```

    public static UserLogin getUserLoginFromUserEntity(UserEntity
userEntity) {
        return new UserLogin(
            userEntity.getCorporateEmail() != null ?
userEntity.getCorporateEmail() : userEntity.getUsername(),
            userEntity.getPassword()
        );
    }
}

```

Г.14 Код файла UserRegister.java

```

package ntukhpi.csit.semit.riv.webappsrivlab4.model.DTO.user;

import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;
import
ntukhpi.csit.semit.riv.webappsrivlab4.validator.common.ValidCorpor
ateEmail;
import
ntukhpi.csit.semit.riv.webappsrivlab4.validator.common.ValidName;
import
ntukhpi.csit.semit.riv.webappsrivlab4.validator.common.ValidPatron
ymic;
import
ntukhpi.csit.semit.riv.webappsrivlab4.validator.common.ValidSurnam
e;
import
ntukhpi.csit.semit.riv.webappsrivlab4.validator.user.ValidPassword
;
import
ntukhpi.csit.semit.riv.webappsrivlab4.validator.user.ValidUsername
;

```

```

/**
 * Data Transfer Object (DTO) for user registration functionality.
 * This class encapsulates registration-related information for
 * creating new user accounts.
 * <p>
 * Key functionalities:
 * - Represents the data required for user registration.
 * - Ensures validation of user input for secure and consistent
 * registration processes.
 * <p>
 * Key attributes:
 * - `surname`: User's surname with validation for proper format.
 * - `name`: User's first name with validation for proper format.
 * - `patronymic`: User's patronymic with validation for proper
 * format.
 * - `corporateEmail`: User's corporate email, which serves as a
 * unique identifier.
 * - `username`: Username chosen by the user, validated for
 * uniqueness and format.
 * - `password`: User's chosen password, validated for security
 * standards.
 * - `confirmPassword`: Confirmation of the password, must match
 * the main password.
 * <p>
 * Validation:
 * - `@ValidSurname`, `@ValidName`, `@ValidPatronymic`: Validate
 * personal name components.
 * - `@ValidCorporateEmail`: Ensures the email follows a corporate
 * format.
 * - `@ValidUsername`: Validates the username for proper format and
 * length.
 * - `@ValidPassword`: Ensures the password meets defined security
 * requirements.
 * <p>
 * Utility Methods:

```

```

* - `getUserRegisterFromUserTestData`: Constructs a `UserRegister`
object from `UserTestData` for testing or default data setup.
* <p>
* Relationships:
* - Supports user registration by integrating with `UserTestData`
and validation mechanisms.
* - Forms the basis for creating `UserEntity` during user creation
processes.
*
* @author Inessa Repeshko CS-222a
* @see UserTestData
* @see ValidCorporateEmail
* @see ValidPassword
* @see ValidUsername
*/

@Data
@AllArgsConstructor
@NoArgsConstructor
public class UserRegister {
    @ValidSurname
    private String surname;

    @ValidName
    private String name;

    @ValidPatronymic
    private String patronymic;

    @ValidCorporateEmail
    private String corporateEmail;

    @ValidUsername
    private String username;

    @ValidPassword

```



```

private String password;

@ValidPassword
private String confirmPassword;

public static UserRegister
getUserRegisterFromUserTestData (UserTestData userTestData) {
    return new UserRegister(
        userTestData.getSurname(),
        userTestData.getName(),
        userTestData.getPatronymic(),
        userTestData.getCorporateEmail(),
        userTestData.getUsername(),
        userTestData.getPassword(),
        userTestData.getPassword()
    );
}
}

```

Г.15 Код файла UserChangeCredentials.java

```

package ntukhpi.csit.semit.riv.webappsrivlab4.model.DTO.user;

import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;
import
ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.user.UserEntity;
import
ntukhpi.csit.semit.riv.webappsrivlab4.validator.user.ValidPassword
;
import
ntukhpi.csit.semit.riv.webappsrivlab4.validator.user.ValidUsername
;

```

```

/**
 * Data Transfer Object (DTO) for managing user credential changes.
 * This class is used to encapsulate data required for updating
 * user credentials such as username and password.
 * <p>
 * Key functionalities:
 * - Represents a user's new username and password during
 * credential updates.
 * - Provides validation for the username and password fields to
 * ensure security and integrity.
 * - Includes a utility method to create a `UserChangeCredentials`
 * instance from a `UserEntity` object.
 * <p>
 * Key attributes:
 * - `username`: New username for the user.
 * - `password`: New password for the user.
 * - `confirmPassword`: Confirmation of the new password.
 * <p>
 * Validation:
 * - Username must comply with the validation rules defined by
 * `@ValidUsername`.
 * - Password fields (`password`, `confirmPassword`) must comply
 * with `@ValidPassword`.
 * <p>
 * Relationships:
 * - Utilizes the `UserEntity` class for generating instances based
 * on existing user data.
 *
 * @author Inessa Repeshko CS-222a
 * @see UserEntity
 * @see ValidUsername
 * @see ValidPassword
 */

@Data
@AllArgsConstructor

```

```

@NoArgsConstructor
public class UserChangeCredentials {
    @ValidUsername
    private String username;

    @ValidPassword
    private String password;

    @ValidPassword
    private String confirmPassword;

    public static UserChangeCredentials
getUserChangeCredentialsFromUserEntity(UserEntity userEntity) {
        return new UserChangeCredentials(
            userEntity.getUsername(),
            userEntity.getPassword(),
            userEntity.getPassword()
        );
    }
}

```

Г.16 Код файла UserForm.java

```

package ntukhpi.csit.semit.riv.webappsrivlab4.model.DTO.user;

import jakarta.validation.Valid;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;
import
ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.user.UserEntity;
import ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.enums.Role;
import ntukhpi.csit.semit.riv.webappsrivlab4.validator.common.*;
import
ntukhpi.csit.semit.riv.webappsrivlab4.validator.user.ValidRole;

```

```

import
ntukhpi.csit.semit.riv.webappsrivlab4.validator.user.ValidUsername
;

import java.util.Arrays;
import java.util.List;

/**
 * Data Transfer Object (DTO) for handling user data in forms.
 * This class encapsulates user-related information for
 * registration, updates, or display in forms.
 * <p>
 * Key functionalities:
 * - Maps user entity data into a form-friendly structure.
 * - Ensures validation for user attributes like name, username,
 * email, and role.
 * - Provides utility methods for formatting field names and
 * converting entities into DTOs.
 * <p>
 * Key attributes:
 * - `id`: Unique identifier for the user.
 * - `surname`, `name`, `patronymic`: Personal details of the user.
 * - `corporateEmail`: Corporate email associated with the user.
 * - `username`: Username for authentication.
 * - `role`: Role assigned to the user in the system.
 * <p>
 * Validation:
 * - Includes various custom validators like `@ValidSurname`,
 * `@ValidName`, `@ValidUsername`, `@ValidCorporateEmail`, and
 * `@ValidRole` to enforce data integrity.
 * <p>
 * Relationships:
 * - Tightly coupled with `UserEntity` for transformations between
 * entity and DTO.
 * - Integrated into processes like user registration and profile
 * management.

```

```

* <p>
* Utility Methods:
* - `getUserFormFromUserEntity`: Converts a `UserEntity` instance
into a `UserForm`.
* - `getFieldNamesAsFormattedStrings`: Returns a list of formatted
field names for use in UI.
*
* @author Inessa Repeshko CS-222a
* @see UserEntity
* @see Role
* @see ValidSurname
* @see ValidName
* @see ValidUsername
* @see ValidCorporateEmail
* @see ValidRole
*/

@Data
@AllArgsConstructor
@NoArgsConstructor
public class UserForm {
    private Long id;

    @ValidSurname
    private String surname;

    @ValidName
    private String name;

    @ValidPatronymic
    private String patronymic;

    @ValidCorporateEmail
    private String corporateEmail;

    @ValidUsername

```

```

private String username;

@ValidRole
private Role role;

public static UserForm getUserFormFromUserEntity(@Valid
UserEntity userEntity) {
    return new UserForm(
        userEntity.getId(),
        userEntity.getSurname(),
        userEntity.getName(),
        userEntity.getPatronymic(),
        userEntity.getCorporateEmail(),
        userEntity.getUsername(),
        userEntity.getRole() != null ? userEntity.getRole()
: Role.STUDENT_VIEWER
    );
}

public static String[][] getFieldNamesAsFormattedStrings() {
    List<String[]> fieldNames = Arrays.asList(
        new String[]{"fullName", "Full Name"},
        new String[]{"role", "Role"},
        new String[]{"username", "Username"},
        new String[]{"corporateEmail", "Corporate Email"}
    );

    return fieldNames.toArray(String[][]::new);
}
}

```

Г.17 Код файла UserTestData.java

```

package ntukhpi.csit.semit.riv.webappsrivlab4.model.DTO.user;

```

```

import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;
import ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.enums.Role;
import
ntukhpi.csit.semit.riv.webappsrivlab4.validator.common.ValidCorporateEmail;
import
ntukhpi.csit.semit.riv.webappsrivlab4.validator.common.ValidName;
import
ntukhpi.csit.semit.riv.webappsrivlab4.validator.common.ValidPatronymic;
import
ntukhpi.csit.semit.riv.webappsrivlab4.validator.common.ValidSurname;
import
ntukhpi.csit.semit.riv.webappsrivlab4.validator.user.ValidPassword;
import
ntukhpi.csit.semit.riv.webappsrivlab4.validator.user.ValidRole;
import
ntukhpi.csit.semit.riv.webappsrivlab4.validator.user.ValidUsername;

/**
 * Data Transfer Object (DTO) for representing test data related to
 * users.
 * This class encapsulates attributes required to simulate user
 * data for testing and validation purposes.
 * <p>
 * Key functionalities:
 * - Provides a structured way to manage user data in testing
 * scenarios.
 * - Ensures all user-related fields adhere to specified validation
 * rules.
 * <p>

```

```

* Key attributes:
* - `username`: A unique identifier for the user, validated for
proper format.
* - `password`: The user's password, validated for security
requirements.
* - `role`: The user's role within the system, ensuring valid
predefined roles.
* - `corporateEmail`: The corporate email address, validated for
proper formatting.
* - `surname`: The user's last name, adhering to valid name
conventions.
* - `name`: The user's first name, validated for format and
length.
* - `patronymic`: The user's patronymic (middle name), ensuring
valid format.
* <p>
* Validation:
* - Ensures adherence to specific constraints using annotations:
* - `@ValidUsername`, `@ValidPassword`, `@ValidRole`,
`@ValidCorporateEmail`, `@ValidSurname`, `@ValidName`,
`@ValidPatronymic`.
* <p>
* Relationships:
* - Related to `Role` enum for defining user roles.
* - Designed to work with other DTOs like `UserRegister`,
`UserLogin`, and `UserForm` for comprehensive user management.
*
* @author Inessa Repeshko CS-222a
* @see Role
* @see ValidUsername
* @see ValidPassword
* @see ValidRole
* @see ValidCorporateEmail
* @see ValidSurname
* @see ValidName
* @see ValidPatronymic

```



```
*/  
  
@Data  
@AllArgsConstructor  
@NoArgsConstructor  
public class UserTestData {  
    @ValidUsername  
    private String username;  
  
    @ValidPassword  
    private String password;  
  
    @ValidRole  
    private Role role;  
  
    @ValidCorporateEmail  
    private String corporateEmail;  
  
    @ValidSurname  
    private String surname;  
  
    @ValidName  
    private String name;  
  
    @ValidPatronymic  
    private String patronymic;  
}
```

ДОДАТОК Д

Код файлів репозиторію Spring Boot проєкту

Д.1 Код файлу EntrantRepository.java

```
package ntukhpi.csit.semit.riv.webappsrivlab4.repository;

import
ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.entrant.Entrant;
import
ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.student.Student;
import org.springframework.data.domain.Sort;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;
import org.springframework.stereotype.Repository;

import java.time.LocalDate;
import java.util.List;
import java.util.Optional;

/**
 * Repository interface for managing `Entrant` entities in the
 * database.
 * Provides CRUD operations and custom queries for handling
 * entrants and their relationships with students.
 * <p>
 * Key functionalities:
 * - Extends `JpaRepository` to provide basic CRUD operations.
 * - Implements custom methods for filtering, sorting, and managing
 * entrants.
 * <p>
 * Key methods:
```

```

* - `getEntrantsBySurnameAndNameAndPatronymic`: Finds entrants by
their full name.
* - `findFilteredAndSortedEntrants`: Retrieves a filtered and
sorted list of entrants based on search criteria and date or score
ranges.
* - `findAllWithoutStudent`: Fetches all entrants who are not
associated with any students.
* - `findStudentIdByEntrantId`: Retrieves the student ID
associated with a specific entrant.
* <p>
* Relationships:
* - Directly interacts with the `Entrant` entity.
* - Associated with the `Student` entity to manage entrant-student
relationships.
* <p>
* Usage:
* - Designed to be used within the service layer for managing
entrant-related workflows.
* - Facilitates efficient querying and manipulation of entrant
data.
*
* @author Inessa Repeshko CS-222a
* @see Entrant
* @see Student
* @see JpaRepository
* @see Sort
*/

@Repository
public interface EntrantRepository extends JpaRepository<Entrant,
Long> {
    List<Entrant> getEntrantsBySurnameAndNameAndPatronymic(String
surname, String name, String patronymic);

    @Query("SELECT e FROM Entrant e ")

```

```

        + "WHERE (:search IS NULL OR UPPER(e.caseNumber) LIKE
UPPER(CONCAT(:search, '%')) "
        + "OR UPPER(CONCAT(e.surname, ' ', e.name, ' ',
e.patronymic)) LIKE UPPER(CONCAT(:search, '%')) "
        + "AND (:birthdayStart IS NULL OR e.birthday >=
:birthdayStart) "
        + "AND (:birthdayEnd IS NULL OR e.birthday <=
:birthdayEnd) "
        + "AND (:ratingScoreMin IS NULL OR e.ratingScore >=
:ratingScoreMin) "
        + "AND (:ratingScoreMax IS NULL OR e.ratingScore <=
:ratingScoreMax) ")
    List<Entrant> findFilteredAndSortedEntrants(@Param("search")
String search,

@Param("birthdayStart") LocalDate birthdayStart,

@Param("birthdayEnd") LocalDate birthdayEnd,

@Param("ratingScoreMin") Double ratingScoreMin,

@Param("ratingScoreMax") Double ratingScoreMax,

Sort sortBy);

    @Query("SELECT e FROM Entrant e WHERE e.id NOT IN (SELECT
s.entrant.id FROM Student s WHERE s.entrant IS NOT NULL)")
    List<Entrant> findAllWithoutStudent(Sort sort);

    @Query("SELECT s.id FROM Student s WHERE s.entrant.id =
:entrantId")
    Optional<Long> findStudentIdByEntrantId(@Param("entrantId")
Long entrantId);
}

```

Д.2 Код файла StudentRepository.java

```
package ntukhpi.csit.semit.riv.webappsrivlab4.repository;

import
ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.enums.FundingType;
import
ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.enums.ScholarshipS
tatus;
import
ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.student.Student;
import org.springframework.data.domain.Sort;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;
import org.springframework.stereotype.Repository;

import java.util.List;

/**
 * Repository interface for managing `Student` entities in the
 * database.
 *
 * Provides CRUD operations and custom queries for filtering and
 * sorting students.
 *
 * <p>
 * Key functionalities:
 *
 * - Extends `JpaRepository` to provide basic CRUD operations.
 * - Implements a custom query for filtering and sorting students
 * based on various criteria.
 *
 * <p>
 * Key methods:
 *
 * - `findFilteredAndSortedStudents`: Filters and sorts students
 * based on search criteria,
 *
 * funding type, and scholarship status. Includes sorting
 * functionality.
```

```

* <p>
* Query Parameters:
* - `search`: Filters students by full name (concatenated surname,
name, and patronymic) containing the search term.
* - `fundingType`: Filters students by their funding type (e.g.,
budget, contract).
* - `scholarshipStatuses`: Filters students by their scholarship
status (e.g., ordinary, enhanced, none).
* - `sortBy`: Specifies sorting order for the query results.
* <p>
* Relationships:
* - Interacts with the `Student` entity and its related `Entrant`
entity.
* - Includes `FundingType` and `ScholarshipStatus` for filtering
based on enum values.
* <p>
* Usage:
* - Designed for use within the service layer to fetch filtered
and sorted student data.
* - Enables flexible and efficient data retrieval for
student-related functionalities.
*
* @author Inessa Repeshko CS-222a
* @see Student
* @see FundingType
* @see ScholarshipStatus
* @see JpaRepository
* @see Sort
*/

@Repository
public interface StudentRepository extends JpaRepository<Student,
Long> {
    @Query("SELECT s FROM Student s "
        + "JOIN s.entrant e ")

```

```

        + "WHERE (:search IS NULL OR UPPER(CONCAT(e.surname, '
', e.name, ' ', e.patronymic)) LIKE UPPER(CONCAT(:search, '%')))"
        + "AND (:fundingType IS NULL OR s.fundingType =
:fundingType) "
        + "AND (:scholarshipStatuses IS NULL OR
s.scholarshipStatus IN :scholarshipStatuses) ")
    List<Student> findFilteredAndSortedStudents(@Param("search")
String search,

@Param("fundingType") FundingType fundingType,

@Param("scholarshipStatuses") List<ScholarshipStatus>
scholarshipStatuses,

Sort sortBy);
}

```

Д.3 Код файла UserRepository.java

```

package ntukhpi.csit.semit.riv.webappsrivlab4.repository;

import ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.enums.Role;
import
ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.user.UserEntity;
import org.springframework.data.domain.Sort;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;
import org.springframework.stereotype.Repository;

import java.util.List;
import java.util.Optional;

/**
 * Repository interface for managing `UserEntity` entities in the
 * database.

```

```

* Provides CRUD operations and custom queries for filtering and
sorting user data.
* <p>
* Key functionalities:
* - Extends `JpaRepository` to enable basic CRUD operations.
* - Includes custom methods for searching users by username,
email, or roles.
* - Implements a custom query for filtering and sorting users
based on search criteria and roles.
* <p>
* Key methods:
* - `findByUsername`: Fetches a user entity by its unique
username.
* - `findByCorporateEmail`: Fetches a user entity by its unique
corporate email.
* - `findByUsernameOrCorporateEmail`: Fetches a user entity using
either its username or corporate email.
* - `existsUserByUsernameOrCorporateEmail`: Checks the existence
of a user by username or corporate email.
* - `findUsersByRole`: Retrieves all users with a specific role.
* - `findFilteredAndSortedUsers`: Filters and sorts users based on
a search term and roles.
* <p>
* Query Parameters:
* - `search`: Filters users by concatenated full name (surname,
name, and patronymic) containing the search term.
* - `roles`: Filters users by their roles (e.g., ADMIN,
STUDENT_VIEWER).
* - `sortBy`: Specifies sorting order for the query results.
* <p>
* Relationships:
* - Works with the `UserEntity` class to represent users.
* - Utilizes the `Role` enum for filtering by user roles.
* <p>
* Usage:

```


ДОДАТОК Е

Код файлів сервісу Spring Boot проєкту

Е.1 Код файлу CustomServiceException.java

```
package ntukhpi.csit.semit.riv.webappsrivlab4.service;

/**
 * Custom exception class for handling service layer errors.
 * This class extends `RuntimeException` and is designed to provide
 * meaningful error
 * messages and encapsulate underlying causes of exceptions
 * occurring in the service layer.
 * <p>
 * Key functionalities:
 * - Allows instantiation with a custom error message.
 * - Supports wrapping of a cause (`Throwable`) alongside the error
 * message.
 * <p>
 * Usage:
 * - Throw this exception in service methods to indicate errors
 * specific to business logic.
 * - Provides consistent error reporting across service
 * implementations.
 * <p>
 * Dependencies:
 * - Extends `RuntimeException` for unchecked exception handling.
 * - Accepts `String` as a message and `Throwable` as an optional
 * cause.
 *
 * @author Inessa Repeshko CS-222a
 * @see RuntimeException
 * @see String
```

```

* @see Throwable
*/

public class CustomServiceException extends RuntimeException {
    public CustomServiceException(String message) {
        super(message);
    }

    public CustomServiceException(String message, Throwable cause)
    {
        super(message, cause);
    }
}

```

E.2 Код файла EntrantService.java

```

package ntukhpi.csit.semit.riv.webappsrivlab4.service.entity;

import jakarta.validation.Valid;
import jakarta.validation.constraints.NotEmpty;
import
ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.entrant.Entrant;
import
ntukhpi.csit.semit.riv.webappsrivlab4.validator.common.ValidId;
import
ntukhpi.csit.semit.riv.webappsrivlab4.validator.entrant.ValidBirth
day;
import
ntukhpi.csit.semit.riv.webappsrivlab4.validator.entrant.ValidRatin
gScore;
import org.springframework.validation.annotation.Validated;

import java.time.LocalDate;
import java.util.List;

```

```

/**
 * Service interface for managing operations on `Entrant` entities.
 * This interface defines methods for CRUD operations, filtering,
 * sorting, and managing
 * relationships with other entities, such as `Student`.
 * <p>
 * Key functionalities:
 * - Fetch all entrants or a filtered and sorted list based on
 * criteria.
 * - Save single or multiple entrants with validation.
 * - Update existing entrants or delete them by ID.
 * - Find entrants without associated students or locate specific
 * entrants by example or ID.
 * - Retrieve the associated student ID for a specific entrant.
 * <p>
 * Validation:
 * - Enforces input validation using annotations like `@Valid`,
 * `@ValidId`, `@ValidBirthday`,
 * and `@ValidRatingScore`.
 * - Throws appropriate exceptions for invalid input or business
 * rule violations.
 * <p>
 * Dependencies:
 * - Annotated with `@Validated` to enable method-level validation.
 * - Relies on custom validators for domain-specific constraints.
 *
 * @author Inessa Repeshko CS-222a
 * @see Entrant
 * @see ValidId
 * @see ValidBirthday
 * @see ValidRatingScore
 * @see Validated
 */

@Validated
public interface EntrantService {

```

```

    List<Entrant> getAllEntrants();

    List<Entrant> getFilteredAndSortedEntrants(String search,
                                                @ValidBirthday
LocalDate birthdayStart,
                                                @ValidBirthday
LocalDate birthdayEnd,
                                                @ValidRatingScore
Double ratingScoreMin,
                                                @ValidRatingScore
Double ratingScoreMax,
                                                String sort);

    List<Entrant> findEntrantsWithoutStudents();

    void saveAllEntrants(@NotEmpty(message = "The entrant list must
not be null.") List<@Valid Entrant> entrants);

    Entrant findEntrantById(@ValidId Long id);

    Entrant findEntrantByExample(@Valid Entrant entrant);

    Long findStudentIdForEntrant(@ValidId Long entrantId);

    Entrant saveEntrant(@Valid Entrant entrant);

    Entrant updateEntrant(@Valid Entrant entrant);

    void deleteEntrantById(@ValidId Long id);
}

```

Е.3 Код файла EntrantServiceImpl.java

```
package
ntukhpi.csit.semit.riv.webappsrivlab4.service.entity.implementatio
n;

import jakarta.validation.ConstraintViolationException;
import jakarta.validation.Valid;
import jakarta.validation.constraints.NotEmpty;
import
ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.entrant.Entrant;
import
ntukhpi.csit.semit.riv.webappsrivlab4.repository.EntrantRepository
;
import
ntukhpi.csit.semit.riv.webappsrivlab4.service.entity.EntrantServic
e;
import
ntukhpi.csit.semit.riv.webappsrivlab4.service.CustomServiceExcepti
on;
import
ntukhpi.csit.semit.riv.webappsrivlab4.validator.entrant.ValidBirth
day;
import
ntukhpi.csit.semit.riv.webappsrivlab4.validator.common.ValidId;
import
ntukhpi.csit.semit.riv.webappsrivlab4.validator.entrant.ValidRatin
gScore;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.dao.DataIntegrityViolationException;
import org.springframework.dao.EmptyResultDataAccessException;
import org.springframework.dao.OptimisticLockingFailureException;
import org.springframework.data.domain.Example;
```

```

import org.springframework.data.domain.Sort;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import org.springframework.validation.annotation.Validated;

import java.time.LocalDate;
import java.util.ArrayList;
import java.util.List;

/**
 * Service implementation for managing `Entrant` entities.
 * This class provides methods for CRUD (Create, Read, Update,
 * Delete) operations
 * and additional functionalities like filtering, sorting, and
 * association handling.
 * <p>
 * Key functionalities:
 * - Fetching all entrants or filtered and sorted lists based on
 * various criteria.
 * - Adding and updating entrants, with proper validation and
 * exception handling.
 * - Deleting entrants while ensuring data integrity.
 * - Managing associations between entrants and students.
 * <p>
 * Exception handling:
 * - Validates input data and throws `CustomServiceException` for
 * validation errors.
 * - Handles specific exceptions like
 * `DataIntegrityViolationException` and
 * `OptimisticLockingFailureException`
 * to manage database-related errors gracefully.
 * <p>
 * Dependencies:
 * - `EntrantRepository`: Provides data access methods for
 * `Entrant` entities.

```

```

* - `Validator` annotations: Ensures data consistency and validity
at the service layer.
* <p>
* Transaction management:
* - Annotated with `@Transactional` to ensure atomicity and
rollback support for critical operations.
*
* @author Inessa Repeshko CS-222a
* @see Entrant
* @see EntrantRepository
* @see CustomServiceException
* @see Transactional
* @see Valid
* @see NotEmpty
* @see DataIntegrityViolationException
* @see EmptyResultDataAccessException
* @see OptimisticLockingFailureException
* @see ValidBirthday
* @see ValidRatingScore
* @see ValidId
*/

@Service
@Transactional
@Validated
public class EntrantServiceImpl implements EntrantService {
    private static final Logger logger =
LoggerFactory.getLogger(EntrantServiceImpl.class);

    private final EntrantRepository entrantRepository;

    @Autowired
    public EntrantServiceImpl(EntrantRepository entrantRepository)
{
        this.entrantRepository = entrantRepository;
    }
}

```



```

@Override
public List<Entrant> getAllEntrants() {
    return entrantRepository.findAll();
}

@Override
public List<Entrant> getFilteredAndSortedEntrants(String
search,

@ValidBirthday LocalDate birthdayStart,

@ValidBirthday LocalDate birthdayEnd,

@ValidRatingScore Double ratingScoreMin,

@ValidRatingScore Double ratingScoreMax,

                                String sort)
{
    String sortPattern =
    "^(id|caseNumber|fullName|surname|name|patronymic|birthday|gender|
ratingScore)-(asc|desc)$";

    if (!sort.trim().matches(sortPattern)) {
        String message = "Invalid sort field provided: " + sort
+ ". It should be in a format: 'fieldName-direction'.";
        logger.warn(message);
        throw new CustomServiceException(message);
    }

    String[] sortParams = sort.split("-");
    String sortField = sortParams[0];
    Sort.Direction sortDirection;

    try {

```

```

        sortDirection =
Sort.Direction.fromString(sortParams[1]);
    } catch (IllegalArgumentException e) {
        logger.error(e.getMessage());
        throw new CustomServiceException(e.getMessage(), e);
    }

    List<Sort.Order> sortOrders = new ArrayList<>();

    switch (sortField) {
        case "caseNumber":
            sortOrders.add(new Sort.Order(sortDirection,
"caseNumber"));
            sortOrders.add(new Sort.Order(sortDirection,
"surname"));
            sortOrders.add(new Sort.Order(sortDirection,
"name"));
            sortOrders.add(new Sort.Order(sortDirection,
"patronymic"));
            break;

        case "surname":
        case "fullName":
            sortOrders.add(new Sort.Order(sortDirection,
"surname"));
            sortOrders.add(new Sort.Order(sortDirection,
"name"));
            sortOrders.add(new Sort.Order(sortDirection,
"patronymic"));
            sortOrders.add(new Sort.Order(sortDirection,
"caseNumber"));
            break;

        default:
            sortOrders.add(new Sort.Order(sortDirection,
sortField));

```

```

        sortOrders.add(new Sort.Order(sortDirection,
"surname"));
        sortOrders.add(new Sort.Order(sortDirection,
"name"));
        sortOrders.add(new Sort.Order(sortDirection,
"patronymic"));
        sortOrders.add(new Sort.Order(sortDirection,
"caseNumber"));
        break;
    }

    Sort sortBy = Sort.by(sortOrders);

    try {
        return entrantRepository.findFilteredAndSortedEntrants(
            search,
            birthdayStart, birthdayEnd,
            ratingScoreMin, ratingScoreMax,
            sortBy);
    } catch (Exception e) {
        String message = "An error occurred while getting the
filtered and sorted entrants. ";
        logger.error(message + "\n" + e.getMessage());
        throw new CustomServiceException(message +
e.getMessage(), e);
    }
}

@Override
public List<Entrant> findEntrantsWithoutStudents() {
    List<Sort.Order> sortOrders = new ArrayList<>();

    sortOrders.add(new Sort.Order(Sort.Direction.ASC,
"surname"));
    sortOrders.add(new Sort.Order(Sort.Direction.ASC, "name"));

```

```

        sortOrders.add(new Sort.Order(Sort.Direction.ASC,
"patronymic"));

        sortOrders.add(new Sort.Order(Sort.Direction.ASC,
"caseNumber"));

        Sort sort = Sort.by(sortOrders);

        return entrantRepository.findAllWithoutStudent(sort);
    }

    @Override
    @Transactional(rollbackFor = {
        CustomServiceException.class,
        DataIntegrityViolationException.class,
        ConstraintViolationException.class,
        OptimisticLockingFailureException.class})
    public void saveAllEntrants(@NotEmpty(message = "The entrant
list must not be null.") List<@Valid Entrant> entrants) {
        try {
            entrantRepository.saveAll(entrants);

        } catch (DataIntegrityViolationException e) {
            /* Handling a duplicate or uniqueness exception */
            String message = "Duplicate or invalid data detected.";
            logger.error(message + "\nData integrity violation: " +
e.getMessage());
            throw new CustomServiceException(message, e);

        } catch (ConstraintViolationException e) {
            /* Handling violations of validation constraints */
            String message = "Validation failed for the DAO.";
            logger.error(message + "\nConstraint violation: " +
e.getMessage());
            throw new CustomServiceException(message + " Check the
input data.", e);

```

```

        } catch (OptimisticLockingFailureException e) {
            /* Handling version conflicts (if optimistic locking is
used) */

            String message = "The DAO could not be saved due to a
version conflict.";

            logger.error(message + "\nOptimistic locking failure: "
+ e.getMessage());

            throw new CustomServiceException(message, e);

        } catch (Exception e) {
            /* General handling of other exceptions */

            String message = "An error occurred while saving the
entities.";

            logger.error(message + "\n" + e.getMessage());

            throw new CustomServiceException(message, e);

        }
    }

    @Override
    public Entrant findById(@ValidId Long id) {
        return entrantRepository.findById(id)
            .orElseThrow(() -> {
                String errorMessage = "No record exists with
id: " + id + ".";

                logger.error(errorMessage);

                return new
CustomServiceException(errorMessage);
            });
    }

    @Override
    public Entrant findEntrantByExample(@Valid Entrant entrant) {
        return entrantRepository.findBy(Example.of(entrant), query
-> query.first().orElse(null));
    }

```

```

@Override
    public Long findStudentIdForEntrant(@ValidId Long entrantId) {
        return
entrantRepository.findStudentIdByEntrantId(entrantId).orElse(null)
;
    }

@Override
@Transactional(rollbackFor = {
    CustomServiceException.class,
    DataIntegrityViolationException.class,
    ConstraintViolationException.class,
    OptimisticLockingFailureException.class})
    public Entrant saveEntrant(@Valid Entrant entrant) {
        try {
            return entrantRepository.save(entrant);

        } catch (DataIntegrityViolationException e) {
            /* Handling a duplicate or uniqueness exception,
foreign key, NOT NULL, CHECK, or length violation */
            String message = "The record with the case number '" +
entrant.getCaseNumber() + "' already exists.";
            logger.error(message + "\nData integrity violation: " +
e.getMessage());
            throw new CustomServiceException(message, e);

        } catch (ConstraintViolationException e) {
            /* Handling violations of validation constraints */
            String message = "Validation failed for the DAO.";
            logger.error(message + "\nConstraint violation: " +
e.getMessage());
            throw new CustomServiceException(message + " Check the
input data.", e);

        } catch (OptimisticLockingFailureException e) {

```

```

        /* Handling version conflicts (if optimistic locking is
used) */

        String message = "The DAO could not be saved due to a
version conflict.";

        logger.error(message + "\nOptimistic locking failure: "
+ e.getMessage());

        throw new CustomServiceException(message, e);

    } catch (Exception e) {
        /* General handling of other exceptions */
        String message = "An error occurred while saving the
DAO. ";

        logger.error(message + "\n" + e.getMessage());
        throw new CustomServiceException(message, e);
    }
}

@Override
@Transactional
public Entrant updateEntrant(@Valid Entrant entrant) {
    if (entrant.getId() == null) {
        String message = "Cannot update a record without an
id.";

        logger.error(message);
        throw new CustomServiceException(message);
    }

    if (!entrantRepository.existsById(entrant.getId())) {
        String message = "No record exists to update.";
        logger.error(message);
        throw new CustomServiceException(message);
    }

    return saveEntrant(entrant);
}

```

```

@Override
@Transactional(rollbackFor = {
    CustomServiceException.class,
    DataIntegrityViolationException.class,
    EmptyResultDataAccessException.class,
    OptimisticLockingFailureException.class})
public void deleteEntrantById(@ValidId Long id) {
    try {
        entrantRepository.deleteById(id);

    } catch (DataIntegrityViolationException e) {
        /* Handling data integrity violations, for example,
when foreign keys are violated */
        String message = "Unable to delete the DAO due to data
integrity constraints. ";
        logger.error(message + e.getMessage());
        throw new CustomServiceException(message, e);

    } catch (EmptyResultDataAccessException e) {
        /* Handling the case when an attempt to delete a
non-existent record is made */
        String message = "The DAO does not exist and cannot be
deleted.";
        logger.error("No DAO found for deletion: " +
e.getMessage());
        throw new CustomServiceException(message, e);

    } catch (OptimisticLockingFailureException e) {
        /* Handling an error when trying to delete a record
with a version conflict */
        String message = "The DAO could not be deleted due to a
version conflict.";
        logger.error("Optimistic locking failure: " +
e.getMessage());
        throw new CustomServiceException(message, e);
    }
}

```



```

        } catch (Exception e) {
            /* General handling of other exceptions */
            String message = "An unexpected error occurred while
deleting the DAO.";
            logger.error(message + "\n" + e.getMessage());
            throw new CustomServiceException(message, e);
        }
    }
}

```

E.4 Код файла StudentService.java

```

package ntukhpi.csit.semit.riv.webappsrivlab4.service.entity;

import jakarta.validation.Valid;
import jakarta.validation.constraints.NotEmpty;
import
ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.student.Student;
import
ntukhpi.csit.semit.riv.webappsrivlab4.validator.common.ValidId;
import org.springframework.validation.annotation.Validated;

import java.util.List;

/**
 * Service interface for managing `Student` entities.
 * This interface provides methods for CRUD operations, filtering,
 * sorting, and retrieving student data.
 * It ensures the proper handling of student records while
 * maintaining data integrity and validation.
 * <p>
 * Key functionalities:
 * - Retrieve all students or filter and sort based on search
 * criteria.
 * - Save multiple student records in bulk.

```

```

* - Find students by their unique ID or example criteria.
* - Create new student records or update existing ones.
* - Delete students by their ID.
* <p>
* Validation:
* - Enforces input validation using annotations like `@Valid` and
  `@NotEmpty`.
* - Ensures compliance with domain-specific constraints for IDs
  and other parameters.
* <p>
* Dependencies:
* - Annotated parameters provide consistent validation and error
  handling.
* - Relies on custom validators for enhanced validation rules.
*
* @author Inessa Repeshko CS-222a
* @see Student
* @see ValidId
* @see Valid
* @see List
*/

@Validated
public interface StudentService {
    List<Student> getAllStudents();

    List<Student> getFilteredAndSortedStudents(String search,
                                                String fundingType,
                                                String
scholarshipStatus,
                                                String sort);

    void saveAllStudents(@NotEmpty(message = "The student list must
not be null.") List<@Valid Student> students);

    Student findStudentById(@ValidId Long id);

```

```

    Student findStudentByExample(@Valid Student student);

    Student saveStudent(@Valid Student student);

    Student updateStudent(@Valid Student student);

    void deleteStudent(@ValidId Long id);
}

```

E.5 Код файла StudentServiceImpl.java

```

package
ntukhpi.csit.semit.riv.webappsrivlab4.service.entity.implementatio
n;

import jakarta.validation.ConstraintViolationException;
import jakarta.validation.Valid;
import jakarta.validation.constraints.NotEmpty;
import
ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.enums.FundingType;
import
ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.enums.Scholarships
tatus;
import
ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.student.Student;
import
ntukhpi.csit.semit.riv.webappsrivlab4.repository.StudentRepository
;
import
ntukhpi.csit.semit.riv.webappsrivlab4.service.entity.EntrantServic
e;
import
ntukhpi.csit.semit.riv.webappsrivlab4.service.entity.StudentServic
e;

```

```

import
ntukhpi.csit.semit.riv.webappsrivlab4.service.CustomServiceExcepti
on;
import
ntukhpi.csit.semit.riv.webappsrivlab4.validator.common.ValidId;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.dao.DataIntegrityViolationException;
import org.springframework.dao.EmptyResultDataAccessException;
import org.springframework.dao.OptimisticLockingFailureException;
import org.springframework.data.domain.Example;
import org.springframework.data.domain.Sort;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import org.springframework.validation.annotation.Validated;

import java.util.ArrayList;
import java.util.List;

/**
 * Service implementation for managing `Student` entities.
 * This class provides methods for creating, updating, retrieving,
 * and deleting students,
 * as well as filtering and sorting them based on various criteria.
 * <p>
 * Key functionalities:
 * - Retrieve all students or filtered/sorted lists of students.
 * - Save or update a single student or a batch of students with
 * validation.
 * - Delete students while ensuring data integrity and exception
 * handling.
 * - Handle relationships between `Student` and `Entrant` entities
 * to maintain consistency.
 * <p>
 * Exception handling:

```

```

* - Throws `CustomServiceException` for business logic errors or
data-related exceptions.
* - Handles specific exceptions like
`DataIntegrityViolationException` and
`ConstraintViolationException`
* for robust error management.
* <p>
* Dependencies:
* - `StudentRepository`: Provides data access methods for
`Student` entities.
* - `EntrantService`: Manages associated entrants and their
relationships with students.
* <p>
* Transaction management:
* - Annotated with `@Transactional` to ensure atomicity and
rollback support for critical operations.
*
* @author Inessa Repeshko CS-222a
* @see Student
* @see StudentRepository
* @see StudentService
* @see EntrantService
* @see Transactional
* @see Valid
* @see NotEmpty
* @see ConstraintViolationException
* @see DataIntegrityViolationException
* @see EmptyResultDataAccessException
* @see OptimisticLockingFailureException
* @see CustomServiceException
* @see FundingType
* @see ScholarshipStatus
* @see Logger
* @see LoggerFactory
*/

```

```

@Service
@Transactional
@Validated
public class StudentServiceImpl implements StudentService {
    private static final Logger logger =
LoggerFactory.getLogger(StudentServiceImpl.class);
    private final StudentRepository studentRepository;
    private final EntrantService entrantService;

    @Autowired
    public StudentServiceImpl(StudentRepository studentRepository,
                             EntrantService entrantService) {
        this.studentRepository = studentRepository;
        this.entrantService = entrantService;
    }

    @Override
    public List<Student> getAllStudents() {
        return studentRepository.findAll();
    }

    @Override
    @Transactional(rollbackFor = {
        CustomServiceException.class,
        DataIntegrityViolationException.class,
        ConstraintViolationException.class,
        OptimisticLockingFailureException.class})
    public void saveAllStudents(@NotEmpty(message = "The student
list must not be null.") List<@Valid Student> students) {
        try {
            students.forEach(student ->
student.setEntrant(entrantService.findEntrantByExample(student.get
Entrant())));
            studentRepository.saveAll(students);
        } catch (DataIntegrityViolationException e) {

```

```

        /* Handling a duplicate or uniqueness exception */
        String message = "Duplicate or invalid data detected.
";
        logger.error(message + "\nData integrity violation: " +
e.getMessage());
        throw new CustomServiceException(message, e);

    } catch (ConstraintViolationException e) {
        /* Handling violations of validation constraints */
        String message = "Validation failed for the DAO.";
        logger.error(message + "\nConstraint violation: " +
e.getMessage());
        throw new CustomServiceException(message + " Check the
input data.", e);

    } catch (OptimisticLockingFailureException e) {
        /* Handling version conflicts (if optimistic locking is
used) */
        String message = "The DAO could not be saved due to a
version conflict.";
        logger.error(message + "\nOptimistic locking failure: "
+ e.getMessage());
        throw new CustomServiceException(message, e);

    } catch (Exception e) {
        /* General handling of other exceptions */
        String message = "An error occurred while saving the
DAO.";
        logger.error(message + "\n" + e.getMessage());
        throw new CustomServiceException(message, e);
    }
}

@Override
public Student findStudentById(@ValidId Long id) {
    return studentRepository.findById(id)

```

```

        .orElseThrow(() -> {
            String errorMessage = "No record exists with
id: " + id + ".";
            logger.error(errorMessage);
            return new
CustomServiceException(errorMessage);
        });
    }

    @Override
    public Student findStudentByExample(@Valid Student student) {
        return studentRepository.findBy(Example.of(student), query
-> query.first().orElse(null));
    }

    @Override
    public List<Student> getFilteredAndSortedStudents(String
search,
                                                    String
fundingType,
                                                    String
scholarshipStatus,
                                                    String sort)
    {
        String sortPattern =
"^(id|caseNumber|fullName|surname|name|patronymic|fundingType|scho
larshipStatus|corporateEmail)-(asc|desc)$";

        if (!sort.trim().matches(sortPattern)) {
            String message = "Invalid sort field provided: " + sort
+ ". It should be in a format: 'fieldName-direction'.";
            logger.warn(message);
            throw new CustomServiceException(message);
        }

        String[] sortParams = sort.split("-");

```



```

String sortField = sortParams[0];
Sort.Direction sortDirection;

try {
    sortDirection =
Sort.Direction.fromString(sortParams[1]);
} catch (IllegalArgumentException e) {
    logger.error(e.getMessage());
    throw new CustomServiceException(e.getMessage(), e);
}

List<Sort.Order> sortOrders = new ArrayList<>();

switch (sortField) {
    case "caseNumber":
        sortOrders.add(new Sort.Order(sortDirection,
"entrant.caseNumber"));
        sortOrders.add(new Sort.Order(sortDirection,
"entrant.surname"));
        sortOrders.add(new Sort.Order(sortDirection,
"entrant.name"));
        sortOrders.add(new Sort.Order(sortDirection,
"entrant.patronymic"));
        break;

    case "surname":
    case "fullName":
        sortOrders.add(new Sort.Order(sortDirection,
"entrant.surname"));
        sortOrders.add(new Sort.Order(sortDirection,
"entrant.name"));
        sortOrders.add(new Sort.Order(sortDirection,
"entrant.patronymic"));
        sortOrders.add(new Sort.Order(sortDirection,
"entrant.caseNumber"));
        break;

```

```

        default:
            sortOrders.add(new Sort.Order(sortDirection,
sortField));
            sortOrders.add(new Sort.Order(sortDirection,
"entrant.surname"));
            sortOrders.add(new Sort.Order(sortDirection,
"entrant.name"));
            sortOrders.add(new Sort.Order(sortDirection,
"entrant.patronymic"));
            sortOrders.add(new Sort.Order(sortDirection,
"entrant.caseNumber"));
            break;
    }

    Sort sortBy = Sort.by(sortOrders);

    String[] statuses = scholarshipStatus != null ?
scholarshipStatus.trim().split("-") : new String[]{};
    List<ScholarshipStatus> scholarshipStatuses = new
ArrayList<>();

    for (String status : statuses) {

scholarshipStatuses.add(ScholarshipStatus.fromValue(status));
    }

    try {
        return studentRepository.findFilteredAndSortedStudents(
            search,
            fundingType != null ?
FundingType.fromValue(fundingType) : null,
            !scholarshipStatuses.isEmpty() ?
scholarshipStatuses : null,
            sortBy);
    } catch (Exception e) {

```

```

        String message = "An error occurred while getting the
filtered and sorted students. ";
        logger.error(message + e.getMessage());
        throw new CustomServiceException(message +
e.getMessage(), e);
    }
}

@Override
@Transactional(rollbackFor = {
    CustomServiceException.class,
    DataIntegrityViolationException.class,
    ConstraintViolationException.class,
    OptimisticLockingFailureException.class})
public Student saveStudent(@Valid Student student) {
    try {
        return studentRepository.save(student);

    } catch (DataIntegrityViolationException e) {
        /* Handling a duplicate or uniqueness exception,
foreign key, NOT NULL, CHECK, or length violation */
        String message = "The record with the corporate email
'" + student.getCorporateEmail() + "' already exists.";
        logger.error(message + "\nData integrity violation: " +
e.getMessage());
        throw new CustomServiceException(message, e);

    } catch (ConstraintViolationException e) {
        /* Handling violations of validation constraints */
        String message = "Validation failed for the DAO.";
        logger.error(message + "\nConstraint violation: " +
e.getMessage());
        throw new CustomServiceException(message + " Check the
input data.", e);

    } catch (OptimisticLockingFailureException e) {

```

```

        /* Handling version conflicts (if optimistic locking is
used) */

        String message = "The DAO could not be saved due to a
version conflict.";

        logger.error(message + "\nOptimistic locking failure: "
+ e.getMessage());

        throw new CustomServiceException(message, e);

    } catch (Exception e) {
        /* General handling of other exceptions */
        String message = "An error occurred while saving the
DAO. ";

        logger.error(message + "\n" + e.getMessage());

        throw new CustomServiceException(message +
e.getMessage(), e);
    }
}

@Override
@Transactional
public Student updateStudent(@Valid Student student) {
    if (student.getId() == null) {
        String message = "Cannot update a record without an
id.";

        logger.error(message);

        throw new CustomServiceException(message);
    }

    if (!studentRepository.existsById(student.getId())) {
        String message = "No record exists to update.";

        logger.error(message);

        throw new CustomServiceException(message);
    }

    return saveStudent(student);
}

```

```

@Override
@Transactional(rollbackFor = {
    CustomServiceException.class,
    DataIntegrityViolationException.class,
    EmptyResultDataAccessException.class,
    OptimisticLockingFailureException.class})
public void deleteStudent(@ValidId Long id) {
    try {
        studentRepository.deleteById(id);

    } catch (DataIntegrityViolationException e) {
        /* Handling data integrity violations, for example,
when foreign keys are violated */
        String message = "Unable to delete the DAO due to data
integrity constraints.";
        logger.error(message + "\n" + e.getMessage());
        throw new CustomServiceException(message, e);

    } catch (EmptyResultDataAccessException e) {
        /* Handling the case when an attempt to delete a
non-existent record is made */
        String message = "The DAO does not exist and cannot be
deleted.";
        logger.error("No DAO found for deletion: " +
e.getMessage());
        throw new CustomServiceException(message, e);

    } catch (OptimisticLockingFailureException e) {
        /* Handling an error when trying to delete a record
with a version conflict */
        String message = "The DAO could not be deleted due to a
version conflict.";
        logger.error(message + "\nOptimistic locking failure: "
+ e.getMessage());
        throw new CustomServiceException(message, e);
    }
}

```

```

        } catch (Exception e) {
            /* General handling of other exceptions */
            String message = "An unexpected error occurred while
deleting the DAO.";
            logger.error(message + e.getMessage());
            throw new CustomServiceException(message, e);
        }
    }
}

```

E.6 Код файла UserService.java

```

package ntukhpi.csit.semit.riv.webappsrivlab4.service.entity;

import jakarta.validation.Valid;
import jakarta.validation.constraints.*;
import ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.enums.Role;
import
ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.user.UserEntity;
import
ntukhpi.csit.semit.riv.webappsrivlab4.validator.common.ValidId;
import
ntukhpi.csit.semit.riv.webappsrivlab4.validator.common.ValidCorporateEmail;
import
ntukhpi.csit.semit.riv.webappsrivlab4.validator.user.ValidRole;
import
ntukhpi.csit.semit.riv.webappsrivlab4.validator.user.ValidUsername
;
import
ntukhpi.csit.semit.riv.webappsrivlab4.validator.user.ValidUsername
OrCorporateEmail;
import org.springframework.validation.annotation.Validated;

```

```

import java.util.List;

/**
 * Service interface for managing `UserEntity` entities.
 * This interface provides methods for CRUD operations, filtering,
 * sorting, and retrieving user data.
 * It ensures proper handling of user records with validation and
 * compliance to business rules.
 * <p>
 * Key functionalities:
 * - Retrieve all users or filter and sort based on role and search
 * criteria.
 * - Save multiple user records in bulk.
 * - Find users by their unique ID, username, corporate email, or
 * example criteria.
 * - Create new user records or update existing ones.
 * - Delete users by their ID.
 * - Retrieve user passwords securely by their ID or corporate
 * email.
 * <p>
 * Validation:
 * - Enforces input validation using annotations like `@Valid`,
 * `@NotEmpty`, and `@ValidId`.
 * - Ensures compliance with domain-specific constraints for roles,
 * IDs, and other parameters.
 * - Relies on custom validators for enhanced validation rules,
 * such as `ValidUsername`, `ValidCorporateEmail`, and `ValidRole`.
 * <p>
 * Dependencies:
 * - Provides consistent validation and error handling through
 * annotated parameters.
 * - Relies on domain objects like `UserEntity` and `Role` to
 * encapsulate user data.
 *
 * @author Inessa Repeshko CS-222a
 * @see UserEntity

```

```

* @see Role
* @see ValidId
* @see ValidUsername
* @see ValidCorporateEmail
* @see ValidRole
* @see ValidUsernameOrCorporateEmail
* @see Valid
* @see NotEmpty
*/

@Validated
public interface UserService {

    List<UserEntity> getAllUsers();

    List<UserEntity> findUsersByRole(@ValidRole Role role);

    void saveAllUsers(@NotEmpty(message = "The user list must not
be null.") List<UserEntity> userEntities);

    UserEntity findUserById(@ValidId Long id);

    UserEntity findUserByExample(@Valid UserEntity userEntity);

    UserEntity findUserByUsername(@ValidUsername String username);

    UserEntity findUserByCorporateEmail(@ValidCorporateEmail String
CorporateEmail);

    String findPasswordById(@ValidId Long id);

    String findPasswordByUserCorporateEmail(@ValidCorporateEmail
String corporateEmail);

    UserEntity
findUserByUsernameOrCorporateEmail(@ValidUsernameOrCorporateEmail
String usernameOrCorporateEmail);

```



```

List<UserEntity> getFilteredAndSortedUsers (String search,
                                           String role,
                                           String sort);

UserEntity saveUser(@Valid UserEntity userEntity);

UserEntity updateUser(@Valid UserEntity userEntity);

void deleteUserById(@ValidId Long id);
}

```

E.7 Код файла UserServiceImpl.java

```

package
ntukhpi.csit.semit.riv.webappsrivlab4.service.entity.implementatio
n;

import jakarta.validation.ConstraintViolationException;
import jakarta.validation.Valid;
import jakarta.validation.constraints.NotEmpty;
import ntukhpi.csit.semit.riv.webappsrivlab4.config.EncoderConfig;
import ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.enums.Role;
import
ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.user.UserEntity;
import
ntukhpi.csit.semit.riv.webappsrivlab4.repository.PasswordResetToke
nRepository;
import
ntukhpi.csit.semit.riv.webappsrivlab4.repository.UserRepository;
import
ntukhpi.csit.semit.riv.webappsrivlab4.service.entity.UserService;
import
ntukhpi.csit.semit.riv.webappsrivlab4.service.CustomServiceExcepti
on;

```

```

import
ntukhpi.csit.semit.riv.webappsrivlab4.validator.common.ValidId;
import
ntukhpi.csit.semit.riv.webappsrivlab4.validator.common.ValidCorporateEmail;
import
ntukhpi.csit.semit.riv.webappsrivlab4.validator.user.ValidRole;
import
ntukhpi.csit.semit.riv.webappsrivlab4.validator.user.ValidUsername
;
import
ntukhpi.csit.semit.riv.webappsrivlab4.validator.user.ValidUsername
OrCorporateEmail;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.dao.DataIntegrityViolationException;
import org.springframework.dao.EmptyResultDataAccessException;
import org.springframework.dao.OptimisticLockingFailureException;
import org.springframework.data.domain.Example;
import org.springframework.data.domain.Sort;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import org.springframework.validation.annotation.Validated;

import java.util.ArrayList;
import java.util.List;

/**
 * Service implementation for managing `UserEntity` entities.
 * This class provides methods for creating, updating, retrieving,
 * and deleting users,
 * as well as filtering and sorting users based on various
 * criteria.
 * <p>
 * Key functionalities:

```

```

* - Retrieve all users or filtered/sorted lists of users.
* - Save or update a single user or a batch of users with
validation.
* - Delete users while maintaining data integrity and exception
handling.
* - Handle relationships between users and other entities like
`PasswordResetToken`.
* <p>
* Exception handling:
* - Throws `CustomServiceException` for business logic errors or
data-related exceptions.
* - Handles specific exceptions like
`DataIntegrityViolationException` and
`ConstraintViolationException`
* for robust error management.
* <p>
* Dependencies:
* - `UserRepository`: Provides data access methods for
`UserEntity` entities.
* - `PasswordResetTokenRepository`: Manages password reset tokens
for user accounts.
* - `EncoderConfig`: Handles password encoding and security
configurations.
* <p>
* Transaction management:
* - Annotated with `@Transactional` to ensure atomicity and
rollback support for critical operations.
*
* @author Inessa Repeshko CS-222a
* @see UserEntity
* @see UserRepository
* @see PasswordResetTokenRepository
* @see UserService
* @see EncoderConfig
* @see Transactional
* @see Valid

```

```

* @see ConstraintViolationException
* @see DataIntegrityViolationException
* @see EmptyResultDataAccessException
* @see OptimisticLockingFailureException
* @see CustomServiceException
* @see ValidId
* @see ValidUsername
* @see ValidCorporateEmail
* @see ValidUsernameOrCorporateEmail
* @see Role
* @see Logger
* @see LoggerFactory
*/

@Service
@Transactional
@Validated
public class UserServiceImpl implements UserService {
    private static final Logger logger =
LoggerFactory.getLogger(UserServiceImpl.class);
    private final UserRepository userRepository;
    private final EncoderConfig encoderConfig;
    private final PasswordResetTokenRepository tokenRepository;

    @Autowired
    public UserServiceImpl(UserRepository userRepository,
                           EncoderConfig encoderConfig,
                           PasswordResetTokenRepository
tokenRepository) {
        this.userRepository = userRepository;
        this.encoderConfig = encoderConfig;
        this.tokenRepository = tokenRepository;
    }

    @Override
    public List<UserEntity> getAllUsers() {

```

```

        return userRepository.findAll();
    }

    @Override
    public List<UserEntity> findUsersByRole(@ValidRole Role role) {
        return userRepository.findUsersByRole(role);
    }

    @Override
    @Transactional(rollbackFor = {
        CustomServiceException.class,
        DataIntegrityViolationException.class,
        ConstraintViolationException.class,
        OptimisticLockingFailureException.class})
    public void saveAllUsers(@NotEmpty(message = "The user list
must not be null.") List<UserEntity> userEntities) {
        try {
            userEntities.forEach(user ->
user.setPassword(encoderConfig.passwordEncoder().encode(user.getPa
ssword())));
            userRepository.saveAll(userEntities);

        } catch (DataIntegrityViolationException e) {
            /* Handling a duplicate or uniqueness exception */
            String message = "Duplicate or invalid data detected.
";
            logger.error(message + "\nData integrity violation: " +
e.getMessage());
            throw new CustomServiceException(message, e);

        } catch (ConstraintViolationException e) {
            /* Handling violations of validation constraints */
            String message = "Validation failed for the DAO.";
            logger.error(message + "\nConstraint violation: " +
e.getMessage());

```

```

        throw new CustomServiceException(message + " Check the
input data.", e);

    } catch (OptimisticLockingFailureException e) {
        /* Handling version conflicts (if optimistic locking is
used) */

        String message = "The DAO could not be saved due to a
version conflict.";

        logger.error(message + "\nOptimistic locking failure: "
+ e.getMessage());

        throw new CustomServiceException(message, e);

    } catch (Exception e) {
        /* General handling of other exceptions */
        String message = "An error occurred while saving the
DAO.";

        logger.error(message + "\n" + e.getMessage());

        throw new CustomServiceException(message, e);
    }
}

@Override
public UserEntity findUserById(@ValidId Long id) {
    return userRepository.findById(id)
        .orElseThrow(() -> {
            String errorMessage = "No record exists with
id: " + id + ".";

            logger.error(errorMessage);

            return new
CustomServiceException(errorMessage);
        });
}

@Override
public UserEntity findUserByUsername(@ValidUsername String
username) {

```

```

        return
userRepository.findByUsername(username).orElseThrow(() -> {
    String message = "No user found for passed username.";
    logger.error(message);
    return new CustomServiceException(message);
});
}

@Override
public UserEntity findUserByCorporateEmail(@ValidCorporateEmail
String corporateEmail) {
    return
userRepository.findByCorporateEmail(corporateEmail).orElseThrow(()
-> {
        String message = "No user found for passed corporate
email.";
        logger.error(message);
        return new CustomServiceException(message);
    });
}

@Override
public String findPasswordById(@ValidId Long id) {
    return userRepository.findById(id)
        .map(UserEntity::getPassword)
        .orElseThrow(() -> {
            String errorMessage = "No record exists with
id: " + id + ".";
            logger.error(errorMessage);
            return new
CustomServiceException(errorMessage);
        });
}

@Override

```

```

    public String
findPasswordByUserCorporateEmail(@ValidCorporateEmail String
corporateEmail) {
    return userRepository.findByCorporateEmail(corporateEmail)
        .map(UserEntity::getPassword)
        .orElseThrow(() -> {
            String errorMessage = "No record exists with
corporate email: " + corporateEmail + ".";
            logger.error(errorMessage);
            return new
CustomServiceException(errorMessage);
        });
}

@Override
public UserEntity
findUserByUsernameOrCorporateEmail(@ValidUsernameOrCorporateEmail
String usernameOrCorporateEmail) {
    return userRepository.findByUsernameOrCorporateEmail(
        usernameOrCorporateEmail,
        usernameOrCorporateEmail)
        .orElseThrow(() -> {
            String message = "No user found for passed user
identifier (username or corporate email).";
            logger.error(message);
            return new CustomServiceException(message);
        });
}

@Override
public UserEntity findUserByExample(@Valid UserEntity
userEntity) {
    return userRepository.findBy(Example.of(userEntity), query
-> query.first().orElse(null));
}

```



```

@Override
public List<UserEntity> getFilteredAndSortedUsers (String
search,
                                                    String role,
                                                    String sort)
{
    String sortPattern =
"^(id|username|role|corporateEmail|fullName|surname|name|patronymi
c)-(asc|desc)$";

    if (!sort.trim().matches(sortPattern)) {
        String message = "Invalid sort field provided: " + sort
+
        ". It should be in a format:
'fieldName-direction'.";
        logger.warn(message);
        throw new CustomServiceException(message);
    }

    String[] sortParams = sort.split("-");
    String sortField = sortParams[0];
    Sort.Direction sortDirection;

    try {
        sortDirection =
Sort.Direction.fromString(sortParams[1]);
    } catch (IllegalArgumentException e) {
        logger.error(e.getMessage());
        throw new CustomServiceException(e.getMessage(), e);
    }

    List<Sort.Order> sortOrders = new ArrayList<>();

    switch (sortField) {
        case "surname":
        case "fullName":

```

```

        sortOrders.add(new Sort.Order(sortDirection,
"surname"));

        sortOrders.add(new Sort.Order(sortDirection,
"name"));

        sortOrders.add(new Sort.Order(sortDirection,
"patronymic"));

        break;

        default:

            sortOrders.add(new Sort.Order(sortDirection,
sortField));

            sortOrders.add(new Sort.Order(sortDirection,
"surname"));

            sortOrders.add(new Sort.Order(sortDirection,
"name"));

            sortOrders.add(new Sort.Order(sortDirection,
"patronymic"));

            break;
    }

    Sort sortBy = Sort.by(sortOrders);

    String[] roles = role != null ? role.trim().split("-") :
new String[]{};

    List<Role> roleList = new ArrayList<>();

    for (String roleName : roles) {
        roleList.add(Role.fromName(roleName));
    }

    try {
        return userRepository.findFilteredAndSortedUsers(
            search,
            !roleList.isEmpty() ? roleList : null,
            sortBy);
    } catch (Exception e) {

```

```

        String message = "An error occurred while getting the
filtered and sorted users. ";

        logger.error(message + e.getMessage());

        throw new CustomServiceException(message +
e.getMessage(), e);
    }
}

@Override
@Transactional(rollbackFor = {
    CustomServiceException.class,
    DataIntegrityViolationException.class,
    ConstraintViolationException.class,
    OptimisticLockingFailureException.class})
public UserEntity saveUser(@Valid UserEntity userEntity) {
    try {
        if (userEntity.getPassword() != null &&
userEntity.getPassword().length() < 60) {

userEntity.setPassword(encoderConfig.passwordEncoder().encode(user
Entity.getPassword()));
        }

        return userRepository.save(userEntity);

    } catch (DataIntegrityViolationException e) {
        /* Handling a duplicate or uniqueness exception,
foreign key, NOT NULL, CHECK, or length violation */
        String message = "The record with the username '" +
userEntity.getUsername()
            + "' or corporate email '" +
userEntity.getCorporateEmail() + "' already exists.";
        logger.error(message + "\nData integrity violation: " +
e.getMessage());
        throw new CustomServiceException(message, e);
    }
}

```

```

        } catch (ConstraintViolationException e) {
            /* Handling violations of validation constraints */
            String message = "Validation failed for the DAO.";
            logger.error(message + "\nConstraint violation: " +
e.getMessage());

            throw new CustomServiceException(message + " Check the
input data.", e);

        } catch (OptimisticLockingFailureException e) {
            /* Handling version conflicts (if optimistic locking is
used) */
            String message = "The DAO could not be saved due to a
version conflict.";
            logger.error(message + "\nOptimistic locking failure: "
+ e.getMessage());
            throw new CustomServiceException(message, e);

        } catch (Exception e) {
            /* General handling of other exceptions */
            String message = "An error occurred while saving the
DAO. ";
            logger.error(message + "\n" + e.getMessage());
            throw new CustomServiceException(message +
e.getMessage(), e);
        }
    }

    @Override
    @Transactional
    public UserEntity updateUser(@Valid UserEntity userEntity) {
        if (userEntity.getId() == null) {
            String message = "Cannot update a record without an
id.";
            logger.error(message);
            throw new CustomServiceException(message);
        }
    }

```

```

        if (!userRepository.existsById(userEntity.getId())) {
            String message = "No record exists to update.";
            logger.error(message);
            throw new CustomServiceException(message);
        }

        if (userEntity.getPassword() == null) {
            userEntity.setPassword(userRepository.findById(userEntity.getId())
                .get().getPassword());
        }

        return saveUser(userEntity);
    }

    @Override
    @Transactional(rollbackFor = {
        CustomServiceException.class,
        DataIntegrityViolationException.class,
        EmptyResultDataAccessException.class,
        OptimisticLockingFailureException.class})
    public void deleteUserById(@ValidId Long id) {
        try {
            tokenRepository.deleteByUserId(id);
            userRepository.deleteById(id);

        } catch (DataIntegrityViolationException e) {
            /* Handling data integrity violations, for example,
            when foreign keys are violated */
            String message = "Unable to delete the user due to data
            integrity constraints. ";
            logger.error(message + e.getMessage());
            throw new CustomServiceException(message, e);

        } catch (EmptyResultDataAccessException e) {

```

```

        /* Handling the case when an attempt to delete a
non-existent record is made */

        String message = "The user does not exist and cannot be
deleted.";

        logger.error("No user found for deletion: " +
e.getMessage());

        throw new CustomServiceException(message, e);

    } catch (OptimisticLockingFailureException e) {

        /* Handling an error when trying to delete a record
with a version conflict */

        String message = "The user could not be deleted due to
a version conflict.";

        logger.error("Optimistic locking failure: " +
e.getMessage());

        throw new CustomServiceException(message, e);

    } catch (Exception e) {

        /* General handling of other exceptions */

        String message = "An unexpected error occurred while
deleting the user.";

        logger.error(message + "\n" + e.getMessage());

        throw new CustomServiceException(message, e);

    }

}

}

```

ДОДАТОК Ж

Код файлів валідатора Spring Boot проєкту

Ж.1 Код файлу BirthdayValidator.java

```
package ntukhpi.csit.semit.riv.webappsrivlab4.validator.validator;

import jakarta.validation.ConstraintValidator;
import jakarta.validation.ConstraintValidatorContext;
import
ntukhpi.csit.semit.riv.webappsrivlab4.validator.entrant.ValidBirth
day;

import java.time.LocalDate;

/**
 * Custom validator for validating a birthday.
 * This validator ensures that the annotated `LocalDate` field
 * represents a valid birthday,
 * where the date must be within the range of 16 to 116 years ago
 * from the current date.
 * <p>
 * Validation rules:
 * - The date must be no more than 116 years in the past (i.e., the
 * person must not be older than 116 years).
 * - The date must be no less than 16 years ago (i.e., the person
 * must be at least 16 years old).
 * <p>
 * The validator uses the current date (`LocalDate.now()`) to
 * compute the valid range for the birthday.
 * If the provided birthday is outside this range, a custom error
 * message is added to the constraint violation context.
 * <p>
```

```

* This validator is applied through the `@ValidBirthday`
annotation.
*
* @author Inessa Repeshko CS-222a
* @see ValidBirthday
* @see LocalDate
* @see ConstraintValidator
*/

public class BirthdayValidator implements
ConstraintValidator<ValidBirthday, LocalDate> {

    @Override
    public boolean isValid(LocalDate value,
ConstraintValidatorContext context) {
        if (value != null) {
            LocalDate today = LocalDate.now();
            LocalDate minDate = today.minusYears(116);
            LocalDate maxDate = today.minusYears(16);

            if (value.isBefore(minDate) || value.isAfter(maxDate))
{
                context.disableDefaultConstraintViolation();
                context.buildConstraintViolationWithTemplate("The
birthday must be between 16 and 110 years ago today.")
                    .addConstraintViolation();
                return false;
            }
        }

        return true;
    }
}

```


Ж.2 Код файла UsernameOrCorporateEmailValidator.java

```
package ntukhpi.csit.semit.riv.webappsrivlab4.validator.validator;

import jakarta.validation.ConstraintValidator;
import jakarta.validation.ConstraintValidatorContext;
import
ntukhpi.csit.semit.riv.webappsrivlab4.validator.user.ValidUsername
OrCorporateEmail;

import java.util.regex.Pattern;

/**
 * Custom validator for validating a user identifier, which can be
 * either a username or a corporate email.
 * This validator ensures that the provided identifier is either a
 * valid username or a valid corporate email.
 * <p>
 * Validation rules:
 * - If the identifier is a username, it must contain Latin letters
 * and may include numbers and special characters such as `.``, `_`,
 * or `-`.
 * - If the identifier is a corporate email, it must match the
 * format `name.surname@faculty.khpi.edu.ua`.
 * - The identifier must not be null or blank.
 * <p>
 * This validator is applied through the
 * `@ValidUsernameOrCorporateEmail` annotation.
 * <p>
 * Dependencies:
 * - The validator uses regular expressions (regex) defined in
 * `ValidationConstants.REGEX_USERNAME` and
 * `ValidationConstants.REGEX_CORPORATE_EMAIL` to validate the
 * username and corporate email formats.
 *

```

```

* @author Inessa Repeshko CS-222a
* @see ValidUsernameOrCorporateEmail
* @see ValidationConstants
* @see Pattern
* @see ConstraintValidator
*/

public class UsernameOrCorporateEmailValidator implements
ConstraintValidator<ValidUsernameOrCorporateEmail, String> {
    private static final Pattern USERNAME_PATTERN =
Pattern.compile(ValidationConstants.REGEX_USERNAME);
    private static final Pattern CORPORATE_EMAIL_PATTERN =
Pattern.compile(ValidationConstants.REGEX_CORPORATE_EMAIL);

    @Override
    public boolean isValid(String value, ConstraintValidatorContext
context) {
        if (value == null || value.isBlank()) {
            context.disableDefaultConstraintViolation();
            context.buildConstraintViolationWithTemplate("The user
identifier must not be blank or null.")
                .addConstraintViolation();
            return false;
        }

        if (!USERNAME_PATTERN.matcher(value).matches() &&
!CORPORATE_EMAIL_PATTERN.matcher(value).matches()) {
            context.disableDefaultConstraintViolation();
            context.buildConstraintViolationWithTemplate("The user
identifier must be a valid username " +
                "(must contain Latin letters and may
contain numbers and special characters '.', '_', '-') " +
                "or corporate email (should match the
format 'name.surname@faculty.khpi.edu.ua').")
                .addConstraintViolation();
            return false;
        }
    }
}

```

```

    }

    return true;
}
}

```

Ж.3 Код файлу ValidationConstants.java

```

package ntukhpi.csit.semit.riv.webappsrivlab4.validator.validator;

import jakarta.validation.constraints.Pattern;
import
ntukhpi.csit.semit.riv.webappsrivlab4.validator.common.ValidCorpor
ateEmail;
import
ntukhpi.csit.semit.riv.webappsrivlab4.validator.common.ValidName;
import
ntukhpi.csit.semit.riv.webappsrivlab4.validator.entrant.ValidCaseN
umber;
import
ntukhpi.csit.semit.riv.webappsrivlab4.validator.user.ValidPassword
;
import
ntukhpi.csit.semit.riv.webappsrivlab4.validator.user.ValidUsername
;

import java.time.format.DateTimeFormatter;

/**
 * This class contains various constants used for validation
 purposes.
 * These constants include regular expressions for validating case
 numbers, names, gender, email addresses, usernames, and passwords.
 * It also defines date formatters used for parsing dates in
 Ukrainian and English formats.

```

```

* <p>
* Key constants:
* - **REGEX_CASE_NUMBER**: Regular expression for validating case
numbers in the format `LLLLLLLLLLNN-NNNN`.
* - **REGEX_NAMES**: Regular expression for validating names
(first name, surname, and patronymic) with support for Ukrainian
characters, spaces, and dashes.
* - **DATE_FORMATTER_UA**: Date formatter for parsing dates in the
format `dd.MM.yyyy` (Ukrainian date format).
* - **DATE_FORMATTER_EN**: Date formatter for parsing dates in the
format `yyyy-MM-dd` (English date format).
* - **REGEX_GENDER**: Regular expression for validating gender
input, allowing values such as `1`, `0`, `true`, `false`, and
their variants.
* - **REGEX_CORPORATE_EMAIL**: Regular expression for validating
corporate email addresses in the format
`name.surname@faculty.khpi.edu.ua`.
* - **REGEX_USERNAME**: Regular expression for validating
usernames, allowing Latin letters, digits, and special characters
such as `.`, `_`, and `-`.
* - **REGEX_PASSWORD**: Regular expression for validating
passwords, ensuring the inclusion of uppercase letters, digits,
and special characters.
* <p>
* These constants are utilized in validation annotations to
enforce consistent and precise validation rules across the
application.
*
* @author Inessa Repeshko CS-222a
* @see DateTimeFormatter
* @see Pattern
* @see ValidCaseNumber
* @see ValidName
* @see ValidCorporateEmail
* @see ValidUsername
* @see ValidPassword

```

```

*/

public final class ValidationConstants {

    public static final String REGEX_CASE_NUMBER =
"^ (?.{5,20}$) [A-ЩЮЯҐЄІЇ]{1,10} \\d{2} [\\-] \\d{1,4}$";

    public static final String REGEX_NAMES =
"^ (?.{1,50}$) [A-ЩЮЯҐЄІЇ] (?:[A-ЯҐЄІЇа-яґєіі']*[A-ЯҐЄІЇа-яґєіі])?(?:
[ \\-] [A-ЩЮЯҐЄІЇ] (?:[A-ЯҐЄІЇа-яґєіі']*[A-ЯҐЄІЇа-яґєіі])?)*$";

    public static final DateTimeFormatter DATE_FORMATTER_UA =
DateTimeFormatter.ofPattern("dd.MM.yyyy");

    public static final DateTimeFormatter DATE_FORMATTER_EN =
DateTimeFormatter.ofPattern("yyyy-MM-dd");

    public static final String REGEX_GENDER =
"^ (1|0|true|false|TRUE|FALSE|True|False)$";

    public static final String REGEX_CORPORATE_EMAIL =
"^ (?.{17,123}$) [a-z] (\\.?.[a-z]+) {1,99} @[a-z]{1,10}\\.khpi\\.edu\\.
.ua$";

    public static final String REGEX_USERNAME =
"^ (?.{8,32}$) [a-zA-Z0-9._-]+$";

    public static final String REGEX_PASSWORD =
"^ (?.{8,50}$) (?.*[A-Z]) (?.*[0-9]) (?.*[!@#$%^&*()_+\\-=\\\\\\\\[\\\\\\\\
\\\\}{;\\':\\\\\\\\\\\\\"|,.<>/?~]) [A-Za-z0-9!@#$%^&*()_+\\-=\\\\\\\\[\\\\\\\\\\\\}{;\\':
\\\\\\\\\\\\\"|,.<>/?~]*$";

}

```

Ж.4 Код файлу ValidBirthday.java

```

package ntukhpi.csit.semit.riv.webappsrivlab4.validator.entrant;

import jakarta.validation.Constraint;
import jakarta.validation.Payload;
import jakarta.validation.constraints.Past;
import
ntukhpi.csit.semit.riv.webappsrivlab4.validator.validator.Birthday
Validator;

```

```

import java.lang.annotation.*;

/**
 * Custom validation annotation for validating birthdays.
 * This annotation ensures that the annotated field or parameter:
 * - Represents a date in the past.
 * - Passes additional validation logic implemented in the
 * `BirthdayValidator` class.
 * <p>
 * Validation rules:
 * - `@Past`: Ensures the date is strictly in the past.
 * - `BirthdayValidator`: Provides custom logic to validate
 * domain-specific birthday constraints.
 * <p>
 * Typical usage:
 * - Annotate fields or method parameters representing birthdays to
 * enforce these validation rules.
 * - Example: `@ValidBirthday private LocalDate birthday;`
 * <p>
 * Dependencies:
 * - Relies on Jakarta Bean Validation for basic date validation.
 * - Custom validation logic is implemented in the
 * `BirthdayValidator` class.
 * <p>
 * This annotation is commonly used in entity or DTO validation
 * layers to ensure consistent formatting of birthdays
 * and adherence to domain-specific rules.
 *
 * @author Inessa Repeshko CS-222a
 * @see Past
 * @see BirthdayValidator
 */
@Constraint(validatedBy = {BirthdayValidator.class})
@Documented

```

```

@Target({ElementType.PARAMETER, ElementType.FIELD,
ElementType.CONSTRUCTOR, ElementType.METHOD})
@Retention(RetentionPolicy.RUNTIME)

@Past(message = "The birthday must be in the past.")

public @interface ValidBirthday {

    String message() default "Invalid birthday";

    Class<?>[] groups() default {};

    Class<? extends Payload>[] payload() default {};

}

```

Ж.5 Код файла ValidCaseNumber.java

```

package ntukhpi.csit.semit.riv.webappsrivlab4.validator.entrant;

import jakarta.validation.Constraint;
import jakarta.validation.Payload;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Pattern;
import jakarta.validation.constraints.Size;
import
ntukhpi.csit.semit.riv.webappsrivlab4.validator.validator.Validati
onConstants;

import java.lang.annotation.*;

/**
 * Custom validation annotation for validating case numbers.
 * This annotation ensures that the annotated field or parameter:
 * - Is not blank.
 * - Falls within the defined size range.
 * - Matches the specified format for a valid case number.

```

```

* <p>
* Validation rules:
* - `@NotBlank`: Ensures the case number is not null or empty.
* - `@Size`: Limits the length of the case number between 5 and 20
characters.
* - `@Pattern`: Enforces a specific format defined by the regular
expression `ValidationConstants.REGEX_CASE_NUMBER`.
* <p>
* Typical usage:
* - Apply this annotation to fields or method parameters
representing case numbers in entities or DTOs.
* - Example: `@ValidCaseNumber private String caseNumber;`
* <p>
* Dependencies:
* - Built-in constraints (`@NotBlank`, `@Size`, and `@Pattern`)
for general validation.
* - Custom regex defined in `ValidationConstants` for
domain-specific format enforcement.
* <p>
* This annotation is commonly used in the validation layers of
entities or DTOs to ensure data consistency
* and adherence to the required format for case numbers.
*
* @author Inessa Repeshko CS-222a
* @see NotBlank
* @see Size
* @see Pattern
* @see ValidationConstants
*/

@Constraint(validatedBy = {})
@Documented
@Target({ElementType.PARAMETER, ElementType.FIELD,
ElementType.CONSTRUCTOR, ElementType.METHOD})
@Retention(RetentionPolicy.RUNTIME)

```



```

@NotBlank(message = "The case number must not be blank.")
@Size(min = 5, max = 20, message = "The case number size should be
between 5 to 20 characters.")
@Pattern(regex = ValidationConstants.REGEX_CASE_NUMBER, message =
"The case number should be in the format 'LLLLLLLLLLNN-NNNN'.")

public @interface ValidCaseNumber {

    String message() default "Invalid case number";

    Class<?>[] groups() default {};

    Class<? extends Payload>[] payload() default {};

}

```

Ж.6 Код файла ValidGender.java

```

package ntukhpi.csit.semit.riv.webappsrivlab4.validator.entrant;

import jakarta.validation.Constraint;
import jakarta.validation.Payload;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.Past;
import org.springframework.format.annotation.DateTimeFormat;

import java.lang.annotation.*;

/**
 * Custom validation annotation for validating gender values.
 * This annotation ensures that the annotated field or parameter:
 * - Is not null.
 * <p>
 * Validation rules:
 * - `@NotNull`: Ensures the gender value is not null.
 * <p>
 * Typical usage:

```

```

* - Apply this annotation to fields or method parameters
representing gender in entities or DTOs.
* - Example: `@ValidGender private Gender gender;`
* <p>
* Purpose:
* - Ensures that gender values are not left undefined in the
application, adhering to domain requirements.
* <p>
* Dependencies:
* - Built-in constraint (`@NotNull`) for null-check validation.
* <p>
* This annotation is commonly used in the validation layers of
entities or DTOs to enforce mandatory gender specification.
*
* @author Inessa Repeshko CS-222a
* @see NotNull
*/

@Constraint(validatedBy = {})
@Documented
@Target({ElementType.PARAMETER, ElementType.FIELD,
ElementType.CONSTRUCTOR, ElementType.METHOD})
@Retention(RetentionPolicy.RUNTIME)

@NotNull(message = "The gender must not be null.")

public @interface ValidGender {
    String message() default "Invalid gender";

    Class<?>[] groups() default {};

    Class<? extends Payload>[] payload() default {};
}

```

Ж.7 Код файла ValidRatingScore.java

```
package ntukhpi.csit.semit.riv.webappsrivlab4.validator.entrant;

import jakarta.validation.Constraint;
import jakarta.validation.Payload;
import jakarta.validation.constraints.DecimalMax;
import jakarta.validation.constraints.DecimalMin;
import jakarta.validation.constraints.Positive;

import java.lang.annotation.*;

/**
 * Custom annotation for validating rating scores.
 * This annotation ensures that the annotated field or parameter
 * meets specific domain constraints:
 * - The rating score must be a positive number.
 * - The value must be greater than 120.000 (exclusive).
 * - The value must be less than or equal to 200.000 (inclusive).
 * <p>
 * Validation rules:
 * - `@DecimalMin`: Enforces the minimum value constraint.
 * - `@DecimalMax`: Enforces the maximum value constraint.
 * - `@Positive`: Ensures the value is a positive number.
 * <p>
 * Purpose:
 * - To validate rating scores used in entities, DTOs, or other
 * domain models, ensuring data consistency.
 * <p>
 * Usage:
 * - Annotate the relevant field or parameter to enforce these
 * constraints.
 * - Example: `@ValidRatingScore private Double ratingScore;`
 * <p>
 * Dependencies:

```

```

* - Uses Jakarta Bean Validation annotations for built-in
constraint validation.
* <p>
* This annotation is ideal for ensuring that rating scores are
within an acceptable range
* and are positive, thus supporting domain-specific logic and data
integrity.
*
* @author Inessa Repeshko CS-222a
* @see DecimalMin
* @see DecimalMax
* @see Positive
* @see Payload
*/

@Constraint(validatedBy = {})
@Documented
@Target({ElementType.PARAMETER, ElementType.FIELD,
ElementType.CONSTRUCTOR, ElementType.METHOD})
@Retention(RetentionPolicy.RUNTIME)

@DecimalMin(value = "120.000", inclusive = false, message = "The
rating score must be greater than 120,000.")
@DecimalMax(value = "200.000", inclusive = true, message = "The
rating score must be less than or equal to 200,000.")
@Positive(message = "The rating score must be a positive number.")

public @interface ValidRatingScore {
    String message() default "Invalid rating score";

    Class<?>[] groups() default {};

    Class<? extends Payload>[] payload() default {};
}

```

Ж.8 Код файла ValidFundingType.java

```
package ntukhpi.csit.semit.riv.webappsrivlab4.validator.student;

import jakarta.validation.Constraint;
import jakarta.validation.Payload;
import jakarta.validation.constraints.NotNull;

import java.lang.annotation.*;

/**
 * Custom annotation for validating funding types.
 * This annotation ensures that the annotated field or parameter
 * meets specific domain constraints:
 * - The funding type must not be null.
 * <p>
 * Validation rules:
 * - `@NotNull`: Ensures the value is not null, preventing the
 * absence of a funding type.
 * <p>
 * Purpose:
 * - To validate funding type fields in entities, DTOs, or other
 * domain models, ensuring that valid funding type data is always
 * provided.
 * <p>
 * Usage:
 * - Annotate the relevant field or parameter to enforce this
 * constraint.
 * - Example: `@ValidFundingType private FundingType fundingType;`
 * <p>
 * Dependencies:
 * - Uses Jakarta Bean Validation annotations for built-in
 * constraint validation.
 * <p>

```

```

* This annotation is ideal for ensuring that funding type fields
are always specified and comply with domain-specific requirements.
*
* @author Inessa Repeshko CS-222a
* @see NotNull
* @see Payload
*/

@Constraint(validatedBy = {})
@Documented
@Target({ElementType.PARAMETER, ElementType.FIELD,
ElementType.CONSTRUCTOR, ElementType.METHOD})
@Retention(RetentionPolicy.RUNTIME)

@NotNull(message = "Funding type must not be null.")

public @interface ValidFundingType {
    String message() default "Invalid funding type";

    Class<?>[] groups() default {};

    Class<? extends Payload>[] payload() default {};
}

```

Ж.9 Код файла ValidScholarshipStatus.java

```

package ntukhpi.csit.semit.riv.webappsrivlab4.validator.student;

import jakarta.validation.Constraint;
import jakarta.validation.Payload;
import jakarta.validation.constraints.NotNull;

import java.lang.annotation.*;

/**

```

```

* Custom annotation for validating scholarship statuses.
* This annotation ensures that the annotated field or parameter
meets specific domain constraints:
* - The scholarship status must not be null.
* <p>
* Validation rules:
* - `@NotNull`: Ensures the value is not null, preventing the
absence of a scholarship status.
* <p>
* Purpose:
* - To validate scholarship status fields in entities, DTOs, or
other domain models, ensuring that valid scholarship status data
is always provided.
* <p>
* Usage:
* - Annotate the relevant field or parameter to enforce this
constraint.
* - Example: `@ValidScholarshipStatus private ScholarshipStatus
scholarshipStatus;`
* <p>
* Dependencies:
* - Uses Jakarta Bean Validation annotations for built-in
constraint validation.
* <p>
* This annotation is ideal for ensuring that scholarship status
fields are always specified and comply with domain-specific
requirements.
*
* @author Inessa Repeshko CS-222a
* @see NotNull
* @see Payload
*/

@Constraint(validatedBy = {})
@Documented

```

```

@Target({ElementType.PARAMETER, ElementType.FIELD,
ElementType.CONSTRUCTOR, ElementType.METHOD})
@Retention(RetentionPolicy.RUNTIME)

@NotNull(message = "Scholarship status must not be null.")

public @interface ValidScholarshipStatus {

    String message() default "Invalid scholarship status";

    Class<?>[] groups() default {};

    Class<? extends Payload>[] payload() default {};

}

```

Ж.10 Код файла ValidEncodedPassword.java

```

package ntukhpi.csit.semit.riv.webappsrivlab4.validator.user;

import jakarta.validation.Constraint;
import jakarta.validation.Payload;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Size;

import java.lang.annotation.*;

/**
 * Custom annotation for validating encoded passwords.
 * This annotation ensures that the annotated field or parameter
 * adheres to the specific format and requirements of an encoded
 * password:
 * - The password must not be blank.
 * - The password must have a fixed length of 60 characters.
 * <p>
 * Validation rules:

```



```

* - @NotBlank: Ensures the password is not blank, preventing
empty or null values.
* - @Size: Enforces that the password length is exactly 60
characters, which is standard for securely hashed passwords (e.g.,
BCrypt).
* <p>
* Purpose:
* - To validate password fields that are expected to be encoded
(e.g., hashed) in entities, DTOs, or other domain models.
* <p>
* Usage:
* - Annotate the relevant field or parameter to enforce this
constraint.
* - Example: @ValidEncodedPassword private String password;
* <p>
* Dependencies:
* - Uses Jakarta Bean Validation annotations for built-in
constraint validation.
* <p>
* This annotation is tailored for applications where password
storage must meet security standards, ensuring that passwords are
securely encoded and comply with domain-specific requirements.
*
* @author Inessa Repeshko CS-222a
* @see NotBlank
* @see Size
* @see Payload
*/

@Constraint(validatedBy = {})
@Documented
@Target({ElementType.PARAMETER, ElementType.FIELD,
ElementType.CONSTRUCTOR, ElementType.METHOD})
@Retention(RetentionPolicy.RUNTIME)

NotBlank(message = "The password must not be blank.")

```

```

@Size(min = 60, max = 60, message = "The password should be 60
characters long.")

public @interface ValidEncodedPassword {

    String message() default "Invalid password";

    Class<?>[] groups() default {};

    Class<? extends Payload>[] payload() default {};

}

```

Ж.11 Код файла ValidPassword.java

```

package ntukhpi.csit.semit.riv.webappsrivlab4.validator.user;

import jakarta.validation.Constraint;
import jakarta.validation.Payload;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Pattern;
import jakarta.validation.constraints.Size;
import
ntukhpi.csit.semit.riv.webappsrivlab4.validator.validator.Validati
onConstants;

import java.lang.annotation.*;

/**
 * Custom annotation for validating passwords.
 * This annotation ensures that the annotated field or parameter
 meets specific password strength and formatting requirements:
 * - The password must not be blank.
 * - The password length must be between 8 and 50 characters.
 * - The password must include:
 * - Latin letters.
 * - At least one uppercase letter.

```

```

* - At least one special character.
* <p>
* Validation rules:
* - `@NotBlank`: Ensures the password is not blank, preventing
empty or null values.
* - `@Size`: Enforces a password length between 8 and 50
characters.
* - `@Pattern`: Validates that the password matches the specified
format in `ValidationConstants.REGEX_PASSWORD`.
* <p>
* Purpose:
* - To validate password fields for security and compliance with
application-defined rules.
* <p>
* Usage:
* - Annotate the relevant field or parameter to enforce this
constraint.
* - Example: `@ValidPassword private String password;`
* <p>
* Dependencies:
* - Requires `ValidationConstants.REGEX_PASSWORD` to define the
password format.
* <p>
* This annotation is essential for applications that require
strong password policies to enhance security and user
authentication robustness.
*
* @author Inessa Repeshko CS-222a
* @see NotBlank
* @see Size
* @see Pattern
* @see Payload
*/

@Constraint(validatedBy = {})
@Documented

```

```

@Target({ElementType.PARAMETER, ElementType.FIELD,
        ElementType.CONSTRUCTOR, ElementType.METHOD})
@Retention(RetentionPolicy.RUNTIME)

@NotBlank(message = "The password must not be blank.")
@Size(min = 8, max = 50, message = "The password length should be
between 8 to 50 characters.")
@Pattern(regexp = ValidationConstants.REGEX_PASSWORD,
        message = "The password must contain Latin letters, at
least one capital letter and one special character.")

public @interface ValidPassword {
    String message() default "Invalid password";

    Class<?>[] groups() default {};

    Class<? extends Payload>[] payload() default {};
}

```

Ж.12 Код файла ValidRole.java

```

package ntukhpi.csit.semit.riv.webappsrivlab4.validator.user;

import jakarta.validation.Constraint;
import jakarta.validation.Payload;
import jakarta.validation.constraints.NotNull;

import java.lang.annotation.*;

/**
 * Custom annotation for validating user roles.
 * This annotation ensures that the annotated field or parameter is
 * not null and complies with the application's role requirements.
 * <p>
 * Validation rules:

```

```

* - @NotNull: Ensures the role value is not null.
* <p>
* Purpose:
* - To validate that a role is assigned and properly configured
  for user entities or parameters requiring a role.
* <p>
* Usage:
* - Annotate the relevant field or parameter to enforce this
  constraint.
* - Example: @ValidRole private Role userRole;
* <p>
* Dependencies:
* - Relies on the Role enumeration or similar logic to define
  valid roles in the application.
* <p>
* This annotation is critical for maintaining data integrity and
  ensuring that user entities or related logic include a valid role,
  preventing null values from causing errors in the application
  flow.
*
* @author Inessa Repeshko CS-222a
* @see NotNull
* @see Payload
*/

@Constraint(validatedBy = {})
@Documented
@Target({ElementType.PARAMETER, ElementType.FIELD,
  ElementType.CONSTRUCTOR, ElementType.METHOD})
@Retention(RetentionPolicy.RUNTIME)

@NotNull(message = "Role must not be null.")

public @interface ValidRole {
    String message() default "Invalid role";
}

```

```

    Class<?>[] groups() default {};

    Class<? extends Payload>[] payload() default {};
}

```

Ж.13 Код файла ValidUsername.java

```

package ntukhpi.csit.semit.riv.webappsrivlab4.validator.user;

import jakarta.validation.Constraint;
import jakarta.validation.Payload;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Pattern;
import jakarta.validation.constraints.Size;
import
ntukhpi.csit.semit.riv.webappsrivlab4.validator.validator.Validation
onConstants;

import java.lang.annotation.Documented;
import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;

/**
 * Custom annotation for validating usernames.
 * This annotation ensures that the annotated field or parameter
 * meets the application's username requirements.
 * <p>
 * Validation rules:
 * - `@NotBlank`: Ensures that the username is not empty or null.
 * - `@Size`: Validates the username length to be between 8 to 32
 * characters.

```

```

* - @Pattern: Enforces the username format to match the
specified regular expression (allowing Latin letters, numbers, and
certain special characters like '.', '_', and '-').
* <p>
* Purpose:
* - To ensure that usernames are valid and comply with the
application's formatting and length constraints.
* <p>
* Usage:
* - Annotate the relevant field or parameter in a user-related
class or method to enforce these constraints.
* - Example: @ValidUsername private String username;
* <p>
* Dependencies:
* - Relies on the ValidationConstants.REGEX_USERNAME to define
the valid pattern for usernames.
* - The pattern allows Latin letters, numbers, and specific
special characters (`.`, `_`, `-`).
* <p>
* This annotation is essential for user registration, login, and
other functionalities involving usernames, ensuring that usernames
are valid and consistent across the application.
*
* @author Inessa Repeshko CS-222a
* @see NotBlank
* @see Size
* @see Pattern
* @see Payload
*/

@Constraint(validatedBy = {})
@Documented
@Target({ElementType.PARAMETER, ElementType.FIELD,
ElementType.CONSTRUCTOR, ElementType.METHOD})
@Retention(RetentionPolicy.RUNTIME)

```

```

@NotBlank(message = "The username must not be blank.")
@Size(min = 8, max = 32, message = "The username length should be
between 8 to 32 characters.")
@Pattern(regexp = ValidationConstants.REGEX_USERNAME, message =
"The username must contain Latin letters and may contain numbers
and special characters '.', '_', '-'")

public @interface ValidUsername {
    String message() default "Invalid username";

    Class<?>[] groups() default {};

    Class<? extends Payload>[] payload() default {};
}

```

Ж.14 Код файла ValidUsernameOrCorporateEmail.java

```

package ntukhpi.csit.semit.riv.webappsrivlab4.validator.user;

import jakarta.validation.Constraint;
import jakarta.validation.Payload;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Size;
import
ntukhpi.csit.semit.riv.webappsrivlab4.validator.validator.Username
OrCorporateEmailValidator;

import java.lang.annotation.*;

/**
 * Custom annotation for validating a user identifier, which can
 * either be a username or a corporate email.
 * This annotation ensures that the annotated field or parameter
 * meets the application's requirements for both username
 * and corporate email formats.

```



```

* <p>
* Validation rules:
* - `@NotBlank`: Ensures that the user identifier is not empty or
null.
* - `@Size`: Validates the length of the identifier, ensuring it
falls between 8 to 123 characters.
* - `@UsernameOrCorporateEmailValidator`: A custom validator that
checks whether the identifier is a valid username or corporate
email.
* <p>
* Purpose:
* - To ensure that user identifiers are valid, either as usernames
or corporate email addresses.
* <p>
* Usage:
* - Annotate the relevant field or parameter in a user-related
class or method to enforce these constraints.
* - Example: `@ValidUsernameOrCorporateEmail private String
userIdentifier;`
* <p>
* Dependencies:
* - The annotation relies on a custom validator
`UsernameOrCorporateEmailValidator` to ensure the identifier is
either a valid username or corporate email.
* <p>
* This annotation is crucial for user authentication scenarios
where either a username or corporate email can be used to identify
a user.
*
* @author Inessa Repeshko CS-222a
* @see UsernameOrCorporateEmailValidator
* @see NotBlank
* @see Size
* @see Payload
*/

```

```

@Constraint(validatedBy =
{UsernameOrCorporateEmailValidator.class})
@Documented
@Target({ElementType.PARAMETER, ElementType.FIELD,
ElementType.CONSTRUCTOR, ElementType.METHOD})
@Retention(RetentionPolicy.RUNTIME)

@NotBlank(message = "The user identifier must not be blank.")
@Size(min = 8, max = 123, message = "The user identifier length
should be between 8 to 123 characters.")

public @interface ValidUsernameOrCorporateEmail {
    String message() default "The user identifier must be a valid
username or corporate email";

    Class<?>[] groups() default {};

    Class<? extends Payload>[] payload() default {};
}

```

Ж.15 Код файла ValidCorporateEmail.java

```

package ntukhpi.csit.semit.riv.webappsrivlab4.validator.common;

import jakarta.validation.Constraint;
import jakarta.validation.Payload;
import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Size;
import
ntukhpi.csit.semit.riv.webappsrivlab4.validator.validator.Validati
onConstants;

import java.lang.annotation.Documented;
import java.lang.annotation.ElementType;

```

```

import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;

/**
 * Custom validation annotation for validating corporate email
 * addresses.
 *
 * This annotation ensures that the annotated field or parameter:
 *
 * - Is not blank.
 * - Has a length between 17 and 123 characters.
 * - Matches the specific format for corporate email addresses.
 *
 * <p>
 * Validation rules:
 *
 * - `@NotBlank`: Ensures the email is not null or empty.
 * - `@Size`: Limits the length of the email to the range 17-123
 * characters.
 * - `@Email`: Validates the format of the email against a custom
 * regex defined in `ValidationConstants.REGEX_CORPORATE_EMAIL`.
 *
 * <p>
 * Typical usage:
 *
 * - Annotate fields or method parameters in DTOs or entities to
 * enforce corporate email validation rules.
 * - Example: `@ValidCorporateEmail private String corporateEmail;`
 *
 * <p>
 * Dependencies:
 *
 * - Utilizes standard Jakarta Bean Validation constraints like
 * `@NotBlank`, `@Size`, and `@Email`.
 * - Relies on the `ValidationConstants.REGEX_CORPORATE_EMAIL` for
 * regex pattern matching.
 *
 *
 * @author Inessa Repeshko CS-222a
 * @see NotBlank
 * @see Size
 * @see Email
 * @see ValidationConstants
 */

```

```

@Constraint(validatedBy = {})
@Documented
@Target({ElementType.PARAMETER, ElementType.FIELD,
ElementType.CONSTRUCTOR, ElementType.METHOD})
@Retention(RetentionPolicy.RUNTIME)

@NotBlank(message = "The email must not be blank.")
@Size(min = 17, max = 123, message = "The corporate email length
should be between 17 to 123 characters.")
>Email(regex = ValidationConstants.REGEX_CORPORATE_EMAIL,
        message = "The corporate email should match the format
'name.surname@faculty.khpi.edu.ua'.")

public @interface ValidCorporateEmail {
    String message() default "Invalid corporate email";

    Class<?>[] groups() default {};

    Class<? extends Payload>[] payload() default {};
}

```

Ж.16 Код файла ValidId.java

```

package ntukhpi.csit.semit.riv.webappsrivlab4.validator.common;

import jakarta.validation.Constraint;
import jakarta.validation.Payload;
import jakarta.validation.constraints.*;

import java.lang.annotation.*;

/**
 * Custom validation annotation for validating entity IDs.
 * This annotation ensures that the annotated field or parameter:

```

```

* - Is not null.
* - Is a positive number.
* <p>
* Validation rules:
* - `@NotNull`: Ensures the ID is not null.
* - `@Positive`: Validates that the ID is a positive number
(greater than zero).
* <p>
* Typical usage:
* - Annotate fields or method parameters representing entity IDs
to enforce these validation rules.
* - Example: `@ValidId private Long id;`
* <p>
* Dependencies:
* - Utilizes standard Jakarta Bean Validation constraints such as
`@NotNull` and `@Positive`.
* <p>
* This annotation is commonly used in service and repository
layers to validate IDs before processing.
*
* @author Inessa Repeshko CS-222a
* @see NotNull
* @see Positive
*/

@Constraint(validatedBy = {})
@Documented
@Target({ElementType.PARAMETER, ElementType.FIELD,
ElementType.CONSTRUCTOR, ElementType.METHOD})
@Retention(RetentionPolicy.RUNTIME)

@NotNull(message = "The id must not be null.")
@Positive(message = "The id must be a positive number.")

public @interface ValidId {
    String message() default "Invalid id";
}

```

```

    Class<?>[] groups() default {};

    Class<? extends Payload>[] payload() default {};
}

```

Ж.17 Код файлу ValidName.java

```

package ntukhpi.csit.semit.riv.webappsrivlab4.validator.common;

import jakarta.validation.Constraint;
import jakarta.validation.Payload;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Pattern;
import jakarta.validation.constraints.Size;
import
ntukhpi.csit.semit.riv.webappsrivlab4.validator.validator.Validati
onConstants;

import java.lang.annotation.*;

/**
 * Custom validation annotation for validating names.
 * This annotation ensures that the annotated field or parameter:
 * - Is not blank.
 * - Has a length between 1 and 50 characters.
 * - Matches the specified pattern for valid names, allowing only
Ukrainian letters,
 * with support for complex names containing spaces or dashes.
 * <p>
 * Validation rules:
 * - `@NotBlank`: Ensures the name is not blank or empty.
 * - `@Size`: Enforces the length of the name to be between 1 and
50 characters.

```

```

* - @Pattern: Validates the name against a regular expression
defined in ValidationConstants.REGEX_NAMES.
* <p>
* Typical usage:
* - Annotate fields or method parameters representing names to
enforce these validation rules.
* - Example: @ValidName private String name;
* <p>
* Dependencies:
* - Uses Jakarta Bean Validation constraints and custom-defined
constants for regular expressions.
* <p>
* This annotation is commonly used in entity or DTO validation
layers for ensuring consistent name formatting.
*
* @author Inessa Repeshko CS-222a
* @see NotBlank
* @see Size
* @see Pattern
*/

@Constraint(validatedBy = {})
@Documented
@Target({ElementType.PARAMETER, ElementType.FIELD,
ElementType.CONSTRUCTOR, ElementType.METHOD})
@Retention(RetentionPolicy.RUNTIME)

@NotBlank(message = "The name must not be blank.")
@Size(min = 1, max = 50, message = "The name length should be
between 1 to 50 characters.")
@Pattern(regex = ValidationConstants.REGEX_NAMES,
        message = "The name can contain only letters of the
Ukrainian alphabet and can be complex using a space or dash.")

public @interface ValidName {
    String message() default "Invalid name";
}

```

```

    Class<?>[] groups() default {};

    Class<? extends Payload>[] payload() default {};
}

```

Ж.18 Код файла ValidPatronymic.java

```

package ntukhpi.csit.semit.riv.webappsrivlab4.validator.common;

import jakarta.validation.Constraint;
import jakarta.validation.Payload;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Pattern;
import jakarta.validation.constraints.Size;
import
ntukhpi.csit.semit.riv.webappsrivlab4.validator.validator.Validation
onConstants;

import java.lang.annotation.*;

/**
 * Custom validation annotation for validating patronymic names.
 * This annotation ensures that the annotated field or parameter:
 * - Is not blank.
 * - Has a length between 1 and 50 characters.
 * - Matches the specified pattern for valid patronymics, allowing
 * only Ukrainian letters,
 * with support for complex patronymics containing spaces or
 * dashes.
 * <p>
 * Validation rules:
 * - `@NotBlank`: Ensures the patronymic is not blank or empty.
 * - `@Size`: Enforces the length of the patronymic to be between 1
 * and 50 characters.

```



```

* - @Pattern: Validates the patronymic against a regular
expression defined in ValidationConstants.REGEX_NAMES.
* <p>
* Typical usage:
* - Annotate fields or method parameters representing patronymics
to enforce these validation rules.
* - Example: @ValidPatronymic private String patronymic;
* <p>
* Dependencies:
* - Uses Jakarta Bean Validation constraints and custom-defined
constants for regular expressions.
* <p>
* This annotation is commonly used in entity or DTO validation
layers to ensure consistent formatting of patronymics.
*
* @author Inessa Repeshko CS-222a
* @see NotBlank
* @see Size
* @see Pattern
*/

@Constraint(validatedBy = {})
@Documented
@Target({ElementType.PARAMETER, ElementType.FIELD,
ElementType.CONSTRUCTOR, ElementType.METHOD})
@Retention(RetentionPolicy.RUNTIME)

@NotBlank(message = "The patronymic must not be blank.")
@Size(min = 1, max = 50, message = "The patronymic length should
be between 1 to 50 characters.")
@Pattern(regexp = ValidationConstants.REGEX_NAMES,
        message = "The patronymic can contain only letters of the
Ukrainian alphabet and can be complex using a space or dash.")

public @interface ValidPatronymic {
    String message() default "Invalid patronymic";
}

```

```

    Class<?>[] groups() default {};

    Class<? extends Payload>[] payload() default {};
}

```

Ж.19 Код файлу ValidSurname.java

```

package ntukhpi.csit.semit.riv.webappsrivlab4.validator.common;

import jakarta.validation.Constraint;
import jakarta.validation.Payload;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Pattern;
import jakarta.validation.constraints.Size;
import
ntukhpi.csit.semit.riv.webappsrivlab4.validator.validator.Validation
onConstants;

import java.lang.annotation.*;

/**
 * Custom validation annotation for validating surnames.
 * This annotation ensures that the annotated field or parameter:
 * - Is not blank.
 * - Has a length between 1 and 50 characters.
 * - Matches the specified pattern for valid surnames, allowing
 * only Ukrainian letters,
 * with support for complex surnames containing spaces or dashes.
 * <p>
 * Validation rules:
 * - `@NotBlank`: Ensures the surname is not blank or empty.
 * - `@Size`: Enforces the length of the surname to be between 1
 * and 50 characters.

```

```

* - @Pattern: Validates the surname against a regular expression
defined in ValidationConstants.REGEX_NAMES.
* <p>
* Typical usage:
* - Annotate fields or method parameters representing surnames to
enforce these validation rules.
* - Example: @ValidSurname private String surname;
* <p>
* Dependencies:
* - Uses Jakarta Bean Validation constraints and custom-defined
constants for regular expressions.
* <p>
* This annotation is commonly used in entity or DTO validation
layers to ensure consistent formatting of surnames.
*
* @author Inessa Repeshko CS-222a
* @see NotBlank
* @see Size
* @see Pattern
*/

@Constraint(validatedBy = {})
@Documented
@Target({ElementType.PARAMETER, ElementType.FIELD,
ElementType.CONSTRUCTOR, ElementType.METHOD})
@Retention(RetentionPolicy.RUNTIME)

@NotBlank(message = "The surname must not be blank.")
@Size(min = 1, max = 50, message = "The surname length should be
between 1 to 50 characters.")
@Pattern(regex = ValidationConstants.REGEX_NAMES,
        message = "The surname can contain only letters of the
Ukrainian alphabet and can be complex using a space or dash.")

public @interface ValidSurname {
    String message() default "Invalid surname";
}

```

```
Class<?>[] groups() default {};  
  
Class<? extends Payload>[] payload() default {};  
}
```

ДОДАТОК И

Код файлів репозиторію Spring Boot проєкту

И.1 Код файлу EntrantServiceTest.java

```
package ntukhpi.csit.semit.riv.webappsrivlab4;

import
ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.entrant.Entrant;
import
ntukhpi.csit.semit.riv.webappsrivlab4.service.CustomServiceExcepti
on;
import
ntukhpi.csit.semit.riv.webappsrivlab4.service.entity.EntrantServic
e;
import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.context.SpringBootTest;

import static org.junit.jupiter.api.Assertions.assertThrows;
import static
org.springframework.test.util.AssertionErrors.assertEquals;
import static
org.springframework.test.util.AssertionErrors.assertNotNull;

/**
 * Unit test class for the `EntrantService` class.
 * This class tests the functionality of the `EntrantService` in
managing `Entrant` entities.
 * It verifies the basic CRUD operations and ensures that
constraints like duplicate case numbers are enforced.
```

```

* <p>
* Key functionalities tested:
* - Creating and saving an entrant.
* - Reading an entrant by ID and verifying that the case number
  matches.
* - Updating an entrant and ensuring the changes are saved
  correctly.
* - Deleting an entrant and ensuring it cannot be found afterward.
* - Checking that attempting to save an entrant with a duplicate
  case number throws an exception.
* <p>
* Each test method ensures that the `EntrantService` behaves as
  expected and performs the required validation and exception
  handling.
* <p>
* Test annotations:
* - `@BeforeEach` to set up the test data before each test method.
* - `@AfterEach` to clean up any test data after each test method.
* <p>
* Dependencies:
* - `EntrantService`: Provides methods for handling `Entrant`
  entities.
*
* @author Inessa Repeshko CS-222a
* @see Entrant
* @see EntrantService
* @see CustomServiceException
*/

@SpringBootTest
class EntrantServiceTest {
    @Autowired
    private EntrantService entrantService;

    private Entrant testEntrant;

```

```

@BeforeEach
void setUp() {
    testEntrant = new Entrant("KH22-9876", "Франко", "Іван",
"Якович", "2002-01-15", "TRUE", "150.0");
}

@AfterEach
void reset() {
    entrantService.deleteEntrantById(testEntrant.getId());
}

@Test
void testCreateEntrant() {
    Entrant savedEntrant =
entrantService.saveEntrant(testEntrant);
    assertNotNull("Entrant should have an ID after saving.",
savedEntrant.getId());
}

@Test
void testReadEntrant() {
    Entrant savedEntrant =
entrantService.saveEntrant(testEntrant);

    Entrant foundEntrant =
entrantService.findEntrantById(savedEntrant.getId());

    assertNotNull("Entrant should be found by ID.",
foundEntrant);

    assertEquals("Case number should match. Current value: '" +
foundEntrant.getCaseNumber() + "'.",
        savedEntrant.getCaseNumber(),
foundEntrant.getCaseNumber());
}

@Test
void testUpdateEntrant() {
    String newName = "Василь";

```

```

        Entrant savedEntrant =
entrantService.saveEntrant(testEntrant);

        savedEntrant.setName(newName);

        Entrant updatedEntrant =
entrantService.saveEntrant(savedEntrant);

        Entrant foundEntrant =
entrantService.findEntrantById(savedEntrant.getId());

        assertNotNull("Updated Entrant should be found by ID.",
foundEntrant);

        assertEquals("Name should be updated. Current value: '" +
foundEntrant.getName() + "'.",
            newName, foundEntrant.getName());
    }

    @Test
    void testDeleteEntrant() {
        Entrant savedEntrant =
entrantService.saveEntrant(testEntrant);

        Long id = savedEntrant.getId();

        entrantService.deleteEntrantById(savedEntrant.getId());

        assertThrows(CustomServiceException.class, () -> {
            entrantService.findEntrantById(id);
        }, "Entrant should not be found after deletion.");
    }

    @Test
    void testSaveEntrantWithDuplicateCaseNumber() {
        entrantService.saveEntrant(testEntrant);

        Entrant duplicateEntrant = new Entrant("KH22-9876",
"Шевченко", "Тарас", "Григорович", "2002-01-16", "FALSE",
"180.0");

        assertThrows(CustomServiceException.class, () -> {
            entrantService.saveEntrant(duplicateEntrant);
        });
    }

```



```

    }, "Duplicate case number should throw
CustomServiceException.");
    }
}

```

И.2 Код файла StudentServiceTest.java

```

package ntukhpi.csit.semit.riv.webappsrivlab4;

import
ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.entrant.Entrant;
import
ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.student.Student;
import
ntukhpi.csit.semit.riv.webappsrivlab4.service.CustomServiceExcepti
on;
import
ntukhpi.csit.semit.riv.webappsrivlab4.service.entity.EntrantServic
e;
import
ntukhpi.csit.semit.riv.webappsrivlab4.service.entity.StudentServic
e;
import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.context.SpringBootTest;

import static org.junit.jupiter.api.Assertions.assertThrows;
import static
org.springframework.test.util.AssertionErrors.assertEquals;
import static
org.springframework.test.util.AssertionErrors.assertNotNull;

/**

```

```

* Unit test class for the `StudentService` class.
* This class tests the functionality of the `StudentService` in
managing `Student` entities, including creating, reading,
updating, and deleting student records.
* It also checks for validation rules, such as ensuring that
duplicate corporate emails are not allowed.
* <p>
* Key functionalities tested:
* - Creating and saving a student, ensuring it has an ID after
saving.
* - Reading a student by ID and verifying that the corporate email
matches.
* - Updating a student's information, specifically updating the
corporate email.
* - Deleting a student and ensuring it cannot be found afterward.
* - Checking that attempting to save a student with a duplicate
corporate email throws an exception.
* <p>
* Each test method ensures that the `StudentService` behaves as
expected and performs the required validation and exception
handling.
* <p>
* Test annotations:
* - `@BeforeEach` to set up the test data before each test method.
* - `@AfterEach` to clean up any test data after each test method.
* <p>
* Dependencies:
* - `StudentService`: Provides methods for handling `Student`
entities.
* - `EntrantService`: Provides methods for handling `Entrant`
entities, which are related to students.
*
* @author Inessa Repeshko CS-222a
* @see Student
* @see Entrant
* @see StudentService

```

```

* @see CustomServiceException
*/

@SpringBootTest
class StudentServiceTest {

    @Autowired
    private StudentService studentService;

    @Autowired
    private EntrantService entrantService;

    private Student testStudent;
    private Entrant testEntrant;

    @BeforeEach
    void setUp() {
        testEntrant = new Entrant("KH22-9876", "Франко", "Іван",
"Якович", "2002-01-15", "TRUE", "150.0");
        testStudent = new Student(testEntrant, "бюджет",
"звичайна", "franko.ivan@cs.khpi.edu.ua");
        entrantService.saveEntrant(testEntrant);
    }

    @AfterEach
    void reset() {
        studentService.deleteStudent(testStudent.getId());
    }

    @Test
    void testCreateStudent() {
        Student savedStudent =
studentService.saveStudent(testStudent);
        assertNotNull("Student should have an ID after saving.",
savedStudent.getId());
    }

    @Test

```

```

    void testReadStudent() {
        Student savedStudent =
studentService.saveStudent(testStudent);

        Student foundStudent =
studentService.findStudentById(savedStudent.getId());

        assertNotNull("Student should be found by ID.",
foundStudent);

        assertEquals("Corporate email should match.",
            savedStudent.getCorporateEmail(),
foundStudent.getCorporateEmail());
    }

    @Test
    void testUpdateStudent() {
        String newEmail = "shevchenko.taras@cs.khpi.edu.ua";
        Student savedStudent =
studentService.saveStudent(testStudent);

        savedStudent.setCorporateEmail(newEmail);

        Student updatedStudent =
studentService.saveStudent(savedStudent);

        Student foundStudent =
studentService.findStudentById(savedStudent.getId());

        assertNotNull("Updated Student should be found by ID.",
foundStudent);

        assertEquals("Corporate email should be updated.",
            newEmail, foundStudent.getCorporateEmail());
    }

    @Test
    void testDeleteStudent() {
        Student savedStudent =
studentService.saveStudent(testStudent);

        Long id = savedStudent.getId();

        studentService.deleteStudent(savedStudent.getId());
    }

```

```

        assertThrows(CustomServiceException.class, () -> {
            studentService.findStudentById(id);
        }, "Student should not be found after deletion.");
    }

    @Test
    void testSaveStudentWithDuplicateCorporateEmail() {
        studentService.saveStudent(testStudent);
        Student duplicateStudent = new Student(testEntrant,
"Бюджет", "звичайна", "franko.ivan@cs.khpi.edu.ua");

        assertThrows(CustomServiceException.class, () -> {
            studentService.saveStudent(duplicateStudent);
        }, "Duplicate corporate email should throw
CustomServiceException.");
    }
}

```

И.3 Код файлы UserServiceTest.java

```

package ntukhpi.csit.semit.riv.webappsrivlab4;

import
ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.user.UserEntity;
import ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.enums.Role;
import
ntukhpi.csit.semit.riv.webappsrivlab4.service.CustomServiceExcepti
on;
import
ntukhpi.csit.semit.riv.webappsrivlab4.service.entity.UserService;
import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.context.SpringBootTest;

```

```

import static org.junit.jupiter.api.Assertions.assertThrows;
import static
org.springframework.test.util.AssertionErrors.assertEquals;
import static
org.springframework.test.util.AssertionErrors.assertNotNull;

/**
 * Unit test class for the `UserService` class.
 * This class tests the functionality of the `UserService` in
managing `UserEntity` entities.
 * It verifies the basic CRUD operations and ensures that
constraints like duplicate usernames or corporate emails are
enforced.
 * <p>
 * Key functionalities tested:
 * - Creating and saving a user.
 * - Reading a user by ID and verifying that the username matches.
 * - Updating a user and ensuring the changes are saved correctly.
 * - Deleting a user and ensuring it cannot be found afterward.
 * - Checking that attempting to save a user with a duplicate
corporate email throws an exception.
 * <p>
 * Each test method ensures that the `UserService` behaves as
expected and performs the required validation and exception
handling.
 * <p>
 * Test annotations:
 * - `@BeforeEach` to set up the test data before each test method.
 * - `@AfterEach` to clean up any test data after each test method.
 * <p>
 * Dependencies:
 * - `UserService`: Provides methods for handling `UserEntity`
entities.
 *
 * @author Inessa Repeshko CS-222a

```

```

* @see UserEntity
* @see UserService
* @see CustomServiceException
*/

@SpringBootTest
class UserServiceTest {

    @Autowired
    private UserService userService;

    private UserEntity testUser;

    @BeforeEach
    void setUp() {
        testUser = new UserEntity(
            "regina.marchuk",
            "Admin123!",
            Role.ADMIN,
            "regina.marchuk@cs.khpi.edu.ua",
            "Марчук",
            "Регіна",
            "Віталіївна");
    }

    @AfterEach
    void reset() {
        if (testUser.getId() != null) {
            userService.deleteUserById(testUser.getId());
        }
    }

    @Test
    void testCreateUser() {
        UserEntity savedUser = userService.saveUser(testUser);
    }
}

```

```

        assertNotNull("User should have an ID after saving.",
savedUser.getId());
    }

    @Test
    void testReadUser() {
        UserEntity savedUser = userService.saveUser(testUser);
        UserEntity foundUser =
userService.findUserById(savedUser.getId());
        assertNotNull("User should be found by ID.", foundUser);
        assertEquals("Username should match.",
            savedUser.getUsername(), foundUser.getUsername());
    }

    @Test
    void testUpdateUser() {
        String newPassword = "Newadm123!";
        UserEntity savedUser = userService.saveUser(testUser);
        savedUser.setPassword(newPassword);
        UserEntity updatedUser = userService.updateUser(savedUser);

        assertEquals("Password should be updated.",
            savedUser.getPassword(),
updatedUser.getPassword());
    }

    @Test
    void testSaveUserWithDuplicateUsername() {
        userService.saveUser(testUser);
        UserEntity duplicateUser = new UserEntity(
            "regina.marchuk",
            "Admin123!",
            Role.ADMIN,
            "regina.marchukkk@cs.khpi.edu.ua",
            "Марчук",
            "Регіна",

```



```

        "Віталіївна");

        assertThrows(CustomServiceException.class, () -> {
            userService.saveUser(duplicateUser);
        }, "Duplicate username should throw
CustomServiceException.");
    }

    @Test
    void testSaveUserWithDuplicateCorporateEmail() {
        userService.saveUser(testUser);
        UserEntity duplicateUser = new UserEntity(
            "regina.marchukkk",
            "Admin123!",
            Role.ADMIN,
            "regina.marchuk@cs.khpi.edu.ua",
            "Марчук",
            "Регіна",
            "Віталіївна");

        assertThrows(CustomServiceException.class, () -> {
            userService.saveUser(duplicateUser);
        }, "Duplicate corporate email should throw
CustomServiceException.");
    }

    @Test
    void testDeleteUser() {
        UserEntity savedUser = userService.saveUser(testUser);
        Long id = savedUser.getId();
        userService.deleteUserById(id);

        assertThrows(CustomServiceException.class, () -> {
            userService.findUserById(id);
        }, "User should not be found after deletion.");
    }

```

}

ДОДАТОК К

Код файлів представлення Spring Boot проєкту

К.1 Код файлу NavigationBar.html

```
<!DOCTYPE html>
<html lang="en" xmlns:th="http://www.thymeleaf.org"
xmlns:sec="http://www.thymeleaf.org/extras/spring-security">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width,
initial-scale=1">
    <title>Header</title>
    <link rel="icon" type="image/x-icon"
th:href="@{/image/icon/favicon_square.png}">
    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootst
rap.min.css"
rel="stylesheet"
integrity="sha384-QWTKZyjpPEjISv5WaRU90FeRpok6YctnYmDr5pNlyT2bRjXh
0JMHjY6hW+ALEwIH"
crossorigin="anonymous">
    <link rel="stylesheet"
href="https://cdn.jsdelivr.net/npm/bootstrap-icons@1.11.3/font/boo
tstrap-icons.min.css">
</head>
<body>
<header th:fragment="header">
    <nav class="container-fluid navbar navbar-expand-lg
bg-info-subtle text-info-emphasis font-monospace">
        <div class="mx-3">
```

```

        <div class="navbar-brand text-info">
            
            <span class="p-0 m-0 ms-2">Student Database
Viewer</span>
        </div>
    </div>

    <div class="mx-3">
        <button class="navbar-toggler" type="button"
data-bs-toggle="dropdown" data-bs-auto-close="outside"
            data-bs-target="#navbarNavAltMarkup"
aria-controls="navbarNavAltMarkup" aria-expanded="false"
            aria-label="Toggle navigation">
            <span class="navbar-toggler-icon"></span>
        </button>
    </div>

    <div class="collapse navbar-collapse mx-3"
id="navbarNavAltMarkup">
        <div class="navbar-nav font-monospace text-info d-flex
align-items-center justify-content-between flex-fill">
            <div class="nav-item me-5 text-info">
                <a class="nav-link link-info link-offset-0
link-underline link-underline-opacity-0
link-underline-opacity-0-hover"
                    aria-current="page" th:href="@{/home}">
                    <i class="bi bi-house-fill"></i>
                    Home
                </a>
            </div>

            <div class="nav-item dropdown me-5
align-content-center text-info">

```

```

sec:authorize="hasRole(T(ntukhpi.csit.semit.riv.webappsrivlab4.mod
el.DAO.enums.Role).ADMIN.name())">
    <div class="nav-item dropdown-toggle"
data-bs-toggle="dropdown" data-bs-auto-close="outside"
    aria-expanded="false">
        <i class="bi bi-person-fill"></i>
        Users
    </div>

    <ul class="dropdown-menu font-monospace
text-start text-secondary-emphasis">
        <li>
            <a class="dropdown-item link-info
link-offset-0 link-underline link-underline-opacity-0
link-underline-opacity-0-hover"
                th:href="@{/users}">User Table</a>
        </li>
        <li>
            <a class="dropdown-item link-info
link-offset-0 link-underline link-underline-opacity-0
link-underline-opacity-0-hover"
                th:href="@{/users/add}">Add User</a>
        </li>
    </ul>
</div>

<div class="nav-item dropdown me-5
align-content-center text-info"
    sec:authorize="hasAnyRole(
T(ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.enums.Role).ENTR
ANT_VIEWER.name(),
T(ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.enums.Role).STUD
ENT_VIEWER.name())">

```

```

        <div class="nav-item dropdown-toggle"
data-bs-toggle="dropdown" data-bs-auto-close="outside"
        aria-expanded="false">
            <i class="bi bi-table"></i>
            Tables
        </div>

        <ul class="dropdown-menu font-monospace
text-start text-secondary-emphasis">
            <li>
sec:authorize="hasRole(T(ntukhpi.csit.semit.riv.webappsrivlab4.mod
el.DAO.enums.Role).ENTRANT_VIEWER.name())">
                <a class="dropdown-item link-info
link-offset-0 link-underline link-underline-opacity-0
link-underline-opacity-0-hover"
                    th:href="@{/entrants}">Entrant
Table</a>
            </li>
            <li>
sec:authorize="hasRole(T(ntukhpi.csit.semit.riv.webappsrivlab4.mod
el.DAO.enums.Role).STUDENT_VIEWER.name())">
                <a class="dropdown-item link-info
link-offset-0 link-underline link-underline-opacity-0
link-underline-opacity-0-hover"
                    th:href="@{/students}">Student
Table</a>
            </li>
        </ul>
    </div>

    <div class="nav-item dropdown me-5
align-content-center text-info"
        sec:authorize="hasAnyRole(
T(ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.enums.Role).ENTR
ANT_MANAGER.name(),

```

```

T(ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.enums.Role).STUDENT_MANAGER.name())">
    <div class="nav-item dropdown-toggle"
data-bs-toggle="dropdown" data-bs-auto-close="outside"
    aria-expanded="false">
        <i class="bi bi-person-plus-fill"></i>
        Add Record
    </div>

    <ul class="dropdown-menu font-monospace
text-start text-secondary-emphasis">
        <li
sec:authorize="hasRole(T(ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.enums.Role).ENTRANT_MANAGER.name())">
            <a class="dropdown-item link-info
link-offset-0 link-underline link-underline-opacity-0
link-underline-opacity-0-hover"
th:href="@{/entrants/add}">Add
Entrant</a>
        </li>
        <li
sec:authorize="hasRole(T(ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.enums.Role).STUDENT_MANAGER.name())">
            <a class="dropdown-item link-info
link-offset-0 link-underline link-underline-opacity-0
link-underline-opacity-0-hover"
th:href="@{/students/add}">Add
Student</a>
        </li>
    </ul>
</div>

    <div class="nav-item dropdown ms-auto
align-content-center text-info"
sec:authorize="isAuthenticated()">

```

```

        <div class="nav-item dropdown-toggle"
data-bs-toggle="dropdown" data-bs-auto-close="outside"
        aria-expanded="false">
            <i class="bi bi-person-circle"></i>
            <span sec:authentication="name"
id="currentUsername"></span>
        </div>

        <ul class="dropdown-menu font-monospace
text-start text-secondary-emphasis">
            <li>
                <a class="dropdown-item link-info
link-offset-0 link-underline link-underline-opacity-0
link-underline-opacity-0-hover"
th:href="@{/profile}">
                    Profile
                </a>
            </li>
            <li>
                <div class="dropdown-item">
                    <form th:action="@{/logout}"
th:method="'post'" class="m-0 p-0">
                        <button type="submit"
id="logout" class="nav-link btn btn-link link-info
link-offset-0 link-underline
link-underline-opacity-0 link-underline-opacity-0-hover
m-0 p-0">
                            Logout
                        </button>
                    </form>
                </div>
            </li>
        </ul>
    </div>
</div>
</div>

```



```

    </nav>
</header>
</body>
</html>

```

К.2 Код файла Footer.html

```

<footer th:fragment="footer"
        class="container-fluid mt-5 p-3 text-center fs-6
bg-info-subtle text-info-emphasis font-monospace">
    <div class="row w-100 align-items-start justify-content-center
p-0 m-0">
        <div class="col justify-content-around" id="footer-author">
            <div class="col mb-2 justify-content-around fw-bold
text-info">Powered by</div>
            <div id="author-name">Inessa Repeshko</div>
            <div id="author-group">CS-222a</div>
            <div id="author-company">
                <span id="company-facility">CSIT </span>
                <span id="company-department">SEMIT</span><br>
                <span id="company-name">NTU "KhPI"</span>
            </div>
        </div>

        <address class="col justify-content-around"
id="footer-address">
            <div class="col mb-2 justify-content-around fw-bold
text-info">Address</div>
            <div id="address-name">National Technical University
"Kharkiv Polytechnic Institute"</div>
            <div id="address-street">2 Kyrpychova</div>
            <div>
                <span id="address-postal-code">61002</span>
                <span id="address-locality">Kharkiv</span>,
                <span id="address-region">Ukraine</span>
            </div>
        </address>
    </div>

```

```

        </div>
    </address>

    <address class="col justify-content-around"
id="footer-contact">
        <div class="col mb-2 justify-content-around fw-bold
text-info">Contact Us</div>
        <div id="contact-telephone">
            <a class="link-info link-offset-2
link-underline-opacity-25 link-underline-opacity-100-hover"
id="contact-telephone-link"
href="tel:+38(057)7076634">+38- (057) -707-66-34</a>
        </div>
        <div id="contact-email">
            <a class="link-info link-offset-2
link-underline-opacity-25 link-underline-opacity-100-hover"
id="contact-email-link"
href="mailto:omsroot@kpi.kharkov.ua">omsroot@kpi.kharkov.ua</a>
        </div>
    </address>
</div>

<div class="row w-100 align-items-center">
    <div class="col justify-content-around"
id="footer-copyright">
        <p class="p-0 m-0">Copyright &copy; 2024 All Rights
Reserved</p>
    </div>
</div>
</footer>

```

К.3 Код файла ErrorToast.html

```

<div th:fragment="errorToast" th:if="{errorMessage != null}">

```

```

    <div role="alert" class="alert alert-danger alert-dismissible
fade show z-1000 fixed-top top-0 start-50
    translate-middle-x border-2 border-danger-subtle">
    <div class="alert-heading">
        <p class="text-center fw-bold">
            <i class="bi bi-exclamation-octagon-fill"></i>
            <span th:text="{errorTitle}"></span>
        </p>
        <button type="button" class="btn-close"
data-bs-dismiss="alert" aria-label="Close"></button>
    </div>
    <hr>
    <p class="mb-0 text-center" th:text="{errorMessage}"></p>
</div>
</div>

```

К.4 Код файла WarningToast.html

```

<div th:fragment="warningToast" th:if="{warningMessage != null}">
    <div role="alert" class="alert alert-warning alert-dismissible
fade show z-1000 fixed-top top-0 start-50
    translate-middle-x border-2 border-warning-subtle">
    <div class="alert-heading">
        <p class="text-center fw-bold">
            <i class="bi bi-exclamation-triangle-fill"></i>
            <span th:text="{warningTitle}"></span>
        </p>
        <button type="button" class="btn-close"
data-bs-dismiss="alert" aria-label="Close"></button>
    </div>
    <hr>
    <p class="mb-0 text-center"
th:text="{warningMessage}"></p>
    </div>
</div>

```

К.5 Код файла SuccessToast.html

```
<div th:fragment="successToast" th:if="{successMessage != null}">
    <div role="alert" class="alert alert-success alert-dismissible
fade show z-1000 fixed-top top-0 start-50
    translate-middle-x border-2 border-success-subtle">
        <div class="alert-heading">
            <p class="text-center fw-bold">
                <i class="bi bi-check-circle-fill"></i>
                <span th:text="{successTitle}"></span>
            </p>
            <button type="button" class="btn-close"
data-bs-dismiss="alert" aria-label="Close"></button>
        </div>
        <hr>
        <p class="mb-0 text-center"
th:text="{successMessage}"></p>
    </div>
</div>
```

К.6 Код файла ErrorPage.html

```
<!DOCTYPE html>
<html lang="en" xmlns:th="http://www.thymeleaf.org"
xmlns:sec="http://www.thymeleaf.org/extras/spring-security">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width,
initial-scale=1">
    <title th:text="{errorTitle}">Error</title>
    <link rel="icon" type="image/x-icon"
th:href="{@/image/icon/favicon_square.png}">
```

```

    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootst
rap.min.css"
    rel="stylesheet"

integrity="sha384-QWTKZyjpPEjISv5WaRU90FeRpok6YctnYmDr5pNlyT2bRjXh
0JMhY6hW+ALEwIH"
    crossorigin="anonymous">
    <link rel="stylesheet"

href="https://cdn.jsdelivr.net/npm/bootstrap-icons@1.11.3/font/boo
tstrap-icons.min.css">
</head>
<body class="d-flex flex-column min-vh-100">

<div th:if="{errorCode != 401}">
    <header th:replace="{component/NavigationBar :: header}"
class="flex-shrink-0"></header>
</div>

<main class="container-fluid flex-fill d-flex align-items-center
justify-content-center">
    <div class="row w-100">
        <div class="card border border-2 border-danger-subtle
            shadow bg-danger-subtle rounded mx-auto my-auto
w-50 p-0">
            <div class="card-header bg-danger-subtle border-bottom
border-2 border-danger-subtle
                font-monospace text-center
text-secondary-emphasis fw-bolder">
                <span class="badge text-bg-danger"
th:text="{errorCode}">500</span>
                <span th:text="{errorTitle}">Error</span>
            </div>

            <div class="card-body bg-light font-monospace">

```

```

        <div class="card-text text-secondary-emphasis
fw-light text-center">
            <p th:text="{errorMessage}">Oops! Something
went wrong.</p>
            <p th:text="{errorException}"
th:if="{errorException != null}"></p>
        </div>
    </div>
</div>

    <div class="d-flex justify-content-center mt-5"
th:switch="{errorCode}">
        <button th:case="401" class="btn btn-outline-info"
type="button" onclick="window.history.back()">
            <i class="bi bi-box-arrow-in-left"></i>
        </button>
        <a th:case="*" th:href="{@{/home}}" class="btn
btn-outline-info" type="button">
            <i class="bi bi-house"></i>
        </a>
    </div>
</div>
</main>

<footer th:replace="{~{component/Footer :: footer}}"
class="flex-shrink-0"></footer>

<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstra
p.bundle.min.js"
integrity="sha384-YvpcrYf0tY3lHB60NNkmXc5s9fDVZLESaAA55NDzOxhy9Gkc
IdslK1eN7N6jIeHz"
crossorigin="anonymous"></script>
</body>
</html>

```

К.7 Код файла Login.html

```

<!DOCTYPE html>
<html lang="en" xmlns:th="http://www.thymeleaf.org"
xmlns:sec="http://www.thymeleaf.org/extras/spring-security">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width,
initial-scale=1">
    <title>Login</title>
    <link rel="icon" type="image/x-icon"
th:href="@{/image/icon/favicon_square.png}">
    <link
href="https://cdn.jsdelivrivr.net/npm/bootstrap@5.3.3/dist/css/bootst
rap.min.css"
        rel="stylesheet"
        integrity="sha384-QWTKZyjpPEjISv5WaRU90FeRpok6YctnYmDr5pNlyT2bRjXh
0JMHjY6hW+ALEwIH"
        crossorigin="anonymous">
    <link rel="stylesheet"
href="https://cdn.jsdelivrivr.net/npm/bootstrap-icons@1.11.3/font/boo
tstrap-icons.min.css">
</head>
<body>
<div th:replace="~{component/ErrorToast :: errorToast}"></div>
<div th:replace="~{component/WarningToast :: warningToast}"></div>
<div th:replace="~{component/SuccessToast :: successToast}"></div>

<div class="container-fluid vw-100 p-0 m-0 flex-column d-flex
align-content-center justify-items-center">
    <div class="row align-content-center justify-content-center
mt-2">

```

```

        <h1 class="w-50 text-info bg-info-subtle border border-3
rounded-pill border-info-subtle
            fs-1 fw-semibold font-monospace text-center
shadow-lg p-2 mb-5 mt-2 bg-body-tertiary rounded">
            
            <span class="p-0 m-0 ms-2">Student Database
Viewer</span>
        </h1>
    </div>

    <div class="row text-center justify-content-center
align-content-center">
        <div class="w-50 card border border-2 border-info-subtle
shadow mb-5 bg-body-tertiary rounded p-0 m-0">
            <div class="card-header bg-info-subtle border-bottom
border-2 border-info-subtle
                font-monospace text-center text-info
fw-bolder fs-4">
                <i class="bi bi-person-circle"></i>
                Sign In
            </div>
            <div class="card-body bg-light font-monospace p-5
pb-0">
                <form id="login-form" th:action="@{/login}"
method="POST" th:object="{user}">
                    <div class="input-group mb-5 border border-1
border-info-subtle rounded">
                        <span class="col-4 input-group-text
text-info">Username or Email</span>
                        <input type="text" class="col-auto
form-control text-dark" required
th:field="*{corporateEmailOrUsername}"
id="inputCorporateEmailOrUsername">

```



```

        </div>

        <div class="input-group mb-4 border border-1
border-info-subtle rounded">
            <span class="col-4 input-group-text
text-info">Password</span>
            <input type="password" class="col-auto
form-control text-dark" required
                th:field="*{password}"
id="inputPassword">
        </div>

        <div class="d-flex justify-content-center
mb-4">
            <div class="form-check form-switch
custom-control custom-switch ">
                <input class="form-check-input btn
btn-info bg-info border border-info focus-ring focus-ring-info
text-end"
                    style="--bs-btn-active-bg:
rgba(var(--bs-info-rgb), .25);"
                    type="checkbox" role="switch"
id="rememberMe" name="remember-me">
                <label class="form-check-label
text-info text-start" for="rememberMe">Remember me</label>
            </div>
        </div>

        <div class="d-flex justify-content-center">
            <button type="submit" class="btn
btn-outline-info" name="login">
                <i class="bi
bi-box-arrow-in-right"></i>
            </button>
        </div>
    </form>

```

```

    </div>

    <div class="card-footer bg-light font-monospace px-5
pb-4 pt-4 m-0 border-0 d-flex justify-content-evenly">
        <a class="link-info link-offset-2
link-underline-opacity-25 link-underline-opacity-100-hover"
th:href="@{/register}">
            <i class="bi bi-box-arrow-in-up-right"></i>
            New around here? Sign up
        </a>
        <a class="link-info link-offset-2
link-underline-opacity-25 link-underline-opacity-100-hover"
th:href="@{/reset-password}">
            <i class="bi bi-key"></i>
            Forgot password?
        </a>
    </div>
</div>
</div>

<div class="mt-auto p-0">
    <footer th:replace="~{component/Footer ::
footer}"></footer>
</div>
</div>

<script type="text/javascript">
    document.addEventListener("DOMContentLoaded", function () {
        setEmailInputValidity();
        setPasswordInputValidity()

        const form = document.getElementById('login-form');

        form.addEventListener('submit', function (event) {
            event.preventDefault();

```

```

        if (!form.checkValidity()) {
            event.stopPropagation();
            form.reportValidity();
        } else {
            form.submit();
        }
    });
});

function setEmailInputValidity() {
    const regexUsername = /^(?=.{8,32}$)[a-zA-Z0-9\W]+$;/;
    const regexCorporateEmail =
/^[a-z](\\\.?[a-z]{1,99})@[a-z]{1,10}\\.khpi\\.edu\\.ua$/;

    const message = "Enter corporate email or username. " +
        "The corporate email should match the format  

'name.surname@faculty.khpi.edu.ua'. " +
        "The username must contain Latin letters and may  

contain numbers and special characters '.', '_', '-.";

    document.getElementById('inputCorporateEmailOrUsername').addEventListener('input', function () {
        const value = this.value;

        // if (this.validity.patternMismatch) {
            if (!regexCorporateEmail.test(value) &&
!regexUsername.test(value)) {
                this.setCustomValidity(message);
            } else {
                this.setCustomValidity('');
            }

            this.reportValidity();
        });
    }
}

```

```

function setPasswordInputValidity() {
    const regexPassword =
/^((?=.*{8,50}$) (?=.*[A-Z]) (?=.*[0-9]) (?=.*[\W]) [A-Za-z0-9\W]*$)/;

document.getElementById('inputPassword').setAttribute('pattern',
regexPassword.source);

    const message = "The password must contain Latin letters,
at least one capital letter and one special character.";

document.getElementById('inputPassword').addEventListener('input',
function () {
    if (this.validity.patternMismatch) {
        this.setCustomValidity(message);
    } else {
        this.setCustomValidity('');
    }

    this.reportValidity();
});
}
</script>

<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstra
p.bundle.min.js"

integrity="sha384-YvpcrYf0tY3lHB60NNkmXc5s9fDVZLESaAA55NDzOxhy9Gkc
IdslK1eN7N6jIeHz"

crossorigin="anonymous"></script>
</body>
</html>

```

К.8 Код файла Register.html

```

<!DOCTYPE html>
<html lang="en" xmlns:th="http://www.thymeleaf.org"
xmlns:sec="http://www.thymeleaf.org/extras/spring-security">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width,
initial-scale=1">
    <title>Register</title>
    <link rel="icon" type="image/x-icon"
th:href="@{/image/icon/favicon_square.png}">
    <link
href="https://cdn.jsdelivrivr.net/npm/bootstrap@5.3.3/dist/css/bootst
rap.min.css"
        rel="stylesheet"
        integrity="sha384-QWTKZyjpPEjISv5WaRU90FeRpok6YctnYmDr5pNlyT2bRjXh
0JMHjY6hW+ALEwIH"
        crossorigin="anonymous">
    <link rel="stylesheet"
href="https://cdn.jsdelivrivr.net/npm/bootstrap-icons@1.11.3/font/boo
tstrap-icons.min.css">
</head>
<body>

<div th:replace="~{component/ErrorToast :: errorToast}"></div>
<div th:replace="~{component/WarningToast :: warningToast}"></div>
<div th:replace="~{component/SuccessToast :: successToast}"></div>

<div class="container-fluid vw-100 m-0 p-0 text-center
justify-content-center">
    <div class="row align-content-center justify-content-center
mt-2">

```

```

        <h1 class="w-50 text-info bg-info-subtle border border-3
rounded-pill border-info-subtle
            fs-1 fw-semibold font-monospace text-center
shadow-lg p-2 mb-1 mt-2 bg-body-tertiary rounded">
            
            <span class="p-0 m-0 ms-2">Student Database
Viewer</span>
        </h1>
    </div>

    <div class="row text-center justify-content-center
align-content-center">
        <div class="card w-50 border border-2 border-info-subtle shadow
bg-body-tertiary rounded m-0 mt-5 p-0">
            <div class="card-header bg-info-subtle border-bottom
border-2 border-info-subtle
                font-monospace text-center text-info fw-bolder
fs-4" id="formHeader">
                <i class="bi bi-person-circle"></i>
                Sign Up
            </div>

            <form class="card-body bg-light font-monospace p-5 m-0"
id="userForm"
                th:object="{user}"
                th:action="@{/register}"
                th:method="'post'">

                <div class="card-text mb-4">
                    <div class="input-group border border-1
border-info-subtle rounded">
                        <label for="inputSurname" class="col-4
input-group-text text-info">Surname</label>

```

```

        <input id="inputSurname" class="col-auto
form-control text-dark" type="text" required
        th:field="*{surname}"
        th:minlength="1" th:maxlength="50">
    </div>
</div>

<div class="card-text mb-4">
    <div class="input-group border border-1
border-info-subtle rounded">
        <label for="inputName" class="col-4
input-group-text text-info">Name</label>
        <input id="inputName" class="col-auto
form-control text-dark" type="text" required
        th:field="*{name}"
        th:minlength="1" th:maxlength="50">
    </div>
</div>

<div class="card-text mb-4">
    <div class="input-group border border-1
border-info-subtle rounded ">
        <label for="inputPatronymic" class="col-4
input-group-text text-info">Patronymic</label>
        <input id="inputPatronymic" class="col-auto
form-control text-dark" type="text" required
        th:field="*{patronymic}"
        th:minlength="1" th:maxlength="50">
    </div>
</div>

<div class="card-text mb-4">
    <div class="input-group border border-1
border-info-subtle rounded">
        <label for="inputCorporateEmail" class="col-4
input-group-text text-info">Corporate Email</label>

```

```

        <input id="inputCorporateEmail" class="col-auto
form-control text-dark" type="text" required
        th:field="*{corporateEmail}"
        th:minlength="17" th:maxlength="123">
    </div>
</div>

<div class="card-text mb-4">
    <div class="input-group border border-1
border-info-subtle rounded">
        <label for="inputUsername" class="col-4
input-group-text text-info">Username</label>
        <input id="inputUsername" class="col-auto
form-control text-dark" type="text" required
        th:field="*{username}"
        th:minlength="8" th:maxlength="32">
    </div>
</div>

<div class="card-text mb-4">
    <div class="input-group border border-1
border-info-subtle rounded">
        <label for="inputPassword" class="col-4
input-group-text text-info">Password</label>
        <input id="inputPassword" class="col-auto
form-control text-dark" type="password"
        th:field="*{password}"
        th:minlength="8" th:maxlength="50">
    </div>
</div>

<div class="card-text mb-4">
    <div class="input-group border border-1
border-info-subtle rounded">
        <label for="inputConfirmPassword" class="col-4
input-group-text text-info">Confirm Password</label>

```



```

        <input id="inputConfirmPassword"
class="col-auto form-control text-dark" type="password"
        th:field="*{confirmPassword}"
        th:minlength="8" th:maxlength="50">
    </div>
</div>

<div class="d-flex justify-content-evenly">
    <button type="submit" class="btn btn-outline-info
border-2"
        th:title="'Save'">
        <i class="bi bi-floppy"></i>
    </button>

    <button type="reset" class="btn btn-outline-info
border-2"
        onclick="form.reset()"
        th:title="'Reset fields'">
        <i class="bi bi-x-lg"></i>
    </button>
</div>
</form>
</div>
</div>

<div class="d-flex justify-content-center mt-5">
    <a class="btn btn-outline-info" type="button"
        th:title="'Go to Login'"
        th:href="@{/login}">
        <i class="bi bi-arrow-left"></i>
    </a>
</div>
</div>

<footer th:replace="~{component/Footer :: footer}"></footer>

```

```

<script type="text/javascript">
    document.addEventListener("DOMContentLoaded", function () {
        fillUsernameOnCorpEmail();
        setAllInputsValidity();
    });

    function fillUsernameOnCorpEmail() {
        const corporateEmailInput =
document.getElementById("inputCorporateEmail");
        const usernameInput =
document.getElementById("inputUsername");

        corporateEmailInput.addEventListener("input", function () {
            const emailValue = corporateEmailInput.value;
            const atIndex = emailValue.indexOf("@");

            if (atIndex > 0) {
                usernameInput.value = emailValue.substring(0,
atIndex).trim();
            } else {
                usernameInput.value = "";
            }
        });
    }

    function setAllInputsValidity() {
        setPatternAttr();
        setAllStringInputValidity();

        const form = document.getElementById('userForm');

        form.addEventListener('submit', function (event) {
            event.preventDefault();

            if (!form.checkValidity()) {
                event.stopPropagation();
            }
        });
    }

```

```

        form.reportValidity();
    } else {
        form.submit();
    }
});

function setPatternAttr() {
    const regexNames = new
RegExp("^(?={1,50}$)[А-ЩЮЯҐЄІİ](?:[А-ЯҐЄІİа-яґєіі']*[А-ЯҐЄІİа-яґєіі])?(?:[\\-][А-ЩЮЯҐЄІİ](?:[А-ЯҐЄІİа-яґєіі']*[А-ЯҐЄІİа-яґєіі])?)*$");
    const regexCorporateEmail = new
RegExp("^[a-z](\\.?[a-z]+){1,99}@[a-z]{1,10}\\.khpi\\.edu\\.ua$");
    const regexUsername = /^(?={8,32}$)[a-zA-Z0-9\\W]+$/;
    const regexPassword =
/^(?={8,50}$)(?=.*[A-Z])(?=.*[0-9])(?=.*[\\W])[A-Za-z0-9\\W]*$/;

    document.getElementById('inputSurname').setAttribute('pattern',
regexNames.source);

    document.getElementById('inputName').setAttribute('pattern',
regexNames.source);

    document.getElementById('inputPatronymic').setAttribute('pattern',
regexNames.source);

    document.getElementById('inputCorporateEmail').setAttribute('patte
rn', regexCorporateEmail.source);

    document.getElementById('inputUsername').setAttribute('pattern',
regexUsername.source);

```

```

document.getElementById('inputPassword').setAttribute('pattern',
regexPassword.source);

document.getElementById('inputConfirmPassword').setAttribute('patt
ern', regexPassword.source);
    }

    function setAllStringInputValidity() {
        const messages = [
            "Value must be Ukrainian char string between 1 and
50 chars long, including space and apostrophe.",
            "Value must be Ukrainian char string between 1 and
50 chars long, including space and apostrophe.",
            "Value must be Ukrainian char string between 1 and
50 chars long, including space and apostrophe.",
            "The corporate email should match the format
'name.surname@faculty.khpi.edu.ua'.",
            "The username must contain Latin letters and may
contain numbers and special characters '.', '_', '-'.",
            "The password must contain Latin letters, at least
one capital letter and one special character.",
            "The password must contain Latin letters, at least
one capital letter and one special character.",
        ];

        document.getElementById('userForm')
            .querySelectorAll('input[pattern]')
            .forEach((input, i) => {
                input.addEventListener('input', function () {
                    if (this.validity.patternMismatch) {
                        this.setCustomValidity(messages[i]);
                    } else {
                        this.setCustomValidity('');
                    }
                });
            });
    }

```

```

        this.reportValidity();
    });
});
}
}
</script>

<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"

integrity="sha384-YvpcrYf0tY3lHB60NNkmXc5s9fDVZLESaAA55NDzOxhy9Gkc
IdslK1eN7N6jIeHz"
crossorigin="anonymous"></script>
</body>
</html>

```

К.9 Код файла Home.html

```

<!DOCTYPE html>
<html lang="en" xmlns:th="http://www.thymeleaf.org"
xmlns:sec="http://www.thymeleaf.org/extras/spring-security">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width,
initial-scale=1">
    <title>Home</title>
    <link rel="icon" type="image/x-icon"
th:href="@{/image/icon/favicon_square.png}">
    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
rel="stylesheet"

```

```

integrity="sha384-QWTKZyjpPEjISv5WaRU9OFeRpok6YctnYmDr5pNlyT2bRjXh
0JMhY6hW+ALEwIH"
    crossorigin="anonymous">
    <link rel="stylesheet"

href="https://cdn.jsdelivr.net/npm/bootstrap-icons@1.11.3/font/boo
tstrap-icons.min.css">
</head>
<body>

<header th:replace="~{component/NavigationBar ::
header}"></header>

<div th:replace="~{component/ErrorToast :: errorToast}"></div>
<div th:replace="~{component/WarningToast :: warningToast}"></div>
<div th:replace="~{component/SuccessToast :: successToast}"></div>

<div class="container" style="flex: 90%; max-width: 90%">
    <div class="bg-info-subtle border border-3 rounded-pill
border-info-subtle
        fs-1 fw-semibold font-monospace text-info
text-center shadow-lg p-3 my-5 bg-body-tertiary rounded mb-5">
        <span th:text="{userLogin != null} ? ('Welcome, ' +
${userLogin} + '!') : _">Welcome to Student Database
Viewer!</span>
    </div>

    <div class="col m-0 p-0 d-flex flex-wrap gap-4"
        th:classappend="

${#authorization.expression('hasRole('ROLE_ENTRANT_MANAGER') or
hasRole('ROLE_STUDENT_MANAGER')')} ? 'justify-content-between'
        :

($#{#authorization.expression('hasRole('ROLE_ENTRANT_VIEWER') or

```

```

hasRole('ROLE_STUDENT_VIEWER'))} ? 'justify-content-evenly' :
    '');">
    <div class="row gap-4 align-self-start" style="flex: 33%;
max-width: 33%">
        <div class="card p-0 border border-2 border-info-subtle
shadow bg-body-tertiary rounded"
            sec:authorize="isAuthenticated()">
            <div class="card-header bg-info-subtle
border-bottom border-2 border-info-subtle
                font-monospace text-center text-info
fw-bolder">
                Software developer info
            </div>
            <div class="card-body bg-light font-monospace
text-dark fw-light">
                <p class="card-text d-flex">
                    <span class="flex-grow-1">Name:</span>
                    <span class="text-info-emphasis fw-medium
fst-italic ms-3">Repeshko Inessa</span>
                </p>
                <p class="card-text d-flex">
                    <span class="flex-grow-1">Group:</span>
                    <span class="text-info-emphasis fw-medium
fst-italic ms-3">CS-222a</span>
                </p>
                <p class="card-text d-flex">
                    <span
class="flex-grow-1">Department:</span>
                    <span class="text-info-emphasis fw-medium
fst-italic ms-3">SEMIT</span>
                </p>
                <p class="card-text d-flex">
                    <span class="flex-grow-1">Year:</span>
                    <span class="text-info-emphasis fw-medium
fst-italic ms-3">2024</span>
                </p>

```

```

        <p class="card-text d-flex">
            <span class="flex-grow-1">Page access
time:</span>

            <span class="text-info-emphasis fw-medium
fst-italic ms-3"

th:text="${#dates.format(#dates.createNow(), 'dd.MM.yyyy
HH:mm:ss')}">

                </span>
            </p>
        </div>
    </div>

    <div class="card p-0 border border-2 border-info-subtle
shadow bg-body-tertiary rounded"

sec:authorize="hasRole(T(ntukhpi.csit.semit.riv.webappsrivlab4.mod
el.DAO.enums.Role).ADMIN.name())">

        <div class="card-header bg-info-subtle
border-bottom border-2 border-info-subtle
font-monospace text-center text-info
fw-bolder">

            <a th:href="@{/users}"
class="link-info link-offset-2
link-underline-opacity-0 link-underline-opacity-75-hover">
                User Table
            </a>
        </div>

        <div class="card-body bg-light font-monospace">
            <div class="card-text text-secondary-emphasis
fw-light">

                <p>Table with data on users of the Student
Database Viewer of the

                    National Technical University "Kharkiv
Polytechnic Institute".</p>

```



```

        </div>

        <div class="border-start border-info-subtle
border-5 bg-body-secondary p-3 mb-3">
            <p class="p-0 m-0">The table contains the
following information about users:</p>
            <ul>
                <li><code
class="text-info">String</code> full name of the personal
information owner;</li>
                <li><code class="text-info">Role</code>
user role in the Student Database Viewer (enum);
                <li><code
class="text-info">String</code> username in the Student Database
Viewer;</li>
                </li>
                <li><code
class="text-info">String</code> address of corporate mail in NTU
"KhPI".</li>
            </ul>
        </div>

        <div class="d-flex justify-content-center">
            <a th:href="@{/entrants}" class="btn
btn-outline-info" role="button">
                <i class="bi bi-table"></i>
            </a>
        </div>
    </div>
</div>

    <div class="row gap-4 align-self-start" style="flex: 33%;
max-width: 33%"

```

```

sec:authorize="hasRole(T(ntukhpi.csit.semit.riv.webappsrivlab4.mod
el.DAO.enums.Role).ENTRANT_VIEWER.name())">
    <div class="card p-0 border border-2 border-info-subtle
shadow bg-body-tertiary rounded">
        <div class="card-header bg-info-subtle
border-bottom border-2 border-info-subtle
font-monospace text-center text-info
fw-bolder">
            <a th:href="@{/entrants}"
            class="link-info link-offset-2
link-underline-opacity-0 link-underline-opacity-75-hover">
                Entrant Table
            </a>
        </div>

        <div class="card-body bg-light font-monospace">
            <div class="card-text text-secondary-emphasis
fw-light">
                <p>Table with data on entrants to
                National Technical University "Kharkiv
Polytechnic Institute".</p>
            </div>

            <div class="border-start border-info-subtle
border-5 bg-body-secondary p-3 mb-3">
                <p class="p-0 m-0">The table contains the
following information about entrants:</p>
                <ul>
                    <li><code
class="text-info">String</code> case number of the personal
file;</li>
                    <li><code
class="text-info">String</code> full name of the entrant;</li>

```

```

        <li><code
class="text-info">LocalDate</code> date of birth of the
entrant;</li>

        <li><code
class="text-info">Boolean</code> gender of the entrant;</li>

        <li><code
class="text-info">Double</code> rating score of the entrant.</li>
    </ul>
</div>

    <div class="d-flex justify-content-center">
        <a th:href="@{/entrants}" class="btn
btn-outline-info" role="button">
            <i class="bi bi-table"></i>
        </a>
    </div>
</div>
</div>

    <div class="row gap-4 align-self-start" style="flex: 33%;
max-width: 33%"

    sec:authorize="hasRole(T(ntukhpi.csit.semit.riv.webappsrivlab4.mod
el.DAO.enums.Role).STUDENT_VIEWER.name())">

        <div class="card p-0 border border-2 border-info-subtle
shadow bg-body-tertiary rounded">
            <div class="card-header bg-info-subtle
border-bottom border-2 border-info-subtle
font-monospace text-center text-info
fw-bolder">
                <a th:href="@{/students}"
                    class="link-info link-offset-2
link-underline-opacity-0 link-underline-opacity-75-hover">

```

```

        Student Table
    </a>
</div>

<div class="card-body bg-light font-monospace">
    <div class="card-text text-secondary-emphasis
fw-light">
        <p>Table with data on students of
            National Technical University "Kharkiv
Polytechnic Institute".</p>
    </div>

    <div class="border-start border-info-subtle
border-5 bg-body-secondary p-3 mb-3">
        <p class="p-0 m-0">The table contains the
following information about students:</p>
        <ul>
            <li><code
class="text-info">String</code> full name of the personal
information owner;</li>
            <li><code
class="text-info">FundingType</code> funding type under the study
agreement
                (contract or budget);
            </li>
            <li><code
class="text-info">ScholarshipStatus</code> scholarship accrual
status of the
                student (none, ordinary or
enhanced);
            </li>
            <li><code
class="text-info">String</code> address of corporate mail in NTU
"KhPI".</li>
        </ul>
    </div>

```



```

<title>Profile</title>
<link rel="icon" type="image/x-icon"
th:href="@{/image/icon/favicon_square.png}">
<link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootst
rap.min.css"
rel="stylesheet"

integrity="sha384-QWTKZyjpPEjISv5WaRU9OFeRpok6YctnYmDr5pNlyT2bRjXh
0JMhjY6hW+ALEwIH"
crossorigin="anonymous">
<link rel="stylesheet"

href="https://cdn.jsdelivr.net/npm/bootstrap-icons@1.11.3/font/boo
tstrap-icons.min.css">
</head>
<body>

<header th:replace="~{component/NavigationBar ::
header}"></header>

<div th:replace="~{component/ErrorToast :: errorToast}"></div>
<div th:replace="~{component/WarningToast :: warningToast}"></div>
<div th:replace="~{component/SuccessToast :: successToast}"></div>

<div class="container w-75 text-center justify-content-center">
<div class="card border border-3 border-info-subtle shadow
bg-body-tertiary rounded m-0 mt-5 p-0">
<div class="card-header bg-info-subtle border-bottom
border-3 border-info-subtle

font-monospace text-center text-info fw-bolder
fs-4" id="formHeader">
<i class="bi bi-person-circle"></i>
Profile
</div>

```

```

    <form class="card-body bg-light font-monospace p-5 m-0"
id="profileForm"
    th:object="${user}"
    th:action="@{/profile}"
    th:method="'post'">
    <div class="d-none card-text mb-4">
        <div class="input-group border border-1
border-info-subtle rounded">
            <label for="inputUserId" class="col-4
input-group-text text-info">Id</label>
            <input id="inputUserId" class="col-auto
form-control text-secondary" type="hidden" required
                th:field="*{id}"
                th:minlength="1" th:maxlength="100"
                th:readonly="${true}">
        </div>
    </div>

    <div class="d-none card-text mb-4">
        <div class="input-group border border-1
border-info-subtle rounded">
            <label for="inputSurname" class="col-3
input-group-text text-info">Surname</label>
            <input id="inputSurname" class="col-auto
form-control text-secondary" type="hidden" required
                th:field="*{surname}"
                th:minlength="1" th:maxlength="50"
                th:readonly="${true}">
        </div>
    </div>

    <div class="d-none card-text mb-4">
        <div class="input-group border border-1
border-info-subtle rounded">
            <label for="inputName" class="col-3
input-group-text text-info">Name</label>

```

```

        <input id="inputName" class="col-auto
form-control text-secondary" type="hidden" required
        th:field="*{name}"
        th:minlength="1" th:maxlength="50"
        th:readonly="{true}">

    </div>
</div>

    <div class="d-none card-text mb-4">
        <div class="input-group border border-1
border-info-subtle rounded ">
            <label for="inputPatronymic" class=" col-3
input-group-text text-info">Patronymic</label>
            <input id="inputPatronymic" class="col-auto
form-control text-secondary" type="hidden" required
            th:field="*{patronymic}"
            th:minlength="1" th:maxlength="50"
            th:readonly="{true}">

        </div>
    </div>

    <div class="card-text mb-4">
        <div class="input-group border border-1
border-info-subtle rounded ">
            <label for="inputFullname" class=" col-4
input-group-text text-info">Full Name</label>
            <input id="inputFullname" class="col-auto
form-control text-secondary" type="text" required
            th:name="'fullname'"
            th:value="{user?.surname + ' ' +
user?.name + ' ' + user?.patronymic}"
            th:minlength="1" th:maxlength="200"
            th:readonly="{true}">

        </div>
    </div>

```



```

        <div class="card-text mb-4">
            <div class="input-group border border-1
border-info-subtle rounded">
                <label for="inputCorporateEmail" class="col-4
input-group-text text-info">Corporate Email</label>
                <input id="inputCorporateEmail" class="col-auto
form-control text-secondary" type="text" required
                    th:field="*{corporateEmail}"
                    th:minlength="17" th:maxlength="123"
                    th:readonly="{true}">
            </div>
        </div>

        <div class="card-text mb-4">
            <div class="input-group border border-1
border-info-subtle rounded">
                <label for="inputUsername" class="col-4
input-group-text text-info">Username</label>
                <input id="inputUsername" class="col-auto
form-control text-secondary" type="text" required
                    th:field="*{username}"
                    th:minlength="8" th:maxlength="32"
                    th:readonly="{true}">
            </div>
        </div>

        <div class="card-text d-flex align-content-center
justify-content-start">
            <div class="col-4 border border-1
border-info-subtle rounded d-inline-flex p-0 m-0 me-3">
                <span class="d-flex flex-fill input-group-text
text-info">Role</span>
            </div>
            <div class="col-auto text-center
align-content-center" th:switch="{user?.role?.name()}">

```

```

        <button
th:case="${T(ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.enums
.Role).ADMIN.name()}"
        th:text="${user?.role?.getValue()}"
        style="color: #fd7e14; border-color:
#fd7e14;"
        class="btn btn-outline-info border-2
fw-bolder"
        disabled
data-bs-toggle="button"></button>
        <button
th:case="${T(ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.enums
.Role).ENTRANT_MANAGER.name()}"
        th:text="${user?.role?.getValue()}"
        style="color: #198754; border-color:
#198754;"
        class="btn btn-outline-info border-2
fw-bolder"
        disabled
data-bs-toggle="button"></button>
        <button
th:case="${T(ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.enums
.Role).STUDENT_MANAGER.name()}"
        th:text="${user?.role?.getValue()}"
        style="color: #6f42c1; border-color:
#6f42c1;"
        class="btn btn-outline-info border-2
fw-bolder"
        disabled
data-bs-toggle="button"></button>
        <button
th:case="${T(ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.enums
.Role).ENTRANT_VIEWER.name()}"
        th:text="${user?.role?.getValue()}"
        style="color: #20c997; border-color:
#20c997;"

```

```

        class="btn btn-outline-info border-2
fw-bolder"

        disabled
data-bs-toggle="button"></button>

        <button
th:case="${T(ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.enums
.Role).STUDENT_VIEWER.name()}"

        th:text="${user?.role?.getValue()}"
        style="color: #6610f2; border-color:
#6610f2;"

        class="btn btn-outline-info border-2
fw-bolder"

        disabled
data-bs-toggle="button"></button>

    </div>
</div>

    <div class="d-flex justify-content-evenly mt-5">
        <a th:href="@{/profile/change-credentials}"
type="button" class="btn btn-outline-info border-2"
        th:title="'Change Credentials'">
            <i class="bi bi-key"></i>
            Change Credentials
        </a>
    </div>
</form>
</div>

<div class="d-flex justify-content-center mt-5">
    <a class="btn btn-outline-info" type="button"
        th:title="'Go to Home'"
        th:href="@{/home}">
        <i class="bi bi-house"></i>
    </a>
</div>
</div>

```

```

<footer th:replace="~{component/Footer :: footer}"></footer>

<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstra
p.bundle.min.js"

integrity="sha384-YvpcrYf0tY3lHB60NNkmXc5s9fDVZLESaAA55NDzOxhy9Gkc
Ids1K1eN7N6jIeHz"

    crossorigin="anonymous"></script>
</body>
</html>

```

К.11 Код файла ChangeCredentialsForm.html

```

<!DOCTYPE html>
<html lang="en" xmlns:th="http://www.thymeleaf.org"
xmlns:sec="http://www.thymeleaf.org/extras/spring-security">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width,
initial-scale=1">
    <title>Change Credentials</title>
    <link rel="icon" type="image/x-icon"
th:href="@{/image/icon/favicon_square.png}">
    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootst
rap.min.css"
        rel="stylesheet"

integrity="sha384-QWTKZyjpPEjISv5WaRU90FeRpok6YctnYmDr5pNlyT2bRjXh
0JMHjY6hW+ALEwIH"

        crossorigin="anonymous">
    <link rel="stylesheet"

```

```

href="https://cdn.jsdelivr.net/npm/bootstrap-icons@1.11.3/font/boo
tstrap-icons.min.css">
</head>
<body>

<header th:replace="~{component/NavigationBar ::
header}"></header>

<div th:replace="~{component/ErrorToast :: errorToast}"></div>
<div th:replace="~{component/WarningToast :: warningToast}"></div>
<div th:replace="~{component/SuccessToast :: successToast}"></div>

<div class="container w-75 text-center justify-content-center">
    <div class="card border border-3 border-info-subtle shadow
bg-body-tertiary rounded m-0 mt-5 p-0">
        <div class="card-header bg-info-subtle border-bottom
border-3 border-info-subtle
            font-monospace text-center text-info fw-bolder
fs-4" id="formHeader">
            <i class="bi bi-unlock-fill"></i>
            Change Credentials
        </div>

        <form class="card-body bg-light font-monospace p-5 m-0"
id="changeCredentialsForm"
            th:object="{userChangeCredentials}"
            th:action="@{/profile/change-credentials}"
            th:method="'post'">
            <div class="card-text mb-4">
                <div class="input-group border border-1
border-info-subtle rounded">
                    <label for="inputUsername" class="col-3
input-group-text text-info">Username</label>
                    <input id="inputUsername" class="col-auto
form-control text-dark" required

```

```

        type="text" name="username"
th:minLength="8" th:maxLength="32"
        th:field="{username}">
    </div>
</div>

<div class="card-text mb-4">
    <div class="input-group border border-1
border-info-subtle rounded">
        <label for="inputPassword" class="col-3
input-group-text text-info">Password</label>
        <input id="inputPassword" class="col-auto
form-control text-dark" required
            type="password" name="password"
th:minLength="8" th:maxLength="50"
            th:field="{password}">
    </div>
</div>

<div class="card-text mb-4">
    <div class="input-group border border-1
border-info-subtle rounded">
        <label for="inputConfirmPassword" class="col-3
input-group-text text-info">Confirm Password</label>
        <input id="inputConfirmPassword"
class="col-auto form-control text-dark" required
            type="password" name="confirmPassword"
th:minLength="8" th:maxLength="50"
            th:field="{confirmPassword}">
    </div>
</div>

<div class="d-flex justify-content-evenly">
    <button type="submit" class="btn btn-outline-info
border-2"
        th:title="'Save'">

```

```

        <i class="bi bi-floppy"></i>
    </button>

    <button type="reset" class="btn btn-outline-info
border-2"

        onclick="form.reset() "
        th:title="'Reset fields'">
        <i class="bi bi-x-lg"></i>
    </button>
</div>
</form>
</div>

<div class="d-flex justify-content-center mt-5">
    <a class="btn btn-outline-info" type="button"
        th:title="'Go to Home'"
        th:href="@{/home}">
        <i class="bi bi-arrow-left"></i>
    </a>
</div>
</div>

<footer th:replace="~{component/Footer :: footer}"></footer>

<script type="text/javascript">
    document.addEventListener("DOMContentLoaded", function () {
        setPatternAttr();
        setAllStringInputValidity();

        const form =
document.getElementById('changeCredentialsForm');

        form.addEventListener('submit', function (event) {
            event.preventDefault();

            if (!form.checkValidity()) {

```

```

        event.stopPropagation();
        form.reportValidity();
    } else {
        form.submit();
    }
});

document.getElementById('inputUsername').setAttribute('value',
document.getElementById('currentUsername').innerText);

});

function setPatternAttr() {
    const regexUsername = /^(?=.{8,32}$)[a-zA-Z0-9\W]+$;/;
    const regexPassword =
/^(?=.{8,50}$)(?=.*[A-Z])(?=.*[0-9])(?=.*[\W])[A-Za-z0-9\W]*$/;

document.getElementById('inputUsername').setAttribute('pattern',
regexUsername.source);

document.getElementById('inputPassword').setAttribute('pattern',
regexPassword.source);

document.getElementById('inputConfirmPassword').setAttribute('patt
ern', regexPassword.source);
}

function setAllStringInputValidity() {
    const messages = [
        "The username must contain Latin letters and may
contain numbers and special characters '.', '_', '-'.",
        "The password must contain Latin letters, at least one
capital letter and one special character.",

```



```

        "The password must contain Latin letters, at least one
        capital letter and one special character.",
    ];

    document.getElementById('changeCredentialsForm')
        .querySelectorAll('input[pattern]')
        .forEach((input, i) => {
            input.addEventListener('input', function () {
                if (this.validity.patternMismatch) {
                    this.setCustomValidity(messages[i]);
                } else {
                    this.setCustomValidity('');
                }

                this.reportValidity();
            });
        });
    }
</script>

<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap
p.bundle.min.js"

integrity="sha384-YvpcrYf0tY3lHB60NNkmXc5s9fDVZLESaAA55NDzOxhy9Gkc
IdslK1eN7N6jIeHz"

crossorigin="anonymous"></script>
</body>
</html>

```

К.12 Код файла EntrantForm.html

```

<!DOCTYPE html>
<html lang="en" xmlns:th="http://www.thymeleaf.org"
xmlns:sec="http://www.thymeleaf.org/extras/spring-security">

```

```

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width,
initial-scale=1">
  <title th:text="${mode.getValue() + ' Entrant'}">Entrant
Form</title>
  <link rel="icon" type="image/x-icon"
th:href="@{/image/icon/favicon_square.png}">
  <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootst
rap.min.css"
      rel="stylesheet"
      integrity="sha384-QWTKZyjpPEjISv5WaRU90FeRpok6YctnYmDr5pNlyT2bRjXh
0JMhY6hW+ALEwIH"
      crossorigin="anonymous">
  <link rel="stylesheet"
href="https://cdn.jsdelivr.net/npm/bootstrap-icons@1.11.3/font/boo
tstrap-icons.min.css">
</head>
<body>

<header th:replace="~{component/NavigationBar ::
header}"></header>

<div th:replace="~{component/ErrorToast :: errorToast}"></div>
<div th:replace="~{component/WarningToast :: warningToast}"></div>
<div th:replace="~{component/SuccessToast :: successToast}"></div>

<div class="container w-50 text-center justify-content-center">
  <div class="card border border-3 border-info-subtle shadow
bg-body-tertiary rounded m-0 mt-5 p-0"
      th:with="isEditOrDelete = ${mode eq
T(ntukhpi.csit.semit.riv.webappsrivlab4.controller.Mode).EDIT

```

```

                                or mode eq
T(ntukhpi.csit.semit.riv.webappsrivlab4.controller.Mode).DELETE},
                                isDeleteOrView = ${mode eq
T(ntukhpi.csit.semit.riv.webappsrivlab4.controller.Mode).DELETE
                                or mode eq
T(ntukhpi.csit.semit.riv.webappsrivlab4.controller.Mode).VIEW_FORM
},
                                isNotDeleteAndNotView = ${mode ne
T(ntukhpi.csit.semit.riv.webappsrivlab4.controller.Mode).EDIT
                                and mode ne
T(ntukhpi.csit.semit.riv.webappsrivlab4.controller.Mode).VIEW_FORM
}">

    <div class="card-header bg-info-subtle border-bottom
border-3 border-info-subtle
        font-monospace text-center text-info fw-bolder"
        th:text="${mode.getValue() + ' Entrant'}">Entrant Form
    </div>

    <form class="card-body bg-light font-monospace p-5 m-0"
id="entrantForm"
        th:object="${entrant}"
        th:with="actionUrl=
                                ${mode eq
T(ntukhpi.csit.semit.riv.webappsrivlab4.controller.Mode).ADD ?
'/entrants/add' :
                                (mode eq
T(ntukhpi.csit.semit.riv.webappsrivlab4.controller.Mode).VIEW_FORM
? '/entrants/view' :
                                (mode eq
T(ntukhpi.csit.semit.riv.webappsrivlab4.controller.Mode).EDIT ?
('/entrants/' + entrant?.id + '/edit') :
                                (mode eq
T(ntukhpi.csit.semit.riv.webappsrivlab4.controller.Mode).DELETE ?
('/entrants/' + entrant?.id + '/delete') : '')}}}"
        th:action="@{${actionUrl}}" th:method="'post'">

```

```

        <div class="d-none card-text mb-4">
            <div class="input-group border border-1
border-info-subtle rounded ">
                <label for="inputEntrantId" class="col-3
input-group-text text-info">Id</label>
                <input id="inputEntrantId" class="col-auto
form-control text-dark" type="hidden" required
                    th:field="*{id}" th:minlength="1"
th:maxlength="100"
                    th:value="{isEditOrDelete ? entrant?.id
: null}"
                    th:readonly="{isDeleteOrView}">
            </div>
        </div>

        <div class="card-text mb-4">
            <div class="input-group border border-1
border-info-subtle rounded ">
                <label for="inputCaseNumber" class="col-3
input-group-text text-info">Case Number</label>
                <input id="inputCaseNumber" class="col-auto
form-control text-dark" type="text" required
                    th:field="*{caseNumber}"
th:minlength="5" th:maxlength="20"
                    th:readonly="{isDeleteOrView}">
            </div>
        </div>

        <div class="card-text mb-4">
            <div class="input-group border border-1
border-info-subtle rounded ">
                <label for="inputSurname" class="col-3
input-group-text text-info">Surname</label>
                <input id="inputSurname" class="col-auto
form-control text-dark" type="text" required

```

```

th:field="*{surname}" th:minlength="1"
th:maxlength="50"
th:readonly="{isDeleteOrView}">
    </div>
</div>

<div class="card-text mb-4">
    <div class="input-group border border-1
border-info-subtle rounded ">
        <label for="inputName" class=" col-3
input-group-text text-info">Name</label>
        <input id="inputName" class="col-auto
form-control text-dark" type="text" required
th:field="*{name}" th:minlength="1"
th:maxlength="50"
th:readonly="{isDeleteOrView}">
    </div>
</div>

<div class="card-text mb-4">
    <div class="input-group border border-1
border-info-subtle rounded ">
        <label for="inputPatronymic" class=" col-3
input-group-text text-info">Patronymic</label>
        <input id="inputPatronymic" class="col-auto
form-control text-dark" type="text" required
th:field="*{patronymic}"
th:minlength="1" th:maxlength="50"
th:readonly="{isDeleteOrView}">
    </div>
</div>

<div class="card-text mb-4">
    <div class="input-group border border-1
border-info-subtle rounded ">

```

```

        <label for="inputBirthday" class=" col-3
input-group-text text-info">Birthday</label>

        <input id="inputBirthday" class="col-auto
form-control text-dark" type="date" required
            th:name="${'birthday'}"

th:value="${#temporals.format(entrant?.birthday, 'yyyy-MM-dd')}"
            th:readonly="${isDeleteOrView}">

    </div>
</div>

    <div class="card-text d-flex align-content-center
justify-content-start mb-4">

        <div class="col-3 border border-1
border-info-subtle rounded d-inline-flex p-0 m-0 me-3">

            <span class="d-flex flex-fill input-group-text
text-info">Gender</span>

        </div>

        <div class="col-auto btn-group" role="group"
aria-label="Gender radio toggle button group">

            <input type="hidden"
                th:field="*{gender}"
                th:value="${entrant?.gender}"
                th:if="${mode eq
T(ntukhpi.csit.semit.riv.webappsrivlab4.controller.Mode).DELETE}">

            <input id="inputGenderMale" type="radio"
class="btn-check" autocomplete="off"
                th:field="*{gender}"
                th:value="${true}"
                th:disabled="${isDeleteOrView}">

            <label class="btn btn-outline-info"
for="inputGenderMale">Male</label>

```

```

        <input id="inputGenderFemale" type="radio"
class="btn-check" autocomplete="off"
        th:field="*{gender}"
        th:value="{false}"
        th:disabled="{isDeleteOrView}">
        <label class="btn btn-outline-info"
for="inputGenderFemale">Female</label>
    </div>
</div>

<div class="card-text mb-4">
    <div class="input-group border border-1
border-info-subtle rounded ">
        <label for="inputRatingScore" class="col-3
input-group-text text-info">Rating score</label>
        <input id="inputRatingScore" class="col-auto
form-control text-dark" type="number" required
        th:min="120.001" th:max="200.000"
th:step="0.001"
        th:field="*{ratingScore}"
        th:readonly="{isDeleteOrView}">
    </div>
</div>

<div class="d-flex justify-content-evenly">
    <button type="submit" class="btn btn-outline-info
border-2"
        th:title="'Save record'"
        th:if="{isNotDeleteAndNotView}">
        <i class="bi bi-floppy"></i>
    </button>

    <button type="reset" class="btn btn-outline-info
border-2"
        onclick="form.reset()"
        th:title="'Reset fields'"

```

```

        th:if="{isNotDeleteAndNotView}">
        <i class="bi bi-x-lg"></i>
    </button>

    <button type="button" class="btn btn-outline-info
border-2"

        onclick="submitConfirmation()"
        th:title="'Delete record'"
        th:if="{mode eq
T(ntukhpi.csit.semit.riv.webappsrivlab4.controller.Mode).DELETE}">
        <i class="bi bi-trash3"></i>
    </button>
</div>
</form>
</div>

<div class="d-flex justify-content-center mt-5">
    <button class="btn btn-outline-info me-5" type="button"
        th:title="'Go Back'"
        onclick="window.history.back()"
        th:if="{mode eq
T(ntukhpi.csit.semit.riv.webappsrivlab4.controller.Mode).VIEW_FORM
}">

        <i class="bi bi-box-arrow-in-left"></i>
    </button>

    <a class="btn btn-outline-info" type="button"
        th:title="'Go to Entrant Table'"
        th:href="@{/entrants}">
        <i class="bi bi-table"></i>
    </a>
</div>
</div>

<footer th:replace="~{component/Footer :: footer}"></footer>

```



```

<script type="text/javascript">
    document.addEventListener("DOMContentLoaded", function () {
        setAllInputsValidity();
    });

    function setAllInputsValidity() {
        setPatternAttr();
        setBirthdayLimit();

        setStringInputValidity();
        setBirthdayValidity();
        setRatingScoreValidity();

        const form = document.getElementById('entrantForm');

        form.addEventListener('submit', function (event) {
            event.preventDefault();

            if (!form.checkValidity()) {
                event.stopPropagation();
                form.reportValidity();
            } else {
                form.submit();
            }
        });

        function setPatternAttr() {
            const regexCaseNumber = new
RegExp ("^(?=.{5,20}$) [А-ЩЮЯҒҒІİ]{1,10} \\d{2} [\\-] \\d{1,4}$");
            const regexNames = new
RegExp ("^(?=.{1,50}$) [А-ЩЮЯҒҒІİ] (?:[А-ЯҒҒІİа-яҒҒіі']*[А-ЯҒҒІİа-яҒҒіі
і])?(?:[
\\-] [А-ЩЮЯҒҒІİ] (?:[А-ЯҒҒІİа-яҒҒіі']*[А-ЯҒҒІİа-яҒҒіі])?)*$");

```

```

document.getElementById('inputCaseNumber').setAttribute('pattern',
regexCaseNumber.source);

document.getElementById('inputSurname').setAttribute('pattern',
regexNames.source);

document.getElementById('inputName').setAttribute('pattern',
regexNames.source);

document.getElementById('inputPatronymic').setAttribute('pattern',
regexNames.source);
    }

    function setBirthdayLimit() {

document.getElementById('inputBirthday').setAttribute('min',
getBirthdayMinValue());

document.getElementById('inputBirthday').setAttribute('max',
getBirthdayMaxValue());

        function getBirthdayMinValue() {
            const today = new Date();
            return new Date(today.getFullYear() - 110,
today.getMonth(), today.getDate()).toISOString().split('T')[0];
        }

        function getBirthdayMaxValue() {
            const today = new Date();
            return new Date(today.getFullYear() - 16,
today.getMonth(), today.getDate()).toISOString().split('T')[0];
        }
    }

    function setBirthdayValidity() {

```

```

        const regexpDate = new
RegExp("(?=.{10}$)\\d{4}\\-\\d{2}\\-\\d{2}");
        const messages = [
            "Birthday cannot be empty.",
            "Value must be date from 110 to 16 years ago
today."
        ];

document.getElementById('inputBirthday').addEventListener('submit'
, function () {
    const minDate = new Date(this.min);
    const maxDate = new Date(this.max);
    const selectedDate = new Date(this.value);

    if (!regexpDate.test(selectedDate.toString())) {
        this.setCustomValidity(messages[0]);
    } else if (selectedDate < minDate || selectedDate >
maxDate) {
        this.setCustomValidity(messages[1]);
    } else {
        this.setCustomValidity("");
    }

    this.reportValidity();
});
}

function setRatingScoreValidity() {
    const message = "Value must be decimal number in range
from 120.001 to 200.000 inclusive.";

document.getElementById('inputRatingScore').addEventListener('inpu
t', function () {
    const minVal = parseFloat(this.min);

```

```

        const maxVal = parseFloat(this.max);
        const selectedVal = parseFloat(this.value);

        if (selectedVal < minVal || selectedVal > maxVal) {
            this.setCustomValidity(message);
        } else {
            this.setCustomValidity("");
        }

        this.reportValidity();
    });
}

function setStringInputValidity() {
    const messages = [
        "Value must be Ukrainian char string between 5 and
20 chars long in format 'LLLLLLLLLLNN-NNNN'.",
        "Value must be Ukrainian char string between 1 and
50 chars long, including space and apostrophe.",
        "Value must be Ukrainian char string between 1 and
50 chars long, including space and apostrophe.",
        "Value must be Ukrainian char string between 1 and
50 chars long, including space and apostrophe."
    ];

    document.getElementById('entrantForm')
        .querySelectorAll('input[pattern]')
        .forEach((input, i) => {
            input.addEventListener('input', function () {
                if (this.validity.patternMismatch) {
                    this.setCustomValidity(messages[i]);
                } else {
                    this.setCustomValidity('');
                }
            });

            this.reportValidity();
        });
}

```

```

        });
    });
}

}

function submitConfirmation() {
    const message = 'Are you sure you want to delete that
record?';

    if (confirm(message) === true) {
        document.getElementById('entrantForm').submit();
    }
}
</script>

<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap
p.bundle.min.js"

integrity="sha384-YvpcrYf0tY3lHB60NNkmXc5s9fDVZLESaAA55NDzOxhy9Gkc
IdslK1eN7N6jIeHz"

crossorigin="anonymous"></script>

</body>
</html>

```

К.13 Код файла EntrantTable.html

```

<!DOCTYPE html>
<html lang="en" xmlns:th="http://www.thymeleaf.org"
xmlns:sec="http://www.thymeleaf.org/extras/spring-security">
<head>
    <meta charset="UTF-8">

```

```

    <meta name="viewport" content="width=device-width,
initial-scale=1">
    <title>Entrant Table</title>
    <link rel="icon" type="image/x-icon"
th:href="@{/image/icon/favicon_square.png}">
    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootst
rap.min.css"
        rel="stylesheet"
integrity="sha384-QWTKZyjpPEjISv5WaRU90FeRpok6YctnYmDr5pNlyT2bRjXh
0JMHjY6hW+ALEwIH"
        crossorigin="anonymous">
    <link rel="stylesheet"
href="https://cdn.jsdelivr.net/npm/bootstrap-icons@1.11.3/font/boo
tstrap-icons.min.css">
</head>
<body>

<header th:replace="~{component/NavigationBar ::
header}"></header>

<div th:replace="~{component/ErrorToast :: errorToast}"></div>
<div th:replace="~{component/WarningToast :: warningToast}"></div>
<div th:replace="~{component/SuccessToast :: successToast}"></div>

<div class="mx-3 mt-3 mb-5 font-monospace">
    <div class="row">
        <h1 class="text-info fs-1 my-3 fw-semibold
text-center">Entrant Table</h1>
    </div>

    <div class="row align-items-start">
        <div class="col-2 mx-2">

```

```

        <div class="card row p-2 mb-4 border border-3
border-info-subtle bg-light shadow">
            <div class="card-body col">
                <div class="card-title row">
                    <p class="text-info fs-5 fw-bold p-0
m-0">Search</p>
                    <p class="text-secondary fw-lighter p-0
m-0" style="font-size: 0.8rem;">
                        Search by case number, full name
(matching start, case-insensitive)
                    </p>
                </div>

                <form class="card-text col mt-3"
id="formSearch"
                    th:method="'get'"
th:action="@{/entrants}">
                    <div class="row mb-3">
                        <label for="inputSearch"
class="visually-hidden">Search</label>
                        <input type="search"
class="form-control border border-2 border-info-subtle"
                            id="inputSearch" name="search"
placeholder="Type to search...">
                    </div>

                    <div class="col d-flex
justify-content-evenly align-content-center text-center p-0 m-0">
                        <a type="button" id="btn-search"
onclick="applySearch(this) "
                            class="btn btn-outline-info
border-2" th:alt-title="'Apply Search'">
                            <i class="bi bi-search"></i>
                        </a>

```

```

        <a type="button" id="btn-search-reset"
onclick="resetSearch(this)"
        class="btn btn-outline-info border-2
button-reset" th:title="'Reset Search'">
            <i class="bi bi-x-lg"></i>
        </a>
    </div>
</form>
</div>
</div>

<div class="card row p-2 mb-4 border border-3
border-info-subtle bg-light shadow">
    <div class="card-body col">
        <div class="card-title row">
            <p class="text-info fs-5 fw-bold p-0
m-0">Filters</p>
        </div>

        <div class="card-subtitle row mt-2"
id="filterBirthday">
            <p class="text-info fs-6 fw-bold p-0
m-0">Birthday</p>
        </div>

        <form class="card-text col mt-2"
id="formFilterBirthday"
            th:method="'get'"
th:action="@{/entrants}">
            <div class="row mb-2">
                <label for="inputBirthdayStart"
class="form-label text-secondary fw-lighter py-0 px-1 m-0"
                    style="font-size:
0.8rem;">Start</label>
                <input type="date" class="form-control
border border-2 border-info-subtle"

```



```

                                id="inputBirthdayStart"
name="birthdayStart">
                                </div>

                                <div class="row mb-3">
                                    <label for="inputBirthdayEnd"
class="form-label text-secondary fw-lighter py-0 px-1 m-0"
                                style="font-size:
0.8rem;">End</label>
                                    <input type="date" class="form-control
border border-2 border-info-subtle"
                                id="inputBirthdayEnd"
name="birthdayEnd">
                                    </div>

                                <div class="col d-flex
justify-content-evenly align-content-center text-center p-0 m-0">
                                    <a type="button"
id="btn-filter-birthday" onclick="applyFilterBirthday(this)"
                                class="btn btn-outline-info
border-2" th:title="'Apply Filter'">
                                        <i class="bi bi-funnel"></i>
                                    </a>

                                    <a type="button"
id="btn-filter-birthday-reset" onclick="resetFilterBirthday(this)"
                                class="btn btn-outline-info border-2
button-reset" th:title="'Reset Filter'">
                                        <i class="bi bi-x-lg"></i>
                                    </a>
                                </div>
                                </form>

                                <div class="card-subtitle row mt-4"
id="filterRatingScore">

```

```

        <p class="text-info fs-6 fw-bold p-0
m-0">RatingScore</p>
    </div>

    <form class="card-text col mt-2"
id="formFilterRatingScore"
        th:method="'get'"
th:action="@{/entrants}">
        <div class="row mb-2">
            <label for="inputRatingScoreMin"
class="form-label text-secondary fw-lighter py-0 px-1 m-0"
                style="font-size:
0.8rem;">Start</label>
            <input type="number"
class="form-control border border-2 border-info-subtle"
                id="inputRatingScoreMin"
name="ratingScoreMin"
                th:min="120.001"
th:max="200.000" th:step="0.001">
            </div>

            <div class="row mb-3">
                <label for="inputRatingScoreMax"
class="form-label text-secondary fw-lighter py-0 px-1 m-0"
                    style="font-size:
0.8rem;">End</label>
                <input type="number"
class="form-control border border-2 border-info-subtle"
                    id="inputRatingScoreMax"
name="ratingScoreMax"
                    th:min="120.001"
th:max="200.000" th:step="0.001">
                </div>

            <div class="col d-flex
justify-content-evenly align-content-center text-center p-0 m-0">

```

```

        <a type="button"
id="btn-filter-rating-score"
onclick="applyFilterRatingScore(this)"
        class="btn btn-outline-info
border-2" th:title="'Apply Filter'">
            <i class="bi bi-funnel"></i>
        </a>

        <a type="button"
id="btn-filter-rating-score-reset"
onclick="resetFilterRatingScore(this)"
        class="btn btn-outline-info border-2
button-reset" th:title="'Reset Filter'">
            <i class="bi bi-x-lg"></i>
        </a>
    </div>
</form>
</div>
</div>

<div class="card row p-2 mb-4 border border-3
border-info-subtle bg-light shadow"

sec:authorize="hasRole(T(ntukhpi.csit.semit.riv.webappsrivlab4.mod
el.DAO.enums.Role).ENTRANT_MANAGER.name())">
    <div class="card-body col">
        <div class="card-title row">
            <p class="text-info fs-5 fw-bold p-0
m-0">Add record</p>
            <p class="text-secondary fw-lighter p-0
m-0" style="font-size: 0.8rem;">
                Add new record to the end of the table
            </p>
        </div>
    </div>

```

```

        <form class="card-text col mt-3"
id="formInsert"
        th:method="'get'"
th:action="@{/entrants}">
        <div class="col d-flex
justify-content-evenly align-content-center text-center p-0 m-0">
            <a th:href="@{/entrants/add}"
id="btn-add-entrant"
                class="btn btn-outline-info
border-2" role="button" th:title="'Add record'">
                <i class="bi bi-plus-lg"></i>
            </a>
        </div>
    </form>
</div>
</div>
</div>

<div class="col">
    <table id="entrantTable" class="table table-sm
table-hover font-monospace
        table-bordered border border-3
border-info-subtle shadow-lg">
        <thead class="table-info text-center fw-medium
text-secondary-emphasis">
            <tr>
                <th class="text-center align-content-center
text-info">#</th>

                <div class="p-0 m-0" th:each="fieldName :
${fields}">
                    <th class="text-center
align-content-center">
                        <div class="d-flex p-0 m-0">
                            <div class="flex-grow-1
align-content-center justify-content-center p-0 m-0">

```

```

                <span class="">
                    <a class="sortLink
link-info link-offset-2 link-offset-3-hover link-underline
link-underline-opacity-0 link-underline-opacity-75-hover"
                th:id="${fieldName[0]}"
th:text="${fieldName[1]}"

onclick="applySort(this)"></a>

                </span>
            </div>
            <div class="d-none flex-shrink-1
align-content-center justify-content-end ms-2 p-0 m-0"
                style="width: 20px;">
                <button type="button"
class="btn bg-transparent p-0 m-0" title="Sort records">
                    <i class="bi
bi-sort-down-alt sortingAsc" th:title="'Ascending Sort'"></i>
                    <i class="bi bi-sort-down
sortingDesc" th:title="'Descending Sort'"></i>
                </button>
            </div>
        </div>
    </th>
</div>

    <th class="text-center align-content-center
text-info"

sec:authorize="hasRole(T(ntukhpi.csit.semit.riv.webappsrivlab4.mod
el.DAO.enums.Role).ENTRANT_MANAGER.name())
                and
hasRole(T(ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.enums.Ro
le).STUDENT_VIEWER.name())">Operations</th>

    <th class="text-center align-content-center
text-info"

```

```

sec:authorize="!hasRole(T(ntukhpi.csit.semit.riv.webappsrivlab4.mo
del.DAO.enums.Role).ENTRANT_MANAGER.name())
        and
hasRole(T(ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.enums.Ro
le).STUDENT_VIEWER.name())">Student Info</th>
        </tr>
    </thead>

    <tbody class="table-light fw-light
text-primary-emphasis" th:each="entrant, iterStat : ${entrants}">
        <tr>
            <td class="text-center align-content-center"
th:text="${iterStat.index + 1}"></td>
            <td class="text-start align-content-center"
th:text="${entrant?.caseNumber}"></td>
            <td class="text-start align-content-center"
                th:text="${entrant?.surname} + ' ' +
${entrant?.name} + ' ' + ${entrant?.patronymic}"></td>
            <td class="text-center align-content-center"
th:text="${#temporals.format(entrant?.birthday,
'dd.MM.yyyy')}"></td>
            <td class="text-center align-content-center"
                th:style="'color:' + (${entrant?.gender} ?
'#0d6efd' : '#d63384')"
                th:text="${entrant?.gender} ? 'чоловік' :
'жінка'"></td>
            <td class="text-end align-content-center"
th:text="${#numbers.formatDecimal(entrant?.ratingScore, 3, 3,
'COMMA')}"></td>
            <td class="text-center align-content-center"
sec:authorize="hasRole(T(ntukhpi.csit.semit.riv.webappsrivlab4.mod
el.DAO.enums.Role).STUDENT_VIEWER.name())">

```

```

<div class="d-flex justify-content-evenly
p-0 m-0"

sec:authorize="hasRole(T(ntukhpi.csit.semit.riv.webappsrivlab4.mod
el.DAO.enums.Role).STUDENT_VIEWER.name())">
    <a type="button"
th:href="@{/students/__$${entrant?.studentId}__/view}"
th:if="$${entrant?.studentId ne
null}"
class="col-1 p-0 m-0 border border-0
bg-transparent button-linked-entrant"
th:title="'Student info'">
    <i class="bi bi-person-rolodex
text-info"></i>
    </a>
    <span th:if="$${entrant?.studentId eq
null}" class="col-1"></span>
    <a type="button"
th:href="@{/entrants/__$${entrant?.id}__/edit}"
sec:authorize="hasRole(T(ntukhpi.csit.semit.riv.webappsrivlab4.mod
el.DAO.enums.Role).ENTRANT_MANAGER.name())"
class="col-1 p-0 m-0 border border-0
bg-transparent button-update"
th:title="'Edit record'">
    <i class="bi bi-pencil
text-warning"></i>
    </a>
    <a type="button"
th:href="@{/entrants/__$${entrant?.id}__/delete}"
sec:authorize="hasRole(T(ntukhpi.csit.semit.riv.webappsrivlab4.mod
el.DAO.enums.Role).ENTRANT_MANAGER.name())"
class="col-1 p-0 m-0 border border-0
bg-transparent button-delete"
th:title="'Delete record'">

```

```

                                <i class="bi bi-trash3
text-danger"></i>

                                </a>
                                </div>
                                </td>
                                </tr>
                                </tbody>
                                </table>
                                <p th:if="{entrants == null or entrants.isEmpty()}"
class="text-dark text-center">No data available</p>
                                <br>
                                </div>
                                </div>

                                <div class="d-flex justify-content-center">
                                    <a th:href="{@{/home}}" class="btn btn-outline-info"
type="button">
                                        <i class="bi bi-house"></i>
                                    </a>
                                </div>
                                </div>
                                </div>

                                <footer th:replace="{~{component/Footer :: footer}}"></footer>

                                <script type="text/javascript">
                                    document.addEventListener("DOMContentLoaded", function () {
                                        setSearchPatternAttr();
                                        setSearchInputValidity();

                                        setBirthdayLimit('inputBirthdayStart');
                                        setBirthdayLimit('inputBirthdayEnd');
                                        setBirthdayValidity('inputBirthdayStart');
                                        setBirthdayValidity('inputBirthdayEnd');

                                        setRatingScoreValidity('inputRatingScoreMin');
                                        setRatingScoreValidity('inputRatingScoreMax');

```



```

        setInputValueFromGetParams();
    });

    function setSearchPatternAttr() {
        const regexSearch = new
RegExp("[А-ЩЬЮЯЄІІҐа-щьюяєііґ'\\"-\\d ]{1,200}$");

document.getElementById('inputSearch').setAttribute('pattern',
regexSearch.source);
    }

    function setSearchInputValidity() {
        const message = "Value is a Ukrainian a string between 1
and 200 chars long, optionally including numbers, dash,
apostrophe, space.";

document.getElementById('inputSearch').addEventListener('input',
function () {
    if (this.validity.patternMismatch) {
        this.setCustomValidity(message);
    } else {
        this.setCustomValidity('');
    }

    this.reportValidity();
});
    }

    function setBirthdayLimit(inputId) {
        document.getElementById(inputId).setAttribute('min',
getBirthdayMinValue());

        document.getElementById(inputId).setAttribute('max',
getBirthdayMaxValue());
    }

```

```

function getBirthdayMinValue() {
    const today = new Date();
    return new Date(
        today.getFullYear() - 110,
        today.getMonth(),
        today.getDate())
        .toISOString().split('T')[0];
}

function getBirthdayMaxValue() {
    const today = new Date();
    return new Date(today.getFullYear() - 16,
today.getMonth(), today.getDate()).toISOString().split('T')[0];
}

function setBirthdayValidity(inputId) {
    const regexpDate = new
RegExp("(?=.{10}$)\\d{4}\\-\\d{2}\\-\\d{2}");
    const messages = [
        "Invalid date format. Check the entered value.",
        "Value is a date from 110 to 16 years ago today."
    ];

document.getElementById(inputId).addEventListener('mouseleave',
function () {
    const minDate = new Date(this.min);
    const maxDate = new Date(this.max);
    const selectedDate = regexpDate.test(this.value) ? new
Date(this.value) : null;

    if (!selectedDate) {
        this.setCustomValidity(messages[0]);
    } else if (selectedDate < minDate || selectedDate >
maxDate) {

```

```

        this.setCustomValidity(messages[1]);
    } else {
        this.setCustomValidity("");
    }

    this.reportValidity();
});
}

function setRatingScoreValidity(inputId) {
    const message = "Value is a decimal number in range from
120.001 to 200.000 inclusive.";

    document.getElementById(inputId).addEventListener('input',
function () {
        const minVal = parseFloat(this.min);
        const maxVal = parseFloat(this.max);
        const selectedVal = parseFloat(this.value);

        if (selectedVal < minVal || selectedVal > maxVal) {
            this.setCustomValidity(message);
        } else {
            this.setCustomValidity("");
        }

        this.reportValidity();
    });
}

function setInputValueFromGetParams() {
    const currentURL = new URL(window.location.href);
    const params = new
URLSearchParams(currentURL.searchParams);

    const inputFields = document.getElementsByTagName("input");

    for (const input of inputFields) {

```

```

        if (params.has(input.name)) {
            input.value = params.get(input.name);
        }
    }
}

function applySort(element) {
    const currentURL = new URL(window.location.href);
    const params = new
URLSearchParams(currentURL.searchParams);

    const paramName = "sort";
    const paramFieldName = element.id;
    let paramSortOrder = 'asc';
    const currentSortValue = params.get(paramName);

    if (currentSortValue) {
        const currentSortFieldName =
currentSortValue.split("-")[0];
        const currentSortOrder = currentSortFieldName ===
paramFieldName ? currentSortValue.split("-")[1] : '';
        paramSortOrder = currentSortOrder === 'asc' ? 'desc' :
'asc';
    }

    params.set(paramName, paramFieldName + '-' +
paramSortOrder);

    const newURL = currentURL.pathname + '?' +
params.toString();
    element.setAttribute('href', newURL);
}

function applySearch(linkButton) {
    const currentURL = new URL(window.location.href);

```

```

        const params = new
URLSearchParams(currentURL.searchParams);

        setFilteringParameter(params,
document.getElementById("inputSearch"));

        const newURL = currentURL.pathname + '?' +
params.toString();
        linkButton.setAttribute('href', newURL);
    }

    function resetSearch(linkButton) {
        const currentURL = new URL(window.location.href);
        const params = new
URLSearchParams(currentURL.searchParams);

        resetFilteringParameter(params,
document.getElementById("inputSearch"));

        const newURL = currentURL.pathname + '?' +
params.toString();
        document.getElementById("formSearch").reset();
        linkButton.setAttribute('href', newURL);
    }

    function applyFilterBirthday(linkButton) {
        const currentURL = new URL(window.location.href);
        const params = new
URLSearchParams(currentURL.searchParams);

        setFilteringParameter(params,
document.getElementById("inputBirthdayStart"));
        setFilteringParameter(params,
document.getElementById("inputBirthdayEnd"));
    }

```

```

        const newURL = currentURL.pathname + '?' +
params.toString();
        linkButton.setAttribute('href', newURL);
    }

    function resetFilterBirthday(linkButton) {
        const currentURL = new URL(window.location.href);
        const params = new
URLSearchParams(currentURL.searchParams);

        resetFilteringParameter(params,
document.getElementById("inputBirthdayStart"));
        resetFilteringParameter(params,
document.getElementById("inputBirthdayEnd"));

        const newURL = currentURL.pathname + '?' +
params.toString();
        document.getElementById("formFilterBirthday").reset();
        linkButton.setAttribute('href', newURL);
    }

    function applyFilterRatingScore(linkButton) {
        const currentURL = new URL(window.location.href);
        const params = new
URLSearchParams(currentURL.searchParams);

        setFilteringParameter(params,
document.getElementById("inputRatingScoreMin"));
        setFilteringParameter(params,
document.getElementById("inputRatingScoreMax"));

        const newURL = currentURL.pathname + '?' +
params.toString();
        linkButton.setAttribute('href', newURL);
    }

```

```

function resetFilterRatingScore(linkButton) {
    const currentURL = new URL(window.location.href);
    const params = new
URLSearchParams(currentURL.searchParams);

    resetFilteringParameter(params,
document.getElementById("inputRatingScoreMin"));
    resetFilteringParameter(params,
document.getElementById("inputRatingScoreMax"));

    const newURL = currentURL.pathname + '?' +
params.toString();
    document.getElementById("formFilterRatingScore").reset();
    linkButton.setAttribute('href', newURL);
}

function setFilteringParameter(params, inputElement) {
    const paramName = inputElement.name;
    const paramValue = inputElement.value.toString().trim();

    if (paramValue.length > 0) {
        params.set(paramName, paramValue);
    } else {
        params.delete(paramName);
    }
}

function resetFilteringParameter(params, inputElement) {
    const paramName = inputElement.name;
    params.delete(paramName);
}
</script>

<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstra
p.bundle.min.js"

```

```

integrity="sha384-YvpcrYf0tY3lHB60NNkmXc5s9fDVZLESaAA55NDzOxhy9Gkc
Ids1K1eN7N6jIeHz"
        crossorigin="anonymous"></script>

</body>
</html>

```

К.14 Код файла StudentForm.html

```

<!DOCTYPE html>
<html lang="en" xmlns:th="http://www.thymeleaf.org"
xmlns:sec="http://www.thymeleaf.org/extras/spring-security">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width,
initial-scale=1">
    <title th:text="${mode.getValue() + ' Student'}">Student
Form</title>
    <link rel="icon" type="image/x-icon"
th:href="@{/image/icon/favicon_square.png}">
    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootst
rap.min.css"
        rel="stylesheet"
integrity="sha384-QWTKZyjpPEjISv5WaRU90FeRpok6YctnYmDr5pNlyT2bRjXh
0JMHjY6hW+ALEwIH"
        crossorigin="anonymous">
    <link rel="stylesheet"
href="https://cdn.jsdelivr.net/npm/bootstrap-icons@1.11.3/font/boo
tstrap-icons.min.css">
</head>
<body>

```



```

<header th:replace="~{component/NavigationBar ::
header}"></header>

<div th:replace="~{component/ErrorToast :: errorToast}"></div>
<div th:replace="~{component/WarningToast :: warningToast}"></div>
<div th:replace="~{component/SuccessToast :: successToast}"></div>

<div class="container w-75 text-center justify-content-center">
    <div class="card border border-3 border-info-subtle shadow
bg-body-tertiary rounded m-0 mt-5 p-0"
        th:with="isEditOrDelete = ${mode eq
T(ntukhpi.csit.semit.riv.webappsrivlab4.controller.Mode).EDIT
                                or mode eq
T(ntukhpi.csit.semit.riv.webappsrivlab4.controller.Mode).DELETE},
                                isDeleteOrView = ${mode eq
T(ntukhpi.csit.semit.riv.webappsrivlab4.controller.Mode).DELETE
                                or mode eq
T(ntukhpi.csit.semit.riv.webappsrivlab4.controller.Mode).VIEW_FORM
                                },
                                isNotDeleteAndNotView = ${mode ne
T(ntukhpi.csit.semit.riv.webappsrivlab4.controller.Mode).DELETE
                                and mode ne
T(ntukhpi.csit.semit.riv.webappsrivlab4.controller.Mode).VIEW_FORM
                                }">

        <div class="card-header bg-info-subtle border-bottom
border-3 border-info-subtle
                                font-monospace text-center text-info fw-bolder"
id="formHeader"
                th:text="${mode.getValue() + ' Student'}">
                    Student Form
            </div>

            <form class="card-body bg-light font-monospace p-5 m-0"
id="studentForm"
                th:object="${student}"

```

```

        th:with="actionUrl=
            ${mode eq
T(ntukhpi.csit.semit.riv.webappsrivlab4.controller.Mode).ADD ?
'/students/add' :
            (mode eq
T(ntukhpi.csit.semit.riv.webappsrivlab4.controller.Mode).VIEW_FORM
? '/students/view' :
            (mode eq
T(ntukhpi.csit.semit.riv.webappsrivlab4.controller.Mode).EDIT ?
('/students/' + student?.id + '/edit') :
            (mode eq
T(ntukhpi.csit.semit.riv.webappsrivlab4.controller.Mode).DELETE ?
('/students/' + student?.id + '/delete') : '')}}}"
        th:action="@{{actionUrl}}"
        th:method="'post'">

        <div class="d-none card-text mb-4">
            <div class="input-group border border-1
border-info-subtle rounded ">
                <label for="inputStudentId" class="col-4
input-group-text text-info">Id</label>
                <input id="inputStudentId" class="col-auto
form-control text-dark" type="hidden" required
                    th:field="*{id}"
                    th:minlength="1" th:maxlength="100"
                    th:value="${isEditOrDelete ? student?.id
: null}"
                    th:readonly="${isDeleteOrView}">
            </div>
        </div>

        <div class="card-text d-flex align-content-center
justify-content-start mb-4">
            <div class="col-3 border border-1
border-info-subtle rounded d-inline-flex p-0 m-0 me-3">

```

```

        <span class="d-flex flex-fill input-group-text
text-info">Entrant</span>
    </div>

    <div class="col">
        <select class="form-select border border-1
border-info-subtle" aria-label="Entrant list select"
            th:field="*{entrant.id}" required
            th:disabled="{isDeleteOrView}">
            <option th:selected="true"
                th:value="{mode ne
T(ntukhpi.csit.semit.riv.webappsrivlab4.controller.Mode).ADD
? student?.entrant?.id :
null}"
                th:text="{mode ne
T(ntukhpi.csit.semit.riv.webappsrivlab4.controller.Mode).ADD
? student?.entrant?.surname +
' ' + student?.entrant?.name + ' ' + student?.entrant?.patronymic
+ ', ' + student?.entrant?.caseNumber
: 'Select an Entrant'}">
                Select an Entrant
            </option>

            <div th:each="entrant : {entrantList}">
                <option th:value="{entrant?.id}"
                    th:text="{entrant?.surname + '
' + entrant?.name + ' ' + entrant?.patronymic + ', ' +
entrant?.caseNumber}">
                    Entrant
                </option>
            </div>
        </select>
    </div>
</div>

```

```

        <div class="card-text d-flex align-content-center
justify-content-start mb-4">
            <div class="col-3 border border-1
border-info-subtle rounded d-inline-flex p-0 m-0 me-3">
                <span class="d-flex flex-fill input-group-text
text-info">Funding Type</span>
            </div>

            <div class="col-auto">
                <input type="hidden"
                    th:field="*{fundingType}"
                    th:value="{student?.fundingType.getValue()}"
                    th:if="{mode eq
T(ntukhpi.csit.semit.riv.webappsrivlab4.controller.Mode).DELETE}">

                <input id="fundingTypeBudget" type="radio"
class="btn-check" autocomplete="off" required
                    th:field="*{fundingType}"
                    th:value="{ 'BUDGET' }"
                    th:disabled="{isDeleteOrView}"
                    onclick="setScholarshipStatus(this)">
                <label for="fundingTypeBudget" class="btn
btn-outline-info">Budget</label>

                <input id="fundingTypeContract" type="radio"
class="btn-check" autocomplete="off" required
                    th:field="*{fundingType}"
                    th:value="{ 'CONTRACT' }"
                    th:disabled="{isDeleteOrView}"
                    onclick="setScholarshipStatus(this)">
                <label for="fundingTypeContract" class="btn
btn-outline-info">Contract</label>
            </div>
        </div>
    </div>

```

```

        <div class="card-text d-flex align-content-center
justify-content-start mb-4">
            <div class="col-3 border border-1
border-info-subtle rounded d-inline-flex p-0 m-0 me-3">
                <span class="d-flex flex-fill input-group-text
text-info">Scholarship Status</span>
            </div>

            <div class="col-auto">
                <input type="hidden"
                    th:field="*{scholarshipStatus}"
                    th:value="{student?.scholarshipStatus.getValue()}"
                    th:if="{mode eq
T(ntukhpi.csit.semit.riv.webappsrivlab4.controller.Mode).DELETE}">

                <input id="scholarshipStatusNone" type="radio"
class="btn-check" autocomplete="off" required
                    th:field="*{scholarshipStatus}"
                    th:value="{ 'NONE' }"
                    th:disabled="{isDeleteOrView}">
                <label for="scholarshipStatusNone" class="btn
btn-outline-info">None</label>

                <input id="scholarshipStatusOrdinary"
type="radio" class="btn-check" autocomplete="off" required
                    th:field="*{scholarshipStatus}"
                    th:value="{ 'ORDINARY' }"
                    th:disabled="{isDeleteOrView}">
                <label for="scholarshipStatusOrdinary"
class="btn btn-outline-info">Ordinary</label>

                <input id="scholarshipStatusEnhanced"
type="radio" class="btn-check" autocomplete="off" required
                    th:field="*{scholarshipStatus}"
                    th:value="{ 'ENHANCED' }"

```

```

            th:disabled="{isDeleteOrView}">
            <label for="scholarshipStatusEnhanced"
class="btn btn-outline-info">Enhanced</label>
        </div>
    </div>

    <div class="card-text mb-4">
        <div class="input-group border border-1
border-info-subtle rounded ">
            <label for="inputCorporateEmail" class="col-3
input-group-text text-info">
                Corporate Email
            </label>
            <input id="inputCorporateEmail" class="col-auto
form-control text-dark" type="text" required
                th:field="*{corporateEmail}"
                th:minlength="17" th:maxlength="123"
                th:value="{isEditOrDelete ?
student?.corporateEmail : null}"
                th:readonly="{isDeleteOrView}">
        </div>
    </div>

    <div class="d-flex justify-content-evenly">
        <button type="submit" class="btn btn-outline-info
border-2"
            th:title="'Save record'"
            th:if="{isNotDeleteAndNotView}">
            <i class="bi bi-floppy"></i>
        </button>

        <button type="reset" class="btn btn-outline-info
border-2"
            onclick="form.reset()"
            th:title="'Reset fields'"
            th:if="{isNotDeleteAndNotView}">

```

```

        <i class="bi bi-x-lg"></i>
    </button>

    <button type="button" class="btn btn-outline-info
border-2"

        onclick="submitConfirmation() "
        th:title="'Delete record'"
        th:if="{mode eq
T(ntukhpi.csit.semit.riv.webappsrivlab4.controller.Mode).DELETE}">
        <i class="bi bi-trash3"></i>
    </button>
</div>
</form>
</div>

<div class="d-flex justify-content-center mt-5">
    <button class="btn btn-outline-info me-5" type="button"
        th:title="'Go Back'"
        onclick="window.history.back() "
        th:if="{mode eq
T(ntukhpi.csit.semit.riv.webappsrivlab4.controller.Mode).VIEW_FORM
}">

        <i class="bi bi-box-arrow-in-left"></i>
    </button>

    <a class="btn btn-outline-info" type="button"
        th:title="'Go to Student Table'"
        th:href="{@{/students}}">
        <i class="bi bi-table"></i>
    </a>
</div>
</div>

<footer th:replace="{component/Footer :: footer}"></footer>

<script type="text/javascript">

```

```

document.addEventListener("DOMContentLoaded", function () {
    setAllInputsValidity();

    const title =
document.getElementById('formHeader').innerText;

    if (title.match(new RegExp("Add Student|Edit Student"))) {

setScholarshipStatus(document.getElementById('fundingTypeBudget'))
;

setScholarshipStatus(document.getElementById('fundingTypeContract'
));
    }
});

function setAllInputsValidity() {
    setEmailInputValidity();

    const form = document.getElementById('studentForm');

    form.addEventListener('submit', function (event) {
        event.preventDefault();

        if (!form.checkValidity()) {
            event.stopPropagation();
            form.reportValidity();
        } else {
            form.submit();
        }
    });

    function setEmailInputValidity() {
        const regexCorporateEmail = new
RegExp("^[a-z](\\.|?|_|+){1,99}@[a-z]{1,10}\\.|\\.khpi\\.\\.edu\\.\\.ua$");
    }
}

```



```

document.getElementById('inputCorporateEmail').setAttribute('pattern', regexCorporateEmail.source);

    const message = "The corporate email should match the format 'name.surname@faculty.khpi.edu.ua'.";

document.getElementById('inputCorporateEmail').addEventListener('input', function () {
    if (this.validity.patternMismatch) {
        this.setCustomValidity(message);
    } else {
        this.setCustomValidity('');
    }

    this.reportValidity();
});
}

function setScholarshipStatus(inputElement) {
    const inputId = inputElement.id;
    const enhancedId = 'scholarshipStatusEnhanced';
    const ordinaryId = 'scholarshipStatusOrdinary';
    const inputEnhanced = document.getElementById(enhancedId);
    const inputOrdinary = document.getElementById(ordinaryId);
    const lableEnhanced =
document.querySelector(`label[for="${enhancedId}"]`);
    const lableOrdinary =
document.querySelector(`label[for="${ordinaryId}"]`);

    if (inputId === 'fundingTypeContract' &&
inputElement.checked) {
        inputEnhanced.checked = false;
        inputEnhanced.disabled = true;

```

```

        labelEnhanced.classList.replace('btn-outline-info',
'btn-outline-secondary');

        inputOrdinary.checked = false;
        inputOrdinary.disabled = true;
        labelOrdinary.classList.replace('btn-outline-info',
'btn-outline-secondary');

document.getElementById('scholarshipStatusNone').checked = true;
    }

    if (inputId === 'fundingTypeBudget' &&
inputElement.checked) {
        inputEnhanced.disabled = false;

labelEnhanced.classList.replace('btn-outline-secondary',
'btn-outline-info');

        inputOrdinary.disabled = false;

labelOrdinary.classList.replace('btn-outline-secondary',
'btn-outline-info');
    }
}

function submitConfirmation() {
    const message = 'Are you sure you want to delete that
record?';

    if (confirm(message) === true) {
        document.getElementById('studentForm').submit();
    }
}
</script>

```

```

<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstra
p.bundle.min.js"

integrity="sha384-YvpcrYf0tY3lHB60NNkmXc5s9fDVZLESaAA55NDzOxhy9Gkc
Ids1K1eN7N6jIeHz"

        crossorigin="anonymous"></script>
</body>
</html>

```

К.15 Код файла StudentTable.html

```

<!DOCTYPE html>
<html lang="en" xmlns:th="http://www.thymeleaf.org"
xmlns:sec="http://www.thymeleaf.org/extras/spring-security">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width,
initial-scale=1">
    <title>Student Table</title>
    <link rel="icon" type="image/x-icon"
th:href="@{/image/icon/favicon_square.png}">
    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootst
rap.min.css"
        rel="stylesheet"

integrity="sha384-QWTKZyjpPEjISv5WaRU9OFeRpok6YctnYmDr5pNlyT2bRjXh
0JMhjY6hW+ALEwIH"
        crossorigin="anonymous">
    <link rel="stylesheet"

href="https://cdn.jsdelivr.net/npm/bootstrap-icons@1.11.3/font/boo
tstrap-icons.min.css">
</head>

```

```

<body>

<header th:replace="~{component/NavigationBar ::
header}"></header>

<div th:replace="~{component/ErrorToast :: errorToast}"></div>
<div th:replace="~{component/WarningToast :: warningToast}"></div>
<div th:replace="~{component/SuccessToast :: successToast}"></div>

<div class="mx-3 mt-3 mb-5 font-monospace">
    <div class="row">
        <h1 class="text-info fs-1 my-3 fw-semibold
text-center">Student Table</h1>
    </div>

    <div class="row align-items-start">
        <div class="col-2 mx-2">
            <div class="card row p-2 mb-4 border border-3
border-info-subtle bg-light shadow">
                <div class="card-body col">
                    <div class="card-title row">
                        <p class="text-info fs-5 fw-bold p-0
m-0">Search</p>
                        <p class="text-secondary fw-lighter p-0
m-0" style="font-size: 0.8rem;">
                            Search by full name (matching start,
case-insensitive)
                        </p>
                    </div>
                </div>

                <form class="card-text col mt-3"
id="formSearch"
                    th:method="'get'"
                    th:action="@{/students}">
                    <div class="row mb-3">

```

```

        <label for="inputSearch"
class="visually-hidden">Search</label>

        <input type="search"
class="form-control border border-2 border-info-subtle"
        id="inputSearch" name="search"
placeholder="Type to search...">
    </div>

    <div class="col d-flex
justify-content-evenly align-content-center text-center p-0 m-0">
        <a type="button" id="btn-search"
onclick="applySearch(this) "
        class="btn btn-outline-info
border-2" th:alt-title="'Apply Search'">
            <i class="bi bi-search"></i>
        </a>

        <a type="button" id="btn-search-reset"
onclick="resetSearch(this) "
        class="btn btn-outline-info border-2
button-reset" th:title="'Reset Search'">
            <i class="bi bi-x-lg"></i>
        </a>
    </div>
</form>
</div>
</div>

<div class="card row p-2 mb-4 border border-3
border-info-subtle bg-light shadow">
    <div class="card-body col">
        <div class="card-title row">
            <p class="text-info fs-5 fw-bold p-0
m-0">Filters</p>
        </div>
    </div>

```

```

        <div class="card-subtitle row mt-2"
id="filterFundingType">
            <p class="text-info fs-6 fw-bold p-0
m-0">Funding Type</p>
        </div>

        <form class="card-text col mt-2"
id="formFilterFundingType"
            th:method="'get'"
th:action="@{/students}">
            <div class="mt-2 mb-3">
                <div class="form-check">
                    <input class="form-check-input"
type="radio" id="fundingTypeBudget"
                        name="fundingType"
th:value="{T(ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.enum
s.FundingType).BUDGET.getValue()}">
                        <label class="form-check-label"
for="fundingTypeBudget" style="color: #6f42c1;">
                            Budget
                        </label>
                    </div>

                    <div class="form-check">
                        <input class="form-check-input"
type="radio" id="fundingTypeContract"
                            name="fundingType"
th:value="{T(ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.enum
s.FundingType).CONTRACT.getValue()}">
                        <label class="form-check-label"
for="fundingTypeContract" style="color: #fd7e14;">
                            Contract
                        </label>
                    </div>

```

```

        </div>

        <div class="col d-flex
justify-content-evenly align-content-center text-center p-0 m-0">
            <a type="button"
id="btn-filter-funding-type"
onclick="applyFilterFundingType(this) "
            class="btn btn-outline-info
border-2" th:title="'Apply Filter'">
                <i class="bi bi-funnel"></i>
            </a>

            <a type="button"
id="btn-filter-funding-type-reset"
onclick="resetFilterFundingType(this) "
            class="btn btn-outline-info border-2
button-reset" th:title="'Reset Filter'">
                <i class="bi bi-x-lg"></i>
            </a>
        </div>
    </form>

    <div class="card-subtitle row mt-4"
id="filterScholarshipStatus">
        <p class="text-info fs-6 fw-bold p-0
m-0">Scholarship Status</p>
    </div>

    <form class="card-text col mt-2"
id="formFilterScholarshipStatus"
        th:method="'get'"
th:action="@{/students}">
        <div class="mt-2 mb-3">
            <div class="form-check">
                <input class="form-check-input"
type="checkbox" id="scholarshipStatusEnhanced"

```

```

name="scholarshipStatus"

th:value="${T(ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.enums.ScholarshipStatus).ENHANCED.getValue()}">
    <label class="form-check-label
fw-lighter py-0 px-1 m-0" for="scholarshipStatusEnhanced"
        style="color: #20c997;">
        Enhanced
    </label>
</div>
<div class="form-check">
    <input class="form-check-input"
type="checkbox" id="scholarshipStatusOrdinary"
        name="scholarshipStatus"

th:value="${T(ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.enums.ScholarshipStatus).ORDINARY.getValue()}">
    <label class="form-check-label
fw-lighter py-0 px-1 m-0" for="scholarshipStatusOrdinary"
        style="color: #0d6efd;">
        Ordinary
    </label>
</div>
<div class="form-check">
    <input class="form-check-input"
type="checkbox" id="scholarshipStatusNone"
        name="scholarshipStatus"

th:value="${T(ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.enums.ScholarshipStatus).NONE.getValue()}">
    <label class="form-check-label
fw-lighter py-0 px-1 m-0" for="scholarshipStatusNone"
        style="color: #6c757d;">
        None
    </label>
</div>

```



```

        </div>

        <div class="col d-flex
justify-content-evenly align-content-center text-center p-0 m-0">
            <a type="button"
id="btn-filter-scholarship-status"

onclick="applyFilterScholarshipStatus(this)"
            class="btn btn-outline-info
border-2" th:title="'Apply Filter'">
                <i class="bi bi-funnel"></i>
            </a>

            <a type="button"
id="btn-filter-scholarship-status-reset"

onclick="resetFilterScholarshipStatus(this)"
            class="btn btn-outline-info border-2
button-reset" th:title="'Reset Filter'">
                <i class="bi bi-x-lg"></i>
            </a>
        </div>
    </form>
</div>
</div>

    <div class="card row p-2 mb-4 border border-3
border-info-subtle bg-light shadow"

sec:authorize="hasRole(T(ntukhpi.csit.semit.riv.webappsrivlab4.mod
el.DAO.enums.Role).STUDENT_MANAGER.name())">
        <div class="card-body col">
            <div class="card-title row">
                <p class="text-info fs-5 fw-bold p-0
m-0">Add record</p>

```

```

        <p class="text-secondary fw-lighter p-0
m-0" style="font-size: 0.8rem;">
            Add new record to the end of the table
        </p>
    </div>

    <form class="card-text col mt-3"
id="formInsert"
        th:method="'get'"
th:action="@{/students}">
        <div class="col d-flex
justify-content-evenly align-content-center text-center p-0 m-0">
            <a th:href="@{/students/add}"
id="btn-add-student"
                class="btn btn-outline-info
border-2" role="button" th:title="'Add record'">
                <i class="bi bi-plus-lg"></i>
            </a>
        </div>
    </form>
</div>
</div>
</div>

<div class="col">
    <table id="studentTable" class="table table-sm
table-hover font-monospace
        table-bordered border border-3
border-info-subtle shadow-lg">
        <thead class="table-info text-center fw-medium
text-secondary-emphasis">
            <tr>
                <th class="text-center align-content-center
text-info">#</th>

```

```

        <div class="p-0 m-0" th:each="fieldName :
${fields}">
            <th class="text-center
align-content-center">
                <div class="d-flex p-0 m-0">
                    <div class="flex-grow-1
align-content-center justify-content-center p-0 m-0">
                        <span class="">
                            <a class="sortLink
link-info link-offset-2 link-offset-3-hover link-underline
link-underline-opacity-0 link-underline-opacity-75-hover"
                                th:id="${fieldName[0]}"
                                th:text="${fieldName[1]}"
                                onclick="applySort(this)"></a>
                            </span>
                        </div>
                        <div class="d-none flex-shrink-1
align-content-center justify-content-end ms-2 p-0 m-0"
                                style="width: 20px;">
                            <button type="button"
                                class="btn bg-transparent p-0 m-0" title="Sort records">
                                <i class="bi
bi-sort-down-alt sortingAsc" th:title="'Ascending Sort'"></i>
                                <i class="bi bi-sort-down
sortingDesc" th:title="'Descending Sort'"></i>
                            </button>
                        </div>
                    </div>
                </th>
            </div>

        <th class="text-center align-content-center
text-info"

```

```

sec:authorize="hasRole(T(ntukhpi.csit.semit.riv.webappsrivlab4.mod
el.DAO.enums.Role).STUDENT_MANAGER.name())
and
hasRole(T(ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.enums.Ro
le).ENTRANT_VIEWER.name())">Operations</th>
<th class="text-center align-content-center
text-info"

sec:authorize="!hasRole(T(ntukhpi.csit.semit.riv.webappsrivlab4.mo
del.DAO.enums.Role).STUDENT_MANAGER.name())
and
hasRole(T(ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.enums.Ro
le).ENTRANT_VIEWER.name())">Entrant Info</th>
</tr>
</thead>

<tbody class="table-light fw-light
text-primary-emphasis" th:each="student, iterStat : ${students}">
<tr>
<td class="text-center align-content-center"
th:text="${iterStat.index + 1}"></td>
<td class="text-start align-content-center"
th:text="${student?.entrant?.surname} + ' ' +
${student?.entrant?.name} + ' ' +
${student?.entrant?.patronymic}"></td>
<td class="text-center align-content-center"
th:switch="${student?.fundingType?.name()}">
<span
th:case="${T(ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.enums
.FundingType).CONTRACT.name()}"
style="color: #fd7e14;"
th:text="${student?.fundingType?.getValue()}"></span>
<span
th:case="${T(ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.enums
.FundingType).BUDGET.name()}"

```

```

                style="color: #6f42c1;"
th:text="${student?.fundingType?.getValue()}"</span>
            </td>
            <td class="text-center align-content-center"
th:switch="${student?.scholarshipStatus?.name()}">
                <span
th:case="${T(ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.enums
.ScholarshipStatus).ENHANCED.name()}"
                    style="color: #20c997;"
th:text="${student?.scholarshipStatus?.getValue()}"></span>
                <span
th:case="${T(ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.enums
.ScholarshipStatus).ORDINARY.name()}"
                    style="color: #0d6efd;"
th:text="${student?.scholarshipStatus?.getValue()}"></span>
                <span
th:case="${T(ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.enums
.ScholarshipStatus).NONE.name()}"
                    style="color: #6c757d;"
th:text="${student?.scholarshipStatus?.getValue()}"></span>
            </td>
            <td class="text-start align-content-center"
th:text="${student?.corporateEmail}"></td>
            <td class="text-center align-content-center"

sec:authorize="hasRole(T(ntukhpi.csit.semit.riv.webappsrivlab4.mod
el.DAO.enums.Role).ENTRANT_VIEWER.name())">
                <div class="d-flex justify-content-evenly
p-0 m-0"

sec:authorize="hasRole(T(ntukhpi.csit.semit.riv.webappsrivlab4.mod
el.DAO.enums.Role).ENTRANT_VIEWER.name())">
                    <a type="button"
th:href="@{/entrants/___${student?.entrant?.id}___/view}"
                        class="p-0 m-0 border border-0
bg-transparent button-linked-entrant"

```

```

th:title="'Entrant info'">
    <i class="bi bi-person-rolodex
text-info"></i>

    </a>
    <a type="button"
th:href="@{/students/__$${student?.id}__/_edit}"

sec:authorize="hasRole(T(ntukhpi.csit.semit.riv.webappsrivlab4.mod
el.DAO.enums.Role).STUDENT_MANAGER.name())"
        class="p-0 m-0 border border-0
bg-transparent button-update" th:title="'Edit record'">
        <i class="bi bi-pencil
text-warning"></i>

        </a>
        <a type="button"
th:href="@{/students/__$${student?.id}__/_delete}"

sec:authorize="hasRole(T(ntukhpi.csit.semit.riv.webappsrivlab4.mod
el.DAO.enums.Role).STUDENT_MANAGER.name())"
        class="p-0 m-0 border border-0
bg-transparent button-delete" th:title="'Delete record'">
        <i class="bi bi-trash3
text-danger"></i>

        </a>
    </div>
</td>
</tr>
</tbody>
</table>
<p th:if="${students == null or students.isEmpty()}"
class="text-dark text-center">No data available</p>
    <br>
</div>
</div>

<div class="d-flex justify-content-center">

```

```

        <a th:href="@{/home}" class="btn btn-outline-info"
type="button">
            <i class="bi bi-house"></i>
        </a>
    </div>
</div>

<footer th:replace="~{component/Footer :: footer}"></footer>

<script type="text/javascript">
    document.addEventListener("DOMContentLoaded", function () {
        setSearchPatternAttr();
        setSearchInputValidity();

        setInputValueFromGetParams();
    });

    function setSearchPatternAttr() {
        const regexSearch = new
RegExp("[А-ЩЬЮЯЄІЇҐа-щьюяєііґ'\\"-\\d ]{1,200}$");

document.getElementById('inputSearch').setAttribute('pattern',
regexSearch.source);
    }

    function setSearchInputValidity() {
        const message = "Value is a Ukrainian a string between 1
and 200 chars long, optionally including numbers, dash,
apostrophe, space.";

document.getElementById('inputSearch').addEventListener('input',
function () {
        if (this.validity.patternMismatch) {
            this.setCustomValidity(message);
        } else {

```

```

        this.setCustomValidity('');
    }

    this.reportValidity();
});
}

function setInputValueFromGetParams() {
    const currentURL = new URL(window.location.href);
    const params = new
URLSearchParams(currentURL.searchParams);
    const inputFields = document.getElementsByTagName('input');

    for (const input of inputFields) {
        if (params.has(input.name)) {
            switch (input.type) {
                case 'radio':
                    if (input.value ===
params.get(input.name).trim()) {
                        input.setAttribute('checked', 'true');
                    }
                    break;
                case 'checkbox':
                    const paramValues =
params.get(input.name).trim().split('-');
                    paramValues.forEach(value => {
                        if (input.value === value) {
                            input.setAttribute('checked',
'true');
                        }
                    });
                    break;
                default:
                    input.value = params.get(input.name);
                    break;
            }
        }
    }
}

```



```

    }

    }

}

function applySort(element) {
    const currentURL = new URL(window.location.href);
    const params = new
URLSearchParams(currentURL.searchParams);

    const paramName = 'sort';
    const paramFieldName = element.id;
    let paramSortOrder = 'asc';
    const currentSortValue = params.get(paramName);

    if (currentSortValue) {
        const currentSortFieldName =
currentSortValue.split('-')[0];
        const currentSortOrder = currentSortFieldName ===
paramFieldName ? currentSortValue.split('-')[1] : '';
        paramSortOrder = currentSortOrder === 'asc' ? 'desc' :
'asc';
    }

    params.set(paramName, paramFieldName + '-' +
paramSortOrder);

    const newURL = currentURL.pathname + '?' +
params.toString();
    element.setAttribute('href', newURL);
}

function applySearch(linkButton) {
    const currentURL = new URL(window.location.href);
    const params = new
URLSearchParams(currentURL.searchParams);

```

```

        setFilteringParameter(params,
document.getElementById('inputSearch'));

        const newURL = currentURL.pathname + '?' +
params.toString();
        linkButton.setAttribute('href', newURL);
    }

    function resetSearch(linkButton) {
        const currentURL = new URL(window.location.href);
        const params = new
URLSearchParams(currentURL.searchParams);

        resetFilteringParameter(params,
document.getElementById('inputSearch'));

        const newURL = currentURL.pathname + '?' +
params.toString();
        document.getElementById('formSearch').reset();
        linkButton.setAttribute('href', newURL);
    }

    function applyFilterFundingType(linkButton) {
        const currentURL = new URL(window.location.href);
        const params = new
URLSearchParams(currentURL.searchParams);

        const inputContract =
document.getElementById('fundingTypeContract');
        const inputBudget =
document.getElementById('fundingTypeBudget');

        console.log("inputContract.checked: ",
inputContract.checked);
        console.log("inputBudget.checked: ", inputBudget.checked);
    }

```

```

        if (inputContract.checked) setFilteringParameter(params,
inputContract);

        if (inputBudget.checked) setFilteringParameter(params,
inputBudget);

        const newURL = currentURL.pathname + '?' +
params.toString();
        console.log(newURL);
        linkButton.setAttribute('href', newURL);
    }

    function resetFilterFundingType(linkButton) {
        const currentURL = new URL(window.location.href);
        const params = new
URLSearchParams(currentURL.searchParams);

        resetFilteringParameter(params,
document.getElementById('fundingTypeContract'));
        resetFilteringParameter(params,
document.getElementById('fundingTypeBudget'));

        const newURL = currentURL.pathname + '?' +
params.toString();
        document.getElementById('formFilterFundingType').reset();
        linkButton.setAttribute('href', newURL);
    }

    function applyFilterScholarshipStatus(linkButton) {
        const currentURL = new URL(window.location.href);
        const params = new
URLSearchParams(currentURL.searchParams);

        function setFilteringParameterForScholarship(params,
inputElement) {
            const paramName = inputElement.name;

```

```

        const paramValue =
inputElement.value.toString().trim();

        if (paramValue.length > 0) {
            if (params.has(paramName)) {
                const newParamValue = params.get(paramName) +
"-" + paramValue;
                params.set(paramName, newParamValue);
            } else {
                params.set(paramName, paramValue);
            }
        } else {
            params.delete(paramName);
        }
    }

    const inputEnhanced =
document.getElementById('scholarshipStatusEnhanced');
    const inputOrdinary =
document.getElementById('scholarshipStatusOrdinary');
    const inputNone =
document.getElementById('scholarshipStatusNone');

    params.delete(inputEnhanced.name);

    if (inputEnhanced.checked)
setFilteringParameterForScholarship(params, inputEnhanced);
    if (inputOrdinary.checked)
setFilteringParameterForScholarship(params, inputOrdinary);
    if (inputNone.checked)
setFilteringParameterForScholarship(params, inputNone);

    const newURL = currentURL.pathname + '?' +
params.toString();
    linkButton.setAttribute('href', newURL);
}

```

```

function resetFilterScholarshipStatus(linkButton) {
    const currentURL = new URL(window.location.href);
    const params = new
URLSearchParams(currentURL.searchParams);

    resetFilteringParameter(params,
document.getElementById('scholarshipStatusEnhanced'));
    resetFilteringParameter(params,
document.getElementById('scholarshipStatusOrdinary'));
    resetFilteringParameter(params,
document.getElementById('scholarshipStatusNone'));

    const newURL = currentURL.pathname + '?' +
params.toString();

document.getElementById('formFilterScholarshipStatus').reset();
    linkButton.setAttribute('href', newURL);
}

function setFilteringParameter(params, inputElement) {
    const paramName = inputElement.name;
    const paramValue = inputElement.value.toString().trim();

    if (paramValue.length > 0) {
        params.set(paramName, paramValue);
    } else {
        params.delete(paramName);
    }
}

function resetFilteringParameter(params, inputElement) {
    const paramName = inputElement.name;
    params.delete(paramName);
}

```

```

        if (inputElement.type === 'radio' || inputElement.type ===
'checkbox') {
            inputElement.removeAttribute('checked');
        }
    }
}
</script>

<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstra
p.bundle.min.js"

integrity="sha384-YvpcrYf0tY3lHB60NNkmXc5s9fDVZLESaAA55NDzOxhy9Gkc
Ids1K1eN7N6jIeHz"

crossorigin="anonymous"></script>

</body>
</html>

```

К.16 Код файла UserForm.html

```

<!DOCTYPE html>
<html lang="en" xmlns:th="http://www.thymeleaf.org"
xmlns:sec="http://www.thymeleaf.org/extras/spring-security">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width,
initial-scale=1">
    <title th:text="${mode.getValue() + ' User'}">User Form</title>
    <link rel="icon" type="image/x-icon"
th:href="@{/image/icon/favicon_square.png}">
    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootst
rap.min.css"
rel="stylesheet"

```

```

integrity="sha384-QWTKZyjpPEjISv5WaRU9OFeRpok6YctnYmDr5pNlyT2bRjXh
0JMhY6hW+ALEwIH"
    crossorigin="anonymous">
    <link rel="stylesheet"

href="https://cdn.jsdelivr.net/npm/bootstrap-icons@1.11.3/font/boo
tstrap-icons.min.css">
</head>
<body>

<header th:replace="~{component/NavigationBar ::
header}"></header>

<div th:replace="~{component/ErrorToast :: errorToast}"></div>
<div th:replace="~{component/WarningToast :: warningToast}"></div>
<div th:replace="~{component/SuccessToast :: successToast}"></div>

<div class="container w-75 text-center justify-content-center">
    <div class="card border border-3 border-info-subtle shadow
bg-body-tertiary rounded m-0 mt-5 p-0"
        th:with="
            isEditOrDelete=${mode != null and (mode eq
T(ntukhpi.csit.semit.riv.webappsrivlab4.controller.Mode).EDIT
            or mode eq
T(ntukhpi.csit.semit.riv.webappsrivlab4.controller.Mode).DELETE)},
            isDeleteOrView=${mode != null and (mode eq
T(ntukhpi.csit.semit.riv.webappsrivlab4.controller.Mode).DELETE
            or mode eq
T(ntukhpi.csit.semit.riv.webappsrivlab4.controller.Mode).VIEW_FORM
            ) },
            isNotDeleteAndNotView=${mode != null and (mode ne
T(ntukhpi.csit.semit.riv.webappsrivlab4.controller.Mode).DELETE
            and mode ne
T(ntukhpi.csit.semit.riv.webappsrivlab4.controller.Mode).VIEW_FORM
            ) }">

```



```

th:readonly="{isDeleteOrView}">
    </div>
</div>

<div class="card-text mb-4">
    <div class="input-group border border-1
border-info-subtle rounded">
        <label for="inputCorporateEmail" class="col-3
input-group-text text-info">Corporate Email</label>
        <input id="inputCorporateEmail" class="col-auto
form-control text-dark" type="text" required
            th:field="{corporateEmail}"
            th:minlength="17" th:maxlength="123"
            th:readonly="{isDeleteOrView}">
    </div>
</div>

<div class="card-text mb-4">
    <div class="input-group border border-1
border-info-subtle rounded">
        <label for="inputUsername" class="col-3
input-group-text text-info">Username</label>
        <input id="inputUsername" class="col-auto
form-control text-dark" type="text" required
            th:field="{username}"
            th:minlength="8" th:maxlength="32"
            th:readonly="{isDeleteOrView}">
    </div>
</div>

<div class="card-text mb-4">
    <div class="input-group border border-1
border-info-subtle rounded">
        <label for="inputSurname" class="col-3
input-group-text text-info">Surname</label>

```

```

        <input id="inputSurname" class="col-auto
form-control text-dark" type="text" required
        th:field="*{surname}"
        th:minlength="1" th:maxlength="50"
        th:readonly="{isDeleteOrView}">

    </div>
</div>

    <div class="card-text mb-4">
        <div class="input-group border border-1
border-info-subtle rounded">
            <label for="inputName" class="col-3
input-group-text text-info">Name</label>
            <input id="inputName" class="col-auto
form-control text-dark" type="text" required
            th:field="*{name}"
            th:minlength="1" th:maxlength="50"
            th:readonly="{isDeleteOrView}">

        </div>
    </div>

    <div class="card-text mb-4">
        <div class="input-group border border-1
border-info-subtle rounded ">
            <label for="inputPatronymic" class="col-3
input-group-text text-info">Patronymic</label>
            <input id="inputPatronymic" class="col-auto
form-control text-dark" type="text" required
            th:field="*{patronymic}"
            th:minlength="1" th:maxlength="50"
            th:readonly="{isDeleteOrView}">

        </div>
    </div>

    <div class="card-text d-flex align-content-center
justify-content-start mb-4">

```

```

        <div class="col-3 border border-1
border-info-subtle rounded d-inline-flex p-0 m-0 me-1">
            <span class="d-flex flex-fill input-group-text
text-info">Role</span>
        </div>

        <div class="col">
            <select class="form-select border border-1
border-info-subtle" aria-label="Role list select"
                th:field="*{role}" required
                th:disabled="{isDeleteOrView}">
                <option th:selected="true"
                    th:value="{mode ne
T(ntukhpi.csit.semit.riv.webappsrivlab4.controller.Mode).ADD
? user?.role : null}"
                    th:text="{mode ne
T(ntukhpi.csit.semit.riv.webappsrivlab4.controller.Mode).ADD
?
user?.role?.getFormattedName() : 'Select a Role'}">
                    Select a Role
                </option>

                <div th:each="role : {roleList}">
                    <option th:value="{role?.name()}"
th:text="{role?.getFormattedName()}">
                    </option>
                </div>
            </select>
        </div>
    </div>

    <div class="d-flex justify-content-evenly">
        <button type="submit" class="btn btn-outline-info
border-2"
            th:title="'Save record'"

```

```

        th:if="{isNotDeleteAndNotView}">
        <i class="bi bi-floppy"></i>
    </button>

    <button type="reset" class="btn btn-outline-info
border-2"

        onclick="form.reset()"
        th:title="'Reset fields'"
        th:if="{isNotDeleteAndNotView}">
        <i class="bi bi-x-lg"></i>
    </button>

    <button type="button" class="btn btn-outline-info
border-2"

        onclick="submitConfirmation()"
        th:title="'Delete record'"
        th:if="{mode eq
T(ntukhpi.csit.semit.riv.webappsrivlab4.controller.Mode).DELETE}">
        <i class="bi bi-trash3"></i>
    </button>
</div>
</form>
</div>

<div class="d-flex justify-content-center mt-5">
    <a class="btn btn-outline-info" type="button"
        th:title="'Go to User Table'"
        th:href="{/users}">
        <i class="bi bi-table"></i>
    </a>
</div>
</div>

<footer th:replace="{component/Footer :: footer}"></footer>

<script type="text/javascript">

```

```

document.addEventListener("DOMContentLoaded", function () {
    fillUsernameOnCorpEmail();
    setAllInputsValidity();
});

function fillUsernameOnCorpEmail() {
    const corporateEmailInput =
document.getElementById("inputCorporateEmail");
    const usernameInput =
document.getElementById("inputUsername");

    corporateEmailInput.addEventListener("input", function () {
        const emailValue = corporateEmailInput.value;
        const atIndex = emailValue.indexOf("@");

        if (atIndex > 0) {
            usernameInput.value = emailValue.substring(0,
atIndex);
        } else {
            usernameInput.value = "";
        }
    });
}

function setAllInputsValidity() {
    setPatternAttr();
    setAllStringInputValidity();

    const form = document.getElementById('userForm');

    form.addEventListener('submit', function (event) {
        event.preventDefault();

        if (!form.checkValidity()) {
            event.stopPropagation();
            form.reportValidity();
        }
    });
}

```

```

        } else {
            form.submit();
        }
    });

    function setPatternAttr() {
        const regexUsername = /^(?=.{8,32}$)[a-zA-Z0-9\W]+$/;
        const regexCorporateEmail = new
RegExp("^[a-z](\\.?[a-z]+){1,99}@[a-z]{1,10}\\.khpi\\.edu\\.ua$");
        // const regexUsername = new
RegExp("^(?=.{8,32}$)[a-zA-Z0-9._-]+$");
        const regexNames = new
RegExp("^(?=.{1,50}$)[A-ЩЮЯҐЄІİİ](?:[A-ЯҐЄІİİа-яґєіі]*[A-ЯҐЄІİİа-яґєіі])?(?:[
\\-] [A-ЩЮЯҐЄІİİ](?:[A-ЯҐЄІİİа-яґєіі]*[A-ЯҐЄІİİа-яґєіі])?)*$");

        document.getElementById('inputCorporateEmail').setAttribute('patte
rn', regexCorporateEmail.source);

        document.getElementById('inputUsername').setAttribute('pattern',
regexUsername.source);

        document.getElementById('inputSurname').setAttribute('pattern',
regexNames.source);

        document.getElementById('inputName').setAttribute('pattern',
regexNames.source);

        document.getElementById('inputPatronymic').setAttribute('pattern',
regexNames.source);
    }

    function setAllStringInputValidity() {
        const messages = [

```

```

        "The corporate email should match the format
'name.surname@faculty.khpi.edu.ua'." ,
        "The username must contain Latin letters and may
contain numbers and special characters '.', '_', '-'." ,
        "Value must be Ukrainian char string between 1 and
50 chars long, including space and apostrophe." ,
        "Value must be Ukrainian char string between 1 and
50 chars long, including space and apostrophe." ,
        "Value must be Ukrainian char string between 1 and
50 chars long, including space and apostrophe."
    ];

    document.getElementById('userForm')
        .querySelectorAll('input[pattern]')
        .forEach((input, i) => {
            input.addEventListener('input', function () {
                if (this.validity.patternMismatch) {
                    this.setCustomValidity(messages[i]);
                } else {
                    this.setCustomValidity('');
                }

                this.reportValidity();
            });
        });
    });

    function submitConfirmation() {
        const message = 'Are you sure you want to delete that
record?';

        if (confirm(message) === true) {
            document.getElementById('userForm').submit();
        }
    }
}

```

```

</script>

<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstra
p.bundle.min.js"

integrity="sha384-YvpcrYf0tY3lHB60NNkmXc5s9fDVZLESaAA55NDzOxhy9Gkc
Ids1K1eN7N6jIeHz"

        crossorigin="anonymous"></script>
</body>
</html>

```

К.17 Код файла UserTable.html

```

<!DOCTYPE html>
<html lang="en" xmlns:th="http://www.thymeleaf.org"
xmlns:sec="http://www.thymeleaf.org/extras/spring-security">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width,
initial-scale=1">
    <title>User Table</title>
    <link rel="icon" type="image/x-icon"
th:href="@{/image/icon/favicon_square.png}">
    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootst
rap.min.css"
        rel="stylesheet"

integrity="sha384-QWTKZyjpPEjISv5WaRU90FeRpok6YctnYmDr5pNlyT2bRjXh
0JMhY6hW+ALEwIH"

        crossorigin="anonymous">
    <link rel="stylesheet"

```



```

href="https://cdn.jsdelivr.net/npm/bootstrap-icons@1.11.3/font/boo
tstrap-icons.min.css">
</head>
<body>

<header th:replace="~{component/NavigationBar ::
header}"></header>

<div th:replace="~{component/ErrorToast :: errorToast}"></div>
<div th:replace="~{component/WarningToast :: warningToast}"></div>
<div th:replace="~{component/SuccessToast :: successToast}"></div>

<div class="mx-3 mt-3 mb-5 font-monospace">
  <div class="row">
    <h1 class="text-info fs-1 my-3 fw-semibold
text-center">User Table</h1>
  </div>

  <div class="row align-items-start">
    <div class="col-2 mx-2">
      <div class="card row p-2 mb-4 border border-3
border-info-subtle bg-light shadow">
        <div class="card-body col">
          <div class="card-title row">
            <p class="text-info fs-5 fw-bold p-0
m-0">Search</p>
            <p class="text-secondary fw-lighter p-0
m-0" style="font-size: 0.8rem;">
              Search by full name (matching start,
case-insensitive)
            </p>
          </div>
          <div class="card-text col mt-3"
id="formSearch">

```

```

        th:method="'get'" th:action="@{/users}">
        <div class="row mb-3">
            <label for="inputSearch"
class="visually-hidden">Search</label>
            <input type="search"
class="form-control border border-2 border-info-subtle"
            id="inputSearch" name="search"
placeholder="Type to search...">
        </div>

        <div class="col d-flex
justify-content-evenly align-content-center text-center p-0 m-0">
            <a type="button" id="btn-search"
onclick="applySearch(this)"
            class="btn btn-outline-info
border-2" th:alt-title="'Apply Search'">
                <i class="bi bi-search"></i>
            </a>

            <a type="button" id="btn-search-reset"
onclick="resetSearch(this)"
            class="btn btn-outline-info border-2
button-reset" th:title="'Reset Search'">
                <i class="bi bi-x-lg"></i>
            </a>
        </div>
    </form>
</div>
</div>

    <div class="card row p-2 mb-4 border border-3
border-info-subtle bg-light shadow">
        <div class="card-body col">
            <div class="card-title row">
                <p class="text-info fs-5 fw-bold p-0
m-0">Filters</p>

```

```

        </div>

        <div class="card-subtitle row mt-2"
id="filterRole">
            <p class="text-info fs-6 fw-bold p-0
m-0">Role</p>
        </div>

        <form class="card-text col mt-2"
id="formFilterRole"
            th:method="'get'" th:action="@{/users}">
            <div class="mt-2 mb-3">
                <div class="form-check">
                    <input class="form-check-input"
type="checkbox" name="role"
                        th:id="'role_' +
T(ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.enums.Role).ADMIN.name()"
th:value="'${T(ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.enums.Role).ADMIN.name()}'">
                        <label class="form-check-label"
style="color: #fd7e14;"
                        th:for="'role_' +
T(ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.enums.Role).ADMIN.name()"
th:text="'${T(ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.enums
.Role).ADMIN.getFormattedName()}'">
                        </label>
                    </div>
                </div>

                <div class="form-check">
                    <input class="form-check-input"
type="checkbox" name="role"

```

```

th:id="${'role_' +
T(ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.enums.Role).ENTR
ANT_MANAGER.name()}"

th:value="${T(ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.enum
s.Role).ENTRANT_MANAGER.name()}">
    <label class="form-check-label"
style="color: #198754;"
th:for="${'role_' +
T(ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.enums.Role).ENTR
ANT_MANAGER.name()}"

th:text="${T(ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.enums
.Role).ENTRANT_MANAGER.getFormattedName()}">
    </label>
</div>

<div class="form-check">
    <input class="form-check-input"
type="checkbox" name="role"
th:id="${'role_' +
T(ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.enums.Role).STUD
ENT_MANAGER.name()}"

th:value="${T(ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.enum
s.Role).STUDENT_MANAGER.name()}">
    <label class="form-check-label"
style="color: #6f42c1;"
th:for="${'role_' +
T(ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.enums.Role).STUD
ENT_MANAGER.name()}"

th:text="${T(ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.enums
.Role).STUDENT_MANAGER.getFormattedName()}">
    </label>
</div>

```

```

        <div class="form-check">
            <input class="form-check-input"
type="checkbox" name="role"
                th:id="${'role_' +
T(ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.enums.Role).ENTR
ANT_VIEWER.name()}"

                th:value="${T(ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.enum
s.Role).ENTRANT_VIEWER.name()}">
            <label class="form-check-label"
style="color: #20c997;"
                th:for="${'role_' +
T(ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.enums.Role).ENTR
ANT_VIEWER.name()}"

                th:text="${T(ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.enums
.Role).ENTRANT_VIEWER.getFormattedName()}">
            </label>
        </div>

        <div class="form-check">
            <input class="form-check-input"
type="checkbox" name="role"
                th:id="${'role_' +
T(ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.enums.Role).STUD
ENT_VIEWER.name()}"

                th:value="${T(ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.enum
s.Role).STUDENT_VIEWER.name()}">
            <label class="form-check-label"
style="color: #6610f2;"
                th:for="${'role_' +
T(ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.enums.Role).STUD
ENT_VIEWER.name()}"

```

```

th:text="{T(ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.enums
.Role).STUDENT_VIEWER.getFormattedName()}">
        </label>
    </div>
</div>

    <div class="col d-flex
justify-content-evenly align-content-center text-center p-0 m-0">
        <a type="button"
id="btn-filter-funding-type" onclick="applyFilterRole(this)"
        class="btn btn-outline-info
border-2" th:title="'Apply Filter'">
            <i class="bi bi-funnel"></i>
        </a>

        <a type="button"
id="btn-filter-funding-type-reset" onclick="resetFilterRole(this)"
        class="btn btn-outline-info border-2
button-reset" th:title="'Reset Filter'">
            <i class="bi bi-x-lg"></i>
        </a>
    </div>
</form>
</div>
</div>

    <div class="card row p-2 mb-4 border border-3
border-info-subtle bg-light shadow"

sec:authorize="hasRole(T(ntukhpi.csit.semit.riv.webappsrivlab4.mod
el.DAO.enums.Role).ADMIN.name())">
        <div class="card-body col">
            <div class="card-title row">
                <p class="text-info fs-5 fw-bold p-0
m-0">Add record</p>

```

```

        <p class="text-secondary fw-lighter p-0
m-0" style="font-size: 0.8rem;">
            Add new record to the end of the table
        </p>
    </div>

    <form class="card-text col mt-3"
id="formInsert"
        th:method="'get'" th:action="@{/users}">
        <div class="col d-flex
justify-content-evenly align-content-center text-center p-0 m-0">
            <a th:href="@{/users/add}"
id="btn-add-user"
                class="btn btn-outline-info
border-2" role="button" th:title="'Add record'">
                <i class="bi bi-plus-lg"></i>
            </a>
        </div>
        </form>
    </div>
</div>

<div class="col">
    <table id="userTable" class="table table-sm table-hover
font-monospace
        table-bordered border border-3
border-info-subtle shadow-lg">
        <thead class="table-info text-center fw-medium
text-secondary-emphasis">
            <tr>
                <th class="text-center align-content-center
text-info">#</th>

                <div class="p-0 m-0" th:each="fieldName :
${fields}">

```

```

        <th class="text-center
align-content-center">
            <div class="d-flex p-0 m-0">
                <div class="flex-grow-1
align-content-center justify-content-center p-0 m-0">
                    <span class="">
                        <a class="sortLink
link-info link-offset-2 link-offset-3-hover link-underline
link-underline-opacity-0 link-underline-opacity-75-hover"
th:id="${fieldName[0]}"
th:text="${fieldName[1]}"
onclick="applySort(this)"></a>
                    </span>
                </div>
                <div class="d-none flex-shrink-1
align-content-center justify-content-end ms-2 p-0 m-0"
style="width: 20px;">
                    <button type="button"
class="btn bg-transparent p-0 m-0" title="Sort records">
                        <i class="bi
bi-sort-down-alt sortingAsc" th:title="'Ascending Sort'"></i>
                        <i class="bi bi-sort-down
sortingDesc" th:title="'Descending Sort'"></i>
                    </button>
                </div>
            </div>
        </th>
    </div>

    <th class="text-center align-content-center
text-info"

sec:authorize="hasRole(T(ntukhpi.csit.semit.riv.webappsrivlab4.mod
el.DAO.enums.Role).ADMIN.name())">

```



```

Operations
</th>
</tr>
</thead>

<tbody class="table-light fw-light
text-primary-emphasis" th:each="user, iterStat : ${users}">
  <tr>
    <td class="text-center align-content-center"
th:text="${iterStat.index + 1}"></td>
    <td class="text-start align-content-center"
th:text="${user?.surname} + ' ' +
${user?.name} + ' ' + ${user?.patronymic}"></td>
    <td class="text-center align-content-center"
th:switch="${user?.role?.name()}">
      <span
th:case="${T(ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.enums
.Role).ADMIN.name()}"
style="color: #fd7e14;"
th:text="${user?.role?.getValue()}"></span>
      <span
th:case="${T(ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.enums
.Role).ENTRANT_MANAGER.name()}"
style="color: #198754;"
th:text="${user?.role?.getValue()}"></span>
      <span
th:case="${T(ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.enums
.Role).STUDENT_MANAGER.name()}"
style="color: #6f42c1;"
th:text="${user?.role?.getValue()}"></span>
      <span
th:case="${T(ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.enums
.Role).ENTRANT_VIEWER.name()}"
style="color: #20c997;"
th:text="${user?.role?.getValue()}"></span>

```

```

        <span
th:case="${T(ntukhpi.csit.semit.riv.webappsrivlab4.model.DAO.enums
.Role).STUDENT_VIEWER.name()}"
style="color: #6610f2;"
th:text="${user?.role?.getValue()}"></span>
        </td>
        <td class="text-start align-content-center"
th:text="${user?.username}"></td>
        <td class="text-start align-content-center"
th:text="${user?.corporateEmail}"></td>
        <td class="text-center align-content-center"

sec:authorize="hasRole(T(ntukhpi.csit.semit.riv.webappsrivlab4.mod
el.DAO.enums.Role).ADMIN.name())">
        <div class="d-flex justify-content-evenly
p-0 m-0">
                <a type="button"
th:href="@{/users/__$${user?.id}__/_edit}"
sec:authorize="hasRole(T(ntukhpi.csit.semit.riv.webappsrivlab4.mod
el.DAO.enums.Role).ADMIN.name())"
                class="p-0 m-0 border border-0
bg-transparent button-update" th:title="'Edit record'">
                        <i class="bi bi-pencil
text-warning"></i>
                </a>
                <a type="button"
th:href="@{/users/__$${user?.id}__/_delete}"
sec:authorize="hasRole(T(ntukhpi.csit.semit.riv.webappsrivlab4.mod
el.DAO.enums.Role).ADMIN.name())"
                class="p-0 m-0 border border-0
bg-transparent button-delete" th:title="'Delete record'">
                        <i class="bi bi-trash3
text-danger"></i>
                </a>

```

```

        </div>
    </td>
</tr>
</tbody>
</table>
    <p th:if="{users == null or users.isEmpty()}"
class="text-dark text-center">No data available</p>
    <br>
</div>
</div>

<div class="d-flex justify-content-center">
    <a th:href="{/home}" class="btn btn-outline-info"
type="button">
        <i class="bi bi-house"></i>
    </a>
</div>
</div>

<footer th:replace="{component/Footer :: footer}"></footer>

<script type="text/javascript">
    document.addEventListener("DOMContentLoaded", function () {
        setSearchPatternAttr();
        setSearchInputValidity();

        setInputValueFromGetParams();
    });

    function setSearchPatternAttr() {
        const regexSearch = new
RegExp("^[A-ЩЬЮЯЄІЇҐа-щьюяєіїґ'\\"-\\d ]{1,200}$");

    document.getElementById('inputSearch').setAttribute('pattern',
regexSearch.source);
    }

```

```

function setSearchInputValidity() {
    const message = "Value is a Ukrainian a string between 1
and 200 chars long, optionally including numbers, dash,
apostrophe, space.";

document.getElementById('inputSearch').addEventListener('input',
function () {
    if (this.validity.patternMismatch) {
        this.setCustomValidity(message);
    } else {
        this.setCustomValidity('');
    }

    this.reportValidity();
});
}

function setInputValueFromGetParams() {
    const currentURL = new URL(window.location.href);
    const params = new
URLSearchParams(currentURL.searchParams);
    const inputFields = document.getElementsByTagName('input');

    for (const input of inputFields) {
        if (params.has(input.name)) {
            switch (input.type) {
                case 'radio':
                    if (input.value ===
params.get(input.name).trim()) {
                        input.setAttribute('checked', 'true');
                    }
                    break;
                case 'checkbox':

```

```

        const paramValues =
params.get(input.name).trim().split('-');
        paramValues.forEach(value => {
            if (input.value === value) {
                input.setAttribute('checked',
'true');
            }
        });
        break;
    default:
        input.value = params.get(input.name);
        break;
    }
}
}
}

function applySort(element) {
    const currentURL = new URL(window.location.href);
    const params = new
URLSearchParams(currentURL.searchParams);

    const paramName = 'sort';
    const paramFieldName = element.id;
    let paramSortOrder = 'asc';
    const currentSortValue = params.get(paramName);

    if (currentSortValue) {
        const currentSortFieldName =
currentSortValue.split('-')[0];
        const currentSortOrder = currentSortFieldName ===
paramFieldName ? currentSortValue.split('-')[1] : '';
        paramSortOrder = currentSortOrder === 'asc' ? 'desc' :
'asc';
    }
}

```

```

        params.set(paramName, paramFieldName + '-' +
paramSortOrder);

        const newURL = currentURL.pathname + '?' +
params.toString();
        element.setAttribute('href', newURL);
    }

    function applySearch(linkButton) {
        const currentURL = new URL(window.location.href);
        const params = new
URLSearchParams(currentURL.searchParams);

        setFilteringParameter(params,
document.getElementById('inputSearch'));

        const newURL = currentURL.pathname + '?' +
params.toString();
        linkButton.setAttribute('href', newURL);
    }

    function resetSearch(linkButton) {
        const currentURL = new URL(window.location.href);
        const params = new
URLSearchParams(currentURL.searchParams);

        resetFilteringParameter(params,
document.getElementById('inputSearch'));

        const newURL = currentURL.pathname + '?' +
params.toString();
        document.getElementById('formSearch').reset();
        linkButton.setAttribute('href', newURL);
    }

    function applyFilterRole(linkButton) {

```

```

        const currentURL = new URL(window.location.href);
        const params = new
URLSearchParams(currentURL.searchParams);

        function setFilteringParamForRole(params, inputElement) {
            const paramName = inputElement.name;
            const paramValue =
inputElement.value.toString().trim();

            if (paramValue.length > 0) {
                if (params.has(paramName)) {
                    const newParamValue = params.get(paramName) +
"-" + paramValue;
                    params.set(paramName, newParamValue);
                } else {
                    params.set(paramName, paramValue);
                }
            } else {
                params.delete(paramName);
            }
        }

        const inputAdmin = document.getElementById('role_ADMIN');
        const inputEntrantManager =
document.getElementById('role_ENTRANT_MANAGER');
        const inputStudentManager =
document.getElementById('role_STUDENT_MANAGER');
        const inputEntrantViewer =
document.getElementById('role_ENTRANT_VIEWER');
        const inputStudentViewer =
document.getElementById('role_STUDENT_VIEWER');

        params.delete(inputAdmin.name);

        if (inputAdmin.checked) setFilteringParamForRole(params,
inputAdmin);

```

```

        if (inputEntrantManager.checked)
setFilteringParamForRole(params, inputEntrantManager);
        if (inputStudentManager.checked)
setFilteringParamForRole(params, inputStudentManager);
        if (inputEntrantViewer.checked)
setFilteringParamForRole(params, inputEntrantViewer);
        if (inputStudentViewer.checked)
setFilteringParamForRole(params, inputStudentViewer);

        const newURL = currentURL.pathname + '?' +
params.toString();
        linkButton.setAttribute('href', newURL);
    }

    function resetFilterRole(linkButton) {
        const currentURL = new URL(window.location.href);
        const params = new
URLSearchParams(currentURL.searchParams);

        resetFilteringParameter(params,
document.getElementById('role_ADMIN'));
        resetFilteringParameter(params,
document.getElementById('role_ENTRANT_MANAGER'));
        resetFilteringParameter(params,
document.getElementById('role_STUDENT_MANAGER'));
        resetFilteringParameter(params,
document.getElementById('role_ENTRANT_VIEWER'));
        resetFilteringParameter(params,
document.getElementById('role_STUDENT_VIEWER'));

        const newURL = currentURL.pathname + '?' +
params.toString();
        document.getElementById('formFilterRole').reset();
        linkButton.setAttribute('href', newURL);
    }

```



```

function setFilteringParameter(params, inputElement) {
    const paramName = inputElement.name;
    const paramValue = inputElement.value.toString().trim();

    if (paramValue.length > 0) {
        params.set(paramName, paramValue);
    } else {
        params.delete(paramName);
    }
}

function resetFilteringParameter(params, inputElement) {
    const paramName = inputElement.name;
    params.delete(paramName);

    if (inputElement.type === 'radio' || inputElement.type ===
'checkbox') {
        inputElement.removeAttribute('checked');
    }
}
</script>

<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstra
p.bundle.min.js"

integrity="sha384-YvpcrYf0tY3lHB60NNkmXc5s9fDVZLESaAA55NDzOxhy9Gkc
IdslK1eN7N6jIeHz"

crossorigin="anonymous"></script>

</body>
</html>

```

ДОДАТОК Л

Результати роботи вебзастосунку

Л.1 Результати роботи вебзастосунку для реєстрації нового користувача та відображення доступного функціоналу для Переглядача студентів

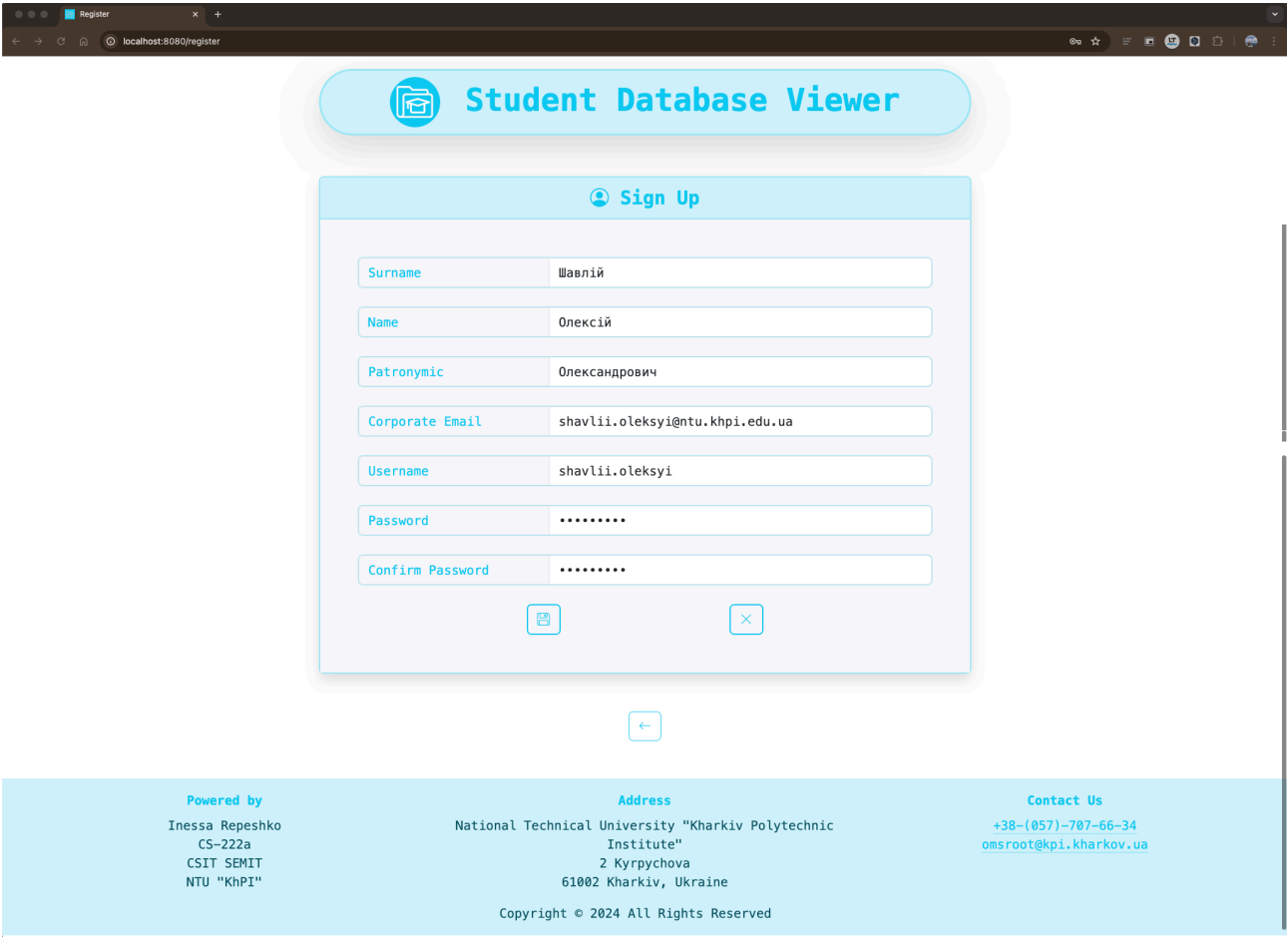


Рисунок Л.1.1 – Сторінка реєстрації з введеними обліковими даними нового користувача

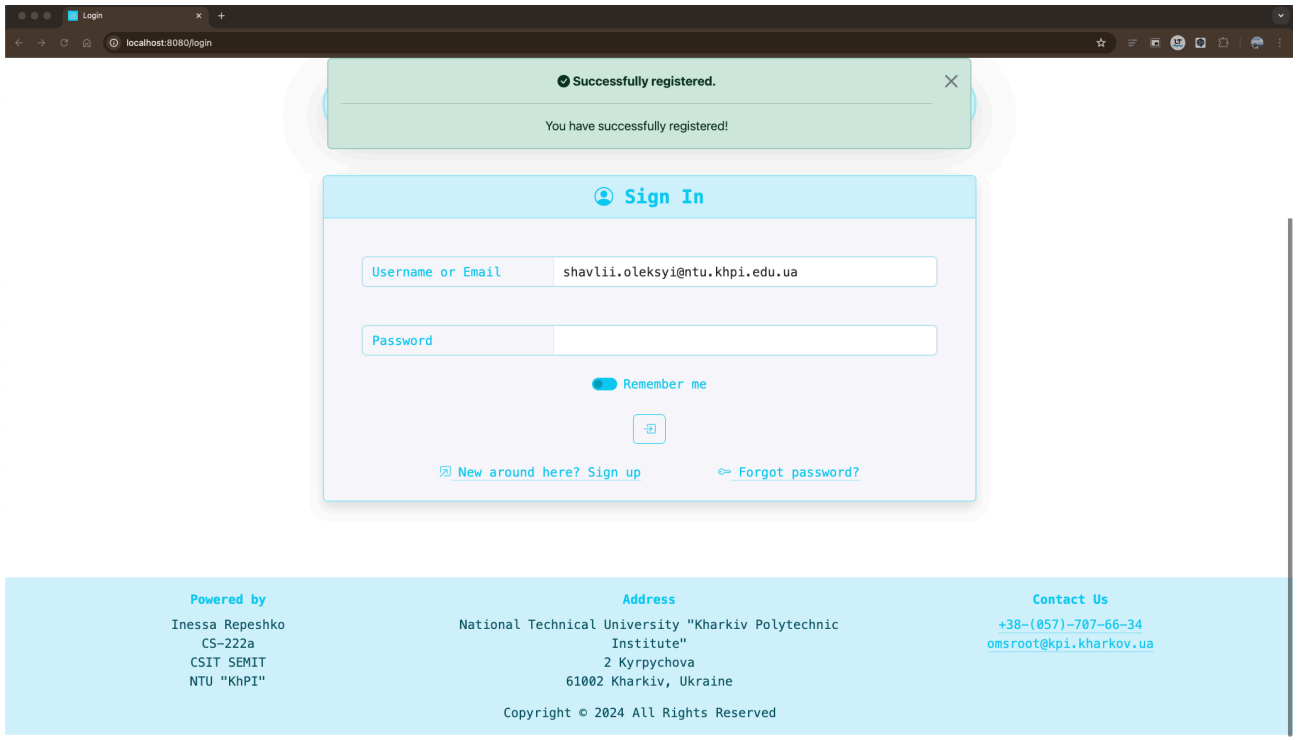


Рисунок Л.1.2 – Сторінка автентифікації з повідомленням про успішну реєстрацію нового користувача

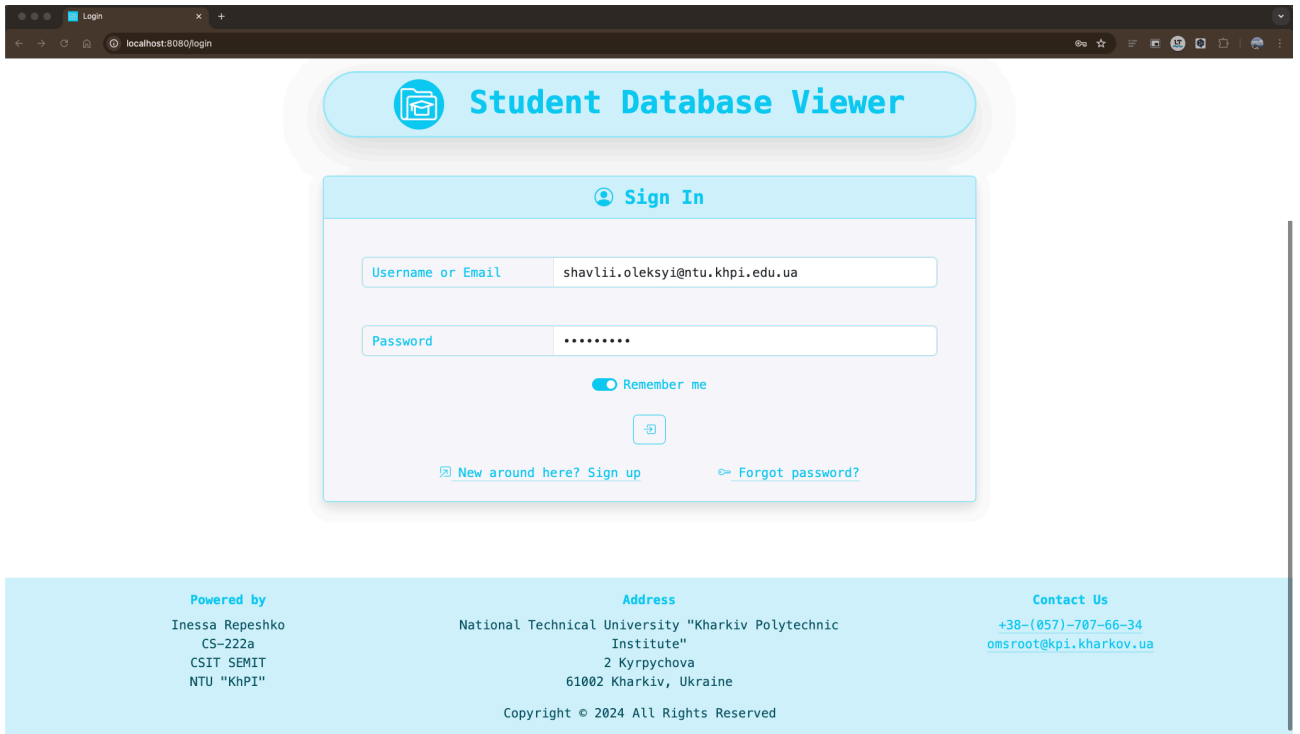


Рисунок Л.1.3 – Сторінка автентифікації з введеними обліковими даними зареєстрованого користувача

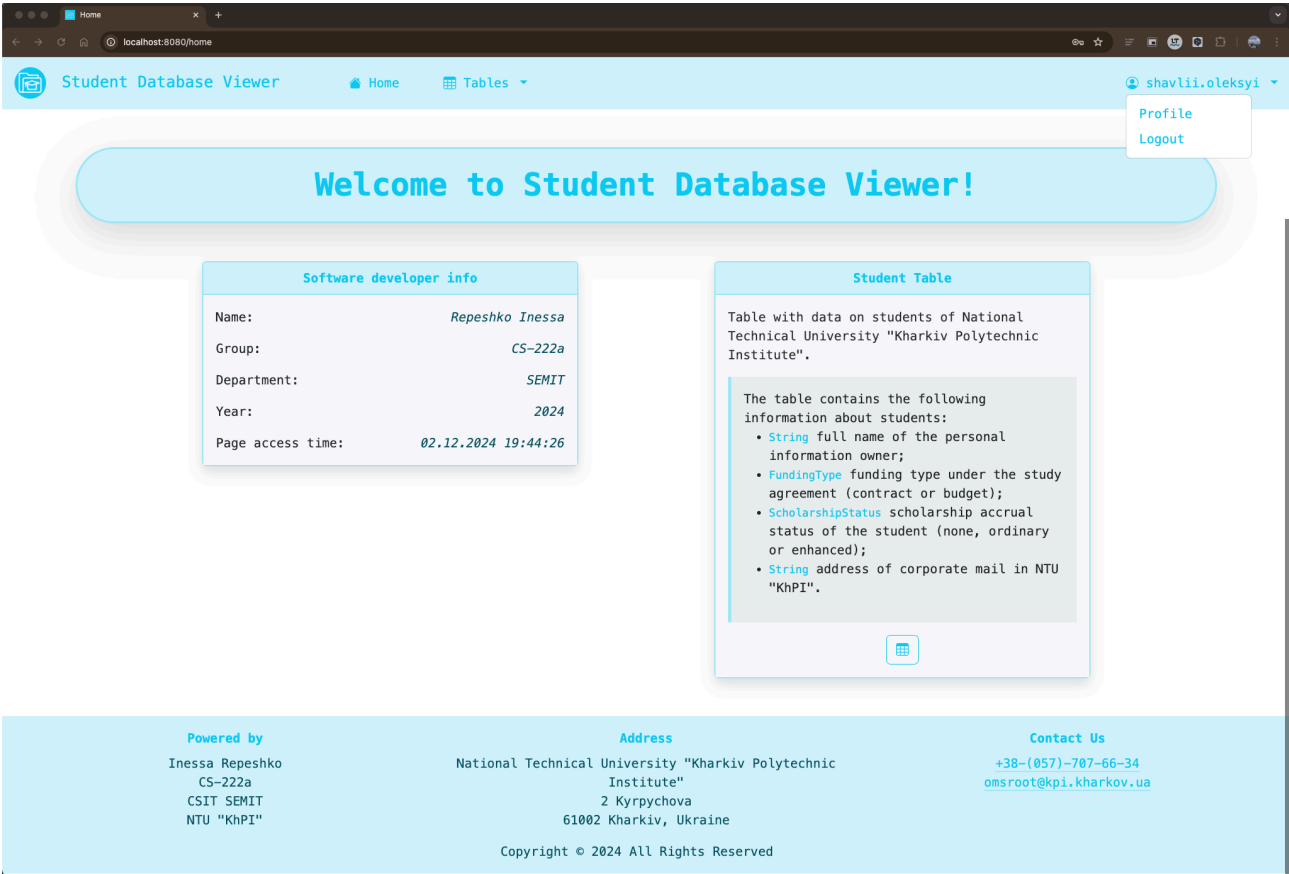


Рисунок Л.1.4 – Домашня сторінка вебзастосунку зареєстрованого користувача
Переглядача студентів

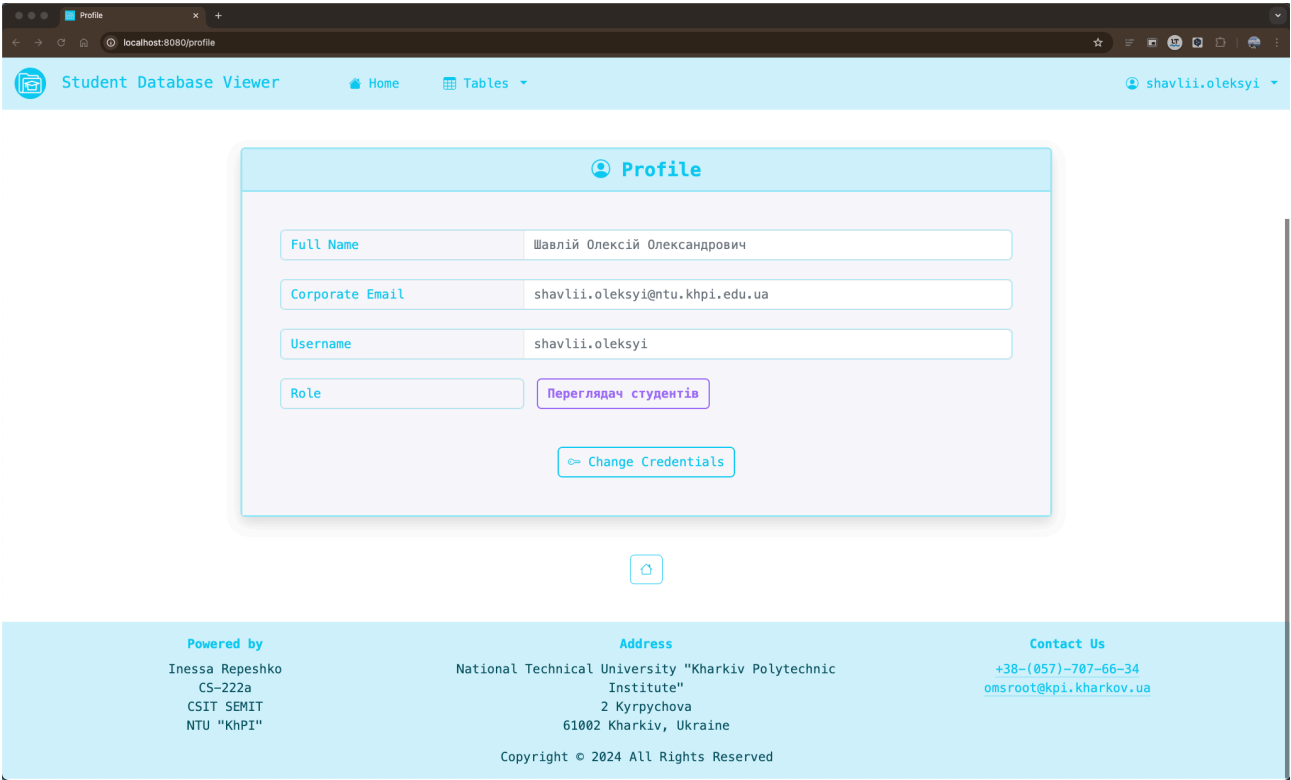


Рисунок Л.1.5 – Сторінка профілю зареєстрованого користувача з роллю Переглядач студентів

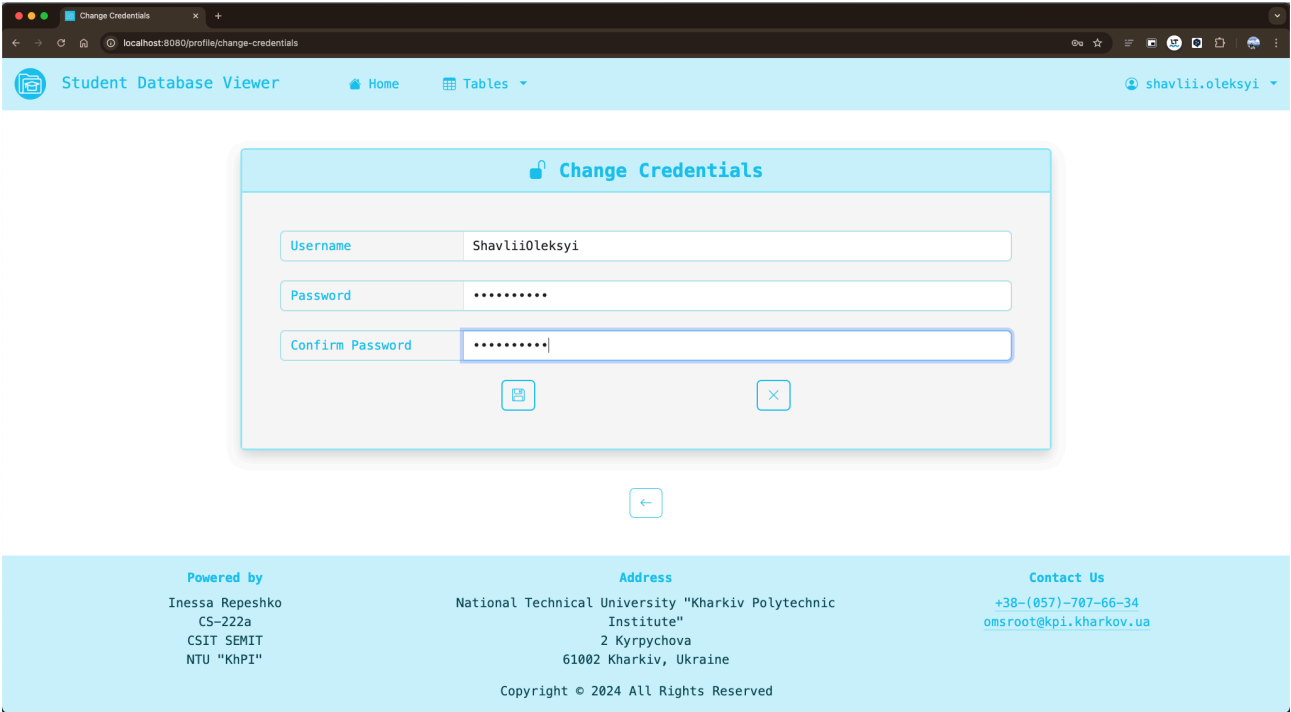


Рисунок Л.1.6 – Сторінка зміни облікових даних користувача

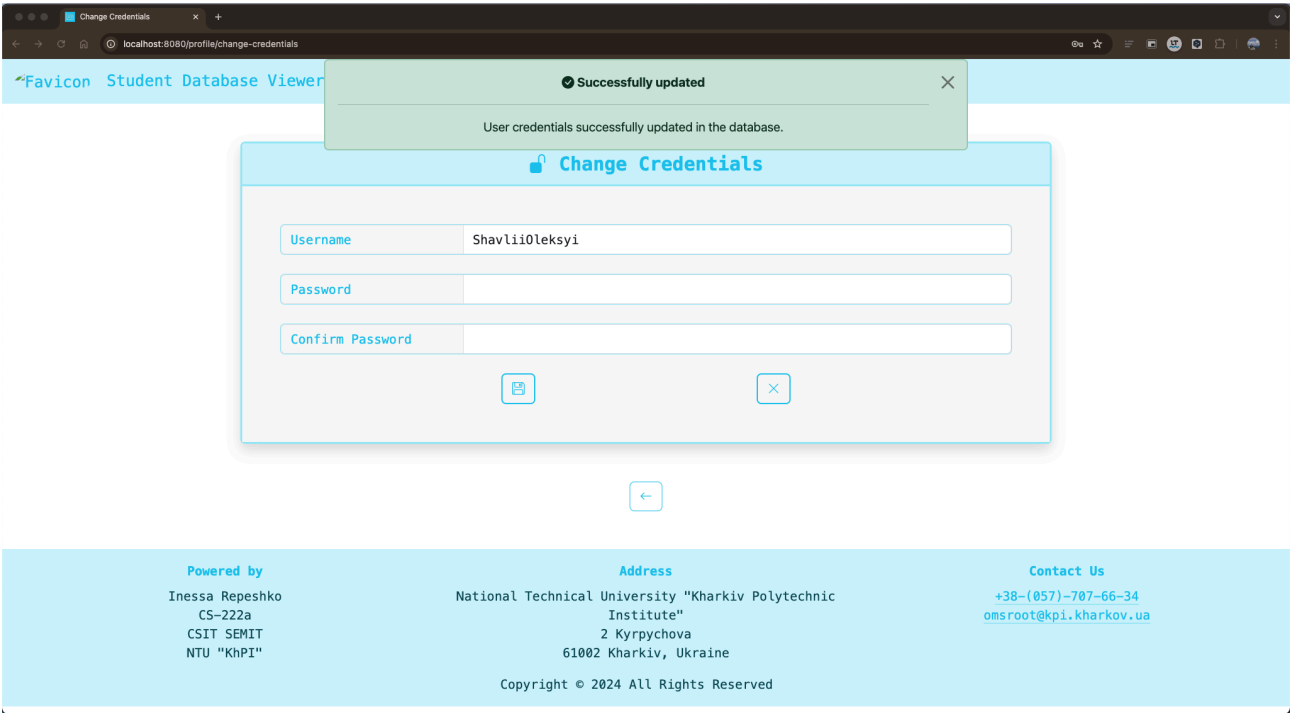


Рисунок Л.1.7 – Сторінка зміни облікових даних з повідомленням про успішне оновлення облікових даних

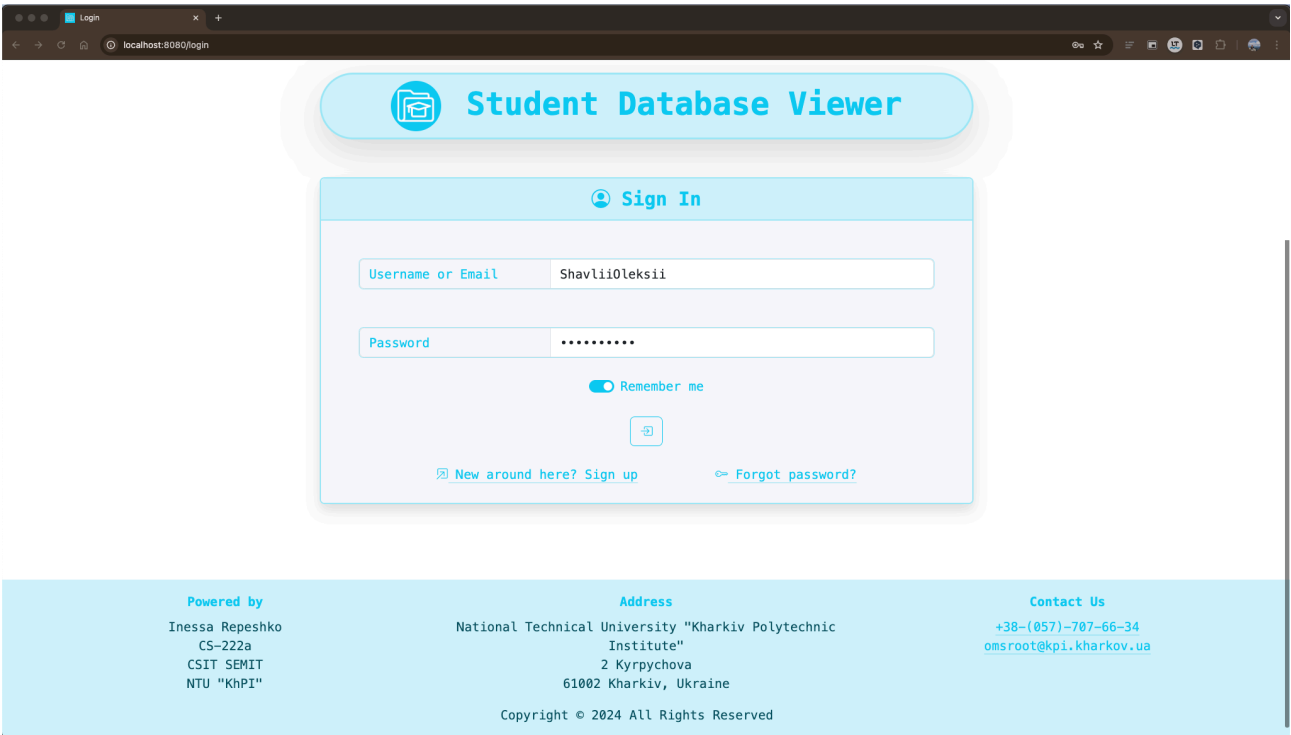


Рисунок Л.1.8 – Сторінка повторної автентифікації після зміни даних з оновленими обліковими даними користувача Переглядача студентів

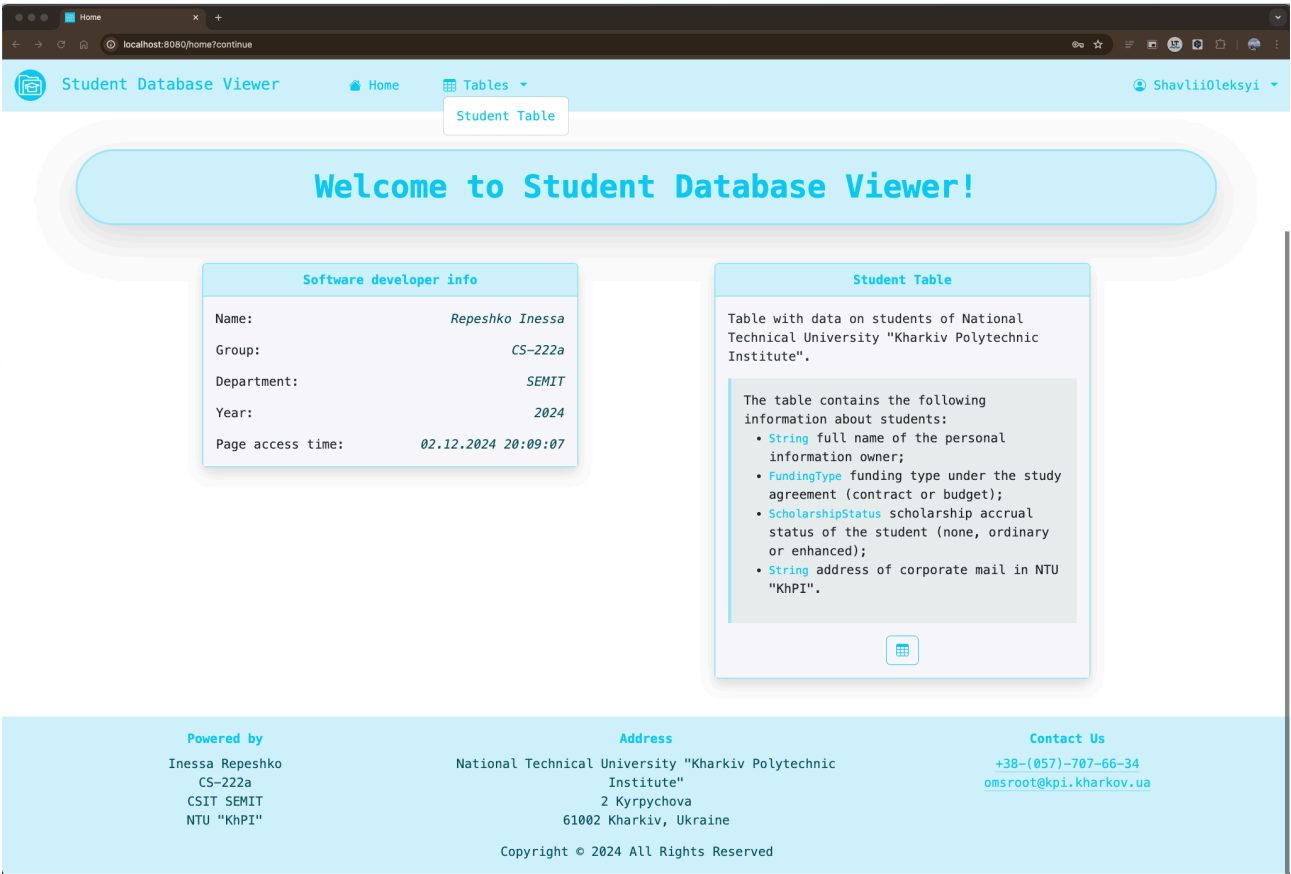


Рисунок Л.1.9 – Домашня сторінка для користувача з оновленими обліковими даними з роллю Переглядач студентів

Student Table

localhost:8080/students

Student Database Viewer

Home

Tables

Shavlii10leksyi

Student Table

Search

Search by full name (matching start, case-insensitive)

Type to search...

🔍

✕

Filters

Funding Type

Budget

Contract

🔼

✕

Scholarship Status

Enhanced

Ordinary

None

🔼

✕

#	Full Name	Funding Type	Scholarship Status	Corporate Email
1	Бабічев Олексій Вікторович	бюджет	звичайна	oleksii.babichiev@cs.khpi.edu.ua
2	Волков Дмитро Олександрович	бюджет	відсутня	dmytro.o.volkov@cs.khpi.edu.ua
3	Геєнко Анна Олексіївна	бюджет	відсутня	anna.hieienko@cs.khpi.edu.ua
4	Голоп'юров Микита Сергійович	бюджет	звичайна	mykyta.holoporov@cs.khpi.edu.ua
5	Гриценко Максим Георгійович	бюджет	підвищена	maksym.hrytsenko@cs.khpi.edu.ua
6	Жарий Вадим Віталійович	контракт	відсутня	vadym.zharyi@cs.khpi.edu.ua
7	Жупанов Андрій Володимирович	бюджет	звичайна	andrii.zhupanov@cs.khpi.edu.ua
8	Кльоз Михайло Романович	контракт	відсутня	mykhailo.kloz@cs.khpi.edu.ua
9	Коваль Микита Артемович	бюджет	відсутня	mykyta.koval@cs.khpi.edu.ua
10	Ковтун Єгор Вячеславович	бюджет	підвищена	yehor.kovtun@cs.khpi.edu.ua
11	Колесниченко Денис Євгенович	бюджет	звичайна	denys.kolesnychenko@cs.khpi.edu.ua
12	Ласкевич Андрій Сергійович	бюджет	звичайна	andrii.laskevych@cs.khpi.edu.ua
13	Лукаш Антон Олександрович	бюджет	звичайна	anton.lukash@cs.khpi.edu.ua
14	Палій Нікіта Андрійович	бюджет	відсутня	nikita.palii@cs.khpi.edu.ua
15	Прокопов Владислав Сергійович	бюджет	звичайна	vladyslav.prokopov@cs.khpi.edu.ua
16	Репешко Інесса Віталіївна	бюджет	підвищена	inessa.repeshko@cs.khpi.edu.ua
17	Скиба Андрій Григорович	контракт	відсутня	andrii.h.skyba@cit.khpi.edu.ua
18	Цибань Ольга Юріївна	бюджет	відсутня	olha.tsyban@cs.khpi.edu.ua
19	Шевченко Данило Валерійович	бюджет	звичайна	danylo.v.shevchenko@cs.khpi.edu.ua
20	Антипенко Катерина Віталіївна	бюджет	звичайна	kateryna.antypenko@cs.khpi.edu.ua
21	Багрянцева Марина Русланівна	бюджет	звичайна	marina.bahriantseva@cs.khpi.edu.ua
22	Бондаренко Ярослав Юрійович	бюджет	підвищена	yaroslav.y.bondarenko@cs.khpi.edu.ua
23	Булгаков Віктор Олександрович	бюджет	відсутня	viktor.bulhakov@cs.khpi.edu.ua
24	Васильєв Михайло Олександрович	бюджет	відсутня	mykhailo.vasyliiev@cs.khpi.edu.ua
25	Деркач Дарина Євгенівна	бюджет	звичайна	daryna.derkach@cs.khpi.edu.ua
26	Ілюхін Тимур Ігорович	контракт	відсутня	tymur.iliukhin@cs.khpi.edu.ua
27	Криженко Аліна Степанівна	бюджет	відсутня	alina.kryzhenko@cs.khpi.edu.ua
28	Посмашний Ярослав Дмитрович	бюджет	звичайна	yaroslav.posmashnyi@cs.khpi.edu.ua
29	Рожкова Вікторія Андріївна	бюджет	підвищена	viktoriia.rozhkova@cs.khpi.edu.ua
30	Сизоненко Єлизавета Олександрівна	контракт	відсутня	yelyzaveta.syzonenko@cs.khpi.edu.ua

🔖

Powered by

Inessa Repeshko

CS-222a

CSIT SEMIT

NTU "KhPI"

Address

National Technical University "Kharkiv Polytechnic Institute"

2 Kyrpychova

61002 Kharkiv, Ukraine

Copyright © 2024 All Rights Reserved

Contact Us

+38-(057)-707-66-34

omsroot@kpi.kharkov.ua

Рисунок Л.1.10 – Сторінка таблиці студентів для Переглядача студентів

Л.2 Результати роботи вебзастосунку для Переглядача абітурієнтів

Student Database Viewer

Sign In

Username or Email: bondar.inna

Password:

☒ Remember me

[New around here? Sign up](#) [Forgot password?](#)

Powered by
Inessa Repeshko
CS-222a
CSIT SEMIT
NTU "KhPI"

Address
National Technical University "Kharkiv Polytechnic
Institute"
2 Kyrpychova
61002 Kharkiv, Ukraine

Contact Us
+38-(057)-707-66-34
omsroot@kpi.kharkov.ua

Copyright © 2024 All Rights Reserved

Рисунок Л.2.1 – Сторінка автентифікації з обліковими даними Переглядача абітурієнтів

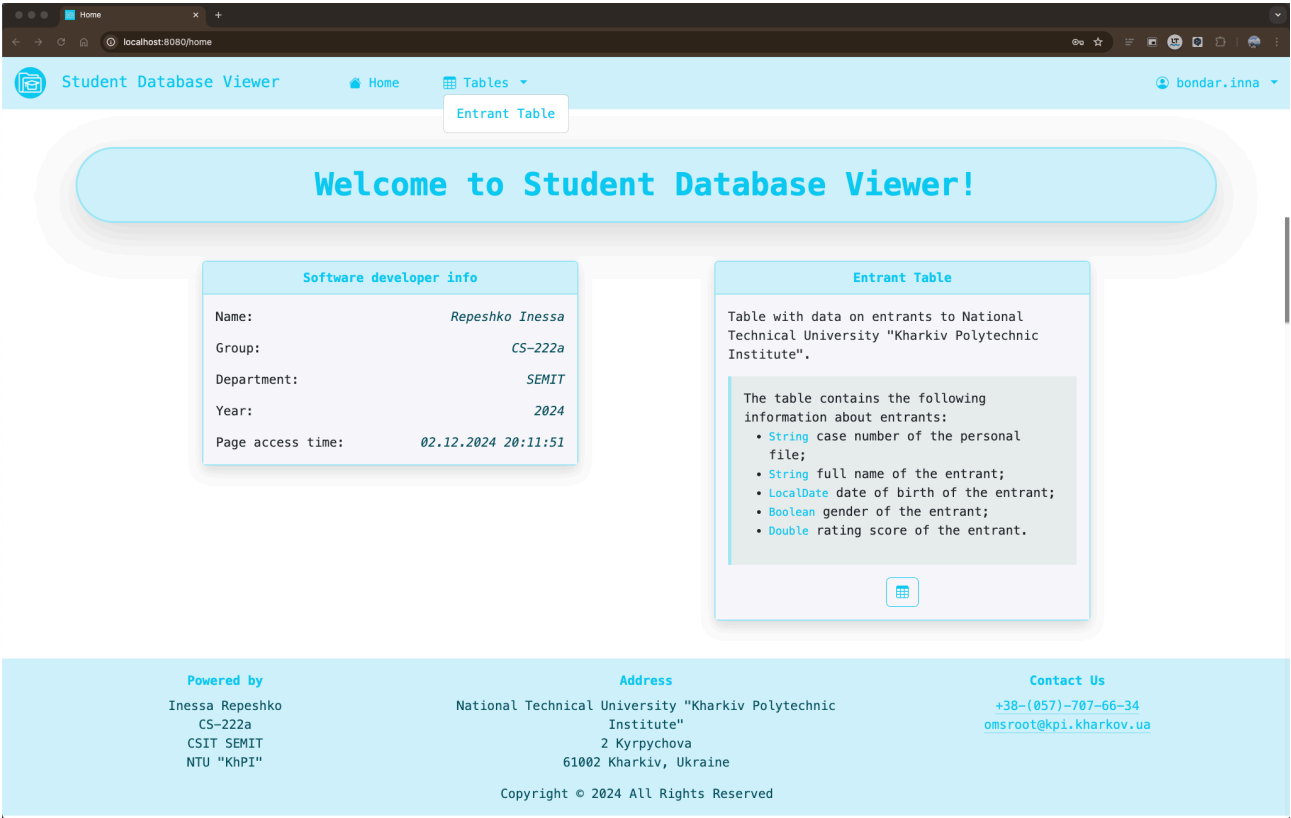


Рисунок Л.2.2 – Домашня сторінка веб-застосунку Переглядача абітурієнтів

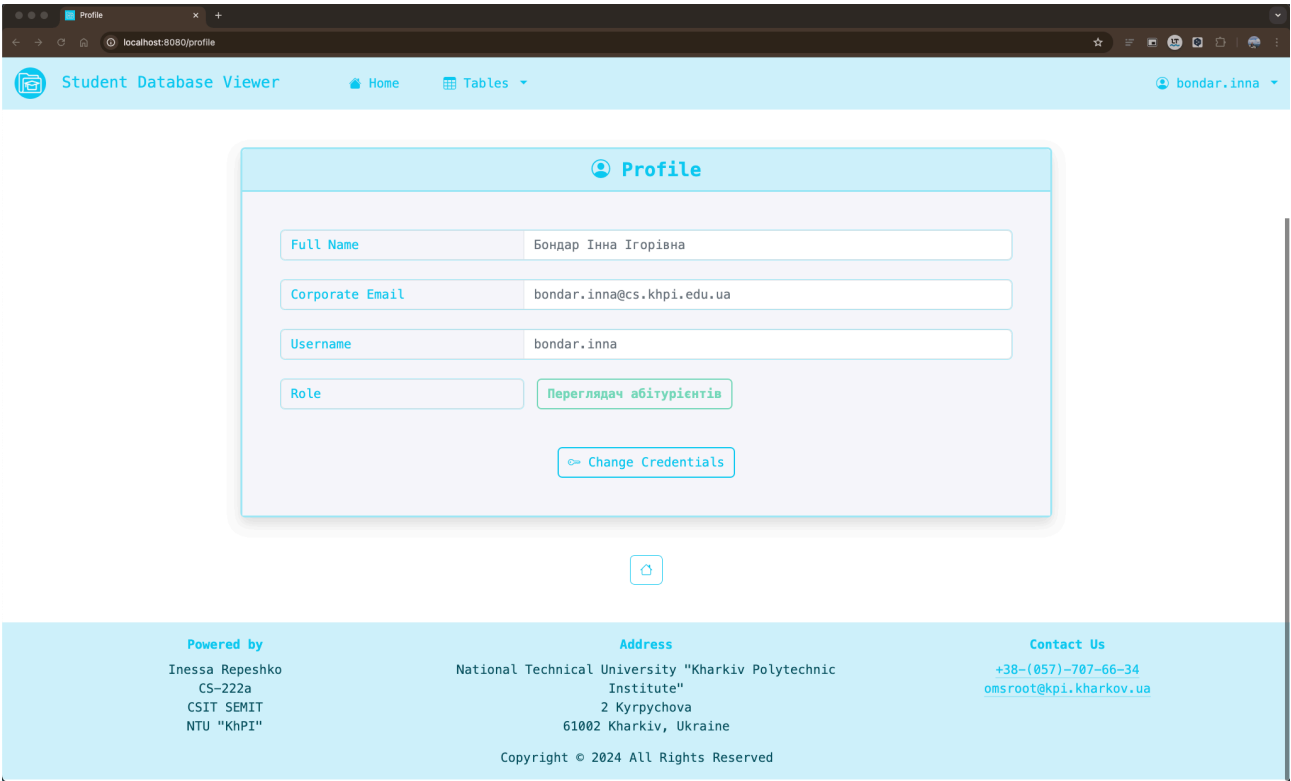


Рисунок Л.2.3 – Сторінка профілю Переглядача абітурієнтів

Student Database Viewer

Home

Tables

bondar.inna

Entrant Table

Search

Search by case number, full name (matching start, case-insensitive)

Type to search...

Filters

Birthday

Start

dd.mm.yyyy

End

dd.mm.yyyy

RatingScore

Start

End

#	Case Number	Full Name	Birthday	Gender	Rating Score
1	KH22-4814	Бабічев Олексій Вікторович	22.05.2005	чоловік	168,111
2	KH22-5835	Волков Дмитро Олександрович	16.07.2005	чоловік	151,202
3	KH22-5511	Гєєнко Анна Олексіївна	03.12.2005	жінка	150,013
4	KH22-9299	Голоп'яров Микита Сергійович	07.11.2005	чоловік	161,484
5	KH22-9012	Гриценко Максим Георгійович	04.12.2005	чоловік	191,945
6	KH22-6636	Жарий Вадим Віталійович	25.10.2005	чоловік	128,346
7	KH22-5140	Жупанов Андрій Володимирович	22.06.2005	чоловік	183,627
8	KH22-4896	Кльоз Михайло Романович	15.01.2005	чоловік	127,288
9	KH22-4694	Коваль Микита Артемович	11.10.2005	чоловік	139,739
10	KH22-5529	Ковтун Егор Вячеславович	10.04.2005	чоловік	187,790
11	KH22-6874	Колесниченко Денис Євгенович	26.03.2005	чоловік	165,641
12	KH22-2947	Ласкевич Андрій Сергійович	05.09.2005	чоловік	175,392
13	KH22-3856	Лукаш Антон Олександрович	24.11.2005	чоловік	184,473
14	KH22-7534	Палій Нікіта Андрійович	05.05.2005	чоловік	168,004
15	KH22-3843	Прокопов Владислав Сергійович	12.12.2004	чоловік	164,665
16	KH22-7819	Репешко Інесса Віталіївна	06.05.2005	жінка	185,306
17	KH22-9115	Скиба Андрій Григорович	26.02.2004	чоловік	125,907
18	KH22-3812	Цибань Ольга Юріївна	19.05.2004	жінка	190,298
19	KH22-1828	Шевченко Данило Валерійович	25.10.2004	чоловік	166,849
20	KH22-9514	Антипенко Катерина Віталіївна	06.01.2005	жінка	168,460
21	KH22-2929	Багрянцева Марина Русланівна	01.10.2005	жінка	157,631
22	KH22-1637	Бондаренко Ярослав Юрійович	24.04.2005	чоловік	199,532
23	KH22-6681	Булгаков Віктор Олександрович	17.05.2005	чоловік	153,683
24	KH22-7557	Васильєв Михайло Олександрович	25.01.2005	чоловік	170,864
25	KH22-8759	Деркач Дарина Євгенівна	03.03.2005	жінка	171,715
26	KH22-7309	Ілюхін Тимур Ігорович	24.01.2005	чоловік	125,776
27	KH22-8254	Криженко Аліна Степанівна	29.08.2005	жінка	138,747
28	KH22-3825	Посмашний Ярослав Дмитрович	25.02.2000	чоловік	160,718
29	KH22-8153	Рожкова Вікторія Андріївна	04.10.1993	жінка	188,309
30	KH22-4788	Сизоненко Єлизавета Олександрівна	17.02.1997	жінка	124,240
31	KH22-8255	Криженко Аліна Степанівна	29.08.2005	жінка	138,747
32	KH22-3826	Посмашний Ярослав Дмитрович	25.02.2000	чоловік	160,718
33	KH22-8154	Рожкова Вікторія Андріївна	04.10.1993	жінка	188,309
34	KH22-6682	Булгаков Віктор Олександрович	17.05.2005	чоловік	153,683
35	KH22-9998	Марчук Регіна Віталіївна	06.06.1996	жінка	200,000

Powered by

Inessa Repeshko
CS-222a
CSIT SEMIT
NTU "KhPI"

Address

National Technical University "Kharkiv Polytechnic Institute"
2 Kyrpychova
61002 Kharkiv, Ukraine

Contact Us

+38-(057)-707-66-34
omsroot@kpi.kharkov.ua

Copyright © 2024 All Rights Reserved

Рисунок Л.2.4 – Сторінка таблиці абітурієнтів для Переглядача абітурієнтів

Л.3 Результати роботи вебзастосунку для Менеджера абітурієнтів

Student Database Viewer

Sign In

Username or Email: burulka.liudmyla@ntu.khpi.edu.ua

Password:

☒ Remember me

[New around here? Sign up](#) [Forgot password?](#)

Powered by
Inessa Repeshko
CS-222a
CSIT SEMIT
NTU "KhPI"

Address
National Technical University "Kharkiv Polytechnic
Institute"
2 Kyrpychova
61002 Kharkiv, Ukraine

Contact Us
+38-(057)-707-66-34
omsroot@kpi.kharkov.ua

Copyright © 2024 All Rights Reserved

Рисунок Л.3.1 – Сторінка автентифікації з обліковими даними Менеджера студентів

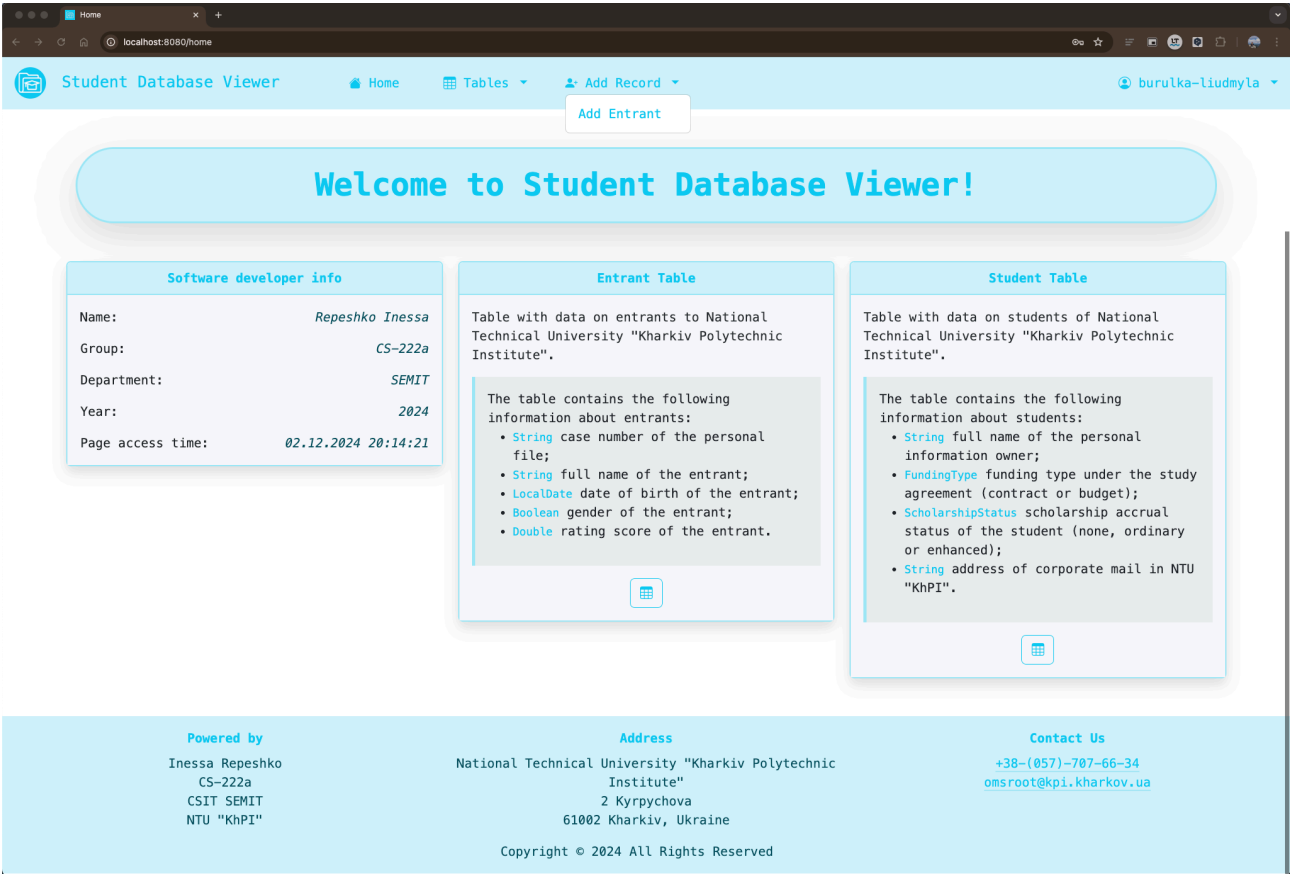


Рисунок Л.3.2 – Домашня сторінка веб-застосунку Менеджера абітурієнтів

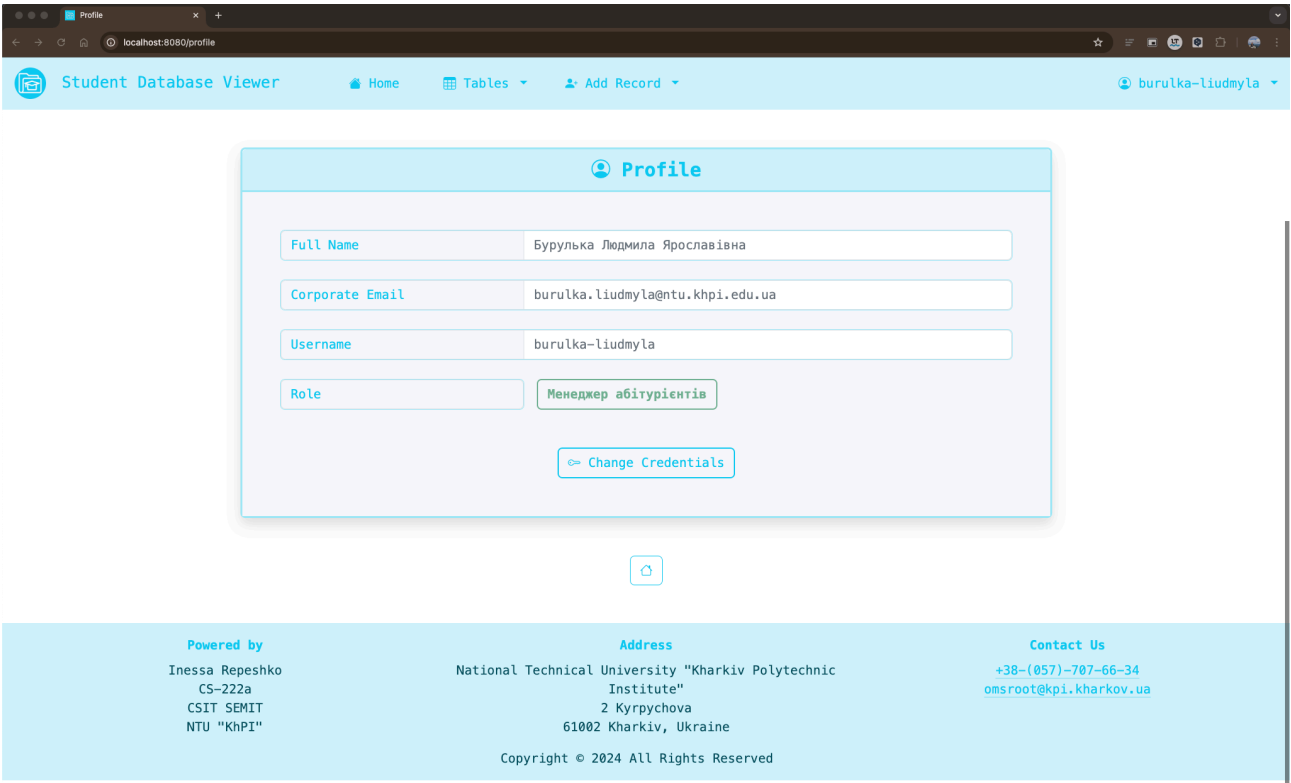


Рисунок Л.3.3 – Сторінка профілю Менеджера абітурієнтів

Student Database Viewer

HomeTablesAdd Record

burulka-liudmyla

Entrant Table

Search

Search by case number, full name (matching start, case-insensitive)

Type to search...

🔍

✖

Filters

Birthdate

Start

dd.mm.yyyy

End

dd.mm.yyyy

🔍

✖

RatingScore

Start

End

🔍

✖

Add record

Add new record to the end of the table

+

#	Case Number	Full Name	Birthdate	Gender	Rating Score	Operations
1	KN22-4814	Бабічев Олексій Вікторович	22.05.2005	чоловік	168,111	
2	KN22-5835	Волков Дмитро Олександрович	16.07.2005	чоловік	151,202	
3	KN22-5511	Геєнко Анна Олексіївна	03.12.2005	жінка	150,013	
4	KN22-9299	Голоп'юров Микита Сергійович	07.11.2005	чоловік	161,484	
5	KN22-9012	Гриценко Максим Георгійович	04.12.2005	чоловік	191,945	
6	KN22-6636	Жарий Вадим Віталійович	25.10.2005	чоловік	128,346	
7	KN22-5140	Жупанов Андрій Володимирович	22.06.2005	чоловік	183,627	
8	KN22-4896	Кльоз Михайло Романович	15.01.2005	чоловік	127,288	
9	KN22-4694	Коваль Микита Артемович	11.10.2005	чоловік	139,739	
10	KN22-5529	Ковтун Єгор Вячеславович	10.04.2005	чоловік	187,790	
11	KN22-6874	Колесниченко Денис Євгенович	26.03.2005	чоловік	165,641	
12	KN22-2947	Ласкевич Андрій Сергійович	05.09.2005	чоловік	175,392	
13	KN22-3856	Лукаш Антон Олександрович	24.11.2005	чоловік	184,473	
14	KN22-7534	Палій Нікіта Андрійович	05.05.2005	чоловік	168,004	
15	KN22-3843	Прокопов Владислав Сергійович	12.12.2004	чоловік	164,665	
16	KN22-7819	Репешко Інесса Віталіївна	06.05.2005	жінка	185,306	
17	KN22-9115	Скиба Андрій Григорович	26.02.2004	чоловік	125,987	
18	KN22-3812	Цибань Ольга Юріївна	19.05.2004	жінка	190,298	
19	KN22-1828	Шевченко Данило Валерійович	25.10.2004	чоловік	166,849	
20	KN22-9514	Антипенко Катерина Віталіївна	06.01.2005	жінка	168,460	
21	KN22-2929	Багрянцева Марина Русланівна	01.10.2005	жінка	157,631	
22	KN22-1637	Бондаренко Ярослав Юрійович	24.04.2005	чоловік	199,532	
23	KN22-6681	Булгаков Віктор Олександрович	17.05.2005	чоловік	153,683	
24	KN22-7557	Васильєв Михайло Олександрович	25.01.2005	чоловік	170,864	
25	KN22-8759	Деркач Дарина Євгенівна	03.03.2005	жінка	171,715	
26	KN22-7309	Ілюхін Тимур Ігорович	24.01.2005	чоловік	125,776	
27	KN22-8254	Криженко Аліна Степанівна	29.08.2005	жінка	138,747	
28	KN22-3825	Посмашний Ярослав Дмитрович	25.02.2000	чоловік	160,718	
29	KN22-8153	Рожкова Вікторія Андріївна	04.10.1993	жінка	188,309	
30	KN22-4788	Сизоненко Єлизавета Олександрівна	17.02.1997	жінка	124,240	
31	KN22-8255	Криженко Аліна Степанівна	29.08.2005	жінка	138,747	
32	KN22-3826	Посмашний Ярослав Дмитрович	25.02.2000	чоловік	160,718	
33	KN22-8154	Рожкова Вікторія Андріївна	04.10.1993	жінка	188,309	
34	KN22-6682	Булгаков Віктор Олександрович	17.05.2005	чоловік	153,683	
35	KN22-9998	Марчук Регіна Віталіївна	06.06.1996	жінка	200,000	

🔍

Powered by

Inessa Repeshko
CS-222a
CSIT SEMIT
NTU "KhPI"

Address

National Technical University "Kharkiv Polytechnic
Institute"
2 Kyrpychova
61002 Kharkiv, Ukraine

Contact Us

+38-(057)-707-66-34
omsroot@kpi.kharkov.ua

Copyright © 2024 All Rights Reserved

Рисунок Л.3.4 – Сторінка таблиці абітурієнтів для Менеджера абітурієнтів

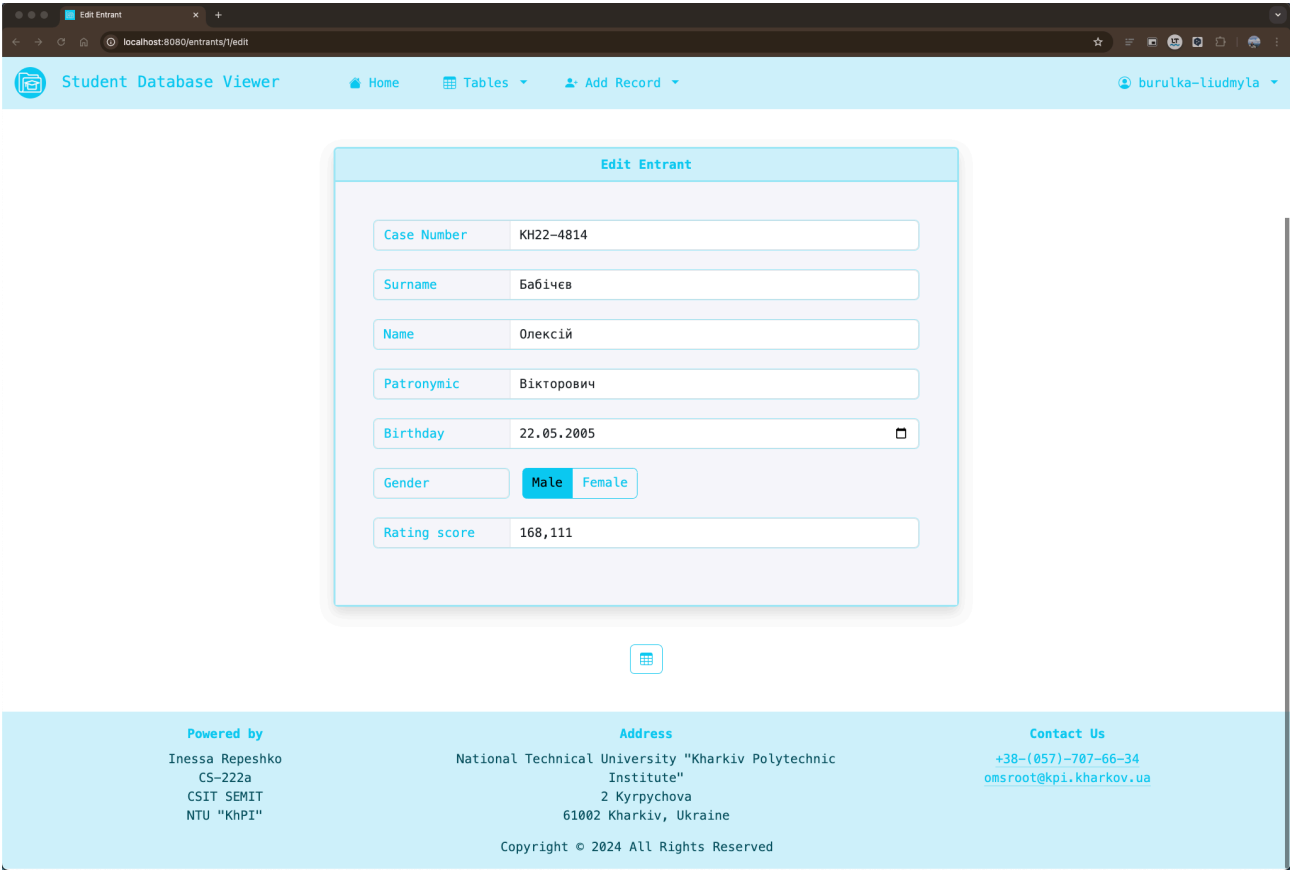


Рисунок Л.3.5 – Сторінка редагування абітурієнта Менеджером абітурієнтів

Student Table

localhost:8080/students

Student Database Viewer

Home

Tables

Add Record

burulka-liudmyla

Student Table

Search

Search by full name (matching start, case-insensitive)

Type to search...

🔍

✖

Filters

Funding Type

Budget

Contract

🔼

🔽

✖

Scholarship Status

Enhanced

Ordinary

None

🔼

🔽

✖

#	Full Name	Funding Type	Scholarship Status	Corporate Email	Entrant Info
1	Бабічев Олексій Вікторович	бюджет	звичайна	oleksii.babichiev@cs.khpi.edu.ua	📄
2	Волков Дмитро Олександрович	бюджет	відсутня	dmytro.o.volkov@cs.khpi.edu.ua	📄
3	Геєнко Анна Олексіївна	бюджет	відсутня	anna.hieienko@cs.khpi.edu.ua	📄
4	Голоп'яров Микита Сергійович	бюджет	звичайна	mykyta.holoporov@cs.khpi.edu.ua	📄
5	Гриценко Максим Георгійович	бюджет	підвищена	maksym.hrytsenko@cs.khpi.edu.ua	📄
6	Жарий Вадим Віталійович	контракт	відсутня	vadym.zharyi@cs.khpi.edu.ua	📄
7	Жупанов Андрій Володимирович	бюджет	звичайна	andrii.zhupanov@cs.khpi.edu.ua	📄
8	Кльоз Михайло Романович	контракт	відсутня	mykhailo.kloz@cs.khpi.edu.ua	📄
9	Коваль Микита Артемович	бюджет	відсутня	mykyta.koval@cs.khpi.edu.ua	📄
10	Ковтун Єгор Вячеславович	бюджет	підвищена	yehor.kovtun@cs.khpi.edu.ua	📄
11	Колесниченко Денис Євгенович	бюджет	звичайна	denys.kolesnychenko@cs.khpi.edu.ua	📄
12	Ласкевич Андрій Сергійович	бюджет	звичайна	andrii.laskevych@cs.khpi.edu.ua	📄
13	Лукаш Антон Олександрович	бюджет	звичайна	anton.lukash@cs.khpi.edu.ua	📄
14	Палій Нікіта Андрійович	бюджет	відсутня	nikita.palii@cs.khpi.edu.ua	📄
15	Прокопов Владислав Сергійович	бюджет	звичайна	vladyslav.prokopov@cs.khpi.edu.ua	📄
16	Репешко Інесса Віталіївна	бюджет	підвищена	inessa.repeshko@cs.khpi.edu.ua	📄
17	Скиба Андрій Григорович	контракт	відсутня	andrii.h.skyba@cit.khpi.edu.ua	📄
18	Цибань Ольга Юріївна	бюджет	відсутня	olha.tsyban@cs.khpi.edu.ua	📄
19	Шевченко Данило Валерійович	бюджет	звичайна	danylo.v.shevchenko@cs.khpi.edu.ua	📄
20	Антипенко Катерина Віталіївна	бюджет	звичайна	kateryna.antypenko@cs.khpi.edu.ua	📄
21	Багрянцева Марина Русланівна	бюджет	звичайна	marina.bahriantseva@cs.khpi.edu.ua	📄
22	Бондаренко Ярослав Юрійович	бюджет	підвищена	yaroslav.y.bondarenko@cs.khpi.edu.ua	📄
23	Булгаков Віктор Олександрович	бюджет	відсутня	viktor.bulhakov@cs.khpi.edu.ua	📄
24	Васильєв Михайло Олександрович	бюджет	відсутня	mykhailo.vasyliev@cs.khpi.edu.ua	📄
25	Деркач Дарина Євгенівна	бюджет	звичайна	daryna.derkach@cs.khpi.edu.ua	📄
26	Ілюхін Тимур Ігорович	контракт	відсутня	tymur.iliukhin@cs.khpi.edu.ua	📄
27	Криженко Аліна Степанівна	бюджет	відсутня	alina.kryzhenko@cs.khpi.edu.ua	📄
28	Посмашний Ярослав Дмитрович	бюджет	звичайна	yaroslav.posmashnyi@cs.khpi.edu.ua	📄
29	Рожкова Вікторія Андріївна	бюджет	підвищена	viktoriia.rozhkova@cs.khpi.edu.ua	📄
30	Сизоненко Єлизавета Олександрівна	контракт	відсутня	yelyzaveta.syzonenko@cs.khpi.edu.ua	📄

🔄

Powered by

Inessa Repeshko
CS-222a
CSIT SEMIT
NTU "KhPI"

Address

National Technical University "Kharkiv Polytechnic Institute"
2 Kyrpychova
61002 Kharkiv, Ukraine

Contact Us

+38-(057)-707-66-34
omsroot@kpi.kharkov.ua

Copyright © 2024 All Rights Reserved

Рисунок Л.3.6 – Сторінка таблиці студентів для Менеджера абітурієнтів

Л.4 Результати роботи вебзастосунку для Менеджера студентів

Student Database Viewer

Sign In

Username or Email: voloshkov-tymur

Password:

☒ Remember me

[New around here? Sign up](#) [Forgot password?](#)

Powered by
Inessa Repeshko
CS-222a
CSIT SEMIT
NTU "KhPI"

Address
National Technical University "Kharkiv Polytechnic
Institute"
2 Kyrpychova
61002 Kharkiv, Ukraine

Contact Us
+38-(057)-707-66-34
omsroot@kpi.kharkov.ua

Copyright © 2024 All Rights Reserved

Рисунок Л.4.1 – Сторінка автентифікації з обліковими даними Менеджера студентів

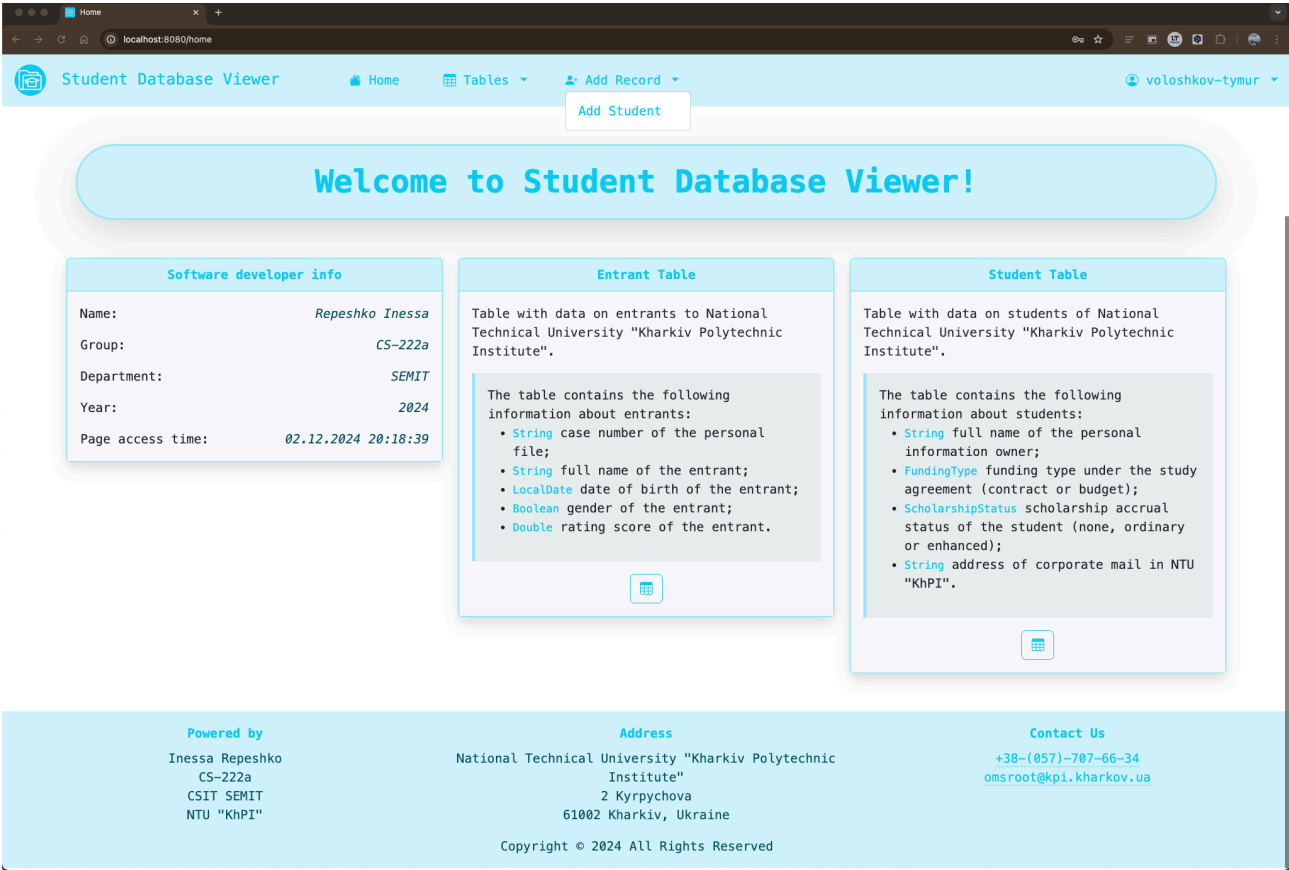


Рисунок Л.4.2 – Домашня сторінка веб-застосунку Менеджера студентів

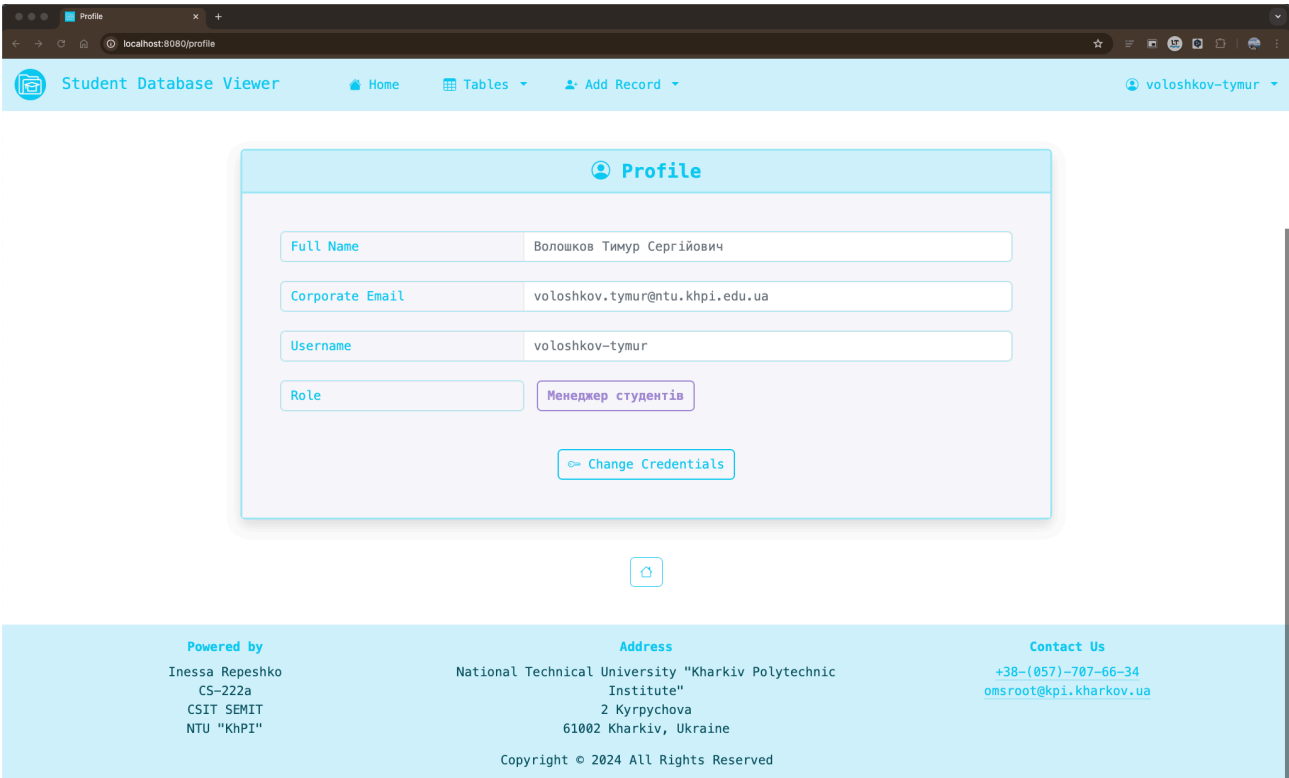


Рисунок Л.4.3 – Сторінка профілю Менеджера студентів

The image shows a web browser window displaying a 'Student Database Viewer' application. The browser's address bar shows 'localhost:8080/entrants'. The application has a light blue header with navigation links: Home, Tables, and Add Record. A user profile 'voloshkov-tymur' is in the top right. The main content area features a large title 'Entrant Table' and a table with 35 rows of student data. The table columns are: #, Case Number, Full Name, Birthday, Gender, Rating Score, and Student Info. To the left of the table are two filter panels. The 'Search' panel has a text input 'Type to search...' and a search button. The 'Filters' panel has two sections: 'Birthday' with 'Start' and 'End' date pickers, and 'RatingScore' with 'Start' and 'End' numeric input fields. Both filter sections have a filter icon and a reset 'X' button. The footer contains three columns of information: 'Powered by' (Inessa Repeshko, CS-222a, CSIT SEMIT, NTU 'KhPI'), 'Address' (National Technical University 'Kharkiv Polytechnic Institute', 2 Kyrpychova, 61002 Kharkiv, Ukraine), and 'Contact Us' (+38-(057)-707-66-34, omsroot@kpi.kharkov.ua). A copyright notice 'Copyright © 2024 All Rights Reserved' is at the bottom center.

Рисунок Л.4.4 – Сторінка таблиці абітурієнтів для Менеджера студентів

Student Table

localhost:8080/students

Student Database Viewer

Home

Tables

Add Record

voloshkov-tymur

Student Table

Search

Search by full name (matching start, case-insensitive)

Type to search...

Filters

Funding Type

Budget

Contract

Scholarship Status

Enhanced

Ordinary

None

Add record

Add new record to the end of the table

#	Full Name	Funding Type	Scholarship Status	Corporate Email	Operations
1	Бабічев Олексій Вікторович	бюджет	звичайна	oleksii.babichiev@cs.khpi.edu.ua	
2	Волков Дмитро Олександрович	бюджет	відсутня	dmytro.o.volkov@cs.khpi.edu.ua	
3	Геєнко Анна Олексіївна	бюджет	відсутня	anna.hieienko@cs.khpi.edu.ua	
4	Голоп'яров Микита Сергійович	бюджет	звичайна	mykyta.holoporov@cs.khpi.edu.ua	
5	Гриценко Максим Георгійович	бюджет	підвищена	maksym.hrytsenko@cs.khpi.edu.ua	
6	Жарий Вадим Віталійович	контракт	відсутня	vadym.zharyi@cs.khpi.edu.ua	
7	Жупанов Андрій Володимирович	бюджет	звичайна	andrii.zhupanov@cs.khpi.edu.ua	
8	Кльоз Михайло Романович	контракт	відсутня	mykhailo.kloz@cs.khpi.edu.ua	
9	Коваль Микита Артемович	бюджет	відсутня	mykyta.koval@cs.khpi.edu.ua	
10	Ковтун Єгор Вячеславович	бюджет	підвищена	yehor.kovtun@cs.khpi.edu.ua	
11	Колесниченко Денис Євгенович	бюджет	звичайна	denys.kolesnychenko@cs.khpi.edu.ua	
12	Ласкевич Андрій Сергійович	бюджет	звичайна	andrii.laskevych@cs.khpi.edu.ua	
13	Лукаш Антон Олександрович	бюджет	звичайна	anton.lukash@cs.khpi.edu.ua	
14	Палій Нікіта Андрійович	бюджет	відсутня	nikita.palii@cs.khpi.edu.ua	
15	Прокопов Владислав Сергійович	бюджет	звичайна	vladyslav.prokopov@cs.khpi.edu.ua	
16	Репешко Інесса Віталіївна	бюджет	підвищена	inessa.repeshko@cs.khpi.edu.ua	
17	Скиба Андрій Григорович	контракт	відсутня	andrii.h.skyba@cit.khpi.edu.ua	
18	Цибань Ольга Юріївна	бюджет	відсутня	olha.tsyban@cs.khpi.edu.ua	
19	Шевченко Данило Валерійович	бюджет	звичайна	danylo.v.shevchenko@cs.khpi.edu.ua	
20	Антипенко Катерина Віталіївна	бюджет	звичайна	kateryna.antypenko@cs.khpi.edu.ua	
21	Багрянцева Марина Русланівна	бюджет	звичайна	marina.bahriantseva@cs.khpi.edu.ua	
22	Бондаренко Ярослав Юрійович	бюджет	підвищена	yaroslav.y.bondarenko@cs.khpi.edu.ua	
23	Булгаков Віктор Олександрович	бюджет	відсутня	viktor.bulhakov@cs.khpi.edu.ua	
24	Васильєв Михайло Олександрович	бюджет	відсутня	mykhailo.vasyliiev@cs.khpi.edu.ua	
25	Деркач Дарина Євгенівна	бюджет	звичайна	daryna.derkach@cs.khpi.edu.ua	
26	Ілюхін Тимур Ігорович	контракт	відсутня	tymur.iliukhin@cs.khpi.edu.ua	
27	Криженко Аліна Степанівна	бюджет	відсутня	alina.kryzhenko@cs.khpi.edu.ua	
28	Посмашний Ярослав Дмитрович	бюджет	звичайна	yaroslav.posmashnyi@cs.khpi.edu.ua	
29	Рожкова Вікторія Андріївна	бюджет	підвищена	viktoriia.rozhkova@cs.khpi.edu.ua	
30	Сизоненко Єлизавета Олександрівна	контракт	відсутня	yelyzaveta.syzonenko@cs.khpi.edu.ua	

Powered by

Inessa Repeshko

CS-222a

CSIT SEMIT

NTU "KhPI"

Address

National Technical University "Kharkiv Polytechnic Institute"

2 Kyrpychova

61002 Kharkiv, Ukraine

Copyright © 2024 All Rights Reserved

Contact Us

+38-(057)-707-66-34

omsroot@kpi.kharkov.ua

Рисунок Л.4.5 – Сторінка таблиці студентів для Менеджера студентів

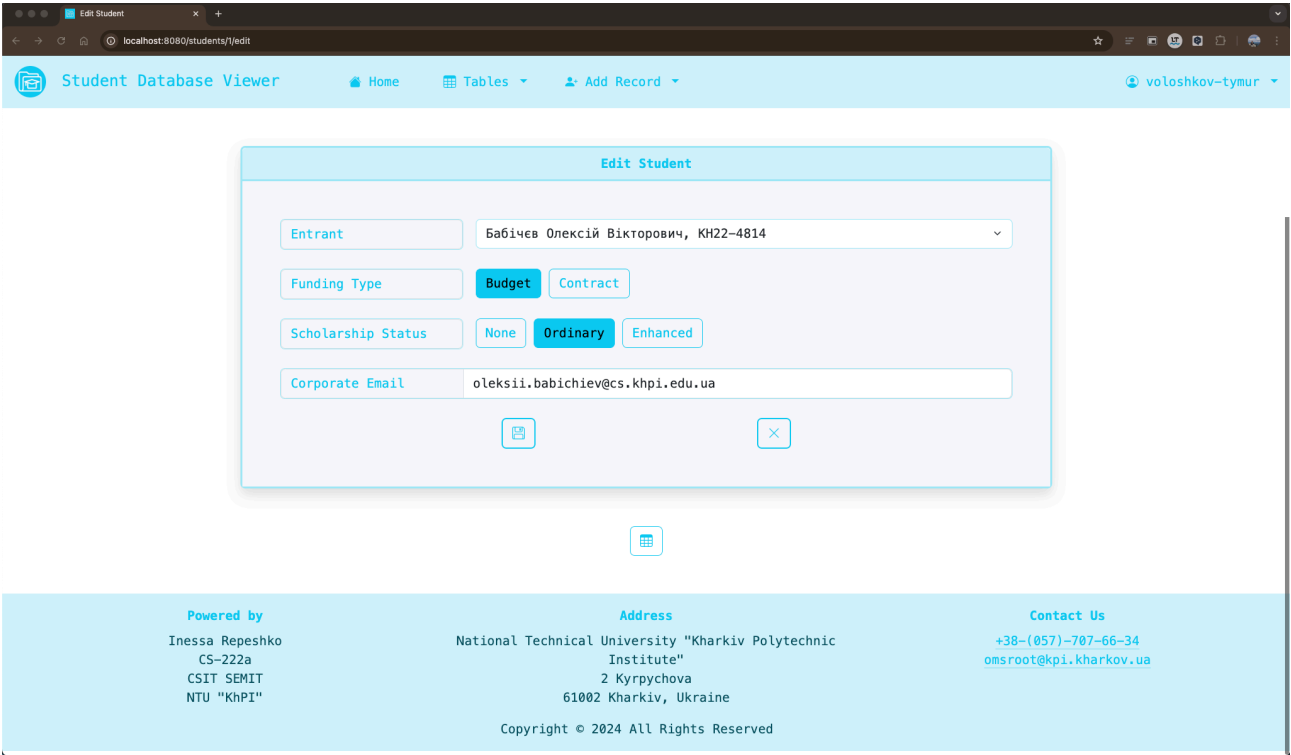


Рисунок Л.4.6 – Сторінка редагування студента Менеджером студентів

Л.5 Результати роботи вебзастосунку для користувача роллю “Адміністратор” – “ADMIN”

Student Database Viewer

Sign In

Username or Email shavlii.oleksii@ntu.khpi.edu.ua

Password

☒ Remember me

Sign In

[New around here? Sign up](#) [Forgot password?](#)

Powered by
Inessa Repeshko
CS-222a
CSIT SEMIT
NTU "KhPI"

Address
National Technical University "Kharkiv Polytechnic
Institute"
2 Kyrpychova
61002 Kharkiv, Ukraine

Contact Us
+38-(057)-707-66-34
omsroot@kpi.kharkov.ua

Copyright © 2024 All Rights Reserved

Рисунок Л.5.1 – Сторінка автентифікації з обліковими даними Адміністратора

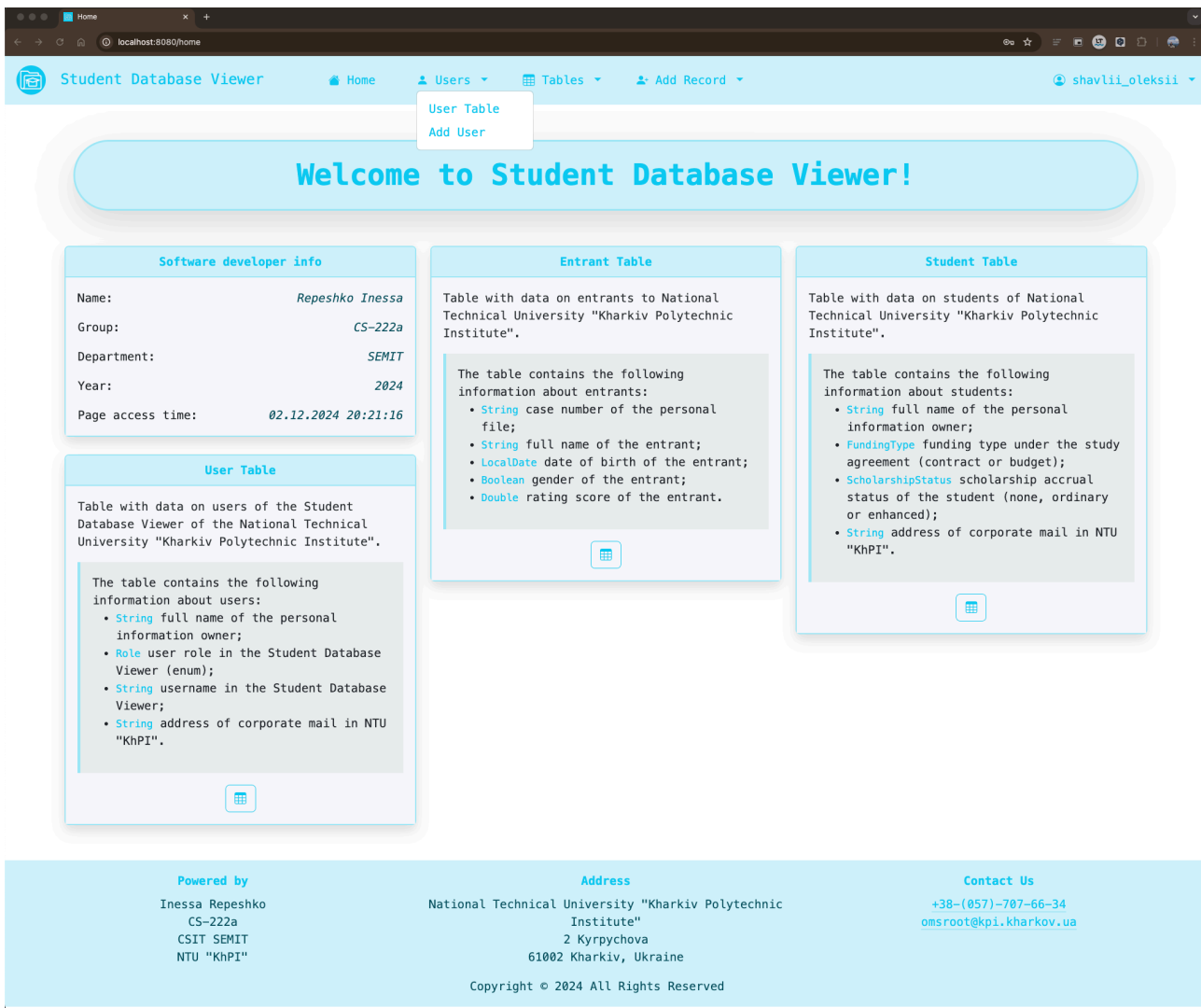


Рисунок Л.5.2 – Домашня сторінка веб-застосунку Адміністратора

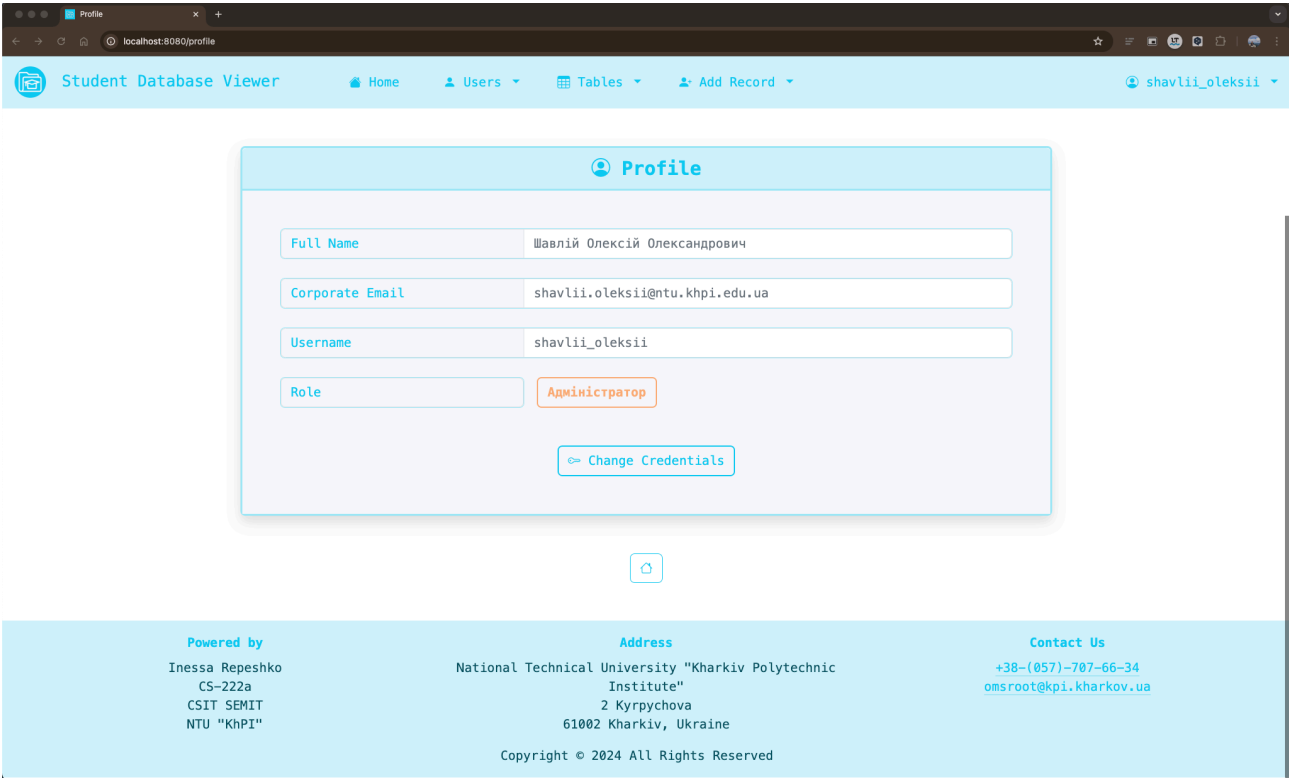


Рисунок Л.5.3 – Сторінка профілю Адміністратора

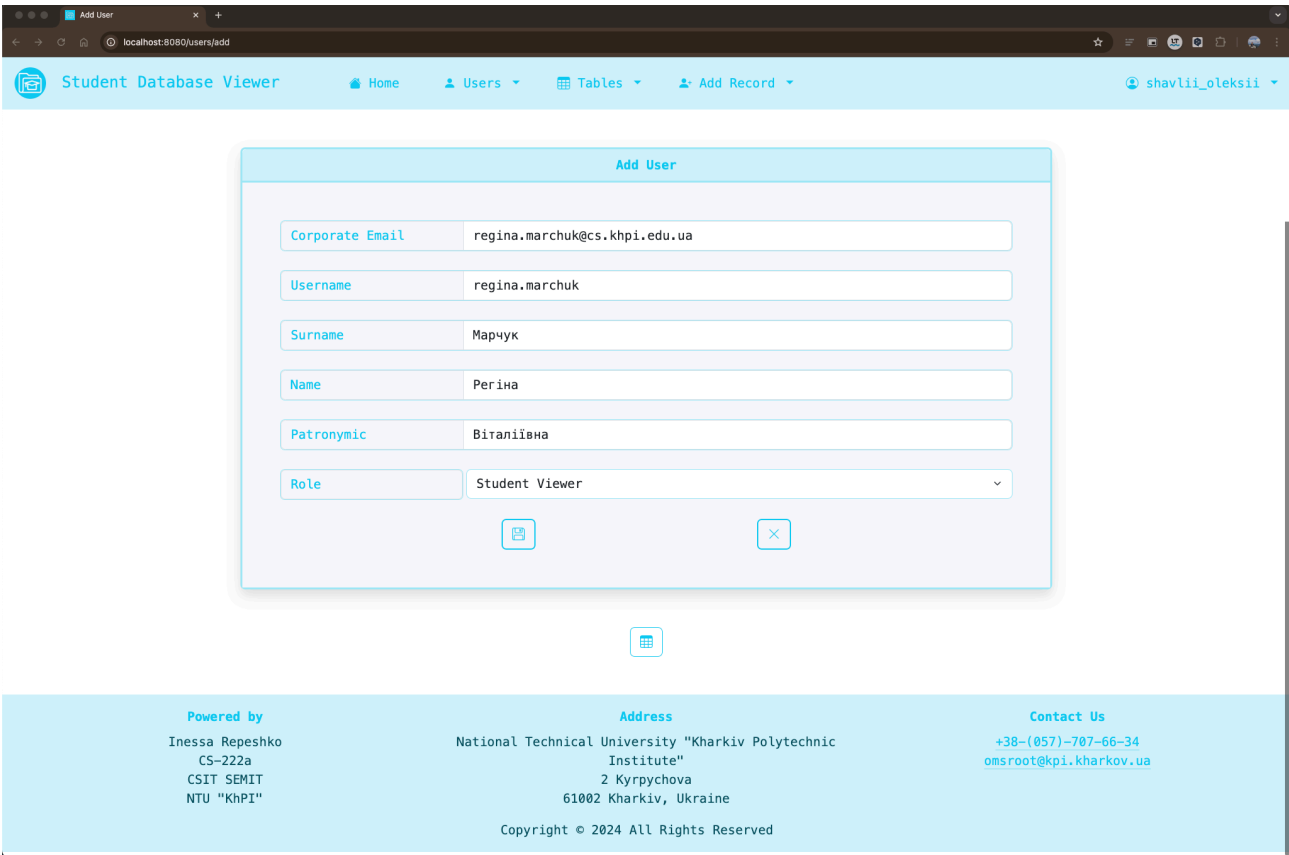


Рисунок Л.5.4 – Сторінка створення нового користувача Адміністратором

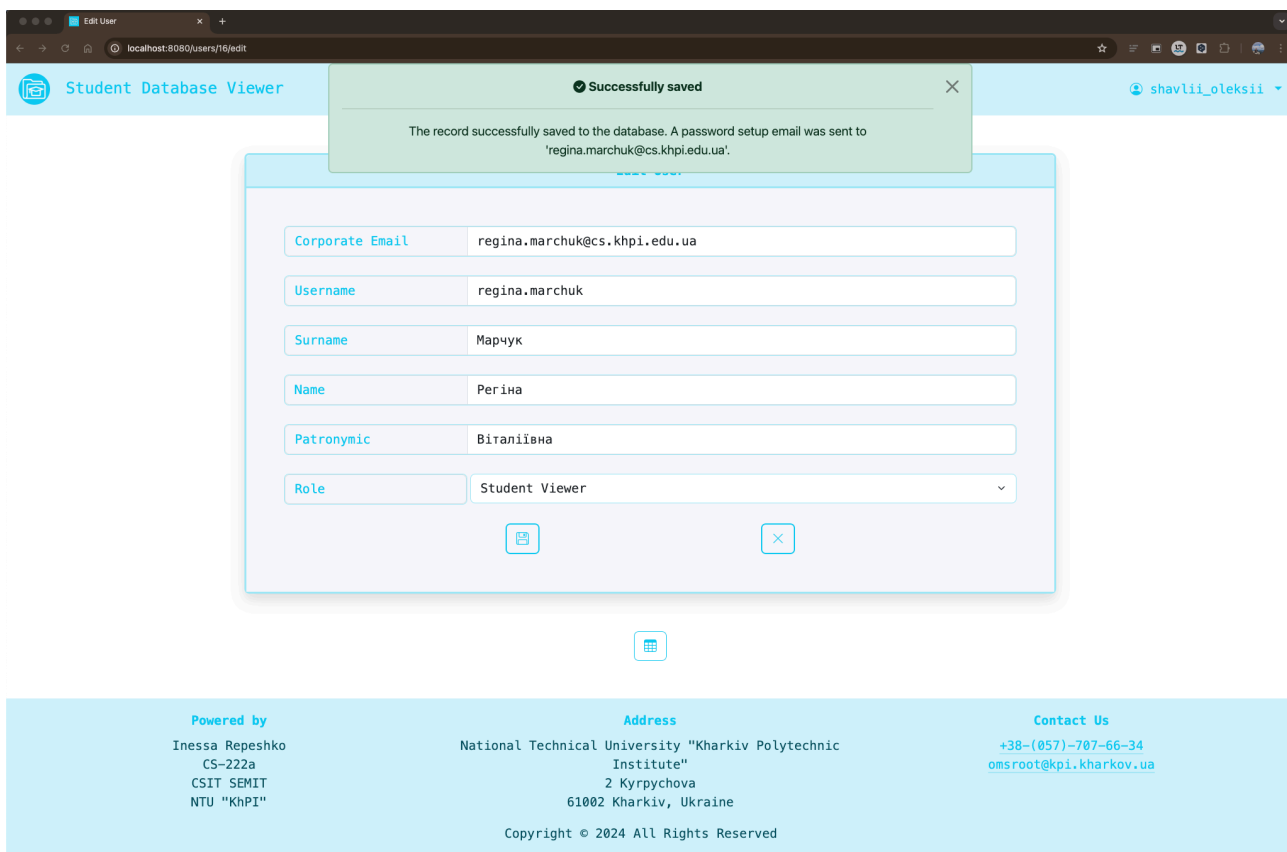


Рисунок Л.5.5 – Сторінка редагування користувача Адміністратором із повідомленням про успішне створення нового користувача

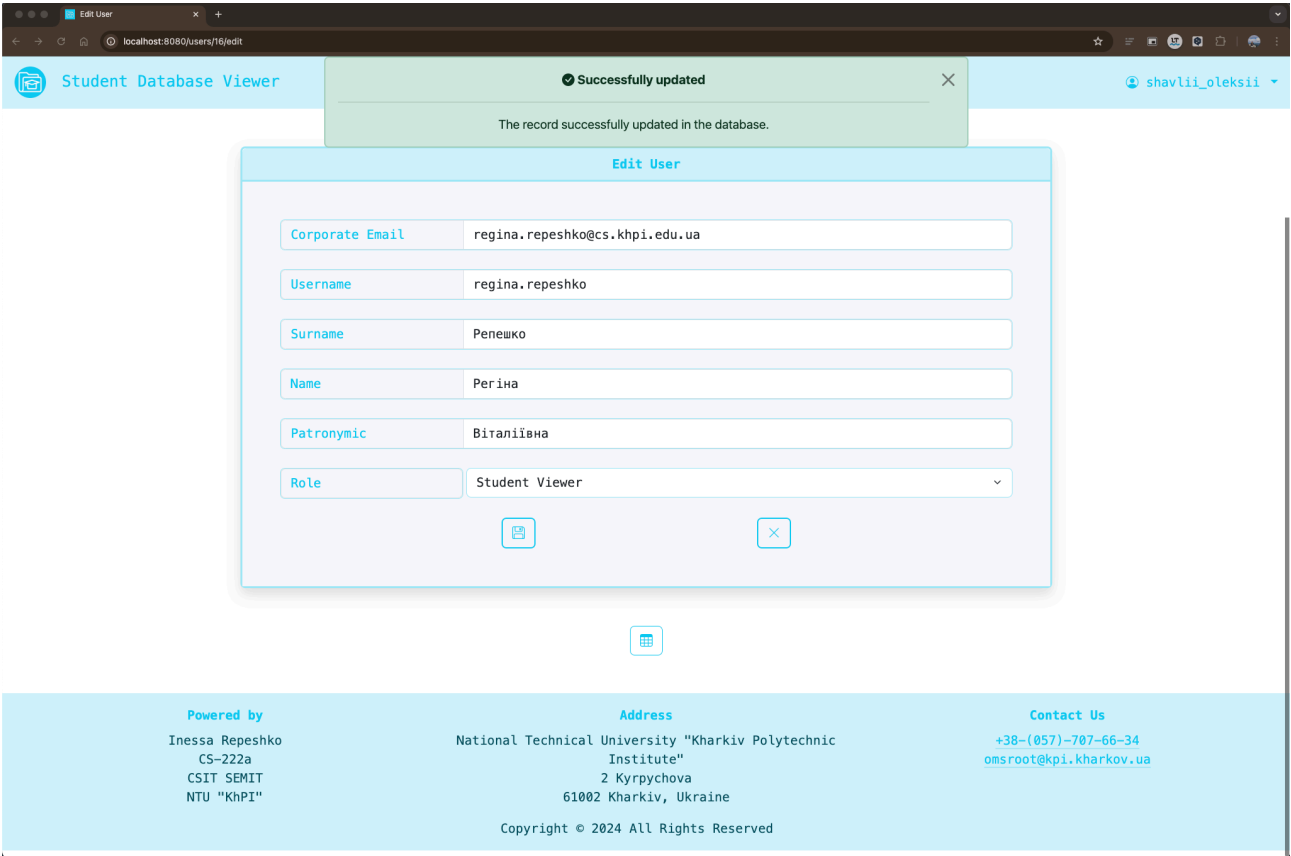


Рисунок Л.5.6 – Сторінка редагування користувача Адміністратором із повідомленням про успішне оновлення даних існуючого користувача

Student Database Viewer

HomeUsersTablesAdd Record

shavlii_oleksii

User Table

Search

Search by full name (matching start, case-insensitive)

Type to search...

Filters

Role

Admin

Entrant Manager

Student Manager

Entrant Viewer

Student Viewer

Add record

Add new record to the end of the table

+

#	Full Name	Role	Username	Corporate Email	Operations
1	Шавлій Олексій Олександрович	Адміністратор	shavlii_oleksii	shavlii.oleksii@ntu.khpi.edu.ua	
2	Бурулька Людмила Ярославівна	Менеджер абітурієнтів	burulka-liudmyla	burulka.liudmyla@ntu.khpi.edu.ua	
3	Волошков Тимур Сергійович	Менеджер студентів	voloshkov-tymur	voloshkov.tymur@ntu.khpi.edu.ua	
4	Куліш Володимир Анатолійович	Переглядач абітурієнтів	kulish.volodymyr	kulish.volodymyr@cs.khpi.edu.ua	
5	Степова Аліна Трофимівна	Переглядач студентів	stepova.alina	stepova.alina@cs.khpi.edu.ua	
6	Іваненко Анна Петрівна	Адміністратор	ivanenko_anna	ivanenko.anna@ntu.khpi.edu.ua	
7	Тарасенко Олег Іванович	Менеджер абітурієнтів	tarasenko-oleh	tarasenko.oleh@ntu.khpi.edu.ua	
8	Мельник Дарина Анатоліївна	Менеджер студентів	melnyk_daryna	melnyk.daryna@ntu.khpi.edu.ua	
9	Горбунів Владислав Михайлович	Переглядач абітурієнтів	horbunov.vladyslav	horbunov.vladyslav@cs.khpi.edu.ua	
10	Романова Олена Олексіївна	Переглядач студентів	romanova-olena	romanova.olena@cs.khpi.edu.ua	
11	Васильченко Михайло Олександрович	Адміністратор	vasylchenko.mykhailo	vasylchenko.mykhailo@ntu.khpi.edu.ua	
12	Шаповал Ксенія Василівна	Менеджер абітурієнтів	shapoval-ksenii	shapoval.ksenii@ntu.khpi.edu.ua	
13	Козак Євгеній Олегович	Менеджер студентів	kozak.yevhenii	kozak.yevhenii@ntu.khpi.edu.ua	
14	Бондар Інна Ігорівна	Переглядач абітурієнтів	bondar.inna	bondar.inna@cs.khpi.edu.ua	
15	Кравченко Артем Богданович	Переглядач студентів	kravchenko-artem	kravchenko.artem@cs.khpi.edu.ua	
16	Репешко Регіна Віталіївна	Переглядач студентів	regina.repeshko	regina.repeshko@cs.khpi.edu.ua	

Powered by

Inessa Repeshko
CS-222a
CSIT SEMIT
NTU "KhPI"

Address

National Technical University "Kharkiv Polytechnic Institute"
2 Kyrpychova
61002 Kharkiv, Ukraine

Contact Us

+38-(057)-707-66-34
omsroot@kpi.kharkov.ua

Copyright © 2024 All Rights Reserved

Рисунок Л.5.7 – Сторінка таблиці користувачів для Адміністратора

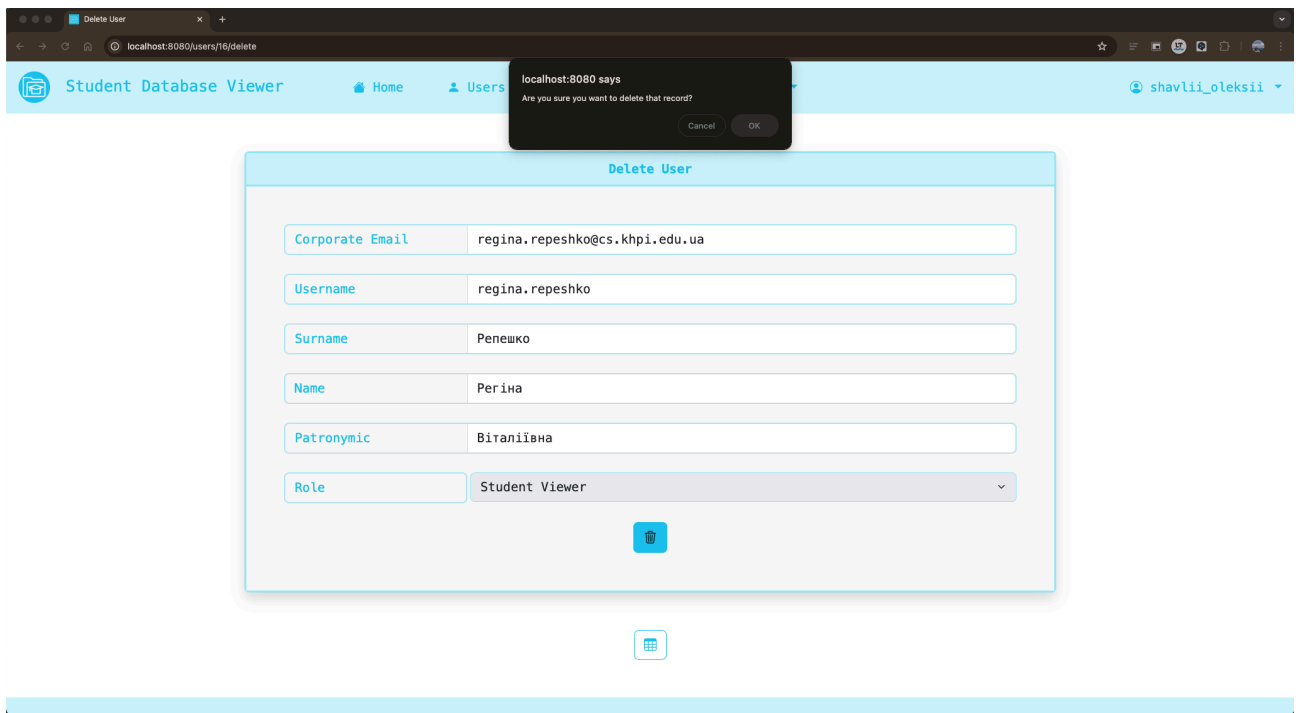


Рисунок Л.5.8 – Сторінка видалення користувача Адміністратором із запитом на підтвердження операції

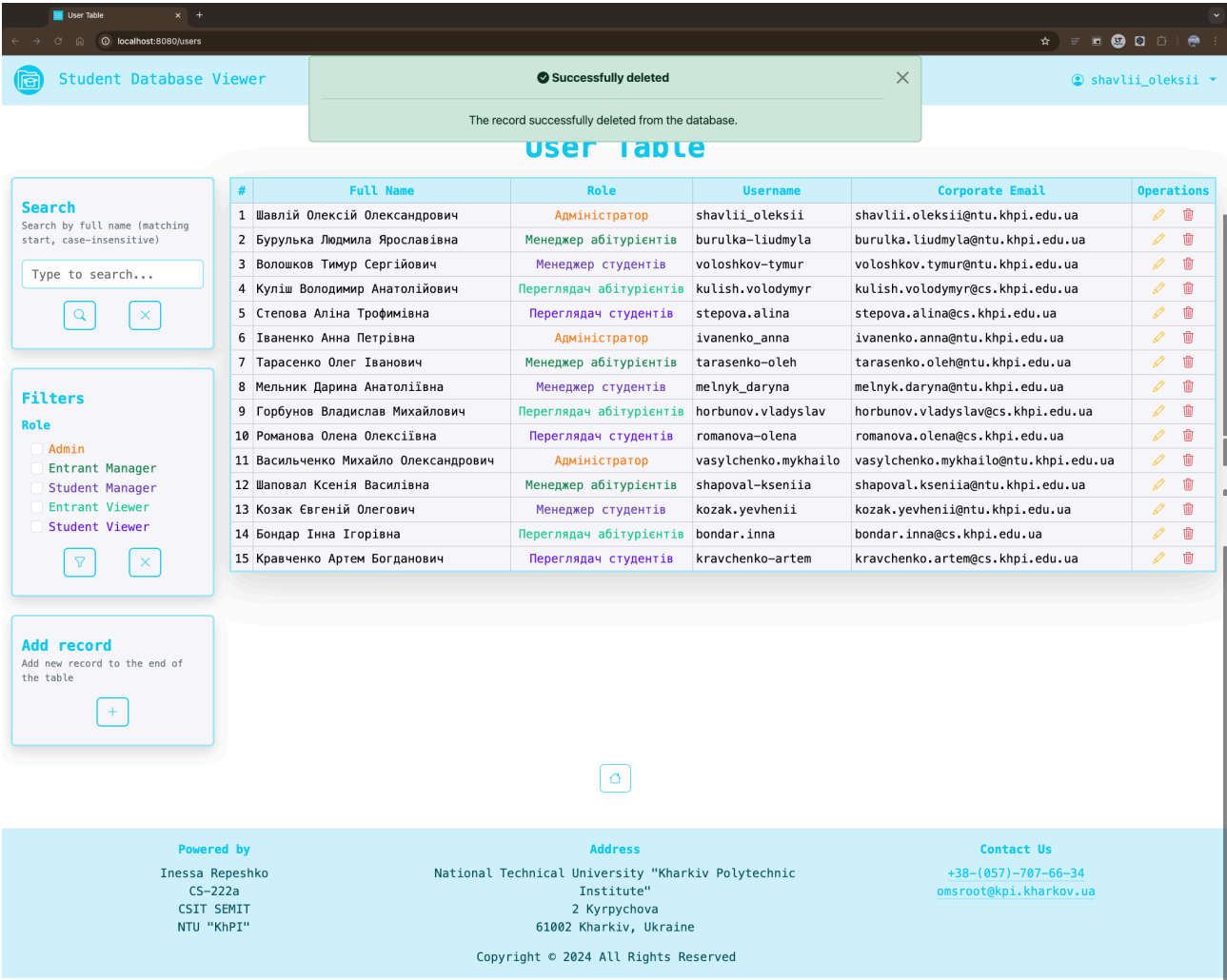


Рисунок Л.5.9 – Сторінка таблиці користувачів для Адміністратора із повідомленням про видалення користувача

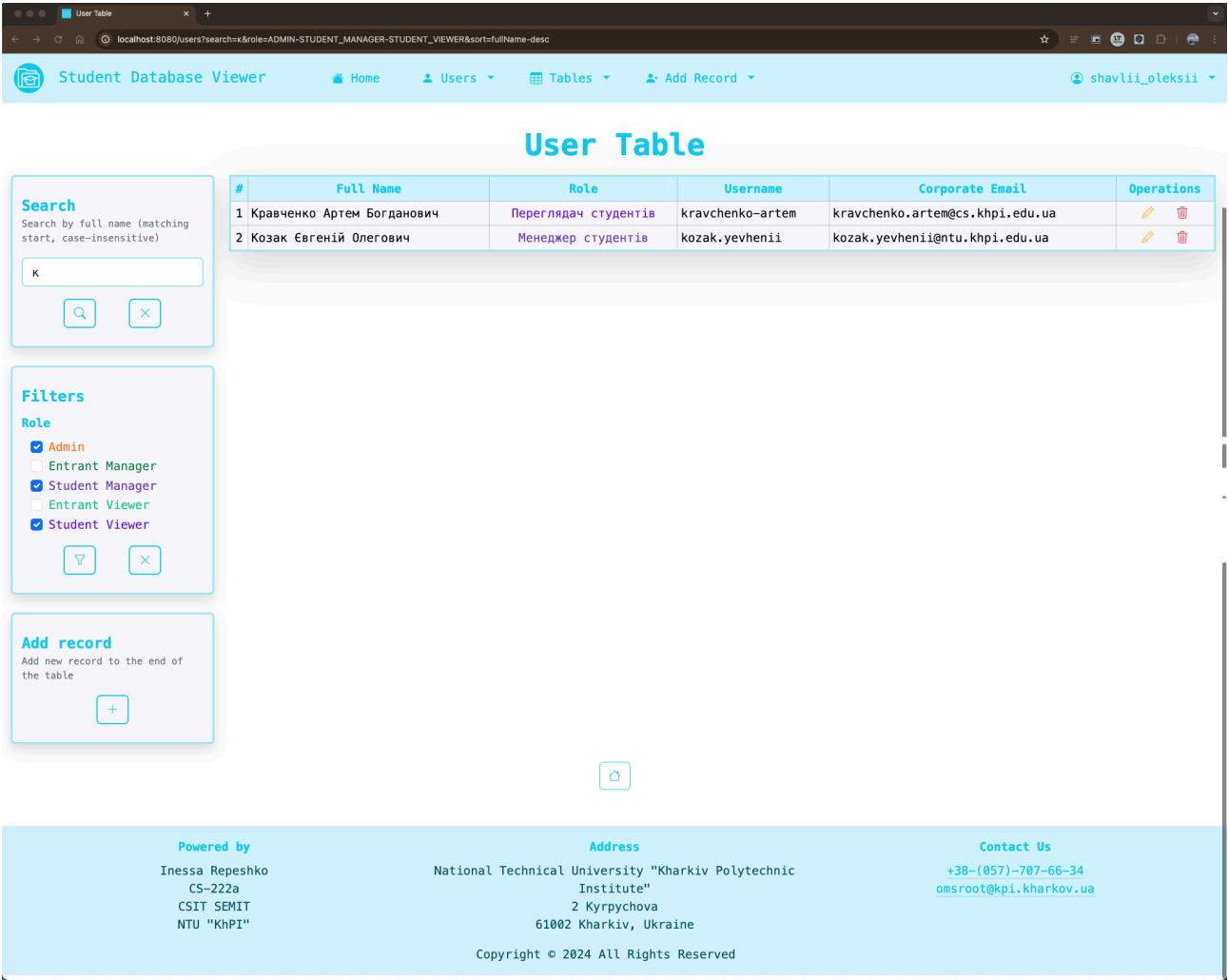


Рисунок Л.5.10 – Сторінка таблиці користувачів для Адміністратора із застосуванням пошуку, фільтрації та сортування